Math 3330: Regression Notes

Kelly Ramsay

2024-06-10

Table of contents

Preface **3**

1 Introduction **4**
1.1 What is the course about? . . . 4
1.1.1 The main question . . . 4
1.1.2 Using our data, how can we determine f? . . . 5
1.1.3 Comparison with means example . . . 5
1.2 Important course information and preparation tasks . . . 8
1.2.1 Prerequisite review . . . 8
1.2.2 Software . . . 9
1.2.3 Outline . . . 9
1.2.4 Homework tasks: . . . 9

2 Review material **10**
2.1 Review of random variables . . . 10
2.1.1 Discrete Random Variables . . . 10
2.1.2 Continuous Random Variables . . . 11
2.1.3 Properties of Random Variables . . . 11
2.1.4 Useful properties of normal and related random variables . . . 13
2.1.5 Central Limit Theorem . . . 14
2.1.6 Homework stop 1 . . . 14
2.2 Review of introductory statistics . . . 14
2.2.1 Basic premise of statistics . . . 15
2.2.2 Confidence intervals . . . 15
2.2.3 Hypothesis tests . . . 16
2.2.4 Homework stop 2 . . . 26
2.3 Review of matrices and linear algebra . . . 27
2.3.1 Matrix properties . . . 27
2.3.2 Important identities . . . 30
2.3.3 Homework stop 3 . . . 31
2.4 Review Random Vectors . . . 31
2.4.1 Definition of random vectors . . . 31
2.4.2 Expected Value and Covariance . . . 31
2.4.3 Properties of expected value and covariance . . . 32
2.4.4 Multivariate normal distribution . . . 33

2.4.5 Homework stop 4 . . . 33

3 Linear Regression **34**

3.1 Basics of linear regression . . . 34
3.1.1 The linear regression model . . . 34
3.1.2 The multiple linear regression model . . . 41
3.1.3 Homework stop 1 . . . 43
3.2 Least Squares . . . 43
3.2.1 Notation . . . 43
3.2.2 Least squares estimation . . . 45
3.2.3 Example . . . 49
3.2.4 Homework stop 2 . . . 52
3.3 Least squares inference . . . 53
3.3.1 Important quantities: Residuals and fitted values . . . 53
3.3.2 Variation decomposition . . . 55
3.3.3 Coefficients of determination . . . 57
3.3.4 The F test . . . 57
3.3.5 Homework stop 3 . . . 61
3.3.6 Significance of one variable . . . 62
3.3.7 Inference for the mean response and prediction intervals . . . 67
3.3.8 Homework stop 4 . . . 69
3.3.9 Partial testing . . . 69
3.3.10 Partial coefficient of determination . . . 71
3.3.11 Another example . . . 76
3.4 What variables should I include? Omitted variable bias, collider bias, mediators, and moderators . . . 82
3.4.1 Frisch-Waugh-Lovell Theorem . . . 82
3.4.2 Omitted variable bias . . . 85
3.4.3 Collider bias, mediators, and moderators . . . 85
3.5 Checking model assumptions . . . 86
3.5.1 Checking normality . . . 87
3.5.2 Checking the other assumptions . . . 92
3.5.3 Homework stop 5 . . . 102
3.6 Simple linear regression . . . 102
3.6.1 Estimated Coefficients . . . 102
3.6.2 Inference in SLR . . . 104
3.6.3 Inference for the correlation coefficient . . . 105
3.6.4 Homework stop 6 . . . 109
3.7 Additional concepts & examples . . . 110
3.7.1 Beware scatter plots in MLR . . . 110
3.7.2 Homework questions . . . 123

4 Residual analysis **124**
4.1 Properties of residuals . . . 124
4.2 Types of residuals . . . 125
4.3 Revisiting checking model assumptions . . . 128
4.3.1 Example . . . 128
4.4 Homework stop . . . 145

5 Transformations **146**
5.1 Variance-stabilizing transformations . . . 146
5.1.1 Inverse transformations . . . 153
5.2 Linearizing the model . . . 178
5.3 Box Cox Transformations . . . 187
5.4 Homework stop . . . 194

6 Indicator Variables **195**
6.1 What are indicator variables? . . . 195
6.2 Interaction effects . . . 201
6.3 Increasing codes and quantitative regressors via dummy variables . . . 207
6.4 A larger scale example . . . 211
6.5 Homework questions . . . 250

7 Leverage and Influence **251**
7.1 Influential observations and leverage . . . 251
7.2 Cook's Distance . . . 253
7.3 Data depth functions . . . 254
7.4 Homework questions . . . 268

8 Multicollinearity **270**
8.1 Multicollinearity and the problems it creates . . . 270
8.2 Multicollinearity Diagnostics . . . 273
8.2.1 VIF . . . 273
8.2.2 Condition number . . . 273
8.3 Homework questions . . . 296

9 Variable/Model Selection **297**
9.1 Variable Selection . . . 297
9.2 Deciding if one subset is better than another . . . 298
9.2.1 Coefficients of determination . . . 298
9.2.2 Mallows C_k criterion . . . 301
9.2.3 Akaike information criterion . . . 307
9.2.4 Bayesian information criterion/Schwartz information criterion . . . 307
9.3 Algorithms for model selection . . . 311
9.3.1 All subsets regression . . . 312

9.3.2 Forward selection . . . 320
9.3.3 Backward elimination/selection . . . 322
9.3.4 Stepwise regression . . . 323
9.4 Cross Validation . . . 332
9.5 LASSO Regression . . . 338
9.6 A final note . . . 342
9.7 Homework questions . . . 343

10 Robust Regression **344**
10.1 M-estimators . . . 344
10.2 Different ρ functions . . . 345
10.2.1 Least Squares . . . 345
10.2.2 Huber's Loss . . . 345
10.2.3 Ramsay's E Function . . . 346
10.2.4 Andrews' Wave Function . . . 346
10.2.5 Tukey's Loss . . . 346
10.3 Computing M-estimators . . . 351
10.3.1 Iterated re-weighted least squares . . . 351
10.3.2 Gradient descent . . . 352
10.4 Homework questions . . . 357

References **358**

Appendices **359**

A Introduction to R software **359**
A.1 Some Basics . . . 359
A.2 Booleans . . . 360
A.3 Vectors . . . 361
A.4 Matrices . . . 364
A.5 Functions . . . 367
A.6 Plotting . . . 369
A.7 If Statements . . . 372
A.8 Loops . . . 374
A.9 Coverage Probability Example . . . 376

Preface

Welcome to the MATH 3330 Notes! Please install R and R studio (Or you can use VSCode if you're comfortable there.)

In order to make the most of these notes, do all of the exercises in the order they appear.

There will likely be typos and some errors, please let me know if you encounter any.

These notes should be used in conjunction with the book:

Title: Introduction to Linear Regression Analysis, *Volume 821 of Wiley Series in Probability and Statistics* Authors: Douglas C. Montgomery, Elizabeth A. Peck, G. Geoffrey Vining Edition: 5, illustrated, reprint Publisher: John Wiley & Sons, 2012 ISBN: 0470542810, 9780470542811

1 Introduction

1.1 What is the course about?

1.1.1 The main question

The whole course is concerned with the following problem: Suppose that X and Y are some attributes of a population. What is the relationship between X and Y. How can we use X to predict Y, or how can we use X to explain Y?

For example, questions of this form include:

- How is location, square feet, parking available related to the price of an Airbnb?
- How is hours played and age related to win rate in League of Legends?
- How are creatine and protein consumption related to deadlift 1RM?
- How is treatment (A or B) related to pain levels of patients?

All of these can be answered with regression!

Exercise 1.1. What is X and what is Y here?

Solution 1.1.

- X: location, square feet, parking available Y: price of an Airbnb
- X: hours played and age Y: win rate in League of Legends
- X: creatine and protein consumption Y: deadlift 1RM
- X: treatment (A or B) Y: pain levels of patients

We suppose at the population level, **on average** that $Y = f(X)$. By on average, we mean that each person may not have exactly $Y = f(X)$,, but if we we average out Y for many people, we will have that the average is approximately $f(X)$. (This will be made more formal later).

For instance, consider the pain level question in the above example. Suppose that $f(A) = 2$ and $f(B) = 5$. Then, if we average the pain level of many patients who take treatment B, it should be close to 5.

Obviously, we cannot observe the whole population, and so we will assume that we have observed X and Y for a set of n individuals. Specifically, we observe some outcome $Y_1, \dots, Y_n$,

which is a real number and some attributes (categorical or numeric) about the n individuals, denoted by $X_1, \ldots, X_n$. Note that here X_i can be vectors or single numbers.

1.1.2 Using our data, how can we determine f?

Other, related questions:

- What is the form of f? Is it linear?
- How can we estimate f, say with $\hat{f}$? What is the best $\hat{f}$? What is the error of $\hat{f}$ on average?
- How can we tell if our model is good? i.e. how does $\hat{f}$ fit the data?
- How can we tell which X values are important? How can we tell if X is related to Y at all?
- What is the effect of correlation of X values?

These are all questions we will answer in this course.

Statistical modelling starts as follows:

1. Question about a population, e.g., "How are hours played and age related to win rate in League of Legends?"
2. Data: $(Y_1, X_1), \ldots, (Y_n, X_n)$
3. Explore data with graphs and summary stats
4. Use exploratory data analysis to posit a model for the population.

Note that step 4 is necessary! Letting f be anything is too general and won't work well, so we need to use the data to give us a hint at the form of f! For instance, we might suppose that f is a linear function! That is, $f \in \{g(X) = X\beta \colon \beta \in \mathbb{R}^d\}$.

Next, we proceed with the following steps:

5. Estimation: How to get an estimate $\hat{\beta}$ of β?
6. Inference: What is the error of $\hat{\beta}$? Is f degenerate? I.e., is $\beta = 0$?
7. Fit: Does our fitted line match up with the data? What about the normality assumption? Do the errors appear normal?
8. Prediction: Predict any values if necessary.

1.1.3 Comparison with means example

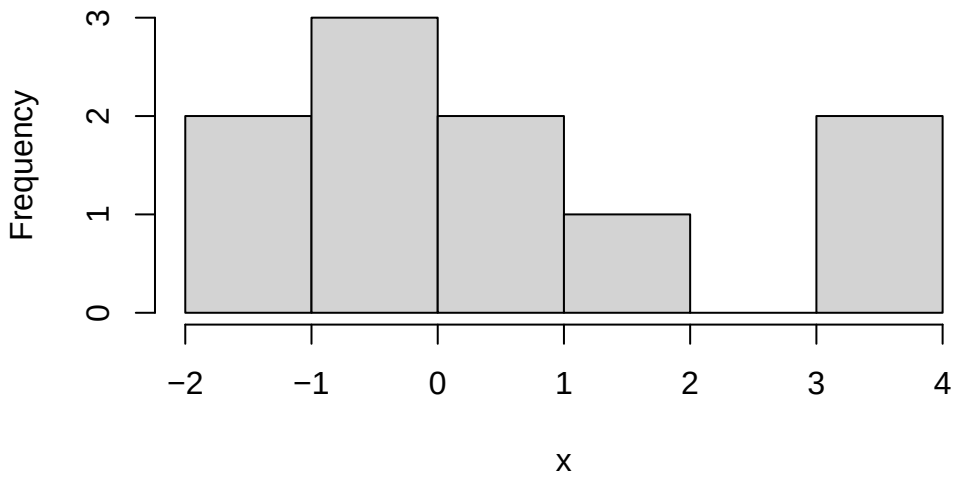
Let's compare to what we learned in previous statistics courses about two sample testing with the above steps in mind. Below we have different hours of extra sleep for two different treatments. Let's see if the sleep for groups 1 and 2 differ.

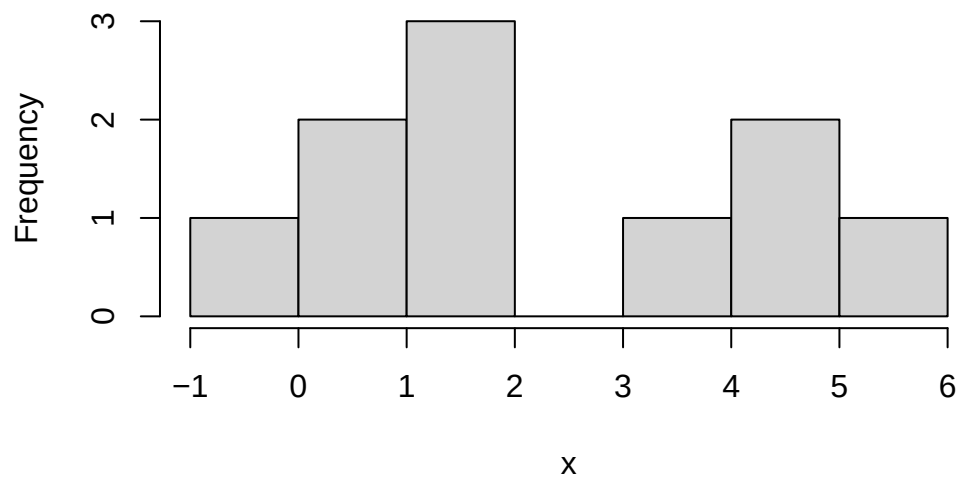
1. Do the counts for A and B differ?

```r
# 2.
data('sleep')
head(sleep)
```

```
  extra group ID
1   0.7     1  1
2  -1.6     1  2
3  -0.2     1  3
4  -1.2     1  4
5  -0.1     1  5
6   3.4     1  6
```

```r
# 3.
aggregate(extra ~ group, data = sleep, FUN = function(x){hist(x,main=names(x))})
```






```
Warning in format.data.frame(if (omit) x[seq_len(n0), , drop = FALSE] else x, :
corrupt data frame: columns will be truncated or padded with NAs
```

```
  group                   extra
1     1   -2, -1, 0, 1, 2, 3, 4
2     2 -1, 0, 1, 2, 3, 4, 5, 6
```

```
summary_stats = aggregate(extra ~ group, data = sleep,  FUN = summary)
print(summary_stats)
```

```
  group extra.Min. extra.1st Qu. extra.Median extra.Mean extra.3rd Qu.
1     1     -1.600        -0.175        0.350      0.750         1.700
2     2     -0.100         0.875        1.750      2.330         4.150
  extra.Max.
1      3.700
2      5.500
```

```
aggregate(extra ~ group, data = sleep, FUN = length)
```

```
  group extra
1     1    10
2     2    10
```

We will assume that the extra hours are normal from the histograms.

Recall then that the pooled standard deviation is $\hat{\sigma}_p = \sqrt{((n_x-1)\hat{\sigma}_x^2 + (n_y-1)\hat{\sigma}_y^2)/(n_x+n_y-2)}$ and the test statistic is:

$$T = \frac{\bar{X}-\bar{Y}}{\hat{\sigma}_p \times \sqrt{1/n_x + 1/n_y}}.$$

In addition, we have that $T \sim t_{n_x+n_y-2}$.

```
# 5 and 6 - here these steps are the same, since we are only doing inference
t.test(sleep$extra[sleep$group==1],sleep$extra[sleep$group==2])
```

```
    Welch Two Sample t-test

data:  sleep$extra[sleep$group == 1] and sleep$extra[sleep$group == 2]
t = -1.8608, df = 17.776, p-value = 0.07939
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -3.3654832  0.2054832
sample estimates:
mean of x mean of y
     0.75      2.33
```

```
# 7 - we checked normality earlier, 8 is not applicable
```

Here, we fail to reject the null hypothesis, and there is not enough evidence to suggest that there is a different between the groups. Notice that the p-value is 0.08, which is moderately low, so there is some evidence of a difference between the groups.

1.2 Important course information and preparation tasks

1.2.1 Prerequisite review

If you have forgotten, you should review the following concepts:

- Sample vs. population, estimates vs. parameters, hypothesis testing and confidence intervals
- Normal theory, random variables, conditional variance and expectation.
- CLT, LLN
- Linear algebra: Matrix operations, inverse, transpose etc.

1.2.2 Software

Download RStudio/R. You can use python, but I'll use R in class. If you are not familiar with R, please follow this tutorial here.

1.2.3 Outline

The course will proceed as follows:

- Review
- Core linear regression concepts
- Special Cases
- Advanced

1.2.4 Homework tasks:

- Download and install RStudio and R Software
- Think of a relationship you would want to model, what is X? what is Y?
- Review prerequisites as stated above

2 Review material

2.1 Review of random variables

Recall that

Definition 2.1. A random variable X is a function which maps outcomes $\omega \in \Omega$ to the real numbers, i.e., $X \colon \Omega \to \mathbb{R}$.

> **Note**
>
> Note that the notation $f \colon A \to B$ means that f is a function whose domain is A and range is B. That is, f takes a value from A and outputs some value in B.

Generally, we will just write X, and ignore the fact that X is a function.

We can categorize a random variable X as follows:

- If $X \colon \Omega \to S$ where S is countable, then X is a *discrete random variable*
- We say X is a *continuous random variable* if $\Pr(X = r) = 0$ for all $r \in \mathbb{R}$.
- Otherwise, X is a *mixed random variable* (which we won't worry about in this course)

2.1.1 Discrete Random Variables

If $X \colon \Omega \to S$ where S is countable, then X is a discrete random variable. S can be finite, but can also be any infinite subset of the integers $\mathbb{Z}$. The distribution of X is given by its PMF, denoted by $f(x)$. For any $x \in S$, $f(x) = \Pr(X = x)$. (Note that '$\in$' means the word "in".)

We must have that:

- $\sum_{x \in S} f(x) = 1$, (This notation means summing over all the elements in S.)
- $\forall x \in S$, $0 \leq f(x) \leq 1$. (This notation means for all x in S, $0 \leq f(x) \leq 1$.)

Examples: Binomial random variables, Poisson random variables and Geometric random variables are all discrete random variables.

Exercise 2.1. What is the PMF of a Binomial random variable? Can two different random variables have the same PMF? Why or why not?

Solution 2.1. First: $\Pr(X = x)\binom{n}{x}p^x(1-p)^{n-x}$ Second: Yes. Two random variables can be different random variables, but have the same distribution.

2.1.2 Continuous Random Variables

We say X is a *continuous random variable* if $\Pr(X = r) = 0$ for all $r \in \mathbb{R}$. If $X\colon \Omega \to S$ and X is a continuous random variable, then S is typically the real numbers, denoted by $\mathbb{R}$, but can be any uncountable subset of $\mathbb{R}$. The distribution of X is given by the PDF $f(x)$. For any interval $(a, b) \subset\in S$, $\Pr(X \in (a, b)) = \int_a^b f(x)dx$.

We must have that:

- $\int_{-\infty}^{\infty} f(x)dx = 1$,
- $\forall x \in \mathbb{R}$, $f(x) \geq 0$.

Examples: Normal random variables, Chi-squared random variables, t random variables, Cauchy random variables, F random variables are all continuous random variables. Generally, we will focus on continuous random variables.

Exercise 2.2. What is the PMF of a Normal random variable?

Solution 2.2. $f(x) = \frac{1}{\sqrt{2\pi}\sigma}e^{-(x-\mu)^2/2\sigma^2}$

2.1.3 Properties of Random Variables

Let X, X_1, X_2 be random variables.

Recall the important quantities $\mathrm{E}(X)$, $\mathrm{var}(X)$, $\mathrm{cov}(X_1, X_2)$, $\mathrm{corr}(X_1, X_2)$. Recall expectation:

Definition 2.2. The expectation of a random variable X is

$$\mathrm{E}(X) = \sum_{x \in S} x \Pr(X = x),$$

if X is discrete and is

$$\mathrm{E}(X) = \int_{-\infty}^{\infty} xf(x)dx,$$

if X is continuous.

This is the "average'' value of the random variable. Note that it is possible for it to be impossible for $X = \mathrm{E}(X)$. Try to come up with an example of this!

Definition 2.3. The variance of a random variable X is

$$\mathrm{Var}\left[X\right] = \mathrm{E}\left[\left|X - \mathrm{E}\left[X\right]\right|^2\right] = \sum_{x \in S} (x - \mathrm{E}\left[X\right])^2 \Pr\left(X = x\right),$$

if X is discrete and is

$$\mathrm{Var}\left[X\right] = \mathrm{E}\left[\left|X - \mathrm{E}\left[X\right]\right|^2\right] = \int_{-\infty}^{\infty} (x - \mathrm{E}\left[X\right])^2 f(x) dx,$$

if X is continuous.

The variance describes the variation of X about its mean. In other words, it describes on "average'', how far is X from its mean.

Definition 2.4. The covariance between two random variables X and Y is

$$\mathrm{cov}\left[X, Y\right] = \mathrm{E}\left[(X - \mathrm{E}\left[X\right])(Y - \mathrm{E}\left[Y\right])\right].$$

The covariance describes the unnormalised linear association between X and Y.

Definition 2.5. The correlation between two random variables X and Y is

$$\mathrm{corr}\left[X, Y\right] = \mathrm{cov}\left[X, Y\right] / \sqrt{\mathrm{Var}\left[X\right] \mathrm{Var}\left[Y\right]}.$$

The correlation describes the normalized linear association between X and Y.

Next, recall that for a random variable X, its cumulative distribution function (CDF) is given by $F_X(x) = \Pr\left(X \leq x\right)$. The joint CDF of X and Y is given by $F_{XY}(x, y) = \Pr\left(X \leq x, Y \leq y\right)$.

Lastly, for a vector of d random variables $\mathbf{X} = (X_1, \ldots, X_d)$, let its CDF by $F_{\mathbf{X}}(\mathbf{x}) = \Pr\left(X_1 \leq x_1, \ldots X_d \leq x_d\right)$, where here $\mathbf{x} \in \mathbb{R}^d$ and $\mathbf{x} = (x_1, \ldots, x_d)$.

We next present the concept of independence of random variables. Let $F_{XY}(x, y)$ be the joint CDF of X and Y and let F_X and F_Y be the univariate CDFs of X and Y, respectively. For two random variables X and Y, we say that X and Y are independent if $F_{XY}(x, y) = F_X(x) F_Y(y)$. More generally, two vectors of random variables $\mathbf{X}$ and $\mathbf{Y}$ are independent if $F_{(\mathbf{X},\mathbf{Y})}(\mathbf{x}, \mathbf{y}) = F_{\mathbf{X}}(\mathbf{x}) F_{\mathbf{Y}}(\mathbf{y})$, where $F_{(\mathbf{X},\mathbf{Y})}$ is the CDF of the vector $(\mathbf{X}, \mathbf{Y})$. A set of random variables $\{X_i\}_{i=1}^n$ are mutually independent if for any two mutually exclusive subsets of $\{X_i\}_{i=1}^n$ are also independent. Note that we write $X \perp Y$ if X is independent of Y.

We have that:

Theorem 2.1. *The following holds:*

- $X_1 \perp X_2 \implies E[X_1 X_2] = E[X_1]\, E[X_2]$
- $X_1 \perp X_2 \implies corr[X_1, X_2] = 0$
- $corr[X_1, X_2] = 0$ *does not imply* $X_1 \perp X_2$

Exercise 2.3. Prove Theorem 2.1 .

Let $X, X_1, X_2, \ldots X_n$ be random variables. Recall the linearity of expectation property:

Theorem 2.2. *For* $a, b \in \mathbb{R}$*, it holds that* $\mathrm{E}(aX + b) = a\mathrm{E}(X) + b$.

Exercise 2.4. Prove Theorem 2.2 .

As a corollary of Theorem 2.2 , we have that

- $\mathrm{E}\left[\sum_{i=1}^n a_i X_i\right] = \sum_{i=1}^n a_i \mathrm{E}\left[X_i\right]$
- $\mathrm{Var}\left[\sum_{i=1}^n a_i X_i\right] = \sum_{i=1}^n a_i^2 \mathrm{Var}\left[X_i\right] + \sum_{i \neq j} a_i a_j \mathrm{cov}\left[X_i, X_j\right]$
- $\mathrm{Var}\left[aX_1 + bX_2 + c\right] = a^2 \mathrm{Var}\left[X_1\right] + b^2 \mathrm{Var}\left[X_2\right] + 2ab\ \mathrm{cov}\left[X_1, X_2\right]$

Exercise 2.5. What happens to $\mathrm{Var}\left[aX_1 + bX_2 + c\right]$ when $\{X_i\}_{i=1}^n$ are mutually independent?

Exercise 2.6. Let $X_1, X_2, \ldots X_n$ be i.i.d. (independent and identically distributed) random variables with expectation (also known as mean) μ and variance σ^2. What is the expectation and variance of

$$\bar{X} = \sum_{i=1}^n X_i / n?.$$

2.1.4 Useful properties of normal and related random variables

Let

- $\mathcal{N}(\mu, \sigma^2)$ represent the normal distribution with mean μ and variance σ^2.
- χ_k^2 be the Chi-squared distribution with k degrees of freedom
- t_n be the student-t distribution with n degrees of freedom
- $F_{m,n}$ be the F distribution with m numerator degrees of freedom and n denominator degrees of freedom

We have the following results:

Theorem 2.3. *Suppose that* $X \sim \mathcal{N}(\mu, \sigma^2)$*, then*

- $Z = \frac{X-\mu}{\sigma} \sim \mathcal{N}(0,1)$
- $Z^2 \sim \chi^2_1$.

Let $[n] = \{1, \dots, n\}$. We also have that

Theorem 2.4.

- *If for* $i \in [n]$ $Y_i \sim \chi^2_{k_i}$ *and* $Y_i \perp Y_j$ *for* $i \neq j$ *then* $\sum_{i=1}^n Y_i \sim \chi^2_{k_1+\dots+k_n}$.
- *If* $Y \sim \chi^2_k$ *and* $Y \perp Z$, *then* $Z/\sqrt{Y/k} \sim t_k$.
- *If* $Y_1 \sim \chi^2_{k_1}$, $Y_2 \sim \chi^2_{k_2}$ *and* $Y_1 \perp Y_2$ *then* $\frac{Y_1/k_1}{Y_2/k_2} \sim F_{k_1,k_2}$.

Define

$$\hat{\sigma}^2 = \frac{1}{n-1}\sum_{i=1}^n (X_i - \bar{X})^2.$$

Theorem 2.5. *Suppose that* $X_1, X_2, \dots X_n \sim \mathcal{N}(\mu, \sigma^2)$ *and are independent, then* $\frac{\bar{X}-\mu}{\sigma/\sqrt{n}} \sim \mathcal{N}(0,1)$, $\bar{X} \perp \hat{\sigma}^2$, $(n-1)\hat{\sigma}^2/\sigma^2 \sim \chi^2_{n-1}$ *and* $\frac{\bar{X}-\mu}{\hat{\sigma}/\sqrt{n}} \sim t_{n-1}$.

2.1.5 Central Limit Theorem

CLT: If $X_1, X_2, \dots X_n$ are i.i.d. with mean μ and variance $\sigma^2 < \infty$, then $\frac{\bar{X}-\mu}{\sigma/\sqrt{n}} \xrightarrow{\text{d}} \mathcal{N}(0,1)$ as $n \to \infty$.

We have that in general, for large n, regardless of the distribution of the random variables, the sample mean is approximately normally distributed.

2.1.6 Homework stop 1

Review your material and complete the above exercises before continuing to the next section.

2.2 Review of introductory statistics

The followings are some concepts that you have learned from prerequisites, and/or we have reviewed in the last two lectures.

- Sample vs. Population
- Observation vs. Random variable
- Statistic vs. Parameter
- Estimate vs. Estimator
- Estimator is a random variable and estimate is a number calculated from data

- Mean and variance of random variable
- Relationships between Normal, t , χ^2, F etc.

2.2.1 Basic premise of statistics

The whole purpose of statistics is to learn something about a population using only a sample of units from that population. A **sample** is a smaller, typically randomly selected, subset of a population. A **population** is a collection of units which we would like to know something about. For example, we may collect a sample of hamburgers from McDonald's if we want to learn something about the population of McDonald's hamburgers.

In general, at least for this course, we assume that we have access to a sample of units from a given population. Furthermore, we assume that that sample is a **random sample**. Specifically, we assume that these units in the sample are realizations of random variables. In addition, we also assume that these random variables are mutually independent. For example, we could assume that our sample $X_1, \dots, X_n$ is Normally distributed with some fixed mean μ and fixed variance σ^2. In this case, μ and σ^2 are unknown **parameters** of the population. A parameter of a population is some quantity that is a function of the distribution of our given sample. For instance, $\mathrm{E}[X_i] = \mu$. Generally, we are concerned with unknown population parameters, which are parts of the distribution that are unknown, and can only ever be estimated. For example, we may know our data is normal, but not know the mean parameter. In that case, we need to use an **estimate** of the parameter. We use a function of the data, typically called the estimator, say T, which produces the estimate, given by T computed at the sample we observed: $T(X_1, \dots, X_n)$.

For example, to estimate μ, we typically use the sample mean. Here, the estimate is given by $\bar{X} = \sum_{i=1}^{n} X_i/n$. To be specific, the estimate is the value of $\bar{X}$ and the estimator T is the function that maps n real numbers to their mean. In general, estimates are used to give our 'best guess' at population parameters.

2.2.2 Confidence intervals

Recall from the previous section that our estimate of a parameter is only that, an estimate. In other words, it is not exactly equal to the population parameter. For instance, if we drew a different sample our estimate would change. A confidence interval is used to acknowledge this phenomenon in the reporting of our statistics. Its used to give a range of estimates that we might have obtained from any "regular" sample we might observe. It is ultimately used to quantify the error (sometimes called uncertainty) in our estimate.

Confidence intervals consist of a level, usually denoted by $(1-\alpha)100\%$ and two end points. For example, you have learned confidence intervals for the population mean. When we say $(-1, 1)$ is 95% confidence interval for the population mean, what does this mean? Colloquially, it means that we expect the sample mean to be somewhere within $(-1, 1)$ with high confidence.

Note that confidence intervals are computed from the data, which means also that for each new sample, we would get a different confidence interval. However, the population parameter never changes. Therefore, the interval is what is varying from sample to sample. This impacts the interpretation of a confidence interval.

Continuing our example, we have that the interval $(-1, 1)$ can be interpreted as: "if we drew many more samples, 95% of the **intervals** will contain the population parameter." We **do not** say that the parameter has a 95% chance of falling in (-1,1), since the parameter is not random, the interval end points are.

For example, we have the formula for a confidence interval for the population mean is given by: $\bar{X} \pm 1.96\hat{\sigma}$. Notice that it is based only on the data. Therefore, it will change if we drew a new sample.

To summarize this section, a confidence interval is used to quantify the uncertainty in our reported estimates. By uncertainty, we specifically mean the uncertainty resulting from the fact that we have only a sample of the population, and our estimate varies depending on the sample.

2.2.3 Hypothesis tests

Hypothesis tests are used to determine whether an effect is spurious or a real property of the population. A spurious effect is one that is specific to the sample we observed, and is not a real property of the population. For example, if the heights of males and female students are measured, and we observe that the sample mean of both male and females are equal, then this would be a spurious effect. We know that the population heights of males and females are substantially different. If we drew a new sample, we would likely observe something that mirrors the population reality (provided it is large enough).

Formally, a hypothesis test compares two competing beliefs about a population parameter, called the null and alternative hypothesis. For instance, we may wish to test whether the population heights of men is greater than women, vs. the heights being less than or equal to that of men.

We write this as follows: $H_0 : \mu_{men} \leq \mu_{women}$ vs. $H_a : \mu_{men} > \mu_{women}$.

The null hypothesis is usually chosen to be one such that if we make a mistake, the error is most serious. However, it is usually clear from the context.

In general, we compute a test statistic and its distribution **under the null hypothesis.** Then we compute how likely it was to see the observed test statistic we saw, if the null hypothesis was true. This likelihood is given by the **p-value**. If it was sufficiently unlikely (in other words, the p-value is less than the threshold α), then we reject the null hypothesis. Otherwise, we fail to reject the null hypothesis. If we fail to reject the null hypothesis then either the null hypothesis is true, it is not true, but there was not enough data collected to show the effect.

There are two types of errors we can make in a hypothesis test: Type 1 and Type 2 error. Type one error occurs when we reject the null hypothesis when it is true. Type two error occurs when we fail to reject the null hypothesis when the alternative is true.

Let's do an example.

Exercise 2.7. In a study about online dating, you are interested in determining the average age of individuals who use online dating platforms. You want to know whether the average age of online daters is significantly different from 30. You have a dataset of 40 ages of people using online dating platforms.

How would you answer this question?

$$H_0 : \mu = 30 \qquad vs. \qquad H_1 : \mu \neq 30.$$

First, we can explore the data:

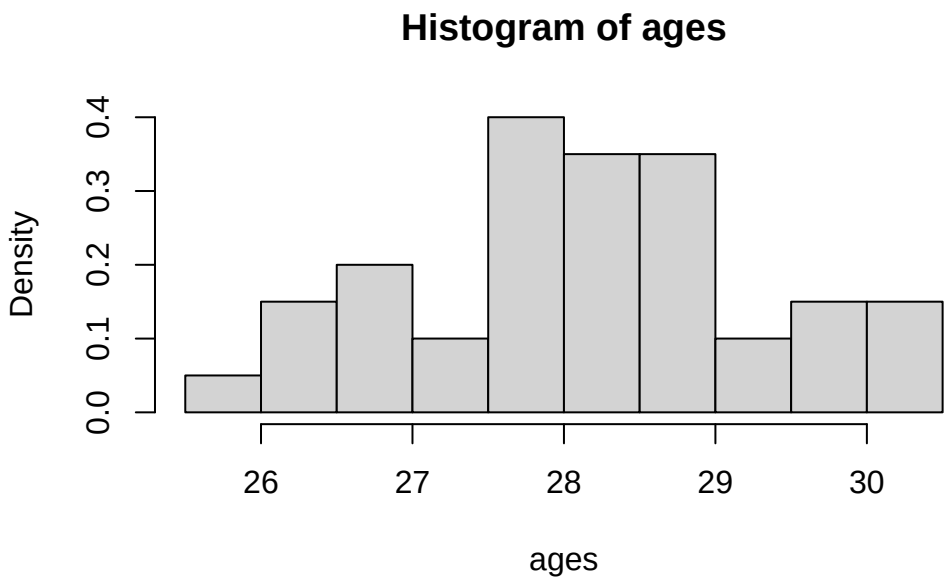
```
getwd()
```

```
[1] "C:/Users/12RAM/OneDrive - York University/Teaching/Courses/Math 3330 Regression/Math 333
```

```
ages=read.csv('C:\\Users\\12RAM\\OneDrive - York University\\Teaching\\Courses\\Math 3330 Reg



hist(ages,freq=F)
```



Now, assume that $X_1, \ldots, X_{40} \sim \mathcal{N}(\mu, \sigma^2)$, and independent. (We can justify normality with the histogram, or we could also invoke the CLT to get normality of the sample mean (not the data itself).) Therefore, we can do a one sample t-test. Recall that, under the null hypothesis, we have $\frac{\bar{X}-30}{\hat{\sigma}/\sqrt{n}} \sim t_{n-1}$. This means that if $\left|\frac{\bar{X}-30}{\hat{\sigma}/\sqrt{n}}\right| \geq t_{n-1,1-\alpha/2}$, then we reject the null hypothesis! Here, $t_{n-1,1-p}$ is the $(1-p)$th quantile of the t_{n-1} distribution. For large n and $p = 0.025$, this is roughly equal to 2.

Now, recall that

$$H_0 : \mu = 30 \qquad vs. \qquad H_1 : \mu \neq 30.$$

```
# Calculate the mean of the 'ages' data and assign it to xbar
xbar = mean(ages)
xbar  # Print the mean
```

```
[1] 28.16378
```

```
# Calculate the variance of the 'ages' data and assign it to ssq
ssq = var(ages)
ssq  # Print the variance
```

```
[1] 1.277377
```

```r
# Calculate the length (number of observations) of the 'ages' data and assign it to n
n = length(ages)
n  # Print the number of observations
```

```
[1] 40
```

```r
# Set the significance level
alpha = 0.05

# print test statistic
ts=(xbar -30)/(sqrt(ssq/n))
print(ts)
```

```
[1] -10.27532
```

```r
# compute p-value
pval=pt(ts,n-1)*2
pval
```

```
[1] 1.179149e-12
```

```r
# Perform a two-sided t-test to check if the mean of 'ages' is significantly different from 3
# t.test() is the function for performing t-tests in R
test = t.test(ages, mu = 30, alternative = 'two.sided')
test  # Print the test result
```

```
    One Sample t-test

data:  ages
t = -10.275, df = 39, p-value = 1.179e-12
alternative hypothesis: true mean is not equal to 30
95 percent confidence interval:
 27.80232 28.52524
sample estimates:
mean of x
 28.16378
```

We have that $\left|\frac{\bar{X}-30}{\hat{\sigma}/\sqrt{n}}\right| = 10.28$. Using R, we get that the p-value is 1.179×10^{-12}.

Here the p-value measures how much evidence there is against the null hypothesis. If the p-value is very small, then this constitutes strong evidence against the null hypothesis. If the p-value is small, but closer to 0.05, then there is evidence against the null. If it is larger, but still small, say 0.1, then this is weak evidence against the null hypothesis. It is not helpful to throw it away if it is above 0.05, therefore we should not just take $\alpha = 0.05$. Choosing α depends on how serious a type 1 error is. If it is not that serious, we can take α larger. If it is very serious, we can take α smaller.

In this example, there is very strong evidence against the null hypothesis.

> **Note**
>
> Note also that we can use the confidence interval method with
>
> $$\bar{X} \pm t_{n-1,1-\alpha/2}\sqrt{\hat{\sigma}^2/n}.$$

```
# Alternative method to calculate the confidence interval
# ci will store the confidence interval values
ci = xbar + c(-1, 1) * qt(1 - alpha / 2, n - 1) * sqrt(ssq / n)
ci  # Print the confidence interval
```

```
[1] 27.80232 28.52524
```

> **Note**
>
> Moving beyond the one-sample testing problem, we might be interested in other population parameters, say $\theta \in \Theta$. Think Lecture 1: $\mathrm{E}[Y|X] = \beta_0 + X\beta_1$, we might want to estimate $\mathrm{E}[Y|X]$, which amounts to $\beta_0, \beta_1 \in \mathbb{R}$! In general, we may estimate θ by $\hat{\theta}$. Then we may compute the variance and distribution of $\hat{\theta}$. From there, we can make confidence intervals and conduct hypothesis tests etc.

Let's do another example:

Exercise 2.8. In a study about online dating, you are interested in determining if the average age of those who identify as men who use online dating platforms differs from those who identify as women. You have a dataset of 20 ages of each group using online dating platforms.

What is the population parameter of interest here? It is $\Delta = \mu_1 - \mu_2$, the difference in means between the two populations. Now, suppose that $X_1, \ldots, X_{20} \sim \mathcal{N}(\mu_1, \sigma^2)$ and $Y_1, \ldots, Y_{20} \sim$

$\mathcal{N}(\mu_2, \sigma^2)$, and are mutually independent. (We could also invoke the CLT instead of assuming normality.) We can estimate those parameters with **estimates.** For instance, $\bar{X},\ \bar{Y}$,

$$\hat{\sigma}^2 = \frac{(n_1 - 1)\hat{\sigma}_1^2 + (n_2 - 1)\hat{\sigma}_2^2}{n_1 + n_2 - 2}.$$

Exercise 2.9. Suppose that $X_1, \ldots, X_{20} \sim \mathcal{N}(\mu_1, \sigma^2)$ and $Y_1, \ldots, Y_{20} \sim \mathcal{N}(\mu_2, \sigma^2)$, and are mutually independent. Compute $\text{Var}\left[\bar{X} - \bar{Y}\right]$.

Solution 2.3. Using independence of $\bar{X}$ and $\bar{Y}$ and the result of the Exercise 2.6 , we have that

$$\text{Var}\left[\bar{X} - \bar{Y}\right] = \text{Var}\left[\bar{X}\right] + \text{Var}\left[\bar{Y}\right] = \sigma_1^2/n_1 + \sigma_1^2/n_2.$$

First, we write down the null and alternative hypothesis:

$$H_0 \colon \Delta = 0 \qquad vs.. \qquad H_1 \colon \Delta \neq 0.$$

Here, we can do a two sample t-test.

Recall that the pooled variance is given by:

$$\hat{\sigma}_p^2 = \frac{(n_1 - 1)\hat{\sigma}_1^2 + (n_2 - 1)\hat{\sigma}_2}{(n_1 + n_2 - 2)}$$

We previously said that a multiple of a one sample standard deviation follows a Chi-squared distribution. It follows that $(n_1 - 1)\hat{\sigma}_1^2/\sigma^2 \sim \chi^2_{n_1 - 1}$ and $(n_2 - 1)\hat{\sigma}_2^2/\sigma^2 \sim \chi^2_{n_2 - 1}$. Using the theory from here, specifically, $(n_1 - 1)\hat{\sigma}_1^2/\sigma^2 + (n_2 - 1)\hat{\sigma}_2^2/\sigma^2$ is a sum of independent Chi-squared random variables, and so we have $(n_1 - 1)\hat{\sigma}_1^2/\sigma^2 + (n_2 - 1)\hat{\sigma}_2^2/\sigma^2 \sim \chi^2_{n_1 + n_2 - 2}$.

Again, using the theory from here, under the null hypothesis, we have that

$$\frac{\bar{X} - \bar{Y}}{\hat{\sigma}_p\sqrt{1/n_1 + 1/n_2}} = \frac{(\bar{X} - \bar{Y})/\sigma\sqrt{1/n_1 + 1/n_2}}{\hat{\sigma}_p/\sigma} \sim t_{n_1 + n_2 - 2}.$$

This follows from 3 facts, first, letting $Z = (\bar{X} - \bar{Y})/\sqrt{\text{Var}\left[\bar{X} - \bar{Y}\right]}$, note that $Z \sim \mathcal{N}(0, 1)$. We have that

$$Z = (\bar{X} - \bar{Y})/\sqrt{\text{Var}\left[\bar{X} - \bar{Y}\right]} = (\bar{X} - \bar{Y})/\sigma\sqrt{1/n_1 + 1/n_2}.$$

Next, we said earlier that $\bar{X}$ is independent of $\hat{\sigma}_1$ and $\bar{Y}$ is independent of $\hat{\sigma}_2$. Now, recall that if two random variables are independent, then any function of them is also independent. In other words, if X and Y are independent, then for real functions f and g, we have that $g(X)$ is

independent of $f(Y)$. It follows that $\bar{X}$ is independent of $\hat{\sigma}_2$ and $\bar{Y}$ is independent of $\hat{\sigma}_1$. It follows that $\bar{X} - \bar{Y}$ is independent of $\hat{\sigma}_p$. Then,

$$\frac{(\bar{X} - \bar{Y})/\sigma\sqrt{1/n_1 + 1/n_2}}{\hat{\sigma}_p/\sigma}$$

is a ratio of a standard normal random variable and the square root of a Chi-squared random variable, divided by its degrees of freedom. Further, the numerator and denominator are independent. Therefore, the above quantity follows a t distribution with $n_1 + n_2 - 2$ degrees of freedom.

This means that if $\left|\frac{\bar{X}-\bar{Y}}{\hat{\sigma}_p\sqrt{1/n_1+1/n_2}}\right| \geq t_{n_1+n_2-2,1-\alpha/2}$, then we reject the null hypothesis.

Let's execute the test in R:

```
# Normally, I will give you a dataset. Here I generate the data
set.seed(440)
female_ages=rnorm(20,28,4)
male_ages=rnorm(20,32,4)



# Check for equal variance
var(female_ages)
```

```
[1] 15.72805
```

```
var(male_ages)
```

```
[1] 26.22371
```

```
## Putting the data in a dataframe
cbind("Age"=c(female_ages,male_ages),"Gender"=rep(c(0,1),each=20))
```

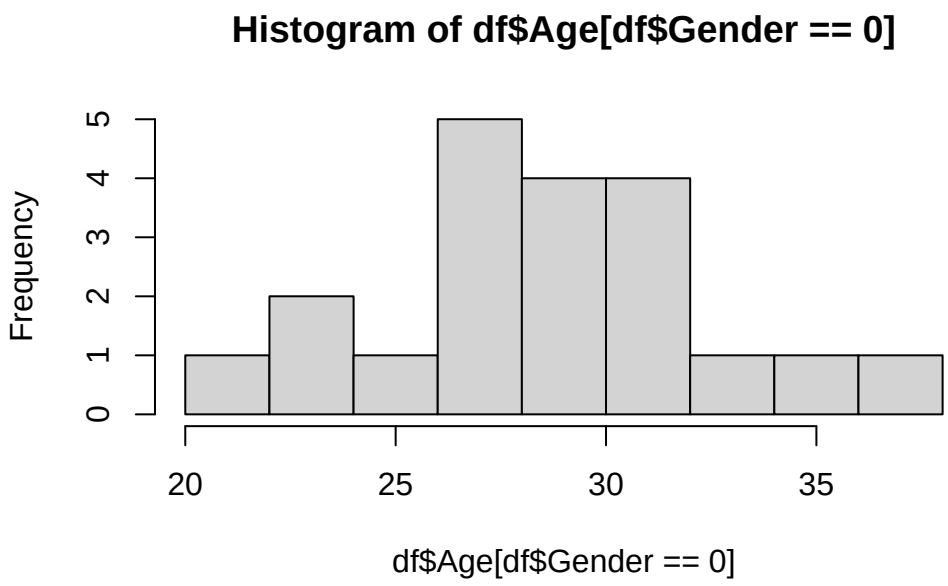
```
           Age Gender
 [1,] 37.19809      0
 [2,] 20.69693      0
 [3,] 27.80284      0
 [4,] 27.69463      0
 [5,] 29.53143      0
 [6,] 29.46190      0
```

```
 [7,] 30.41164      0
 [8,] 33.27790      0
 [9,] 22.65974      0
[10,] 30.73540      0
[11,] 34.08564      0
[12,] 27.58077      0
[13,] 23.26108      0
[14,] 30.94523      0
[15,] 31.52404      0
[16,] 29.13246      0
[17,] 26.95470      0
[18,] 24.80749      0
[19,] 28.60051      0
[20,] 26.76294      0
[21,] 25.94775      1
[22,] 40.16080      1
[23,] 25.58905      1
[24,] 32.16780      1
[25,] 29.87934      1
[26,] 35.46593      1
[27,] 35.71651      1
[28,] 37.76510      1
[29,] 27.23068      1
[30,] 33.41994      1
[31,] 40.43822      1
[32,] 31.04841      1
[33,] 32.66165      1
[34,] 38.28678      1
[35,] 34.72411      1
[36,] 39.57994      1
[37,] 26.85585      1
[38,] 31.87533      1
[39,] 23.71793      1
[40,] 30.54803      1
```

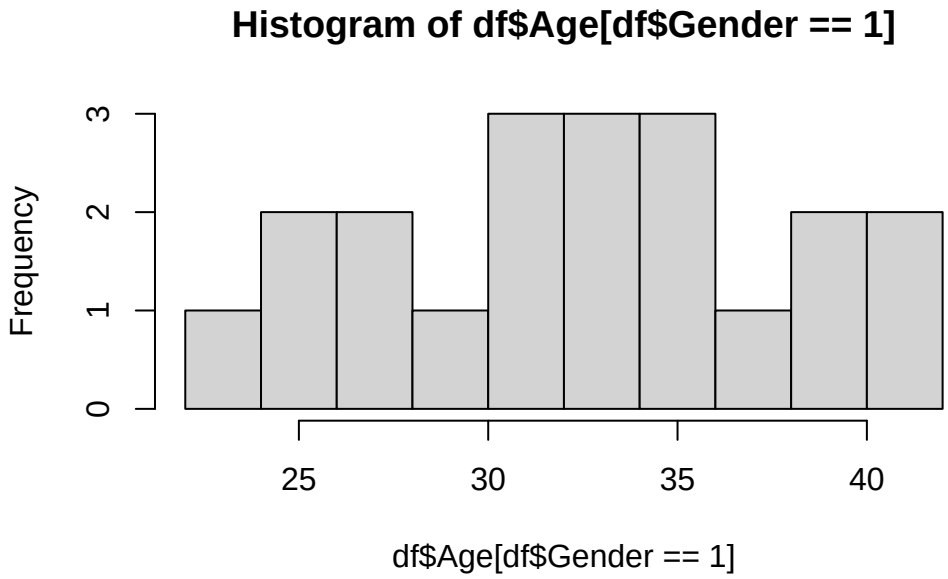
```r
df=data.frame(cbind("Age"=c(female_ages,male_ages),"Gender"=rep(c(0,1),each=20)))

#exploring the data
#hist(x) creates a histogram of the vector x

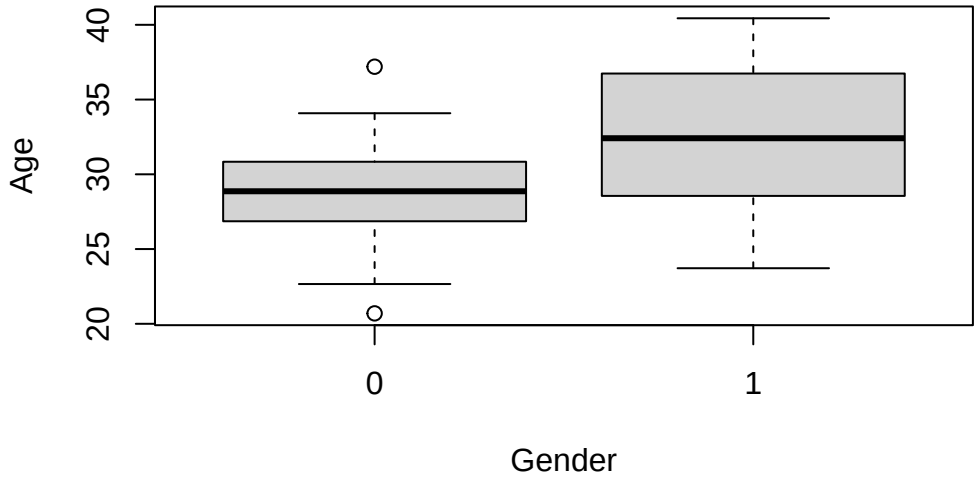
hist(df$Age[df$Gender==0])
```




```
hist(df$Age[df$Gender==1])
```



```r
#boxplot creates boxplots of Age against gender
boxplot(Age~Gender, df)
```




```r
test=t.test(Age~Gender,data=df,var.equal=TRUE)
test
```

```
    Two Sample t-test

data:  Age by Gender
t = -2.7603, df = 38, p-value = 0.008841
alternative hypothesis: true difference in means between group 0 and group 1 is not equal to
95 percent confidence interval:
 -6.929630 -1.065749
sample estimates:
mean in group 0 mean in group 1
       28.65627        32.65396
```

```r
#Interpret the P value, and CI, what are we going to say to a stakeholder?

#e.g.
test$estimate
```

```
mean in group 0 mean in group 1
       28.65627        32.65396
```

> **Note**
>
> Note also that we can use the confidence interval method, meaning that if if 0 is in the interval:
>
> $$\hat{\Delta} \pm t_{n_1+n_2-2,1-\alpha/2}\hat{\sigma}_p\sqrt{1/n_1+1/n_2},$$
>
> then we fail to reject the null hypothesis.

2.2.4 Homework stop 2

Exercise 2.10. IBM Human Resources (HR) department is evaluating job applicants from York University.
They are interested to know if the 2020 ITEC graduating class has an average GPA higher than 6 (i.e. average GPA higher than "B' '). They collected the GPA of 25 ITEC students graduated in 2020.

4.92	4.79	6.76	5.64	6.12	7.37	6.45	6.31	6.68
6.30	4.91	6.95	5.87	6.18	6.60	6.71	6.69	5.62
6.40	5.51	6.44	6.13	8.55	7.94	4.78	-	-

> **Tip**
>
> Use chatGPT to convert the above table to an R vector, so you don't have to waste time!

- For the one sample testing problem, i.e., you have a sample of n normal random variables, with unknown mean and variance and you want to test whether $H_0 : \mu = 0$ vs. $H_0 : \mu \neq 0$, show that $\frac{\bar{X}}{\hat{\sigma}/\sqrt{n}} \sim t_{n-1}$ under the null hypothesis, assuming the observations are normally distributed.
- What is the distribution of each of the following: $\bar{X}, \hat{\sigma}^2$ (sample mean and sample variance) under the assumption of normal data with unknown mean and variance?

Compare and contrast the following concepts. That is, define them and explain the difference between them.

- Sample vs. Population
- Observation vs. Random variable
- Statistic vs. Parameter
- Estimate vs. Estimator

2.3 Review of matrices and linear algebra

Recall that

Definition 2.6. An $(n \times m)$ matrix A takes the form

$$\begin{aligned} A &= \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{pmatrix} \\ &= ((a_{ij})) \qquad i = 1, \ldots, n, \quad j = 1, \ldots, m \end{aligned}$$

and a_{ij} is the element in the ith row and j^{th} column of the matrix A

We also define the following:

- An $(n \times 1)$ matrix is also known as a n dimensional column vector. Note: in this course, a vector means a column vector.
- A $(1 \times m)$ matrix is also known as a m dimensional row vector
- The n dimensional one vector, 1_n, (sometimes the subscript n is suppressed when the dimension is obvious), is an n dimensional column vector with all entries being 1.
- The $(n \times n)$ identity matrix, I_n, is the $(n \times n)$ matrix with diagonal entries set equal to 1 and the off diagonal entries set equal to 0

Throughout this section, we will use the following matrices to demonstrate the numerical calculations:

$$U = \begin{pmatrix} 1 & 2 & 3 \\ -1 & 4 & -2 \end{pmatrix}, \; V = \begin{pmatrix} 2 & 4 \\ 1 & -2 \\ -1 & 0 \end{pmatrix}, \; k = 4$$

2.3.1 Matrix properties

First, we define the transpose of a matrix:

Definition 2.7. Let $A = ((a_{ij}))$ for $i = 1, \ldots, n$ and $j = 1, \ldots, m$, is an $(n \times m)$ matrix. Then $A^\top = A$ transpose $= ((a_{ji}))$ for $j = 1, \ldots, m$ and $i = 1, \ldots, n$, and $A^\top$ is an $(m \times n)$ matrix.

When we transpose a matrix A, the rows of A becomes the columns of $A^\top$ and the columns of A becomes the rows of $A^\top$.

Example 2.1. Using our example matrices, we have that

$$U^\top = \begin{pmatrix} 1 & -1 \\ 2 & 4 \\ 3 & -2 \end{pmatrix}, \; V^\top = \begin{pmatrix} 2 & 1 & -1 \\ 4 & -2 & 0 \end{pmatrix}$$

Definition 2.8. Let $A = ((\, a_{ij} \,))$ and $B = ((\, b_{ij} \,))$ be two $(n \times m)$ matrices. Then

$$A \pm B = ((\, a_{ij} \pm \, b_{ij} \,)).$$

Addition and subtraction of matrices required the matrices to have the same dimension.

Example 2.2. Using our example matrices, we have that: $U + V$ is undefined because they are not of the same dimension, and

$$U + V^\top = \begin{pmatrix} 1+2 & 2+1 & 3+(-1) \\ (-1)+4 & 4+(-2) & (-2)+0 \end{pmatrix} = \begin{pmatrix} 3 & 3 & 3 \\ 3 & 2 & -2 \end{pmatrix}$$

Definition 2.9. Let $A = ((\, a_{ij} \,))$ for $i = 1, \ldots, n$ and $j = 1, \ldots, m$, is an $(n \times m)$ matrix and k is a constant. Then

$$kA = ((\, ka_{ij} \,)) = Ak,$$

i.e. each element of the matrix A is multiplied by k.

Example 2.3. Using our example matrices, we have that:

$$kU^\top = 4 \begin{pmatrix} 1 & -1 \\ 2 & 4 \\ 3 & -2 \end{pmatrix} = \begin{pmatrix} 4(1) & 4(-1) \\ 4(2) & 4(4) \\ 4(3) & 4(-2) \end{pmatrix} = \begin{pmatrix} 4 & -4 \\ 8 & 8 \\ 12 & -2 \end{pmatrix}$$

Definition 2.10. Let A and B be two matrices. Then A multiplied by B, AB, is defined only if (number of columns of A) = (number of rows of B).
The product is a ((number of rows of A) $\times$ (number of columns of B)) matrix.

More precisely, let $A = ((\, a_{ij} \,))$ be an $(n \times m)$ matrix and $B = ((\, b_{ij} \,))$ be an $(m \times p)$ matrix. Then $C = AB = ((\, c_{ij} \,))$ is an $(n \times p)$ matrix with

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{im}b_{mj}$$

> **Note**
>
> In matrix algebra, AB is not necessarily equal to BA.

Example 2.4. Using our example matrices, we have that:

$$\begin{aligned} UV &= \begin{pmatrix} 1 & 2 & 3 \\ -1 & 4 & -2 \end{pmatrix} \begin{pmatrix} 2 & 4 \\ 1 & -2 \\ -1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 1(2)+2(1)+3(-1) & 1(4)+2(-2)+3(0) \\ (-1)(2)+4(1)+(-2)(-1) & (-1)(4)+4(-2)+(-2)(0) \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ 4 & -12 \end{pmatrix} \end{aligned}$$

Assume all the matrix multiplication works. Let I_n be an $(n \times n)$ identity matrix. Then

$$AI_n = A, \quad \textit{and} \quad I_n B = B.$$

Definition 2.11. Let A be an $(n \times n)$ matrix. The inverse of A, A^{-1}, if exists satisfies

$$AA^{-1} = A^{-1}A = I_n$$

and if A^{-1} does not exist, then A is a singular matrix.

> **!** Important
>
> From your linear algebra course, a prerequisite, you have learned the condition(s) for the existence of an inverse, The Invertible Matrix Theorem and you have learned how to obtain an inverse. You should review them.
> Specifically, you should know how to obtain inverse of any diagonal matrix and any (2×2) non-singular matrix, i.e.,
>
> $$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Example 2.5. Using our example matrices, let

$$W = UV = \begin{pmatrix} 1 & 0 \\ 4 & -12 \end{pmatrix}$$

Then

$$W^{-1} = \frac{1}{1(-12)-0(4)} \begin{pmatrix} -12 & 0 \\ -4 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 1/3 & -1/12 \end{pmatrix}.$$

You can verify that $WW^{-1} = W^{-1}W = I_2$.

2.3.2 Important identities

Lastly, we introduce some important identities:

$$X = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \quad and \quad y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

Then

$$X^\top X = \begin{pmatrix} n & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{pmatrix}, \quad \text{and} \quad X^\top y = \begin{pmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i y_i \end{pmatrix}.$$

Also $\bar{y} = \frac{1}{n} 1^\top y$ and $\sum_{i=1}^n y_i = n\bar{y}$ and, finally $\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n y_i^2 - n\bar{y}^2$. These are useful identities that we will use throughout this course.

Exercise 2.11. Prove the previously introduced identities.

Lastly, we recall an important application of matrices. An application of matrices: Suppose that we want to solve for x_1, x_2, x_3 where they satisfy the following set of linear equations:

$$\begin{aligned} 2x_1 + 3x_2 - 4x_3 &= 0 \\ -x_1 + 4x_2 &= -1 \\ 5x_1 + x_2 - 2x_3 &= 4 \end{aligned}$$

We can set it up in matrix form as follows:

$$\begin{pmatrix} 2 & 3 & -4 \\ -1 & 4 & 0 \\ 5 & 1 & -2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \\ 4 \end{pmatrix}$$

Or it can be presented as $Ax = b$. If A is not a singular matrix, then $x = A^{-1}b$. Since $det(A) = 62$, it is not a singular matrix. Solving the above equation this using $x = A^{-1}b$ yields that $x = (1, 0, 0.5)^\top$. Keep this in mind, we will see it return in the next chapter.

Lastly - I will remind you of two definitions:

Definition 2.12. A set of vectors $v_1, \ldots, v_d$ is linearly independent if the only d-dimensional set of scalars $a_1, \ldots, a_d \in \mathbb{R}$ such that $\sum_{i=1}^d a_i v_i = 0$ implies is the d-dimensional 0 vector.

Definition 2.13. A $d \times d$ symmetric matrix A is positive-definite if for all $x \in \mathbb{R}^d \backslash \{0\}$ it holds that $x^\top A x > 0$.

2.3.3 Homework stop 3

Exercise 2.12. Let

$$W = \begin{pmatrix} 3 & 2 \\ -4 & 6 \end{pmatrix}$$

and $x = (2, 1)^\top$. Compute W^{-1}, $xx^\top$ and $x^\top W$. Verify that $WW^{-1} = W^{-1}W = I_2$.

- Prove each of the important identities.
- Verify $X^\top A = (A^\top X)^\top$.
- What is the rank of a matrix? Is a matrix's rank related to whether or not a matrix is invertible? Why?
- Define a positive definite matrix. When is $X^\top X$ positive definite?

2.4 Review Random Vectors

2.4.1 Definition of random vectors

Definition 2.14. Let $Y_1, \cdots, Y_n$ be random variables. Then

$$Y = \begin{pmatrix} Y_1 \\ \vdots \\ Y_n \end{pmatrix}$$

is an n-dimensional random vector.

Similar to a random variable, a random vector also comes with a probability mass function (if all the Y_i are discrete) or a probability density function (if all the Y_i are continuous), or a "mixture'' distribution (if some Y_i are discrete and others are continuous). In general, a random vector is drawn from a multivariate distribution, defined by the PMF or PDF. Just as before, the PMF and PDF range is non-negative, the PMF sums to 1 over all outcomes, and the PDF integrates to 1 over $\mathbb{R}^n$. One discrete multivariate distribution you have learned in 1131 is the Multinomial distribution. We will learn about the multivariate normal distribution soon.

2.4.2 Expected Value and Covariance

Definition 2.15. Let Y be an n-dimensional random vector, then the mean (expected value) of Y is defined as

$$\mathrm{E}(Y) = \begin{pmatrix} \mathrm{E}(Y_1) \\ \vdots \\ \mathrm{E}(Y_n) \end{pmatrix} = \mu$$

and the covariance matrix of Y is defined as

$$\operatorname{cov}[Y] = \mathrm{E}[(Y-\mu)(Y-\mu)^\top] = ((\operatorname{cov}\left[Y_i, Y_j\right]\ )) = \Sigma.$$

Sometimes $\operatorname{cov}[Y]$ is written as $\operatorname{Var}[Y]$.

The following are some facts about Σ:

Σ is an $n \times n$ matrix with the diagonal elements being the variances, $\operatorname{Var}[Y_i]$ for $i = 1, \dots, n$, and the off-diagonal elements being the covariances, $\operatorname{cov}\left[Y_i, Y_j\right]$ for $i, j = 1, \dots, n$ and $i \neq j$. Σ is a symmetric, non-negative definite matrix. In this course, we further restrict it to be a positive definite matrix. Σ is referred to as the **covariance matrix**.

2.4.3 Properties of expected value and covariance

Let $X, Y \in \mathbb{R}^d$ be random vectors with $A \in \mathbb{R}^d$ and $B \in \mathbb{R}^{n \times d}$ be matrices. It holds that

- $\mathrm{E}(X+Y) = \mathrm{E}(X) + \mathrm{E}(Y)$
- $\mathrm{E}(A+BY) = A + B\mathrm{E}(Y)$
- $\operatorname{cov}[A+BY] = B\operatorname{cov}[Y]\, B^\top$.

Exercise 2.13. Let $Y = (Y_1, \dots, Y_n)^\top$ be a random vector, where Y_i are i.i.d. random variables with mean μ and variance σ^2. What are the mean and covariance of Y? Use properties of random vectors to compute the mean and variance of the sample mean.

Solution 2.4. First, $\mathrm{E}(Y) = \mu 1$ and $\operatorname{cov}[Y] = \sigma^2 I$. Note that $\bar{Y} = (Y_1 + \cdots + Y_n)/n = \frac{1}{n} 1^\top Y$. Now, we have

$$\mathrm{E}(\bar{Y}) = \mathrm{E}\left(\frac{1}{n} 1^\top Y\right) = \frac{1}{n}\left(1^\top \mathrm{E}(Y)\right) = \frac{1}{n}(n\mu) = \mu$$

and,

$$\begin{aligned}
\operatorname{cov}\left[\bar{Y}\right] &= \operatorname{cov}\left[\frac{1}{n} 1^\top Y\right] \\
&= \left(\frac{1}{n}\right)^2 \left(1^\top \operatorname{cov}[Y]\, 1\right) \\
&= \left(\frac{1}{n}\right)^2 (n\sigma^2) = \frac{\sigma^2}{n}.
\end{aligned}$$

2.4.4 Multivariate normal distribution

We say that a random vector $X \sim \mathcal{N}_d(\mu, \Sigma)$ follows a multivariate normal distribution if X has PDF:

$$\phi(\mathbf{x}) = \left(\frac{1}{2\pi}\right)^{d/2} |\Sigma|^{-1/2} \exp\left\{-\frac{1}{2}(\mathbf{x}-\mu)'\Sigma^{-1}(\mathbf{x}-\mu)\right\}.$$

If $X \sim \mathcal{N}_d(\mu, \Sigma)$ and $c \in \mathbb{R}^d$, $A \in \mathbb{R}^{m \times d}$ then:

- $AX \sim \mathcal{N}(A\mu, A\Sigma A^\top)$.
- $c^\top X \sim \mathcal{N}(c^\top \mu, c^\top \Sigma c)$.
- Any conditional distribution for a subset of the variables conditional on another subset of variables is a multivariate distribution.

Using random vectors is a simple way of deriving lots of equations for this course. Working with vectors also allows those who are "geometrically gifted" to view the whole regression concepts geometrically! If not, not to worry!

2.4.5 Homework stop 4

Exercise 2.14. For a (full-rank) non-random matrix $X \in \mathbb{R}^{n \times p}$ with $n > p$, and random vector $Y \in \mathbb{R}^{n \times 1}$ with mean μ and covariance Σ, compute the following:

- Expected value and covariance matrix of $(X^\top X)^{-1} X^\top Y$
- Expected value of $Y^\top Y$
- Expected value and covariance matrix of $Z^\top Z$ where Z is a fixed, nonrandom vector of d dimensions

3 Linear Regression

3.1 Basics of linear regression

By the end of this section, you should be able to say what the linear and normal linear regression models are. As well as what it means to assume either of these models.

3.1.1 The linear regression model

Consider the following example.

Example 3.1. It is difficult to accurately determine a person's body fat percentage without immersing them in water. However, we can easily obtain the weight of a person. A researcher would like to know if weight and body fat percentage are related? If so, for a given weight, can the person's body fat percentage be predicted? If so, how accurate is the prediction? This researcher collected the following data:

Individual	1	2	3	4	5	6	7	8	9	10
Weight (lb)	175	181	200	159	196	192	205	173	187	188
Body Fat (%)	6	21	15	6	22	31	32	21	25	30

Individual	11	12	13	14	15	16	17	18	19	20
Weight (lb)	188	240	175	168	246	160	215	159	146	219
Body Fat (%)	10	20	22	9	38	10	27	12	10	28

How can we (as statisticians / data scientists) answer the questions raised by the researcher?

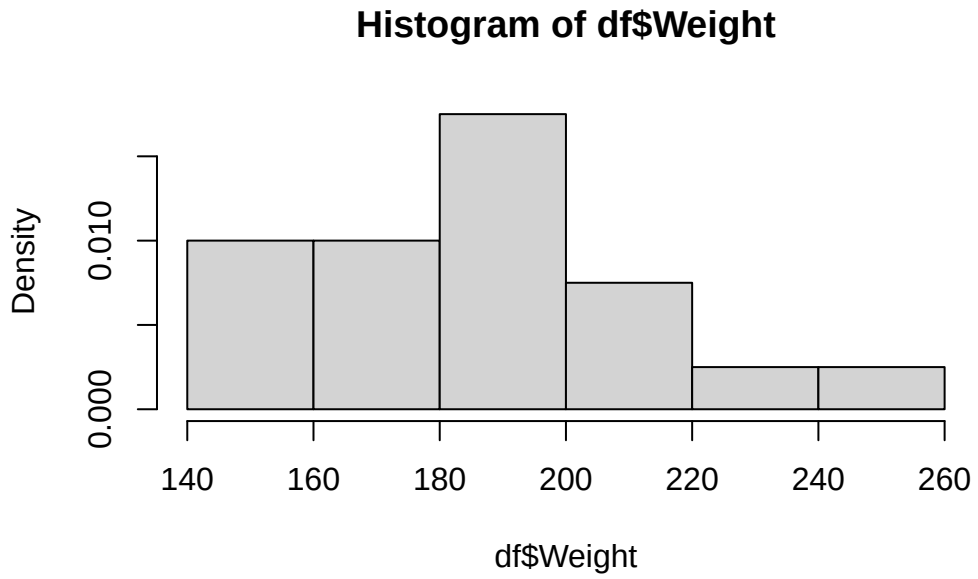
The first thing we might do is explore the data:

```
###################### Exploratory analysis

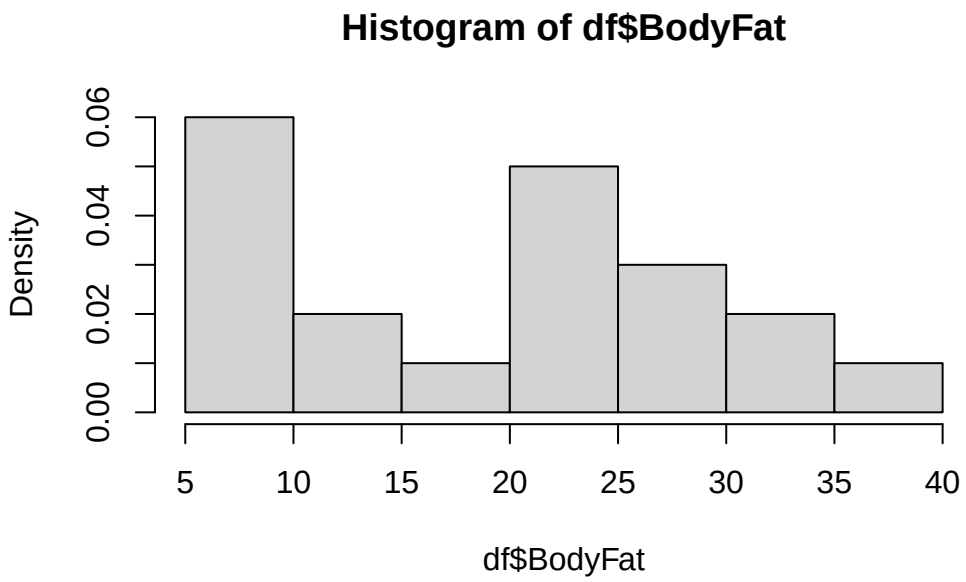
# Make the data frame
Weight=c(175 , 181 , 200 , 159 , 196 , 192 , 205 , 173 , 187 , 188 ,
         188 , 240 , 175 , 168 , 246 , 160 , 215 , 159 , 146 , 219 )
BodyFat =c(6 , 21 , 15 , 6 , 22 , 31 , 32 ,21 , 25 , 30 ,
           10 , 20 , 22 , 9 , 38 , 10 , 27 , 12 , 10 , 28 )

df=data.frame(cbind(Weight=Weight,BodyFat=BodyFat))

# make some histograms
hist(df$Weight,freq=F)
```



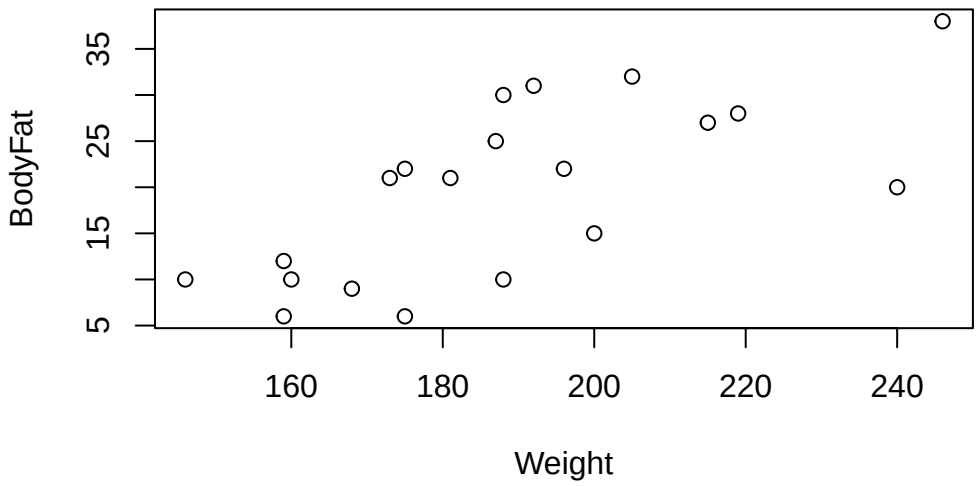

```
hist(df$BodyFat,freq=F)
```




```
# print summary statistics
summary(df)
```

```
     Weight         BodyFat
 Min.   :146.0   Min.   : 6.00
 1st Qu.:171.8   1st Qu.:10.00
 Median :187.5   Median :21.00
 Mean   :188.6   Mean   :19.75
 3rd Qu.:201.2   3rd Qu.:27.25
 Max.   :246.0   Max.   :38.00
```

```
# There seems to be some relationship here
plot(df)
```




```
# Here is the correlation matrix, notice it is high!
cor(df)
```

```
           Weight   BodyFat
Weight  1.0000000 0.6966328
BodyFat 0.6966328 1.0000000
```

We have observed that there is a relatively strong linear relationship between these two variables. What next? We might ask, what is this relationship precisely?

In particular, note that we have observed a sample of vectors $(Y_1, X_1), \ldots, (Y_1, X_n)$. Now, we want to say something about the relationship between X and Y in general. One way to do that is to suppose at the **population** level that

$$\mathrm{E}[Y|X] = f(X).$$

That is, on average, Y is equal to $f(X)$. One way to do that is to assume that $Y|X = f(X) + \epsilon$, where ϵ is a random variable that satisfies $\mathrm{E}[\epsilon] = 0$. This assumption means that, for each Y_i, given X_i, we have that $Y_i = f(X_i) + \epsilon_i$. Note that we do not observe ϵ_i, but we can assume it exists. We can read this as Y_i is equal to $f(X_i)$, plus some random, individual error ϵ_i. The next step is to use the data to determine f.

Using the data analysis steps from the Introduction we can write out the first few steps:

- Question about a population: "How can we use weight to determine body fat percentage?' '
- Data: $(Y_1, X_1), \ldots, (Y_{20}, X_{20})$, (Y_i, X_i) are the body fat percentage and weight of individual $i \in [20]$.

We have explored the data with graphs and summary statistics. Now, we have posited the model $Y|X = f(X) + \epsilon$. Letting f be any function is too general. In fact, we can use the data to learn more about what f might be. Recall that earlier, we saw the scatter plot, where it looked like there was a linear relationship, (with some error), between Y and X. (We can draw a straight line through the middle of the data.)

Let's make some assumptions that make the statistical analysis easier:

1. Assume that $\forall i \in [20]$, it holds that

$$Y_i|X_i = \beta_0 + \beta_1 X_i + \epsilon_i.$$

   This means that we assume that f is a line.

2. Next, we assume $\forall i \in [20]$, $\mathrm{E}[\epsilon_i] = 0$ and $\mathrm{Var}[\epsilon_i] = \sigma^2$. That is, the random error have the same mean and variance for each individual. In addition, the random errors average to 0.

3. We also assume that the individuals' Body fat percentage, weights and random errors are independent, that is, $\epsilon_i \perp \epsilon_j$ for $i \neq j,\ i, j \in [20]$.

This is the **simple linear regression model**. That is, the simple linear regression model is the set of assumptions 1-3 given above.

It is often also assumed:

4. $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$,

but not always. Including the normality assumption is known as the **simple normal linear regression model.**

In general, a model is a set of assumptions about a population. The particular set of assumptions 1-3 is the simple linear regression model.

The following is some terminology used in regression analysis:

- Here, Y_i is the **response variable**, also known as the dependent variable, or the outcome variable.
- Here, X_i is the **covariate**, also known as the explanatory variable, or the independent variable.

Given a "question about a population" which involves regression, you should immediately identify the response variable and the covariates.

Now, how can we interpret this model? That is, what does it mean to assume this model?

First, observe that we assume that $\mathrm{E}[Y|X]$ is a line. This means there is a linear relationship between the average body fat percentage and weight.

Next, observe that for any individual, their actual body fat percentage is given by $Y = \mathrm{E}[Y|X] + \epsilon_i = \beta_0 + \beta_1 X_i + \epsilon_i$. Therefore, their body fat percentage will not fall exactly on the line $\beta_0 + \beta_1 X_i$. Rather, it will fall above or below the line, depending on ϵ_i. Furthermore, if we assume that $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$, then we know from the properties of the Normal distribution that this random error will not exceed 2σ with high probability. Therefore, most of the time, an individual's body fat percentage will fall within 2σ of the line.

Third, notice that this quantity, 2σ, does not depend on X. That is, for any weight, we still expect an individual's body fat percentage to be within 2σ of the line, regardless of the value of weight.

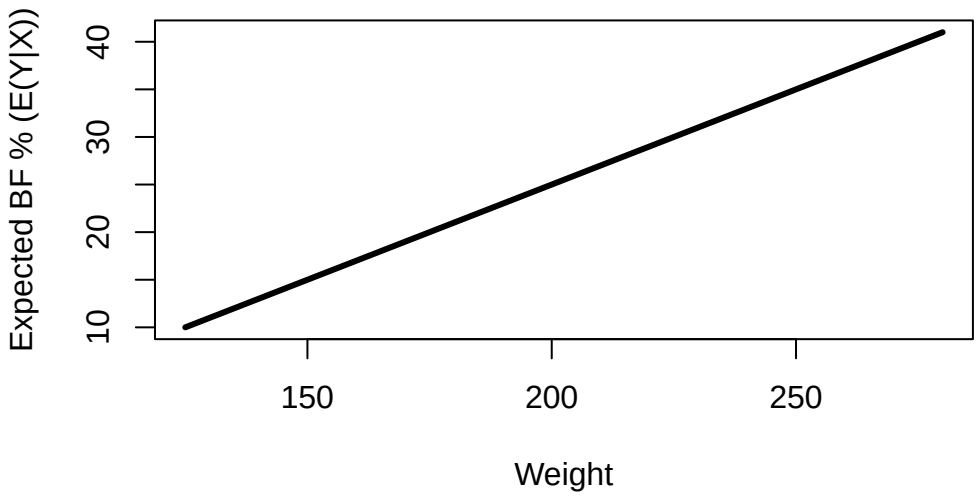
Fourth, if we knew β_0, β_1, then given someone's weight, we could try to predict their body fat percentage given their weight. That is, we could calculate the expected body fat $\mathrm{E}[Y|X]$. There would still be their individual random error ϵ, so we would not be able to predict it exactly. However, if σ^2 isn't too big, then we could produce an accurate prediction.

Therefore, if the model assumptions are correct, we assume there exists some line, around which the body fat percentages are scattered uniformly.

Next, we will simulate data from the normal simple linear regression model to gain a better understanding of this model. Suppose that $\beta_0 = -15$, $\beta_1 = .2$ and $\sigma = 5$. Then we would observe the following.

```
############################ Simulation
set.seed(3252)

# Suppose that beta_0=-15 and beta_1=0.2 and sigma=5,
# then we would have that the mean function E(Y|X) is given by the following line:
curve(-15+.2*x,125,280,lwd=3,xlab="Weight",ylab="Expected BF % (E(Y|X))")
```



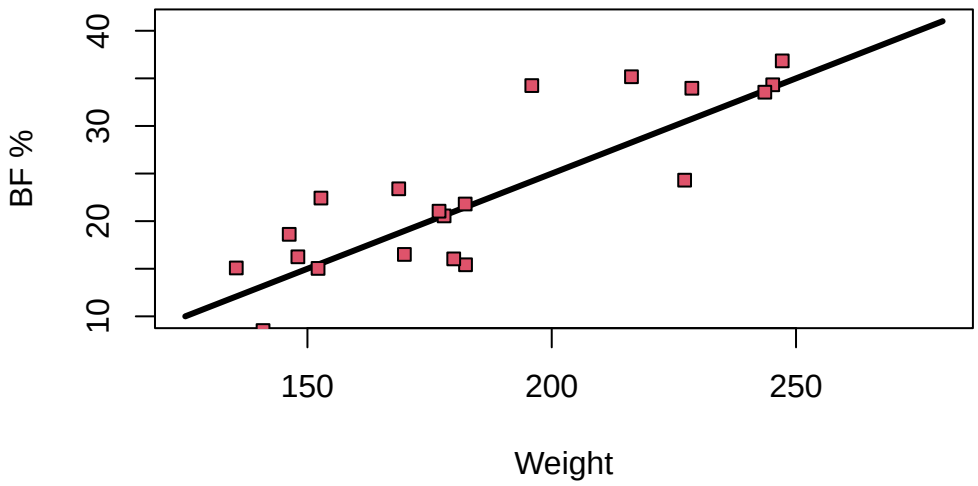

```r
# Next, let's simulate some body weights from the uniform distribution
Weight2=runif(20,135,250)

# Then, we can simulate the population body fat percentages according to the model as follows

# Simulating 20 values of the random error,
epsilons=rnorm(n=20,mean=0,sd=5)

# Computing the simulated Body fat percentages:
Bfs=-15+.2*Weight2+epsilons

# Plot the simulated values, and the mean function
curve(-15+.2*x,125,280,lwd=3,xlab="Weight",ylab="BF %")
points(Weight2,Bfs,pch=22,bg=2)
```



Notice how the data are scatted around the line uniformly? This is what data from a simple linear regression model looks like. Try changing the value in `set.seed()` and re-running the code. Notice how the data changes, but it is always scatted around the line uniformly? This is what we expect to see if the data follow a simple linear regression model.

Notice how the data simulated from our model appears similar to the body fat percentage and weights data we observed? That means this model (set of assumptions) is a good fit for our data.

> **Caution**
>
> In this model, and in regression in general, the response Y is not exactly equal to some function of X given by $f(X)$. The model assumes that **on average** $Y = f(X)$. Therefore, knowing someones "X" value will not exactly give us their Y value, but it would give us a good guess at it. The error ϵ is used to model the fact that someones "X" value will not exactly give us their Y value. Notice above how the actual points are scatted around the line, and not exactly equal to it! This is due to the errors ϵ.

3.1.2 The multiple linear regression model

But what about matrices? Why did we study matrices then? We can write the regression model in terms of matrices and vectors, to make it more compact.

Now, recall the the linear regression model is

$$Y_i|X_i = \beta_0 + \beta_1 X_i + \epsilon_i,$$

with with $\mathrm{E}[\epsilon_i] = 0$, $\mathrm{Var}[\epsilon_i] = \sigma^2$), and all $\epsilon_j \perp \epsilon_j$.
It is more convenient mathematically to let $\mathbf{Y} = (Y_1, ..., Y_n)^\top$,

$$\mathbf{X} = \begin{bmatrix} 1 & X_1 \\ \vdots & \vdots \\ 1 & X_n \end{bmatrix} = \left[1_n \mid (X_1, ..., X_n)^\top\right],$$

$\beta = (\beta_0, \beta_1)^\top$ and $= (\epsilon_1, ..., \epsilon_n)^\top$. Then we can write

$$\mathbf{Y}|\mathbf{X} = \mathbf{X}\beta + .$$

Often, we overload the notation Y, and use Y instead of $\mathbf{Y}$, and X instead of $\mathbf{X}$.

This form allows us to go beyond one explanatory variable very easily! Just add one column to X and one entry to β for each new variable. Observe the following model:

$$Y_i|(X_{i1}, ..., X_{ik}) = \beta_0 + \beta_1 X_{i1} + ... + \beta_k X_{ik} + \epsilon_i,$$

with $\mathrm{E}[\epsilon_i] = 0$, $\mathrm{Var}[\epsilon_i] = \sigma^2$) and $\epsilon_i \perp \epsilon_j$ for $i \neq j,\ i, j \in [n]$. This is known as the **multiple linear regression model** (MLR), or just the linear regression model for short. We can write this model in the same for as above: Let

$$\mathbf{X} = \begin{bmatrix} 1 & X_{11} & X_{1k} \\ \vdots & \vdots & \vdots \\ 1 & X_{n1} ... & X_{nk} \end{bmatrix},$$

and $\beta = (\beta_0, ..., \beta_k)^\top$. Then we can write the MLR as

$$\mathbf{Y}|\mathbf{X} = \mathbf{X}\beta + ,$$

where $\mathrm{E}[\epsilon] = 0$, $\epsilon_i \perp \epsilon_j$ for all $i \neq j \in [n]$ and $\mathrm{Var}[\epsilon] = \sigma^2 I$. Notice how compact this is! As in the simple case, there is also the **normal MLR**, which further assumes that $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$.

We can then study the mathematical properties of

$$Y|X = X\beta + \epsilon$$

for general but fixed k, under either the normal MLR or just the MLR, which will cover many models.

3.1.3 Homework stop 1

Exercise 3.1. Try adjusting the parameters β_0, β_1, σ in the simulation, what happens to the data? What happens to the line?

Exercise 3.2. Is β an estimate or a population parameter? Why?

Exercise 3.3. Come up with another possible form of f that is not linear. Adjust the simulation to include this form of f.

Exercise 3.4. Write down the assumptions of the MLR and the normal MLR. What is the difference between the two models?

3.2 Least Squares

Now that we have settled on a model for the population, the next step is to use the data to estimate the model parameters. In particular, we need to estimate β. That will allow us to estimated $\mathrm{E}[Y|X]$ for any value of X.

Recall that we want to study the **population** model:

$$Y|X = X\beta + \epsilon.$$

3.2.1 Notation

For the model $Y|X = X\beta + \epsilon$, we have

- $Y \in \mathbb{R}^n$ is the response variable (a continuous random variable).
- $X \in \mathbb{R}^{n \times p}$ is the covariate matrix (Note that the first column is often 1_n).
- $X_i \in \mathbb{R}^p$ is the ith observed explanatory variable ($i = 1, \ldots, n$) (not a random variable, in the sense that we condition on it).
- $\beta \in \mathbb{R}^{p \times 1}$ is the coefficient vector .
- $\epsilon \in \mathbb{R}^n$ is the random error (continuous random variable) .

We may also refer to the actual observed values (versus the abstract mathematical concept of a random variable) as follows:

- $y = (y_1, \ldots, y_n)^\top \in \mathbb{R}^n$ is the observed response variable (fixed/observed)
- x_{ij} is the ith observation of the jth explanatory variable (fixed/observed) Data:

Observation	Observed data point
1	$(y_1, x_{11}, x_{12}, \dots x_{1p})$
2	$(y_2, x_{21}, x_{22}, \dots x_{2p})$
$\vdots$	$\vdots$
n	$(y_n, x_{n1}, x_{n2}, \dots x_{np})$

We posit that

$$Y|X = X\beta + \epsilon,$$

where we assume that

- $\forall i \in [n]$, $\mathrm{E}[\epsilon_i] = 0$.
- $\forall i \in [n]$, $\mathrm{Var}[\epsilon_i] = \sigma^2$ (constant variance and is also known as homogeneity.)
- We also would assume that $\epsilon_i \perp \epsilon_j$ for $i \neq j,\ i, j \in [n]$.
- $\beta \in \mathbb{R}^{p\times 1}$ is the unknown, population coefficient vector.
- $X \in \mathbb{R}^{n\times p}$ is a covariate matrix.

Let's talk about β. How do we interpret β? Suppose we know β. Then:

Note that

$$\mathrm{E}[Y_i|X_i] = \mathrm{E}[\beta^\top X_i + \epsilon] = \beta^\top X_i = \beta_1 X_{1,1} + \dots + \beta_p X_{i,p}$$

What does each β_j mean? Suppose that X_j is a continuous covariate.

We can interpret (β_j) as follows:

Holding $X_{i,1}, \dots, X_{i,j-1}, X_{i,j+1}, \dots, X_{i,p}$ constant, a one unit increase in $X_{i,j}$ causes, on average, a β_j unit increase in Y_i.

From another angle, we have that $\partial \mathrm{E}[Y] / \partial X = \beta$, therefore, the rate of change with respect to the jth covariate is β_j.

> **Caution**
>
> The "on average" and "holding other covariates constant" are very important components of the interpretation. First, the on average acknowledges the random error ϵ. In other words, a one unit increase in $X_{i,j}$ will not certainly increase Y_i, but it will on average. Next, the "holding other covariates constant" is used to mention how correlations between covariates are handled by the model. Some of the covariates in the model may be correlated, so increases in a given covariate may often be associated with changes in another covariate. This is not accounted for in the coefficient vectors β. That is why we must specify "holding other covariates constant".
> For instance, if a model includes a terms for years of education attained and income, we know that as the number of years of education increase we expect to see a rise in income

levels. As a result, to interpret the effect of coefficient on income, we must "hold years of education constant", comparing what is expected with income changes but education does not.

Caution

For now, we can assume that all of the covariates X_j are continuous variables. Later in the course, there may be categorical covariates. In this case, the β_j corresponding to the categorical covariates have a different interpretation. We will return to this later.

Recall Example 3.1. We assume $\forall i \in [20]$, it holds that

$$Y_i | X_i = \beta^\top X_i + \epsilon_i,$$

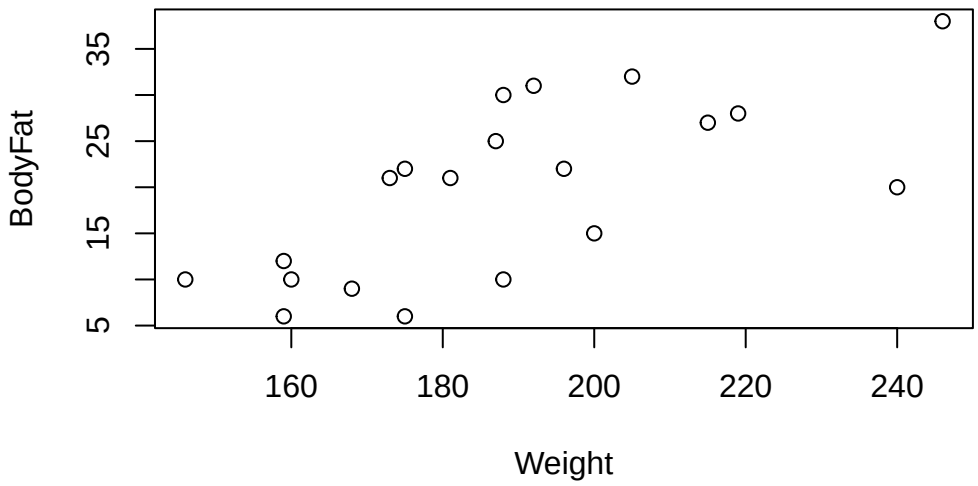
with $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$, $\epsilon_i \perp \epsilon_j$ for $i \neq j$, $i, j \in [20]$. A one unit increase in weight causes, on average, a β_2 unit increase in body fat percentage. Since β_1 is the intercept, it has a special interpretation. β_1 is the average value of Y_i given $X_i = 0$. It is also helpful to note that $\text{cov}(Y) = \sigma^2 I$.

3.2.2 Least squares estimation

Okay, but we don't know β! Just like we estimate the population mean with the sample mean, we need to estimate β. We would like an estimate $\hat{\beta}$, so that we can predict body fat percentage from weight. What is our best guess at β, given the data? One way to answer this, is through the method of **least squares**.

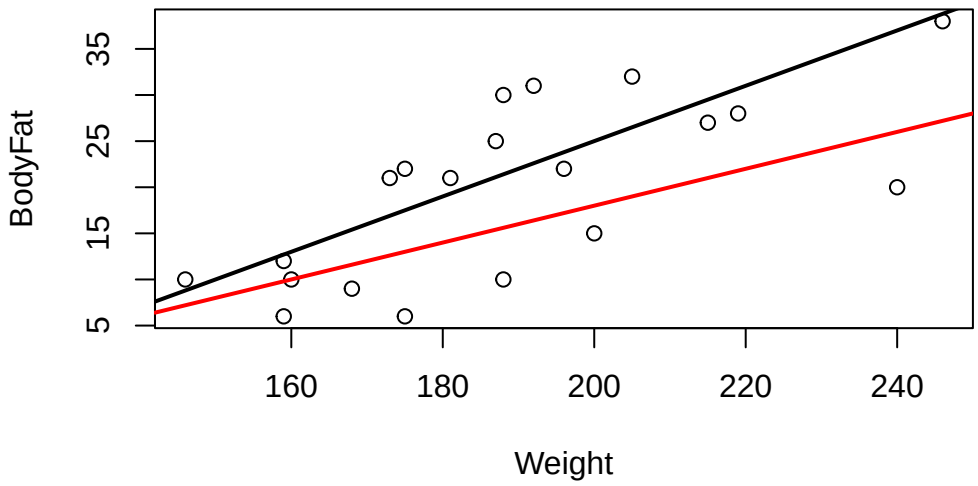
Returning to our example, recall that:

```
plot(df)
```



For example, suppose we want to determine if β is more likely to be $(-35, 0.3)^\top$ or $(-22, 0.2)^\top$. How can we say which line is a better to our data? One way is to graph them on top of the data and determine which one looks better. Let's plot these lines.

```
plot(df)
# plot Y=-35+0.3X
abline(-35,0.3,lwd=2)
# plot Y=-25+0.2X
abline(-22,0.2,col='red',lwd=2)
```



Its not clear which one fits the data better. Even if it was clear, obviously, we cannot plot all possible lines. So how can we determine which line fits the data the "best"?

To do this, we have to define what "best" means quantitatively. For instance, one might ask which line minimizes the sum of the squared distances of the observed data points to the line? This line is then said to be the "best" line. Mathematically, given a proposed value of β, say $\beta_0 \in \mathbb{R}^p$, the signed distance to the hyperplane $X\beta_0$ is $\epsilon_0 = Y - X\beta_0$. The squared distances to the hyperplane $X\beta_0$ is then $\epsilon_0^\top \epsilon_0 = (Y - X\beta_0)^\top (Y - X\beta_0)$. We can then formulate this as a math problem: Which $\beta_0 \in \mathbb{R}^p$ minimizes $\epsilon_0^\top \epsilon_0$? i.e., $\hat{\beta} = \text{argmin}_{\beta_0 \in \mathbb{R}^p} \epsilon_0^\top \epsilon_0$. It is more convenient to just write

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^p}{\text{argmin}} (Y - X\beta)^\top (Y - X\beta).$$

In this framework, the "best" estimate is given by

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^p}{\text{argmin}} (Y - X\beta)^\top (Y - X\beta).$$

Note best is in the sense of minimizing the average squared distance to the hyperplane/line. We could also define best in terms of some other metric, such as average absolute distance to the hyperplane/line. For now, we will stick with this metric.

The next step is to solve:

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^p}{\text{argmin}} (Y - X\beta)^\top (Y - X\beta).$$

How do we minimize a function???

RECALL in calculus, to find the minimum of a function we:

1. Obtain the first two derivatives of the function.
2. Set the first derivative to zero and solve for the critical value.
3. Use the second derivative to verify the critical value minimized the function.

Goal: Compute $\hat{\beta}$ – Minimize $g(\beta) = (Y - X\beta)^\top (Y - X\beta)$. (It may be useful to review taking derivatives with respect to vectors here.

Step 1a: {

$$\begin{aligned}
\frac{\partial g}{\partial \beta} &= \frac{\partial}{\partial \beta}(Y - X\beta)^\top (Y - X\beta) \\
&= \frac{\partial}{\partial \beta}\left[Y^\top Y - 2(X\beta)^\top Y + (X\beta)^\top X\beta\right] \quad \text{(Transpose and distribute)} \\
&= -2\frac{\partial}{\partial \beta}\beta^\top X^\top Y + \frac{\partial}{\partial \beta}\beta^\top X^\top X\beta \quad ((AB)^\top = B^\top A^\top) \\
&= -2X^\top Y + 2X^\top X\beta \quad (\tfrac{\partial}{\partial x}x^\top Ax = 2Ax \text{ if } A \text{ sym.}, \tfrac{\partial}{\partial x}x^\top a = a) \\
&= -2X^\top (Y - X\beta).
\end{aligned}$$

}

Step 1b: (Do this for homework, use eq. (98) in the matrix cookbook.)

$$\frac{\partial^2 g}{\partial \beta \partial \beta^\top} = 2X^\top X.$$

Step 2: We now need $X^\top X$ to be invertible, so we will assume that X is full rank and $n \geq p$.

$$\begin{aligned}
-2X^\top (Y - X\beta) = 0 & \\
&\implies X^\top Y = X^\top X\beta \\
&\implies \beta = (X^\top X)^{-1} X^\top Y.
\end{aligned}$$

Step 3:

Recall that **if the Hessian matrix is positive definite at a critical point, then that critical point is a local minimum.** Since we have assumed X is full rank, this implies that $X^\top X$ is positive definite.

To summarize, the steps have proceeded as follows:

- Step 1a: $\frac{\partial g}{\partial \beta} = -2X^\top (Y - X\beta)$
- Step 1b: $\frac{\partial^2 g}{\partial \beta \partial \beta^\top} = 2X^\top X$ (Do this for homework)

- Step 2: $-2X^\top(Y - X\beta) = 0 \implies X^\top Y = X^\top X\beta \implies \beta = (X^\top X)^{-1}X^\top Y$
- Step 3: $2X^\top X$ is positive definite, and so

$$\hat{\beta} = (X^\top X)^{-1}X^\top Y.$$

The estimate $\hat{\beta}$ is known as the **least squares estimate** of the regression coefficients.

Definition 3.1. The **least squares estimate** of the regression coefficients is

$$\hat{\beta} = (X^\top X)^{-1}X^\top Y.$$

3.2.3 Example

Example 3.2. In the body weight example Example 3.1, write down X, y and compute $\hat{\beta}$. Interpret $\hat{\beta}$.

First, we have that

$$y = (6, 21, 15, 6, 22, 31, 32, 21, 25, 30, 10, 20, 22, 9, 38, 10, 27, 12, 10, 28)^\top$$

$$X = \left[1_{20} \mid (175, 181, 200, 159, 196, 192, 205, 173, 187, 188, 188, 240, 175, 168, 246, 160, 215, 159, 146, 219)^\top\right]$$

> **Note**
>
> For matrices A, B which have the same number of rows, $C = [A|B]$ is horizontal concatenation of A and B. This notation indicates that the matrix C is formed by placing A an B side by side, joining them horizontally. Therefore, X is the matrix whose first column is made up of ones, and second column is made up of the body weights.

Let's use R to compute $\hat{\beta}$.

```
#Define X and Y
X=cbind(rep(1,nrow(df)), df$Weight)
Y=df$BodyFat

# cast to column vec
Y=matrix(Y,ncol=1)

#X'X
X_p_X=t(X)%*%X

#X'X inverse
```

```r
X_p_X_inverse=solve(X_p_X)


#LS
beta_hat= X_p_X_inverse%*%t(X)%*%Y
beta_hat
```

```
            [,1]
[1,] -27.3762623
[2,]   0.2498741
```

```r
# We can also use Rs lm() function to do this:
# This code is essential for the course.
# The first argument is the formula
model=lm(BodyFat ~ Weight, data=df)

#The summary function prints the model output.

summary(model)
```

```
Call:
lm(formula = BodyFat ~ Weight, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-12.5935  -5.7904   0.6536   5.2731  10.4004

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -27.37626   11.54743  -2.371 0.029119 *
Weight        0.24987    0.06065   4.120 0.000643 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.049 on 18 degrees of freedom
Multiple R-squared:  0.4853,    Adjusted R-squared:  0.4567
F-statistic: 16.97 on 1 and 18 DF,  p-value: 0.0006434
```

```
# The least squares estimates are given in the Estimate column of the summary.
```

The `lm()` function is used to fit multiple linear regression models in R. The basic usage involves specifying a formula and a data frame. The syntax is given by `lm(formula, data, ...)`.

The data argument should be the dataframe which contains your data. The formula argument is used to specify the model to be fitted. It provides a symbolic description of the model, indicating the response variable and the predictors/covariates, as well as the relationships between them. The left-hand side should be the name of your response variable, as it is named in your dataframe. To see the names of your variables use the `names()` function, e.g., `names(df)`. The right-hand side contains the covariates you want to include in your model. For instance, above, the formula is given by `BodyFat ~ Weight` . Note that `BodyFat` is the response and `Weight` is the covariate.

We now list some important properties of the least squares estimator.

Exercise 3.5. Compute $\mathrm{E}\left[\widehat{\beta}\right]$ and $\mathrm{cov}(\widehat{\beta})$.

Solution 3.1. It holds that $\mathrm{E}\left[\widehat{\beta}\right] = \beta$ and $\mathrm{cov}(\widehat{\beta}) = \sigma^2(X^\top X)^{-1}$.

Recall that an estimator is **unbiased** if its expectation equals the population parameter it is trying to estimate. After completing Exercise 3.5 you will see that $\widehat{\beta}$ is unbiased for the parameter β.

The least squares estimator is also the "best linear unbiased estimator", or the BLUE. This is known as the Gauss–Markov theorem. This means that under the assumptions of the linear regression model, over any unbiased estimator of β we can construct, which is a linear combination of $Y_1, \ldots, Y_n$, the estimator $\widehat{\beta}$ has the smallest variance (and therefore, the smallest mean squared error. Recall that for an estimator $\hat{\alpha}$, the mean squared error is given by $\mathrm{E}\left[||\beta - \hat{\alpha}||^2\right]$.)

The Gauss–Markov theorem does not require the random error to be normally distributed. If we are willing to assume that $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$, then $\widehat{\beta}$ is also the maximum likelihood estimator and the "uniformly minimum-variance unbiased estimator", or UMVUE. This means that $\widehat{\beta}$ has lower variance than any other unbiased estimator, no matter what the true value of β is.

One might ask, how can we use $\widehat{\beta}$ to predict body fat percentage given weight? The estimate $\widehat{\beta}$ gives us a best guess at the coefficients. Therefore, our best guess at someones body fat is given by

$$Best\ Guess = -27.3762623 + 0.2498741 \times Weight.$$

For instance, for someone who is 170 pounds, we would guess that their body fat percentage is $-27.3762623 + 0.2498741 \times 170 = 15.1023347$.

3.2.4 Homework stop 2

Exercise 3.6. Why do we need $\hat{\beta}$, why not use β?.

Exercise 3.7. Is $\hat{\beta}$ an estimate or a population parameter? What about β?

Exercise 3.8. Compute, X, Y and $\hat{\beta}$ in the following real data example:

It is challenging to assess a student's understanding of a subject without administering an exam. However, we can easily record the number of hours a student studies. A researcher would like to know if the number of hours studied and exam scores are related. This researcher collected the following data:

Student	Hours Studied	Exam Score (%)
1	5	55
2	8	65
3	12	78
4	6	58
5	10	72
6	9	68
7	15	85
8	7	60
9	11	74
10	13	80
11	14	82
12	20	90
13	5	55
14	6	59
15	18	88
16	7	62
17	16	86
18	4	50
19	3	45
20	19	89

To help you, here is some R code the dataset:

```
# Data
study_data <- data.frame(
  Student = 1:20,
  Hours_Studied = c(5, 8, 12, 6, 10, 9, 15, 7, 11, 13, 14, 20, 5, 6, 18, 7, 16, 4, 3, 19),
```

```
  Exam_Score = c(55, 65, 78, 58, 72, 68, 85, 60, 74, 80, 82, 90, 55, 59, 88, 62, 86, 50, 45,
)
```

3.3 Least squares inference

Recall we **estimate** the parameter β using least squares:

Recall that $\hat{\beta} = (X^\top X)^{-1} X^\top Y$. We can predict a new weight $Y_{new}|X = x$ with $\hat{y}_{new} = x^\top \hat{\beta}$. We may be interested in the following questions: How good is $\hat{y}_{new}$ as a prediction, on average? How will new observations vary about the line? For example, given a specific weight, how will does body fat percentage vary around the regression line? How does $\hat{\beta}$ vary around β? Is there strong evidence that Y has a relationship with X? Is X adding information about Y at all?

To answer these questions, we need to look at the variation of our estimates and our data.

3.3.1 Important quantities: Residuals and fitted values

We now introduce some very important quantities: We call the estimated values given our observed X the fitted values: $\hat{Y} = X\hat{\beta}$. The fitted values are what our model would estimate the vector Y to be. We call $\hat{\epsilon} = Y - \hat{Y}$ is the **residual vector**. The ith entry of $\hat{\epsilon}$, say $\hat{\epsilon}_i$, is the ith **residual**. The residuals are the signed distances from the response variable to the estimated regression hyperplane. The **sum of squared error** or **sum of squared residuals** (SSE) is given by $\hat{\epsilon}^\top \hat{\epsilon} = \sum_{i=1}^{n} \hat{\epsilon}_i^2$. Note that since we estimated β using the least squares method, $\hat{\epsilon}^\top \hat{\epsilon}$ is minimized (with respect to varying β).

Example 3.3. Recall Example 3.1. What is the residual of individual 3? How can we interpret this value?

```
residuals=Y-X%*%beta_hat; residuals
```

```
             [,1]
 [1,] -10.3517117
 [2,]   3.1490434
 [3,]  -7.5985652
 [4,]  -6.3537255
 [5,]   0.4009314
 [6,]  10.4004279
 [7,]   8.1520641
 [8,]   5.1480365
 [9,]   5.6497986
```

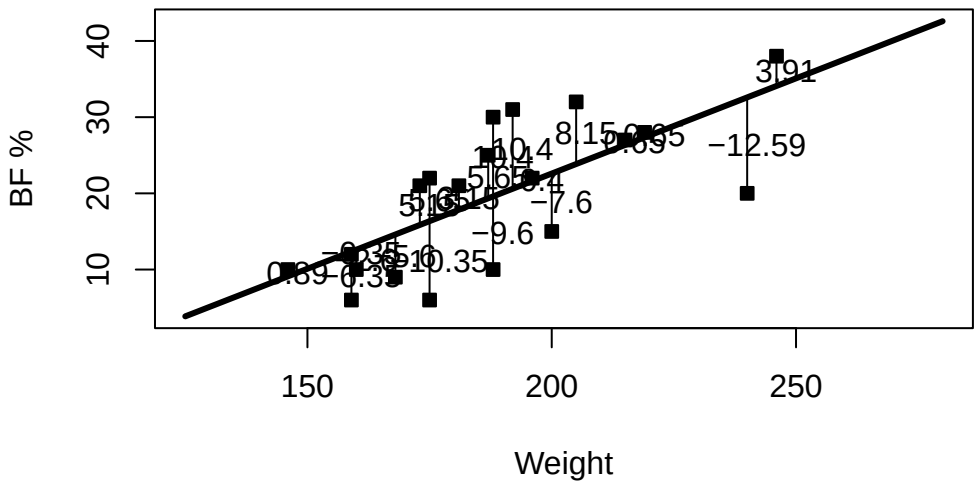
```
[10,]  10.3999245
[11,]  -9.6000755
[12,] -12.5935307
[13,]   5.6482883
[14,]  -5.6025928
[15,]   3.9072245
[16,]  -2.6035997
[17,]   0.6533228
[18,]  -0.3537255
[19,]   0.8946383
[20,]   0.6538262
```

```r
# This means that individual 3's body fat is 7.5 percentage points lower than the fitted line

residuals[3]
```

```
[1] -7.598565
```

```r
# We can go further and and plot all of the residuals
curve(beta_hat[1]+beta_hat[2]*x,125,280,lwd=3,xlab="Weight",ylab="BF %")
points(Weight,BodyFat,pch=22,bg=1)
Yvals=cbind(BodyFat,model$fitted.values)
Xvals=cbind(Weight,Weight)
for(i in 1:nrow(Yvals)){
  lines(Xvals[i,],Yvals[i,])
  text(Xvals[i,1]+2,mean(Yvals[i,]),round(residuals[i],2))
}
```




```
# Then, the population body fat percentages, given weights will look like this:
#
# Bfs=-15+.2*Weight+rnorm(20,0,sd=5)
```

3.3.2 Variation decomposition

Variance decomposition is a fundamental concept that explains how the total variation in the response variable can be partitioned into different sources. This decomposition is crucial for evaluating the performance of the regression model and understanding the contributions of various factors.

The residuals describe one type of variation of the response values. We can also consider the total variation of the response. The total variation of the response, or the **sum of squares total/total sum of squares** (SST) is given by $SST = (n-1)\hat{\sigma}_y = \sum_{i=1}^{n}(Y_i - \bar{Y})^2 = (Y - \bar{Y}1)^\top(Y - \bar{Y}1)$. It can be shown that the SST can be decomposed as follows:

$$SST = (Y-\bar{Y}1)^\top(Y-\bar{Y}1) = (Y-\hat{Y})^\top(Y-\hat{Y}) + (\hat{Y}-\bar{Y})^\top(\hat{Y}-\bar{Y}) = \hat{\epsilon}^\top\hat{\epsilon} + (\hat{Y}-\bar{Y})^\top(\hat{Y}-\bar{Y}).$$

That is, $SST = SSE + SSModel$ where

- $SSModel$, OR SSM measures the total variations of the response explained by the covariates X via the model based on $\hat{\beta}$.

- SSE measures the total variations of the response unexplained by the covariates X via the model based on $\hat{\beta}$.
- Note there are sometimes other names for SSE and $SSModel$, such as $SSRegression$, $SSwithin$ and $SSbetween$, etc.

So, we have that the total variation in the response can be broken down into that which is explained by the X values, and that which is unexplained.

An interesting observation is given as follows: The first column of the X matrix is given by 1_n, which implies that

$$\bar{Y}1 = X \begin{bmatrix} \bar{Y} \\ 0 \end{bmatrix}.$$

This means that if we let $\hat{\beta}_* = (\bar{Y}, 0, ..., 0)^\top$, then $(Y - \bar{Y}1)$ would be the signed distances to (or the residuals of) the regression hyperplane corresponding to $\hat{\beta}_*$. Since $\hat{\beta}$ minimizes the sum of squared residuals, we must have that the hyperplane corresponding to $\hat{\beta}$ has a smaller sum of squared residuals than the regression hyperplane corresponding to $\hat{\beta}_*$. Therefore, we must have that $\hat{\epsilon}^\top \hat{\epsilon} \leq (Y - \bar{Y}1)^\top (Y - \bar{Y}1)$.

Each of these terms in the decomposition is associated with a certain number of **degrees of freedom**.

- Total: $dfT = n - 1$.
- Model: $dfM = \#$ **non-zero** $\beta - 1$.
- Error: $dfE = n - \#$ **non-zero** β.

Intuitively, since the SSE is the variance unexplained by the model/covariates, the SSE is related to the error variance σ^2. In fact, to estimate σ^2, we use

$$\hat{\sigma}^2 = MSE = \frac{SSE}{dfE}.$$

The null model is defined as $Y|X = \beta_0 + \epsilon$. This is the model where the last $p - 1$ terms in the true vector β are 0. This model says that Y does not depend on X. In the null model, we only need to estimate the mean, so $df = n - 1$. Therefore, under the null model,

$$\begin{aligned} \hat{\sigma}^2 = (n-1)^{-1} SST &= \hat{\sigma}^2_Y \\ &= (n-1)^{-1} \sum_{i=1}^{n} (Y_i - \bar{Y})^2 = (n-1)^{-1} (Y - \bar{Y}1)^\top (Y - \bar{Y}1). \end{aligned}$$

Therefore, in the null model, the estimate of σ^2 via the MSE is just the usual estimate of the variance of the response. This is intuitive!

The following table can be used to summarize the variation in the response:

Source	SS	df	MS
Model	SSM	dfM	$MSModel = SSM/dfM$
Residual	SSE	dfE	$MSE = SSE/dfE$
Total	SST	dfT	

> **Note**
>
> It is very important to be able to interpret these terms! The derivation is also important. However, we can use a machine to compute anything for us, so memorizing the formula is not helpful.

3.3.3 Coefficients of determination

A model is a good model if it can explain a fair amount of the variation in the response. (You can think that the model explains "changes" in the response.) In other words, $SSModel$ should be as close to $SSTotal$ as possible; or equivalently, $SSError$ should be as close to 0 as possible. Now, "close" is a relative term, and so we need another value to reference to. This is where the R^2 comes in:

$$R^2 = \frac{SSModel}{SST},$$

and is the proportion of variation explained by the model. It is clear that $0 \leq R^2 \leq 1$, and so rescaling the data will not affect R^2 (like it would affect the sum of squares terms SST, SSE, SSM). If R^2 is close to 1, it is large – "close to 1" is a subjective/area dependent. Generally, the larger the R^2, the better the model!

To compare different models, we could potentially add different covariates and see if R^2 improves. However, every time you add any variable, R^2 will always increase. Therefore, it is common to use the adjusted coefficient of determination:

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n-1}{n-p}.$$

Thus, the (adjusted) coefficient of determination can be used as a measure of how well the regression model fits the data (how much variance is explained). It could also be used to compare models.

3.3.4 The F test

The coefficients of determination are summary statistics which give an idea of the fit of the model. We would also like a significance test that tells us whether the covariates explain Y, or what we observed was simply due to sampling variation.

If $\beta = (\beta_1, \dots, \beta_p)^\top$ then let $\tilde{\beta} = (\beta_2, \dots, \beta_p)^\top$. That is $\tilde{\beta}$ is the regression coefficients without the intercept term. Similarly, let $\hat{\tilde{\beta}} = (\hat{\beta}_2, \dots, \hat{\beta}_p)^\top$. Now, we want to avoid the situation where $\tilde{\beta} = 0$ but $\hat{\tilde{\beta}} \neq 0$ due to sampling variation.

To do this, we perform a significance test:

$$H_0 : \tilde{\beta} = 0 \qquad vs \qquad H_1 : \tilde{\beta} \neq 0.$$

First, we need the normality assumption to perform significance test: Assume $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. With this assumption, the model is then known as the **Normal Multiple Linear Regression Model**. It is important to note that the least squares method does not require this assumption, and this assumption is required only for the significance test to be valid. To test the hypothesis stated above, we use the overall F test and the observed test statistic is $F_{obs} = MSModel/MSE$. Why?

With the extra normality assumption, we have the following holds:

- $Y|X$ is normally distributed.
- We have that $SSM/\sigma^2 \sim \chi^2_{dfM}$ and $SSE/\sigma^2 \sim \chi^2_{dfE}$.
- Furthermore, $SSM \perp SSE$.

Recall that the ratio of two independent χ^2 distributions divided by their respective degrees of freedom follows an F distribution. Therefore, we have that $F_{obs} \sim F_{dfM,dfE}$. The corresponding p-value is $\Pr(W > F_{obs})$ where $W \sim F_{dfM;dfE}$. We can alternatively reject the null hypothesis if $F_{obs} > F_{dfM;dfE,1-\alpha}$, where $F_{obs} > F_{dfM;dfE,1-\alpha}$ is the $1-\alpha$ quantile of the $F_{dfM,dfE}$ distribution.

We can now present the complete ANOVA table

Source	**SS**	**df**	**MS**	**F**	**p-value**
Model	SSM	dfM	$MSModel = \frac{SSR}{dfM}$	$F = \frac{MSModel}{MSE}$	$\Pr(W > F_{obs})$
Residual	SSE	dfE	$MSE = \frac{SSE}{dfE}$		
Total	SST	dfT			

Example 3.4. In Example 3.1, compute and interpret the coefficients of determination. Compute and interpret the ANOVA table. Test whether the regression model is significant. (This means perform the F test.)

```
# recall
head(df)
```

```
  Weight BodyFat
1    175       6
2    181      21
3    200      15
4    159       6
5    196      22
6    192      31
```

```r
# The F test results are given in the summary
summary(model)
```

```
Call:
lm(formula = BodyFat ~ Weight, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-12.5935  -5.7904   0.6536   5.2731  10.4004

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -27.37626   11.54743  -2.371 0.029119 *
Weight        0.24987    0.06065   4.120 0.000643 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.049 on 18 degrees of freedom
Multiple R-squared:  0.4853,    Adjusted R-squared:  0.4567
F-statistic: 16.97 on 1 and 18 DF,  p-value: 0.0006434
```

```r
# The ANOVA table is given below

# First define the null model object using lm()
# This line fits a model with only the intercept term
null_model=lm(BodyFat~1,data=df)

# This line gets the ANOVA table
anova(null_model,model)
```

```
Analysis of Variance Table
```

```
Model 1: BodyFat ~ 1
Model 2: BodyFat ~ Weight
  Res.Df     RSS Df Sum of Sq      F    Pr(>F)
1     19 1737.75
2     18  894.42  1    843.33 16.972 0.0006434 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# We can also do this by hand:


# Store sample size
n=nrow(df)
p=2

# Compute sum of squares
SST=t(Y-mean(Y)*rep(1,n))%*%(Y-mean(Y)*rep(1,n))
Yhat=X%*%beta_hat
res=Y-Yhat
SSE=t(res)%*%res
SSM=SST-SSE

dfe=n-p
dfm=p-1
MSM=SSM/dfm


MSE=SSE/dfe

Fv=MSM/MSE

p.val=1-pf(Fv,dfm,dfe)

# ANOVA Table:
ANOVA_Table=rbind(c(SSM,dfm,MSM,Fv,p.val),c(SSE,dfe,MSE,NA,NA),c(SST,n-1,NA,NA,NA))
rownames(ANOVA_Table)=c("Model","Error","Total")
colnames(ANOVA_Table)=c("SS","df","MS","F","p-value")
ANOVA_Table
```

```
             SS df        MS        F      p-value
Model  843.3252  1 843.32521 16.97164 0.0006434484
Error  894.4248 18  49.69027       NA           NA
```

```
Total 1737.7500 19        NA        NA           NA
```

3.3.5 Homework stop 3

Exercise 3.9. In the following real data example: **Compute and interpret** the coefficient of determination, the adjusted coefficient of determination and perform the F test for model significance. Including printing the ANOVA table, the null and alternative hypothesis, an interpretation of the p-value and the conclusion of the test.

It is challenging to assess a student's understanding of a subject without administering an exam. However, we can easily record the number of hours a student studies. A researcher would like to know if the number of hours studied and exam scores are related. This researcher collected the following data:

Student	Hours Studied	Exam Score (%)
1	5	55
2	8	65
3	12	78
4	6	58
5	10	72
6	9	68
7	15	85
8	7	60
9	11	74
10	13	80
11	14	82
12	20	90
13	5	55
14	6	59
15	18	88
16	7	62
17	16	86
18	4	50
19	3	45
20	19	89

To help you, here is some R code the dataset:

```r
# Data
study_data <- data.frame(
  Student = 1:20,
```

```
  Hours_Studied = c(5, 8, 12, 6, 10, 9, 15, 7, 11, 13, 14, 20, 5, 6, 18, 7, 16, 4, 3, 19),
  Exam_Score = c(55, 65, 78, 58, 72, 68, 85, 60, 74, 80, 82, 90, 55, 59, 88, 62, 86, 50, 45,
)
```

Exercise 3.10. Write down the interpretations of: SSE, MSE, R^2, $\bar{R}^2$, SSM.

Exercise 3.11. What is the interpretation of the p-value in the ANOVA table?

Exercise 3.12. What extra assumption is needed to perform the F-test?

3.3.6 Significance of one variable

So far, we have learned that the least squares method yields the following estimate of $\hat{\beta} = (X^\top X)^{-1} X^\top Y$ with $\mathrm{E}\left[\hat{\beta}\right] = \beta$ and $\mathrm{cov}(\hat{\beta}) = (X^\top X)^{-1}\sigma^2$. Moreover, we use MSE to estimate σ^2. Next, we learned that we can summarize the SS, df, and MS in an ANOVA table. We used the F test and the coefficient of determination to evaluate the quality of the model, i.e., to see the amount of information X provides about Y.

When the model is a significant model, then, at least one of the individual explanatory variables is useful in explaining the response. We may be interested in whether a specific covariate, or set of covariates is useful in explaining the response variable. We now learn how we can test for the significance of each individual explanatory variable separately and how we can test for the significance of a subset of explanatory variables. Note that these tests also require that the random error is normally distributed.

To test for significance and compute confidence intervals of a single variate, we have to compute the distribution of $\hat{\beta}_j$. We first compute the mean and variance of $\hat{\beta}_j$. First, given that $\mathrm{E}(\hat{\beta}) = \beta$, we have $\mathrm{E}(\hat{\beta}_j) = \beta_j$. Next, $\mathrm{Var}\left[\hat{\beta}_j\right]$ is the $(j,j)^{th}$ entry of $\mathrm{cov}(\hat{\beta})$. In addition, we have derived that $\mathrm{cov}(\hat{\beta}) = (X^\top X)^{-1}\sigma^2$.

Now, recall that if Z is multivariate normal, i.e., $Z \sim \mathcal{N}(\mu, \Sigma)$, then $b + AZ \sim \mathcal{N}(b + A\mu, A\Sigma A^\top)$, i.e., $b + AZ$ is also multivariate normal. Therefore, since we have assumed that $\epsilon \sim \mathcal{N}_n(0, \sigma^2 I)$ and that $Y|X = X\beta + \epsilon$, it follows that $Y|X \sim \mathcal{N}_n(X\beta, \sigma^2 I)$. Next, we may recall that $\hat{\beta} = (X^\top X)^{-1} X^\top Y$. Let $A = (X^\top X)^{-1} X^\top$. Then $\hat{\beta} = AY$. It follows that $\hat{\beta}$ is also multivariate normal! Putting everything together, we have that $\hat{\beta} \sim \mathcal{N}_p(\beta, (X^\top X)^{-1}\sigma^2)$.

Theorem 3.1. *Under the assumptions of the* ***normal linear regression model*** *it holds that* $\hat{\beta} \sim \mathcal{N}_p(\beta, (X^\top X)^{-1}\sigma^2)$.

Now that we have the distribution of $\hat{\beta}$, we can use it to compute the confidence intervals for β_js.

Recall from introductory statistics (MATH 1131) that you learned that if we want to compute a confidence interval for the sample mean and the sample variance was unknown, we had to estimate the variance. Similarly, here, the variance of $\hat{\beta}_j$ contains σ, an unknown parameter. Recall that, we estimate σ^2 by MSE, and so we can estimate the variance of $\hat{\beta}_j$ by $\hat{\text{Var}}\left[\hat{\beta}_j\right] = (X^\top X)^{-1}_{j,j} MSE$.

It can be shown that $\hat{\beta} \perp MSE$. Therefore, we have that

$$\frac{\hat{\beta}_j - \beta_j}{\sqrt{\widehat{var}(\hat{\beta}_j)}} \sim t_{dfE}.$$

Now that we know the distribution of $\hat{\beta}_j$, we can perform significance testing and compute confidence intervals.

If we want to test

$$H_0 \colon \beta_j = \beta_j^0 \qquad vs \qquad \beta_j \neq \beta_j^0$$

we can do the following.

The observed test statistic is $TS = \frac{\hat{\beta}_j - \beta_j^0}{\sqrt{\widehat{var}(\hat{\beta}_j)}}$. Note that, under the null hypothesis, we have that $\frac{\hat{\beta}_j - \beta_j^0}{\sqrt{\widehat{var}(\hat{\beta}_j)}} \sim t_{dfE}$. Thus, the corresponding p-value is obtained based on the t_{dfE} distribution. Specifically, we can compute the p-value $\Pr(-|TS| < Z) + \Pr(|TS| > Z) = 2 * \Pr(|TS| > Z)$, where $Z \sim t_{dfE}$.

The test proceeds as follows:

1. State the hypotheses
$$H_0 \colon \beta_j = \beta_j^0 \qquad vs \qquad H_1 : \ \beta_j \neq \beta_j^0.$$
2. Compute the test statistic $\frac{\hat{\beta}_j - \beta_j^0}{\sqrt{\widehat{var}(\hat{\beta}_j)}}$ and the p-value.
3. Interpret the p-value, and use it to decide whether you reject the null hypothesis.

Often, one may choose a threshold α, and reject the null hypothesis if the p-value falls below that threshold. Other times, we use the p-value as a description of evidence against the null. If it is larger than 0.05, but still small, then that still constitutes some evidence against the null hypothesis.

Let's now discuss one-sided hypotheses. First, consider:

$$H_0 \colon \beta_j \leq \beta_j^0 \qquad vs \qquad H_1 : \ \beta_j > \beta_j^0$$

Then, if the alternative hypothesis is true, we expect TS to be positive. The p-value is given by $\Pr(TS > Z)$, where $Z \sim t_{dfE}$. Notice that the p-value is measuring how extremely positive TS is. Using the threshold method, we can also check if $TS > t_{dfE,1-\alpha}$. Next, if we want to test

$$H_0 : \beta_j \geq \beta_j^0 \qquad vs \qquad H_1 : \beta_j < \beta_j^0,$$

then if the alternative hypothesis is true, we expect TS to be negative. The p-value is given by $\Pr(TS < Z)$, where $Z \sim t_{dfE}$. Notice that the p-value is measuring how extremely negative TS is. Using the threshold method, we can also check if $TS < t_{dfE,\alpha}$.

> **Note**
>
> We use $t_{k,p}$ to denote the pth quantile of the t distribution with k degrees of freedom. For $p = 0.025$ and large k, this is approximately equal to 2.

In Example 3.1, test if the coefficient for weight is equal to 1/3 vs. not equal to 1/3. Next, test if the coefficient for weight is less than or equal to 1/3 vs. greater than 1/3. Lastly, test if the coefficient for weight is non-zero.

First, we have that

$$\begin{aligned} H_0 : \beta_1 = 1/3 \qquad & vs \qquad H_1 : \beta_1 \neq 1/3. \\ H_0 : \beta_1 \leq 1/3 \qquad & vs \qquad H_1 : \beta_1 > 1/3. \\ H_0 : \beta_1 = 0 \qquad & vs \qquad H_1 : \beta_1 \neq 0. \end{aligned}$$

Now, let's execute the tests:

```
#changing matrix to scalar
MSE=c(MSE)
hvar_beta=solve(t(X)%*%X)*MSE

print(beta_hat)
```

```
             [,1]
[1,] -27.3762623
[2,]   0.2498741
```

```
TS=(beta_hat[2]-1/3)/sqrt(hvar_beta[2,2])

# not equal
# pt(x,df) is the CDF of a t distributed RV with df degrees of freedom at x.
p_val=2*(1-pt(abs(TS),dfe))
p_val
```

```
[1] 0.1857053
```

```r
# greater than
# pt(x,df) is the CDF of a t distributed RV with df degrees of freedom at x.
p_val=1-pt(TS,dfe)
p_val
```

```
[1] 0.9071474
```

```r
## testing equal to 0
TS=beta_hat[2]/sqrt(hvar_beta[2,2])

# not equal
# pt(x,df) is the CDF of a t distributed RV with df degrees of freedom at x.
p_val=2*(1-pt(abs(TS),dfe))
p_val
```

```
[1] 0.0006434484
```

```r
# We can also use the model object to test if it is not equal to 0:
# The test statistic and the pvalue are given in the t value and Pr(>|t|)  columns, respectiv
summary(model)
```

```
Call:
lm(formula = BodyFat ~ Weight, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-12.5935  -5.7904   0.6536   5.2731  10.4004

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -27.37626   11.54743  -2.371 0.029119 *
Weight        0.24987    0.06065   4.120 0.000643 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.049 on 18 degrees of freedom
Multiple R-squared:  0.4853,    Adjusted R-squared:  0.4567
F-statistic: 16.97 on 1 and 18 DF,  p-value: 0.0006434
```

Based on the concepts that you have learned in 1131, and what we have reviewed in previous lectures, it also follows from the above analysis that a $(1-\alpha)100$ confidence interval for β_j is

$$\hat{\beta}_j \pm t_{dfE,\alpha/2}\sqrt{\widehat{var}(\hat{\beta}_j)}.$$

Example 3.5. In Example 3.1, compute a 99% and a 95% confidence interval for the coefficient for weight. Which one is longer? Why? Interpret these intervals.

```
# By hand
beta_hat[2]+c(-1,1)*qt(0.975,dfe)*sqrt(hvar_beta[2,2])
```

```
[1] 0.1224448 0.3773035
```

```
beta_hat[2]+c(-1,1)*qt(0.995,dfe)*sqrt(hvar_beta[2,2])
```

```
[1] 0.07528522 0.42446306
```

```
# Auto software/using lm:
confint(model,level=0.95)
```

```
                 2.5 %     97.5 %
(Intercept) -51.6365090 -3.1160157
Weight        0.1224448  0.3773035
```

```
confint(model,level=0.99)
```

```
                  0.5 %    99.5 %
(Intercept) -60.61484736 5.8623227
Weight        0.07528522 0.4244631
```

If we took many samples of size 20 and computed a 95% (99%) confidence interval for each sample, then 95% (99%) of then would contain the true coefficient for the weight variable. We can conclude that with 95% (99%) confidence, the true coefficient for weight likely falls within (0.12, 0.38) ((0.08,0.42)).

> **Caution**
>
> The key to understanding a confidence interval is to realize that the end points of the interval depend on the sample, and are therefore, random. On the other hand, the population parameter is not random, it is fixed. Therefore, if we drew a different sample, the interval would move, and there is a $(1-\alpha)100\%$ chance that that interval catches the population parameter. Most of the time it will contain the parameter, but not always.

Recall that the point of computing a confidence interval is to report the uncertainty in our estimate that resulted from drawing a sample. We expect the true parameter to be somewhere in that range, and our best guess at the parameter is given by the center of the interval.

3.3.7 Inference for the mean response and prediction intervals

We may wish to estimate the average response at a specific set of the covariates x. Given x, the theoretical mean response is $x^\top\beta$. Given x, we can estimate the mean response as $x^\top\hat{\beta}$. For instance, what is the average body fat percentage at 160 pounds? How accurate is our estimate? We can use a confidence interval to answer this question.

Note that the expectation and variance of the estimate of the mean response are given by $\mathrm{E}\left[x^\top\hat{\beta}\right] = x^\top\beta$ and $\mathrm{Var}\left[x^\top\hat{\beta}\right] = x^\top(X^\top X)^{-1}x\sigma^2$. Again, we must estimate σ and we can write $\hat{\mathrm{Var}}\left[x^\top\hat{\beta}\right] = x^\top(X^\top X)^{-1}xMSE$.

Exercise 3.13. Under the assumptions of the normal linear regression model, show that for a fixed covariate vector $x \in \mathbb{R}^p$, $x^\top\hat{\beta}$ has a multivariate normal distribution and find it's mean and variance. Argue that $\frac{x^\top\hat{\beta}-x^\top\beta}{\sqrt{\hat{\mathrm{Var}}\left[x^\top\hat{\beta}\right]}} \sim t_{dfE}$.

It can be shown that a $(1-\alpha)100\%$ confidence interval for the mean response $\mathrm{E}[Y|X=x]$ is

$$x^\top\hat{\beta} \pm t_{dfE,\alpha/2}\sqrt{\hat{\mathrm{Var}}\left[x^\top\hat{\beta}\right]}.$$

Similarly, if we want to test

$$H_0 \colon \mathrm{E}[Y|X=x] = \mu_0 \qquad vs \qquad \mathrm{E}[Y|X=x] \neq \mu_0$$

we can do the following:

The observed test statistic is $TS(x,\mu_0) = \frac{x^\top\hat{\beta}-\mu_0}{\sqrt{\hat{\mathrm{Var}}\left[x^\top\hat{\beta}\right]}}$. Observe that under the null hypothesis, we have that $TS(x,\mu_0) \sim t_{dfE}$. Therefore, the p-value is given by $2 * \Pr\left(|TS(x,\mu_0)| > Z\right)$.

Similar to the previous section, we can also perform one-sided tests:

- Right-sided test ($H_1 : x^\top \beta > \mu_0$): p-value $\Pr\left(TS(x, \mu_0) > Z\right)$.
- Left-sided test ($H_1 : x^\top \beta < \mu_0$): p-value $\Pr\left(TS(x, \mu_0) < Z\right)$.

We may also wish to predict what the response will be, given a new set of covariates. On top of that, we may again wish to quantify how much error there is in our prediction. For instance, what is the predicted body fat percentage of someone who is 160 pounds? Note that this differs from the previous section. In the previous section, we were interested in the average body fat percentage of someone who is 160 pounds. Here, we are interested in predicting the body fat percentage of a single, specific person, and not the average of the whole population.

Specifically, suppose that we have a subject whose covariates are given by z, but we do not know the value of the subjects response, which we can denote by Y_{new}. Then the true response is $(Y_{new}|Z = z) \ = z^\top \beta + \epsilon_{new}$.

Suppose we want to predict Y_{new} and give an idea of how much error is in our prediction. We want to predict the response, $\mathrm{E}\left[Y_{new}|Z = z\right] = z^\top \beta$. Of course, we don't know β, so our predicted response will be $\widehat{\mathrm{E}\left[Y_{new}|Z = z\right]} = z^\top \hat{\beta}$. We then model the new variable as $\tilde{Y}_{new} = z^\top \hat{\beta} + \epsilon_{new}$. The variance of this prediction is : $\mathrm{Var}\left[\tilde{Y}_{new}|Z = z\right] = \mathrm{Var}\left[z\hat{\beta}\right] + \mathrm{Var}\left[\epsilon_{new}\right] = z^\top (X^\top X)^{-1} z\sigma^2 + \sigma^2$. Therefore, the variation in a new response is the variation in our estimate of β plus the inherent population variation, σ^2. We have that this can be estimated with: $\hat{\mathrm{Var}}\left[\tilde{Y}_{new}|Z = z\right] = z^\top (X^\top X)^{-1} z MSE + MSE$.

Exercise 3.14. Under the assumptions of the normal linear regression model, show that for a fixed covariate vector $z \in \mathbb{R}^p$, $\tilde{Y}_{new}|Z = z$ has a multivariate normal distribution and find it's mean and variance. Argue that given $Z = z$,

$$\frac{\tilde{Y}_{new} - z^\top \beta}{\sqrt{\hat{\mathrm{Var}}\left[\tilde{Y}_{new}\right]}} \sim t_{dfE}.$$

Therefore, the $(1-\alpha)100\%$ prediction interval for Y_{new} is given by:

$$z\hat{\beta} \pm t_{dfE,\alpha/2} \sqrt{z^\top (X^\top X)^{-1} z MSE + MSE}.$$

Note that the prediction interval is wider than that of the mean response interval for the same covariate vector z. That is because it is more difficult to predict the response for a specific person than it is to estimate a mean of a population. Furthermore, the interpretation of a prediction interval is different. A $(1-\alpha)100\%$ prediction interval can be interpreted it as follows. Given a $(1-\alpha)100\%$ prediction interval for $Y_{new}|Z = z$, say (a, b), we say that the probability Y_{new} is in (a, b) is $(1-\alpha)100\%$. Note that this differs substantially from a confidence interval!

Example 3.6. In Example 3.1, execute the following: What is a 95% confidence interval for the mean of someone who weighs 165 pounds? What is a 95% prediction interval for the BF% of someone who weighs 165 pounds? Interpret these intervals.

```r
# Intervals are given as follows:

z <- data.frame(Weight=165)
predict(model, newdata = z, interval = 'confidence')
```

```
       fit      lwr      upr
1 13.85297 9.379675 18.32627
```

```r
predict(model, newdata = z, interval = 'prediction')
```

```
       fit       lwr      upr
1 13.85297 -1.617547 29.32349
```

We are 95% confident the mean body fat of a person who weighs 165 pounds is in 13.8529704, 9.3796749, 18.3262658. There is a 95% probability that the body fat of a person who weights 165 pounds is in 13.8529704, -1.6175473, 29.323488 . Note that the prediction interval is wider!

3.3.8 Homework stop 4

Exercise 3.15. What is the difference between a prediction interval and an interval for the mean response ?

Exercise 3.16. Code the confidence intervals for the mean response and prediction interval without using the predict function.

Exercise 3.17. Complete the chapter 3 practice problems from the problem list.

3.3.9 Partial testing

We may be interested in executing the following hypothesis test:

$$H_0 : (\beta_1, \dots, \beta_k) = 0 \qquad vs \qquad (\beta_1, \dots, \beta_k) \neq 0.$$

This amounts to testing whether the subset of variables $(\beta_1, \dots, \beta_k)$ adds anything to the model beyond $(\beta_{k+1}, \dots, \beta_p)$. For example, you may be interested in whether location related covariates affect the price of Airbnb. The overall idea is to compare the reduced (null) model with $p - k$ covariates to the complete (saturated, full) model (which contains all covariates).

Let's first review the F-test. We learned about the F test, which compares the following models:

$$Y|X = \beta^\top X + \epsilon \qquad vs \qquad Y|X = \beta_1 + \epsilon.$$

Here, the complete model is given by $Y|X = \beta^\top X + \epsilon$ and the reduced model is given by $Y|X = \beta_1 + \epsilon$. Recall that the test statistic is given by

$$\frac{SSM/dfM}{SSE/dfE} = \frac{(SST - SSE)/(dfT - dfE)}{SSE/dfE},$$

where the degrees of freedom are in terms of the full model (not the null model). We could then rewrite this test statistic as

$$\frac{SSM_C/dfM_C}{SSE_C/dfE_C} = \frac{(SST_C - SSE_C)/(dfT_C - dfE_C)}{SSE_C/dfE_C},$$

where C stands for the complete model. (All that has changed is the notation, we added a C subscript.)

Now, note that $SST = \sum_{i=1}^n (Y_i - \bar{Y})^2$ has nothing to do with what covariates are in the model. In other words, SST is always the same, not matter what covariates are in the model. Therefore, $SST_C = SST_R = SST$, where SST_R stands for the "sum of squares total" in the reduced model. In our example of the F test, the least squares estimate of β_1 in the reduced model is $\hat{\beta}_1 = \bar{Y}$ and the associated residual vector is given by $\hat{\epsilon} = Y - \bar{Y}1_n$. But wait, observe that in this case, we have that $\hat{\epsilon}^\top \hat{\epsilon} = SST$! Therefore, putting everything together, in this example, we have that $SSM_C = SST_C - SSE_C = SSE_R - SSE_C$. That is, the model sum of squares for the complete model is the difference between the sum-squared error in the reduced model and the sum-squared error in the complete model. We can then rewrite the test statistic as

$$\frac{(SSE_R - SSE_C)/(dfT_C - dfE_C)}{SSE_C/dfE_C}.$$

The difference $SSE_R - SSE_C$ can be interpreted as the extra information gained from adding the covariates into the model OR total explained variations lost by going from the full model to the reduced model.

This idea can be generalized to develop a general method for testing hypotheses of the type:

$$H_0 : (\beta_2, \dots, \beta_k) = 0 \qquad vs \qquad (\beta_2, \dots, \beta_k) \neq 0.$$

We complete the test as follows. Given a full model (which contains $\beta_1, \dots, \beta_p$) and reduced model (which contains $\{\beta_1, \beta_{k+1}, \dots, \beta_p\}$), define:

- $SSE_R - SSE_C = SSdrop$
- $dfE_R - dfE_C = dfdrop$
- $MSdrop = SSdrop/dfdrop$

Then the test statistic and p-value are given by: $TS = MSdrop/MSE_C$ and $\Pr\left(F_{dfdrop,dfE_C} \geq TS\right)$, respectively.

We can interpret $SSE_R - SSE_C$ as the extra info gained from adding the extra covariates into the model OR total explained variations lost by going from the full model to the reduced model. In addition, $dfE_R - dfE_C = k - 1$, or the number of covariates dropped from the full model to obtain the reduced model.

> **Note**
>
> If you take $k = 1$, then this is equivalent to the t-test!

3.3.10 Partial coefficient of determination

We can define the **partial coefficient of determination** as follows:

$$\begin{aligned} R^2(X_1, \dots, X_{k-1} | X_k, \dots, X_p) &= (SSE_R - SSE_C)/SSE_R \\ &= SSdrop/SSE_R. \end{aligned}$$

You might also see the partial correlation coefficient:

$$R(X_1, \dots, X_{k-1} | X_k, \dots, X_p) = \sqrt{R^2(X_1, \dots, X_{k-1} | X_k, \dots, X_p)}.$$

This quantity is the extra proportion of variation explained from adding the covariates $X_1, \dots, X_{k-1}$ to the model which already contains $X_k, \dots, X_p$.

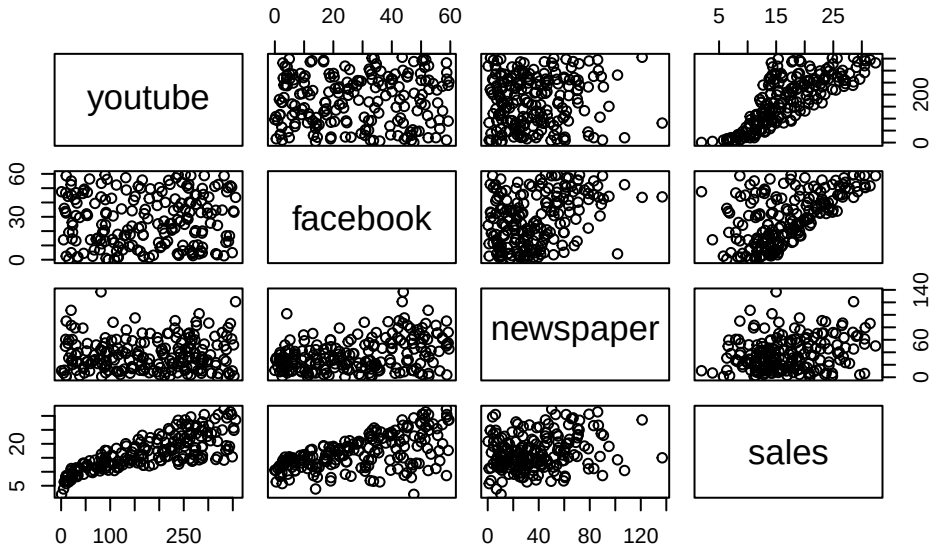
Example 3.7. A researcher ran an experiment to see if YouTube, Facebook and newspaper ads would improve sales. Run the partial F test to see how online advertising affects sales. Compute and interpret the following quantities:

- $SSE_R - SSE_C = SSdrop$
- $dfE_R - dfE_C = dfdrop$
- $MSdrop = SSdrop/dfdrop$
- Test stat: $TS = MSdrop/MSE_C$
- p-value: $\Pr(F_{dfdrop,dfE_C} \geq TS)$
- Partial coefficient of determination

```
# install.packages('datarium')
data("marketing", package = "datarium")
#printing out first few rows
head(marketing, 4)
```

```
  youtube facebook newspaper sales
1  276.12    45.36     83.04 26.52
2   53.40    47.16     54.12 12.48
3   20.64    55.08     83.16 11.16
4  181.80    49.56     70.20 22.20
```

```r
plot(marketing)
```




```r
#setting n to be a variable (sample size)
n=nrow(marketing)


# Estimation: How to get an estimate $\hat\beta$ of $\beta$?
# lm( sales~   , data= marketing)
full_model<- lm(sales ~ youtube+facebook+newspaper, data = marketing)
summary(full_model)
```

```
Call:
lm(formula = sales ~ youtube + facebook + newspaper, data = marketing)

Residuals:
```

```
     Min       1Q   Median       3Q      Max
-10.5932  -1.0690   0.2902   1.4272   3.3951

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.526667   0.374290   9.422   <2e-16 ***
youtube      0.045765   0.001395  32.809   <2e-16 ***
facebook     0.188530   0.008611  21.893   <2e-16 ***
newspaper   -0.001037   0.005871  -0.177     0.86
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.023 on 196 degrees of freedom
Multiple R-squared:  0.8972,    Adjusted R-squared:  0.8956
F-statistic: 570.3 on 3 and 196 DF,  p-value: < 2.2e-16
```

```r
summ=summary(full_model)

full_model$coefficients
```

```
 (Intercept)      youtube     facebook    newspaper
 3.526667243  0.045764645  0.188530017 -0.001037493
```

```r
MSE=summ$sigma^2

SSE_C=sum(summ$residuals^2)



# Inference: What is the error of $\hat\beta$? Is $f$ degenerate? I.e., is $\beta=0$?

#regular ANOVA
summary(full_model)
```

```
Call:
lm(formula = sales ~ youtube + facebook + newspaper, data = marketing)

Residuals:
     Min       1Q   Median       3Q      Max
-10.5932  -1.0690   0.2902   1.4272   3.3951
```

```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.526667   0.374290   9.422   <2e-16 ***
youtube      0.045765   0.001395  32.809   <2e-16 ***
facebook     0.188530   0.008611  21.893   <2e-16 ***
newspaper   -0.001037   0.005871  -0.177     0.86
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.023 on 196 degrees of freedom
Multiple R-squared:  0.8972,    Adjusted R-squared:  0.8956
F-statistic: 570.3 on 3 and 196 DF,  p-value: < 2.2e-16
```

```r
#confidence intervals for beta coefficients
# confint.lm(full_model)

#Partial F Test
model_red=lm(sales ~ newspaper, data = marketing)
sum_reduced=summary(model_red)
MSER=sum_reduced$sigma^2
SSE_R=sum(sum_reduced$residuals^2)

SSdrop=SSE_R-SSE_C

MSEdrop=SSdrop/2
Fstat=MSEdrop/MSE

1-pf(Fstat,2,196)
```

```
[1] 0
```

```r
part_test=anova(model_red,full_model); part_test
```

```
Analysis of Variance Table

Model 1: sales ~ newspaper
Model 2: sales ~ youtube + facebook + newspaper
  Res.Df    RSS Df Sum of Sq      F    Pr(>F)
1    198 7394.1
2    196  801.8  2    6592.3 805.71 < 2.2e-16 ***
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
partial_c_det=SSdrop/SSE_R



SSER=sum(model_red$residuals*model_red$residuals); SSER
```

```
[1] 7394.119
```

```
dfer=model_red$df.residual; dfer
```

```
[1] 198
```

```
SSEC=sum(full_model$residuals*full_model$residuals); SSEC
```

```
[1] 801.8284
```

```
dfeC=full_model$df.residual; dfeC
```

```
[1] 196
```

```
SSdrop=SSER-SSEC; SSdrop
```

```
[1] 6592.29
```

```
dfdrop=dfer-dfeC

MSdrop=SSdrop/dfdrop; MSdrop
```

```
[1] 3296.145
```

```
R_online=SSdrop/SSER; R_online
```

```
[1] 0.8915586
```

```
part_test$F
```

```
[1]       NA 805.7141
```

```
part_test$`Pr(>F)`
```

```
[1]           NA 2.812622e-95
```

```
# Prediction: Predict any values if necessary.
# What if we have a 300$ budget and we only can pick one advertising method?
new_data=marketing[1:3,1:3]
new_data[1:3,]=diag(300,3)
predict(full_model,new_data)
```

```
        1         2         3
17.256061 60.085672  3.215419
```

```
# It's best to put our money in FB... meta?

# What about intervals?

predict(full_model,new_data, interval = 'confidence')
```

```
        fit         lwr       upr
1 17.256061 16.56191879 17.950203
2 60.085672 55.25061022 64.920734
3  3.215419 -0.09445737  6.525296
```

3.3.11 Another example

It's a good time to stop and do another example to review the topics covered so far.

Example 3.8. In the dataset `mtcars` we have the following variables:

- mpg: Miles/(US) gallon
- cyl: Number of cylinders
- disp: Displacement (cu.in.)
- hp: Gross horsepower
- drat: Rear axle ratio

- wt: Weight (1000 lbs)
- qsec: 1/4 mile time
- vs: V/S
- am: Transmission (0 = automatic, 1 = manual)
- gear: Number of forward gears
- carb: Number of carburetors

The data was extracted from the 1974 Motor Trend US magazine, and comprises fuel consumption and 10 aspects of automobile design and performance for 32 automobiles (1973–74 models). Overall, we would like to investigate the relationship between `mpg` and the following variables: `cyl, disp, hp, drat, wt, qsec, gear, carb.` Let's investigate the following questions:

1. Assume the normal MLR model. Store the covariate matrix and response in a variable. Fit a normal MLR model to the data. – That is use `lm()` to fit the model.
2. What are the least squares estimates? What is the MSE?
3. Generate the ANOVA table. Is the model significant?
4. Test if `drat` contributes anything to the model, adjusting for the other covariates. Test if `drat` is related to `mpg`, without adjusting for the other covariates.
5. Test if the subset of variables `gear, carb` contribute to the model jointly, adjusting for the remaining covariates. What is the partial coefficient of determination? Interpret the partial coefficient of determination. Test if the subset of variables `gear, carb` contribute to the model jointly, without adjusting for the remaining covariates.
6. Compute a confidence interval for the mean `mpg` of cars with the following set of covariate values 6.6, 176, 121, 4.29, 2.882, 18.106, 0, 1.1, 4.4, 4.4. Compute a prediction interval for the `mpg` of a car with the above set of covariate values.
7. Compute a confidence interval for the coefficient for `disp`.
8. Compute and interpret the coefficient of determination.

```
data("mtcars")
head(mtcars)
```

```
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

```
dim(mtcars)
```

```
[1] 32 11
```

```r
# 1.
# response~all variables minus the two variables we will not include
model=lm(mpg~.-vs-am,data=mtcars)
summ=summary(model)
summ
```

```
Call:
lm(formula = mpg ~ . - vs - am, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-3.0230 -1.6874 -0.4109  0.9640  5.4400

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 17.88964   17.81996   1.004   0.3259
cyl         -0.41460    0.95765  -0.433   0.6691
disp         0.01293    0.01758   0.736   0.4694
hp          -0.02085    0.02072  -1.006   0.3248
drat         1.10110    1.59806   0.689   0.4977
wt          -3.92065    1.86174  -2.106   0.0463 *
qsec         0.54146    0.62122   0.872   0.3924
gear         1.23321    1.40238   0.879   0.3883
carb        -0.25510    0.81563  -0.313   0.7573
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.622 on 23 degrees of freedom
Multiple R-squared:  0.8596,    Adjusted R-squared:  0.8107
F-statistic:  17.6 on 8 and 23 DF,  p-value: 4.226e-08
```

```r
X=model.matrix(model)
Y=mtcars$mpg
X[1:5,]
```

```
                  (Intercept) cyl disp  hp drat    wt  qsec gear carb
Mazda RX4                   1   6  160 110 3.90 2.620 16.46    4    4
Mazda RX4 Wag               1   6  160 110 3.90 2.875 17.02    4    4
Datsun 710                  1   4  108  93 3.85 2.320 18.61    4    1
Hornet 4 Drive              1   6  258 110 3.08 3.215 19.44    3    1
Hornet Sportabout           1   8  360 175 3.15 3.440 17.02    3    2
```

```r
Y
```

```
 [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
[16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
[31] 15.0 21.4
```

```r
# 2.
LSE=coef(model)
LSE
```

```
(Intercept)         cyl        disp          hp        drat          wt
17.88963741 -0.41459575  0.01293240 -0.02084886  1.10109551 -3.92064847
       qsec        gear        carb
 0.54145693  1.23321026 -0.25509911
```

```r
MSE=summ$sigma^2
MSE
```

```
[1] 6.874941
```

```r
# 3.
null_model=lm(mpg~1,data=mtcars)
anova(null_model,model)
```

```
Analysis of Variance Table

Model 1: mpg ~ 1
Model 2: mpg ~ (cyl + disp + hp + drat + wt + qsec + vs + am + gear +
    carb) - vs - am
  Res.Df     RSS Df Sum of Sq      F    Pr(>F)
1     31 1126.05
2     23  158.12  8    967.92 17.599 4.226e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# 4.
# Notice the p value is 0.5 , not sign.
summ$coefficients['drat',]
```

```
  Estimate Std. Error    t value   Pr(>|t|)
 1.1010955  1.5980601  0.6890201  0.4977032
```

```r
drat=lm(mpg~drat,,data=mtcars)
# Notice the p value is 1.78e-05 , sig! explain this difference!
summary(drat)
```

```
Call:
lm(formula = mpg ~ drat, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-9.0775 -2.6803 -0.2095  2.2976  9.0225

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   -7.525      5.477  -1.374     0.18
drat           7.678      1.507   5.096 1.78e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.485 on 30 degrees of freedom
Multiple R-squared:  0.464, Adjusted R-squared:  0.4461
F-statistic: 25.97 on 1 and 30 DF,  p-value: 1.776e-05
```

```r
# 5.
red_model=lm(mpg~.-vs-am-gear-carb,data=mtcars)
anova(red_model,model)
```

```
Analysis of Variance Table

Model 1: mpg ~ (cyl + disp + hp + drat + wt + qsec + vs + am + gear +
    carb) - vs - am - gear - carb
Model 2: mpg ~ (cyl + disp + hp + drat + wt + qsec + vs + am + gear +
    carb) - vs - am
  Res.Df    RSS Df Sum of Sq      F Pr(>F)
1     25 163.48
2     23 158.12  2    5.3532 0.3893 0.6819
```

```r
ob=anova(red_model,model)
ob$`Sum of Sq`[2]/ob$RSS[1]
```

```
[1] 0.0327457
```

```r
# 3.7% of the variation in mpg is explained from adding the covariate gear and carb to the mo

# 6.
new_ob=c(6.6,176,121,4.29,2.882,18.106,0,1.1,4.4,4.4)
new_ob=matrix(new_ob,nrow=1,ncol=length(new_ob))
colnames(new_ob)=names(mtcars[1,-1])
new_ob=data.frame(new_ob)
predict(model,new_ob, interval = 'confidence')
```

```
       fit      lwr      upr
1 22.43839 18.49468 26.38211
```

```r
predict(model,new_ob, interval = 'prediction')
```

```
       fit     lwr      upr
1 22.43839 15.7322 29.14459
```

```r
# 7.
confint(model)
```

```
                   2.5 %      97.5 %
(Intercept) -18.97375462 54.75302945
cyl          -2.39565252  1.56646102
disp         -0.02343129  0.04929609
hp           -0.06371601  0.02201829
drat         -2.20474377  4.40693480
wt           -7.77195651 -0.06934042
qsec         -0.74362628  1.82654014
gear         -1.66782660  4.13424711
carb         -1.94235037  1.43215215
```

```r
#8.
summ$r.squared
```

```
[1] 0.8595764
```

```
# 85% of the variation in mpg is explained by cyl, disp, hp, drat, wt, qsec, gear and carb
```

Exercise 3.18. Interpret all of the above quantities.

3.4 What variables should I include? Omitted variable bias, collider bias, mediators, and moderators

3.4.1 Frisch-Waugh-Lovell Theorem

One very interesting result is the Frisch-Waugh-Lovell Theorem! This theorem describes how the regression coefficients depend on the other variables in the model.

Consider the MLR with $p-1$ variables and p coefficients. Then, suppose that we regress the jth covariate, X_j, on the remaining covariates. That is, X_j is the response and the remaining covariates are the covariates. From this model, we can obtain residuals, denoted δ_j. The Frisch-Waugh-Lovell Theorem is given as follows:

Theorem 3.2. *The least squares coefficient, $\hat{\beta}_j$ is equivalent to the regression coefficient obtained via regressing Y on δ_j.*

This means that instead of computing $\hat{\beta} = (X'X)^{-1}X'Y$ and taking the jth component to obtain $\hat{\beta}_j$, we can instead do the following:

1. Regress the jth covariate, X_j, on the remaining covariates and compute the residuals: δ_j.
2. Regress Y on δ_j, in particular, the coefficient for δ_j will be equal to $\hat{\beta}_j = \frac{cov(y,\delta_j)}{var(\delta_j)}$.

Here, cov $[y, \delta_j]$ and $var(\delta_j)$ refer to the sample quantities, as in the sample covariance between y and δ_j and the sample variance of δ_j.

The theorem has multiple implications. Importantly, the part of X_j explained by other predictors is removed when computing it's coefficient. See the following simulation. Here Z and W both contribute some information about Y, but nothing beyond what X gives. So once we include X, Z and W no longer meaningfully explain Y. Also, we verify the FWL theorem for the coefficient W:

```r
# sample size
n=1000

# Model - Y is a function of X, and W and Z are just noisy versions of X
X=rnorm(n)
Y=2*X+rnorm(n)

Z=X/2+rnorm(n)/2
W=X/2+rnorm(n)/2


# model without X
summary(lm(Y~W+Z))
```

```
Call:
lm(formula = Y ~ W + Z)

Residuals:
    Min      1Q  Median      3Q     Max
-4.5674 -1.0337 -0.0408  1.0082  5.7001

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.04948    0.04848   1.021    0.308
W            1.34712    0.07972  16.897   <2e-16 ***
Z            1.29004    0.07925  16.277   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.532 on 997 degrees of freedom
Multiple R-squared:  0.5451,    Adjusted R-squared:  0.5442
F-statistic: 597.5 on 2 and 997 DF,  p-value: < 2.2e-16
```

```r
# model with X
summary(lm(Y~X+W+Z))
```

```
Call:
lm(formula = Y ~ X + W + Z)
```

```
Residuals:
    Min      1Q  Median      3Q     Max
-3.3136 -0.6678 -0.0348  0.6717  2.7144

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.01713    0.03137   0.546    0.585
X            1.98209    0.05322  37.243   <2e-16 ***
W            0.02753    0.06257   0.440    0.660
Z           -0.01075    0.06203  -0.173    0.862
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9912 on 996 degrees of freedom
Multiple R-squared:  0.8099,    Adjusted R-squared:  0.8093
F-statistic:  1414 on 3 and 996 DF,  p-value: < 2.2e-16
```

```r
# FWL algorithm, see the coefficients match up
summary(lm(W~Z+X))
```

```
Call:
lm(formula = W ~ Z + X)

Residuals:
     Min       1Q   Median       3Q      Max
-1.61346 -0.34401 -0.00031  0.35200  1.44736

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 0.001964   0.015878   0.124    0.902
Z           0.048142   0.031361   1.535    0.125
X           0.481708   0.022203  21.695   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.5018 on 997 degrees of freedom
Multiple R-squared:  0.5184,    Adjusted R-squared:  0.5174
F-statistic: 536.5 on 2 and 997 DF,  p-value: < 2.2e-16
```

```r
delta_j=lm(W~Z+X)$residuals
summary(lm(Y~delta_j))
```

```
Call:
lm(formula = Y ~ delta_j)

Residuals:
    Min      1Q  Median      3Q     Max
-8.1260 -1.5802  0.0344  1.5193  7.3545

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.05054    0.07182   0.704    0.482
delta_j      0.02753    0.14335   0.192    0.848

Residual standard error: 2.271 on 998 degrees of freedom
Multiple R-squared:  3.695e-05, Adjusted R-squared:  -0.000965
F-statistic: 0.03687 on 1 and 998 DF,  p-value: 0.8478
```

3.4.2 Omitted variable bias

Omitted variable bias occurs when an important variable is omitted. Observe the following. Suppose the true model is

$$Y = \beta_0 + \beta_1 W + \beta_2 Z + \epsilon.$$

Then, if we run a regression model with only W, we will end up with a coefficient that is roughly

$$\tilde{\beta}_1 = \beta_1 + \beta_2 \frac{\text{cov}\left[W, Z\right]}{\text{Var}\left[W\right]}.$$

The term $\beta_2 \frac{\text{cov}[W,Z]}{\text{Var}[W]}$ "biases'' the coefficient, and may even change the sign. This explains why we sometimes see coefficients change dramatically when variables are omitted. Sometimes the sign of the coefficient may even change.

3.4.3 Collider bias, mediators, and moderators

Suppose you are mainly interested in estimating or isolating the total causal effect of X on Y. Then, given the previous section, it's important to be careful what variables we include in our model. Any confounders should be included in the model. Confounders are variables

that affect both X and Y, creating a fake or spurious causal relationship between X and Y or distorting a causal relationship between X and Y.

Blocking variables, or mediators, are ones that "mediate" the relationship between X and Y. They should *not* be included, as they will either partially or fully obscure or block the effect of X. These are intermediate mechanisms, that may be consequences of X, that we do not wish to control for.

We must also be aware of something called **collider bias**. A collider is a variable C such that both X and Y have a causal effect on C. If we include C in the model, then it can create a fake or spurious relationship between X and Y, so these variables should also be omitted.

Example 3.9. Suppose we are interested in exam scores and universities in general, then an example of each of the previous terms:

1. *Confounder:* Prior preparation affects both tutoring participation and exam scores. Leaving it out can distort the estimated effect of tutoring.
2. *Mediator:* Tutoring increases understanding, which improves exam scores. Controlling for understanding removes the pathway through which tutoring works. Therefore, understanding would block the relationship between tutoring and exam scores.
3. *Collider:* Admission to a university depends on both academic achievement and athletic ability. Among admitted students, weaker academic achievement may be compensated for by stronger athletic ability, creating a perceived negative association between athletic ability and academic achievement, even if the two are independent in the applicant population.

Lastly, moderators may change or moderate the effect of X on Y, and we will cover those in a later section, where we discuss interaction effects.

3.5 Checking model assumptions

We learned how to test significance of one or multiple variables, compute confidence intervals for the estimated coefficients, mean response, and predicted response. All the methods rely on the assumptions! Recall that we assume

1. The relationship is linear $Y|X = X\beta + \epsilon$,
2. $\forall i \in [n]$, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$
3. $\epsilon_i \perp \epsilon_j$ for $i \neq j,\ i, j \in [n]$.

We now briefly discuss how to use the data to check if these assumptions are appropriate. We will cover this in more detail in the next chapter.

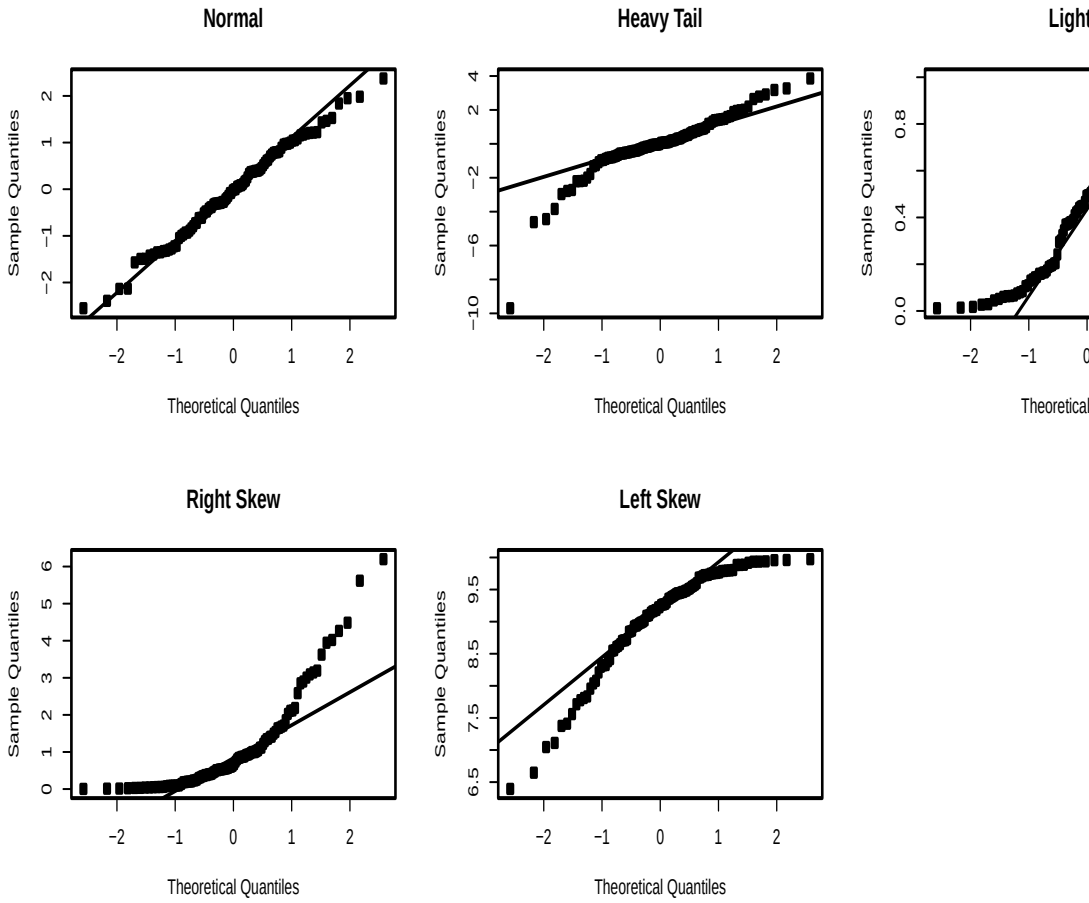
3.5.1 Checking normality

We do not know ϵ, however, we do know $\hat{\epsilon}$, which is our best proxy for the true random error vector ϵ. To check if the true random error vector is normally distributed we can use histograms and quantile-quantile plots. More specifically, if the histogram of the residuals looks more or less bell-shaped, with tails similar to the normal PDF, then the assumption of normality is valid.

Recall that a qq-plot compares the quantiles of the sample to the quantiles of the theoretical normal distribution. The x-axis represents the theoretical quantiles. The y-axis represents the sample quantiles. If the sample follows a normal distribution, the points in the qq-plot will approximately lie on a line.

Interpretation:

- Straight Line: If the points lie on or near the straight line, the sample appears normal.
- Heavy Tails: Points deviating upwards or downwards at the ends suggest the sample has heavier or lighter tails than the normal distribution.
- S-Shape: Points forming an S-shape indicate the sample has lighter tails and a heavier center than the normal distribution.



See below for an example:
Note that you will always have some deviation at the ends of the line in the qq-plot.

Example 3.10. In examples Example 3.1 and Example 3.7, check that the normality assumption is valid.

```
# Make the data frame
Weight=c(175 , 181 , 200 , 159 , 196 , 192 , 205 , 173 , 187 , 188 ,
         188 , 240 , 175 , 168 , 246 , 160 , 215 , 159 , 146 , 219 )
BodyFat =c(6 , 21 , 15 , 6 , 22 , 31 , 32 ,21 , 25 , 30 ,
           10 , 20 , 22 , 9 , 38 , 10 , 27 , 12 , 10 , 28 )

df=data.frame(cbind(Weight=Weight,BodyFat=BodyFat))
model= lm(BodyFat ~Weight, data = df)
summary(model)
```

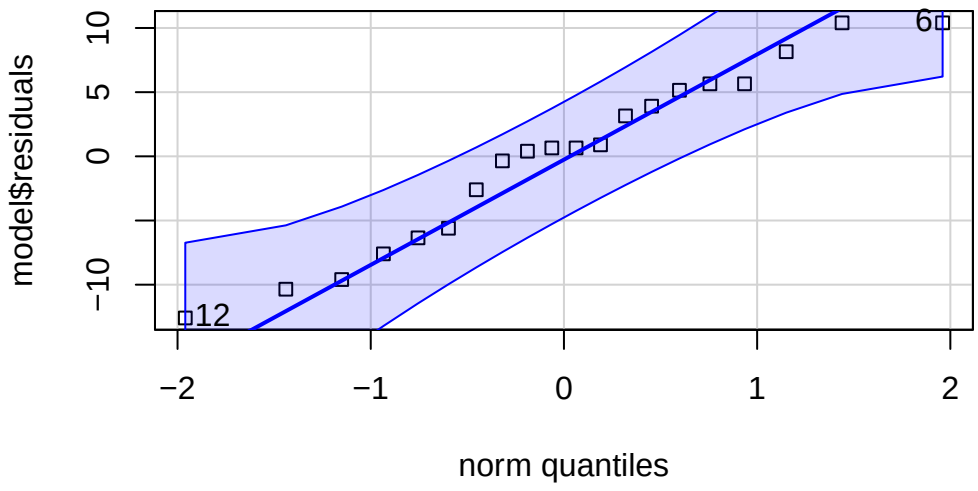
```
Call:
lm(formula = BodyFat ~ Weight, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-12.5935  -5.7904   0.6536   5.2731  10.4004

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -27.37626   11.54743  -2.371 0.029119 *
Weight        0.24987    0.06065   4.120 0.000643 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

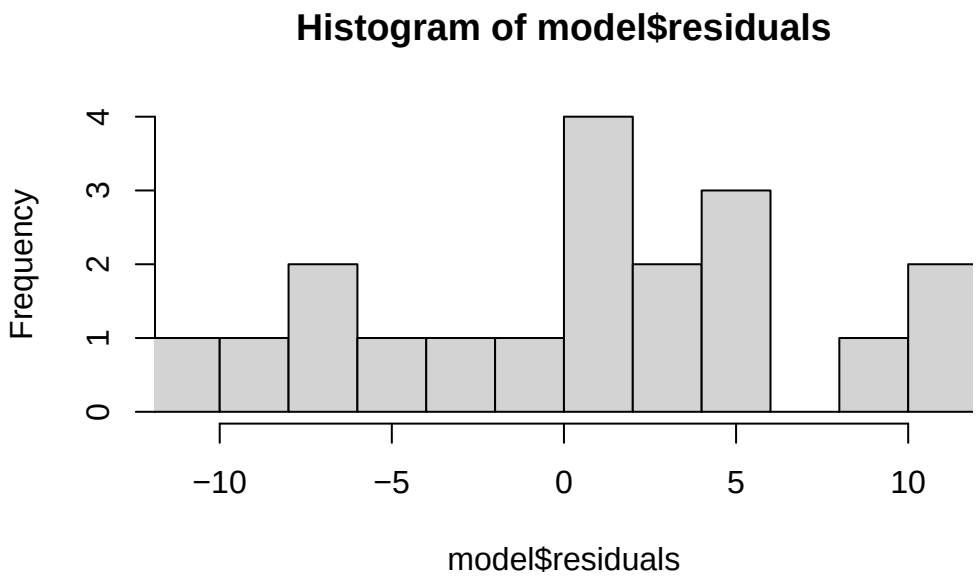
Residual standard error: 7.049 on 18 degrees of freedom
Multiple R-squared:  0.4853,    Adjusted R-squared:  0.4567
F-statistic: 16.97 on 1 and 18 DF,  p-value: 0.0006434
```

```r
car::qqPlot(model$residuals,pch=22)
```




```
[1] 12  6
```

```r
hist(model$residuals,breaks=10,xlim=c(-11,11))
```




```r
# This appears okay!

# Let's do the next example
# install.packages('datarium')
data("marketing", package = "datarium")


# lm( sales~   , data= marketing)
full_model<- lm(sales ~ youtube+facebook+newspaper, data = marketing)
summary(full_model)
```

```
Call:
lm(formula = sales ~ youtube + facebook + newspaper, data = marketing)

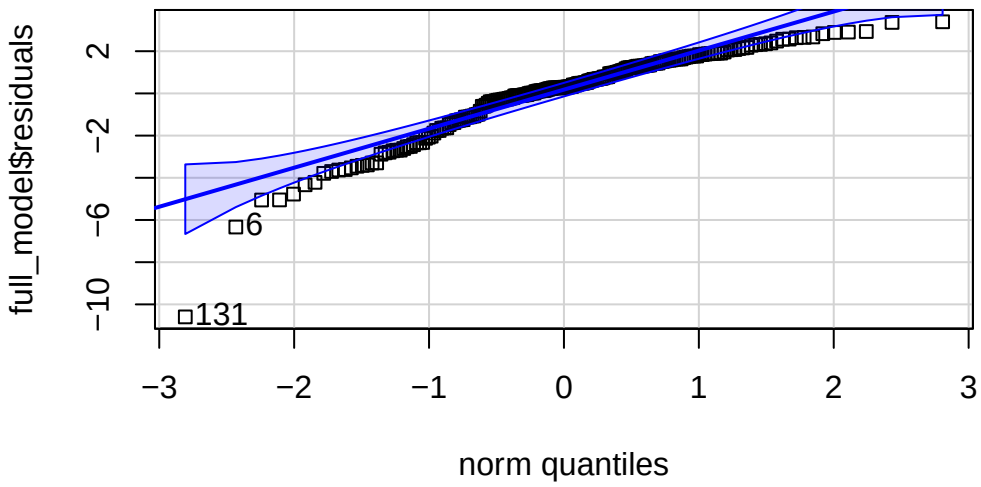
Residuals:
     Min       1Q   Median       3Q      Max
-10.5932  -1.0690   0.2902   1.4272   3.3951

Coefficients:
```

```
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.526667   0.374290   9.422   <2e-16 ***
youtube      0.045765   0.001395  32.809   <2e-16 ***
facebook     0.188530   0.008611  21.893   <2e-16 ***
newspaper   -0.001037   0.005871  -0.177     0.86
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

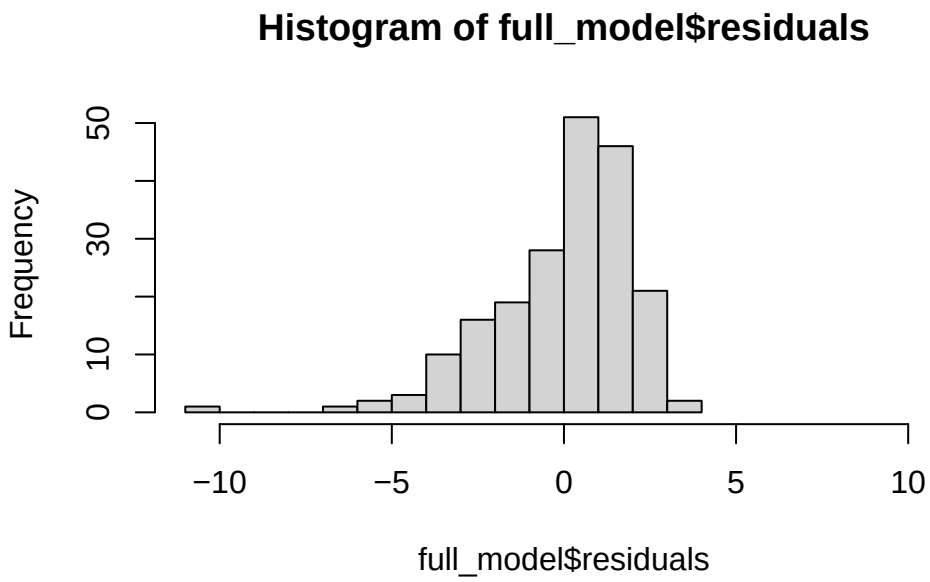
Residual standard error: 2.023 on 196 degrees of freedom
Multiple R-squared:  0.8972,    Adjusted R-squared:  0.8956
F-statistic: 570.3 on 3 and 196 DF,  p-value: < 2.2e-16
```

```
# Not great.
car::qqPlot(full_model$residuals,pch=22)
```




```
[1] 131   6
```

```
hist(full_model$residuals,breaks=10,xlim=c(-11,11))
```



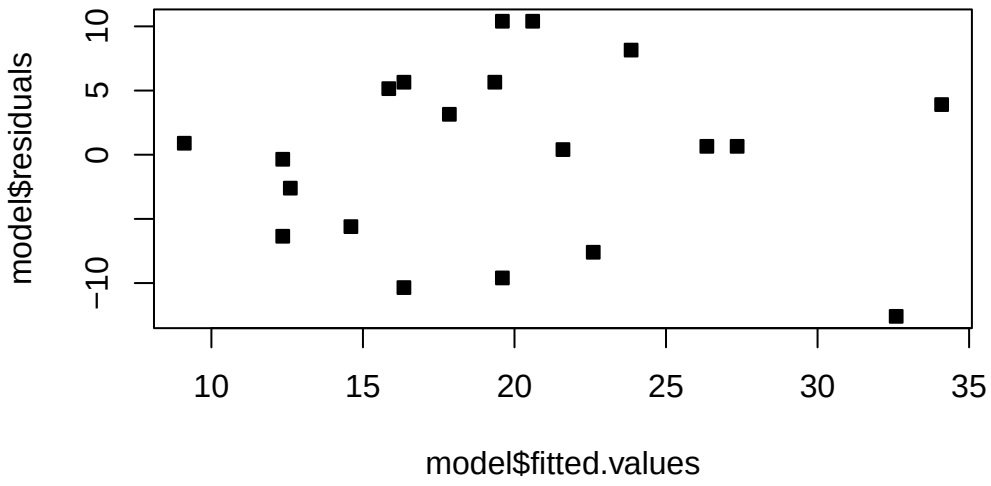
3.5.2 Checking the other assumptions

To check the remaining assumptions (constant variance, independence of residuals, zero mean and linear relationship), we can use some other diagnostic plots.

One plot is that of the fitted values $\hat{Y}$ (x-axis) against the residuals $\hat{\epsilon}$ (y-axis). If the error depends on $\hat{y}$, then the identically distributed assumption on the errors is probably not valid. If the assumptions are valid, we should observe on the plots that at all levels of the response, the mean of the residuals is 0 and the variance remains the same. Thus, we should see a horizontal band centered at 0 containing the observations.

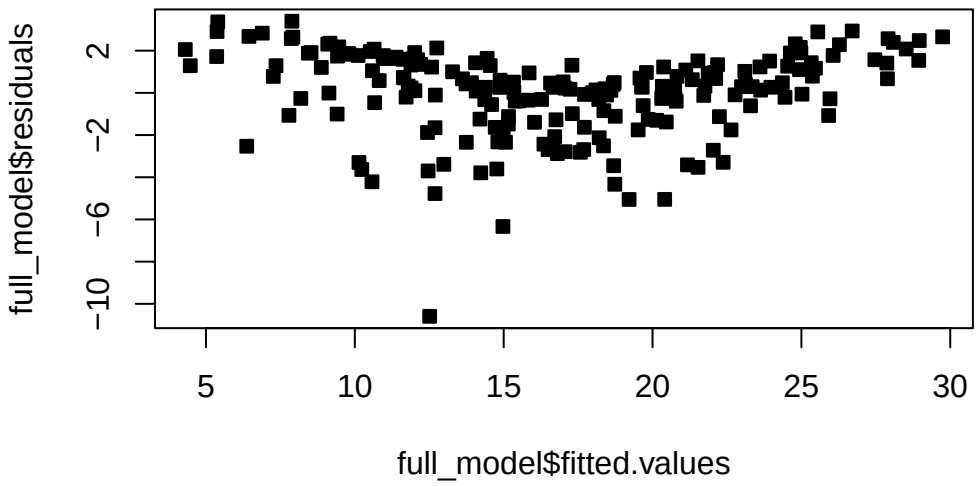
This appears to be the case in the body fat example:

```
plot(model$fitted.values,model$residuals,pch=22,bg=1)
```



Observe that in the marketing example, the residuals admit a pattern. This usually indicates either a non-linear relationship with the covariates, or an important covariate is missing. In this case, we would say the assumption of identically distributed errors is violated.

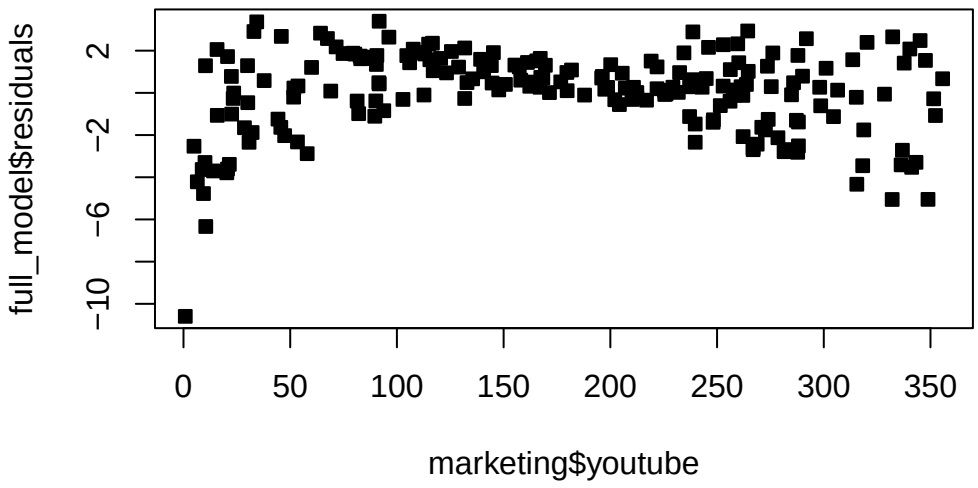
```
plot(full_model$fitted.values,full_model$residuals,pch=22,bg=1)
```



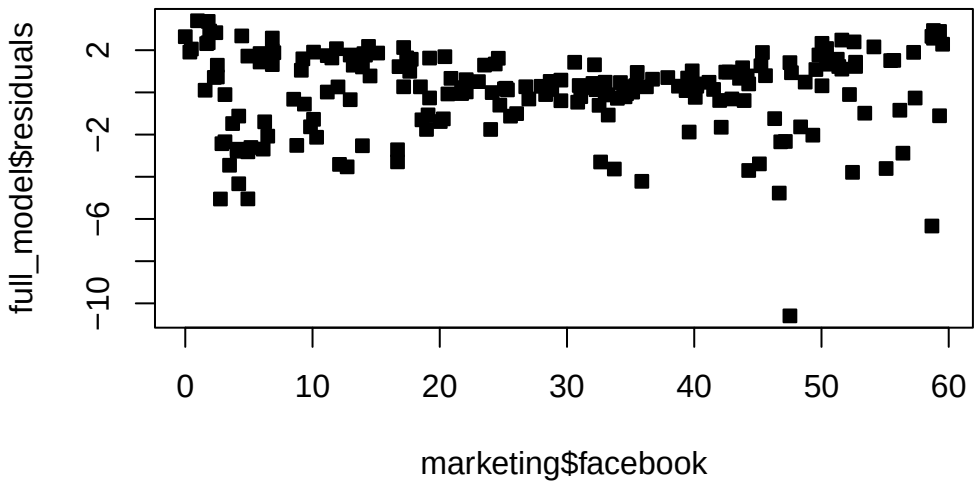
Plotting the residuals against the covariates can reveal dependence between the errors. For instance, if time is a covariate, you can plot the residuals over time to see if they have any relationship with time. If there appears to be dependence among the residuals, then the assumptions of the model are violated. That is, in these plots we should also see a horizontal band centered at 0 containing the observations. If not, then the residuals have a relationship with the given covariate.

Be VERY careful about the scale of your plot, as it can affect your interpretation. Zooming out or in too much can make everything look fine. In addition, the y-axis not being centered at 0 can cause you to misinterpret the plot.

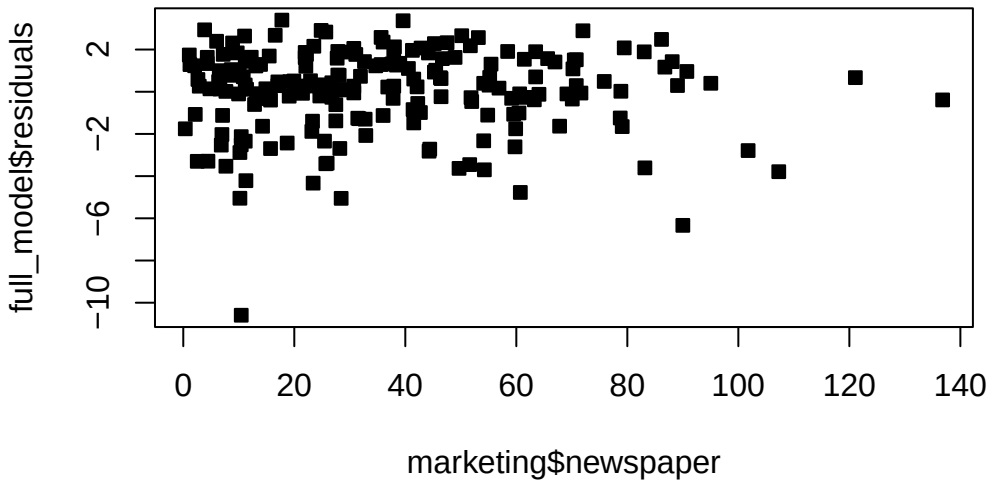
```
plot(marketing$youtube,full_model$residuals,pch=22,bg=1)
```



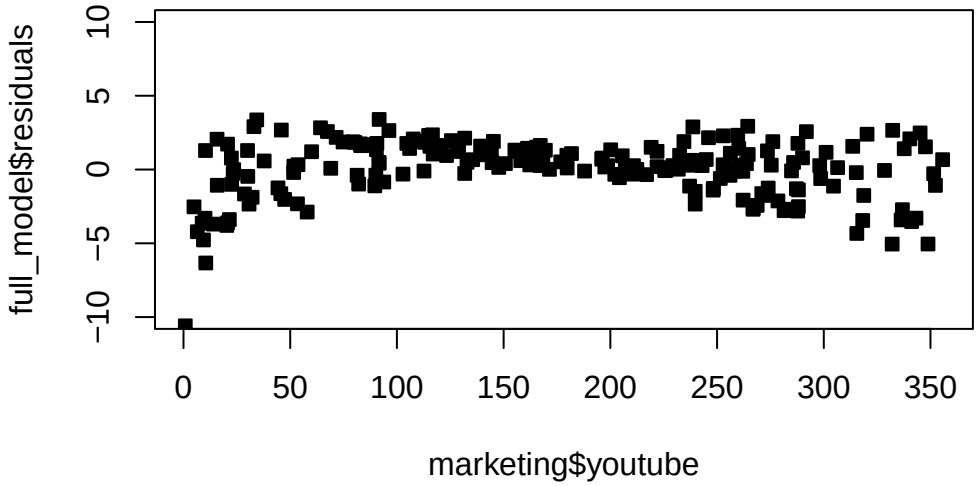

```
plot(marketing$facebook,full_model$residuals,pch=22,bg=1)
```



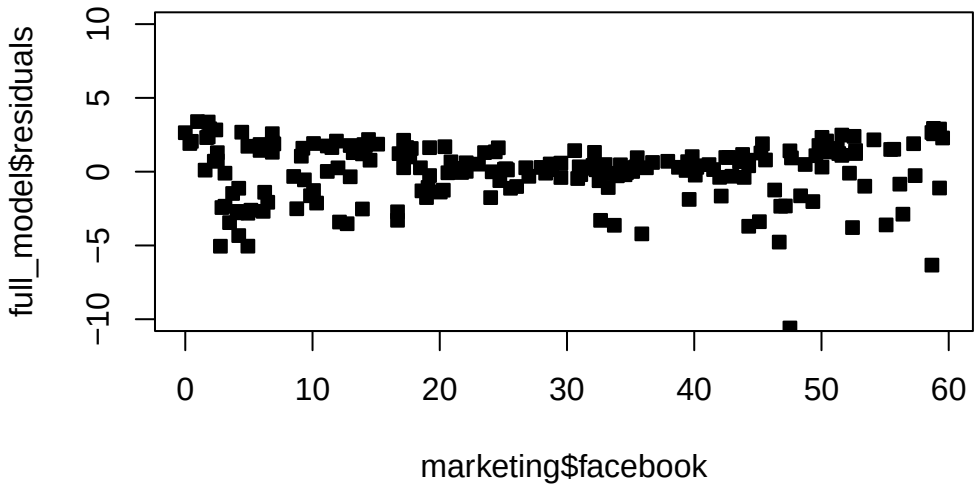
```
plot(marketing$newspaper,full_model$residuals,pch=22,bg=1)
```



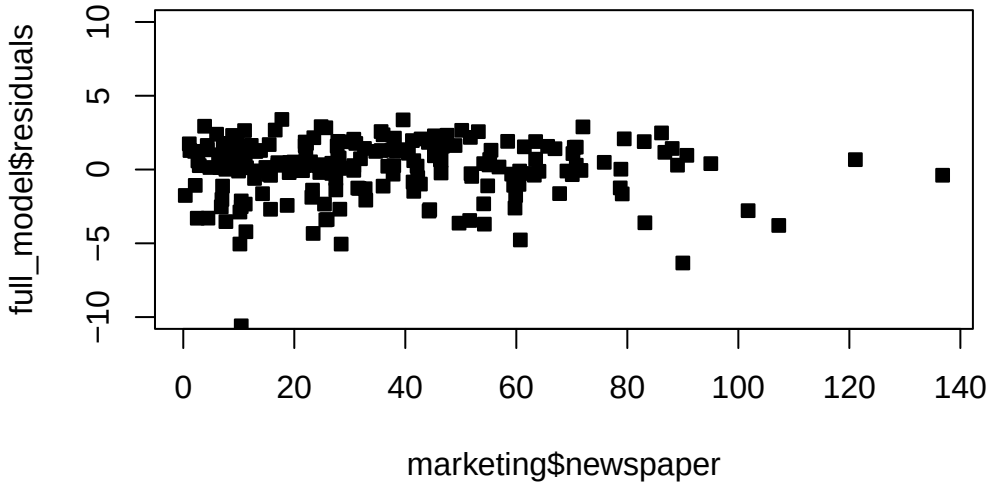

```
plot(marketing$youtube,full_model$residuals,pch=22,bg=1,ylim=c(-10,10))
```



```
plot(marketing$facebook,full_model$residuals,pch=22,bg=1,ylim=c(-10,10))
```



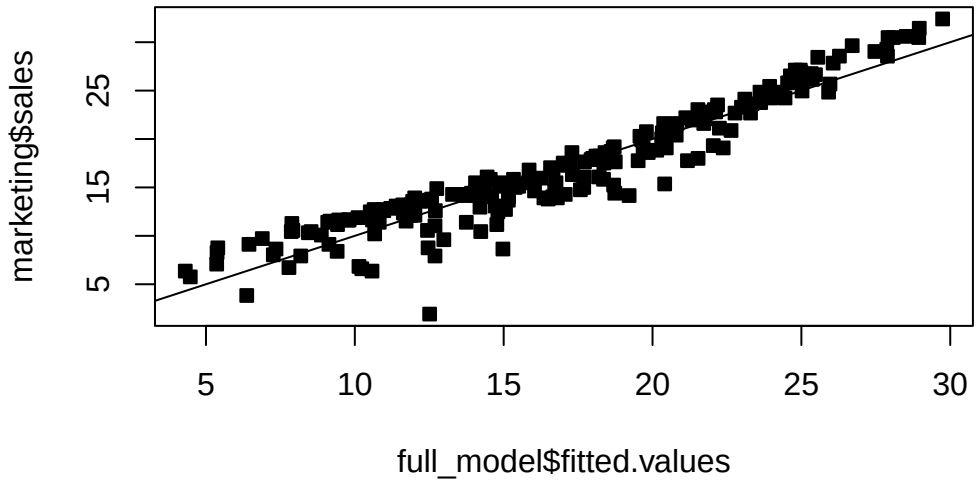

```
plot(marketing$newspaper,full_model$residuals,pch=22,bg=1,ylim=c(-10,10))
```



Notice how the newspaper plot changes with the new axis limits. It appears that the variance of the error is changing with the value of the Facebook and Youtube budgets.

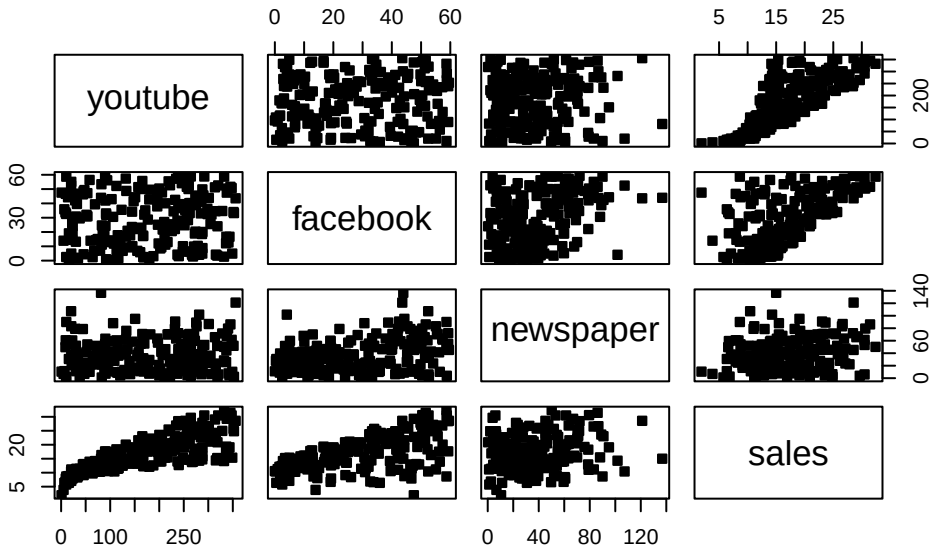
Another plot is that of the fitted values against the response. This gives an idea of the overall fit of the model. We should observe the points scatters around the line $y = x$.

```
plot(full_model$fitted.values,marketing$sales,pch=22,bg=1)
abline(0,1)
```



Notice that the line is slightly curved above the line at the ends. This means that at high and low values, the actual sales are empirically greater than as predicted by the model. Let's plot the actual data.

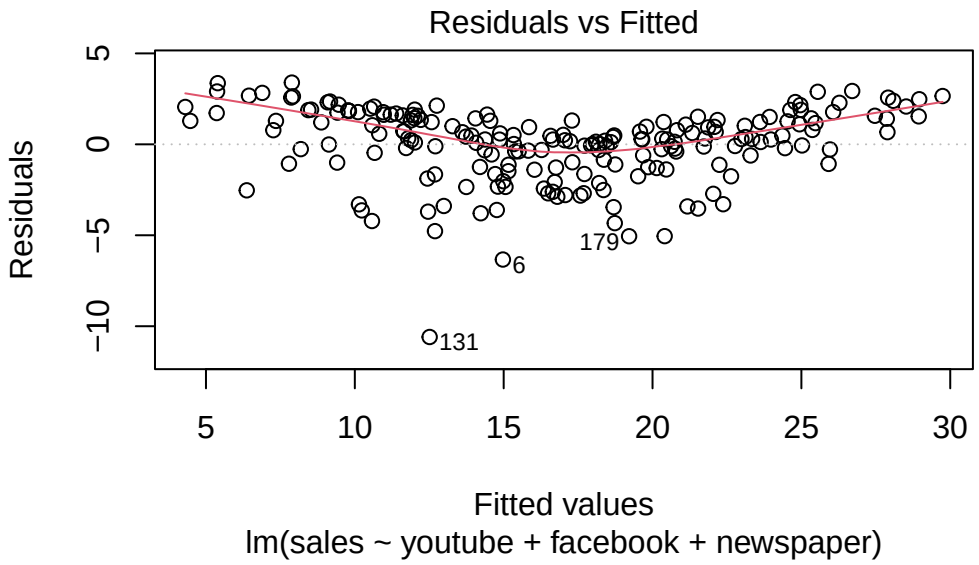
```
plot(marketing,pch=22,bg=1)
```

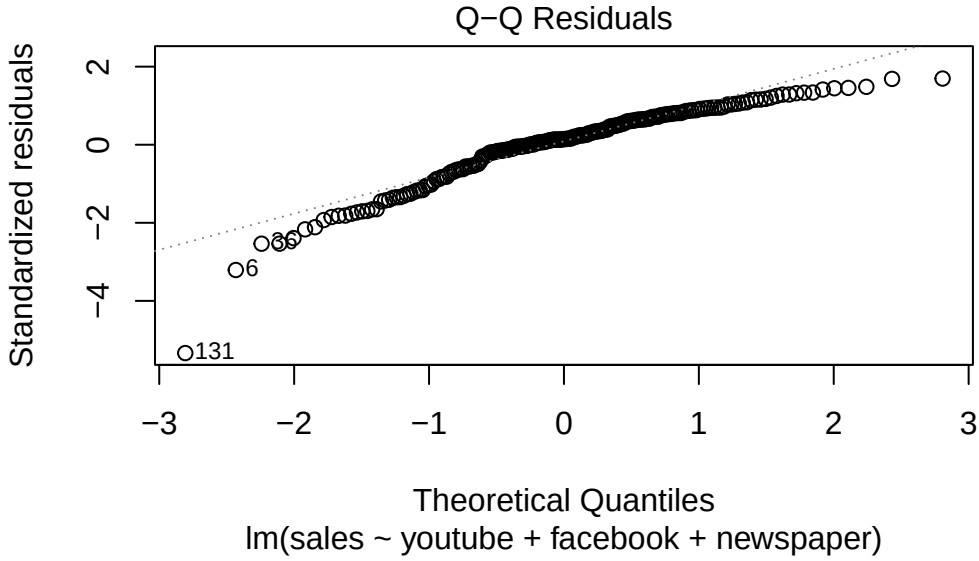


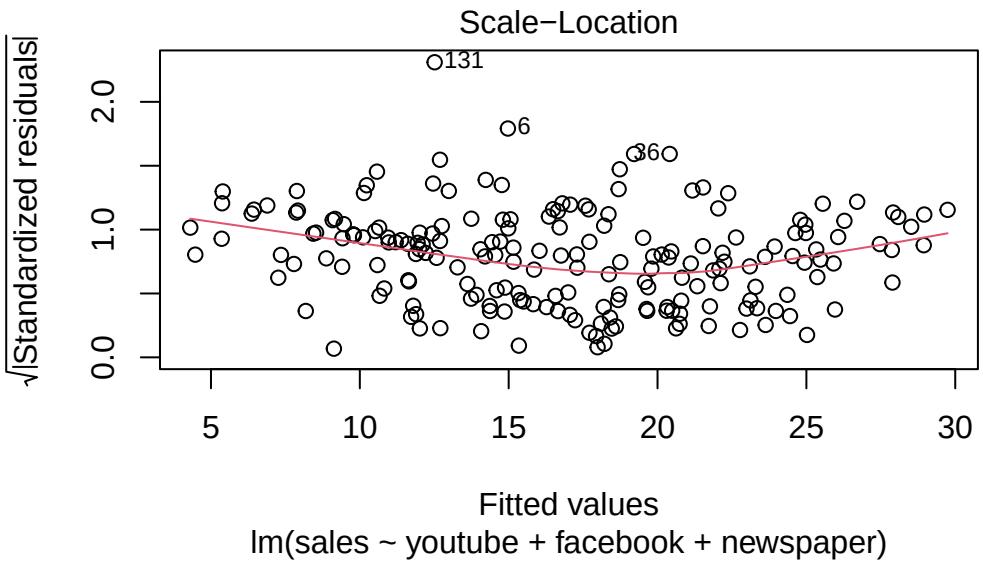
In this case, Youtube and Facebook spending seems to have a nonlinear relationship with sales. We will see how to remedy this in later chapters.

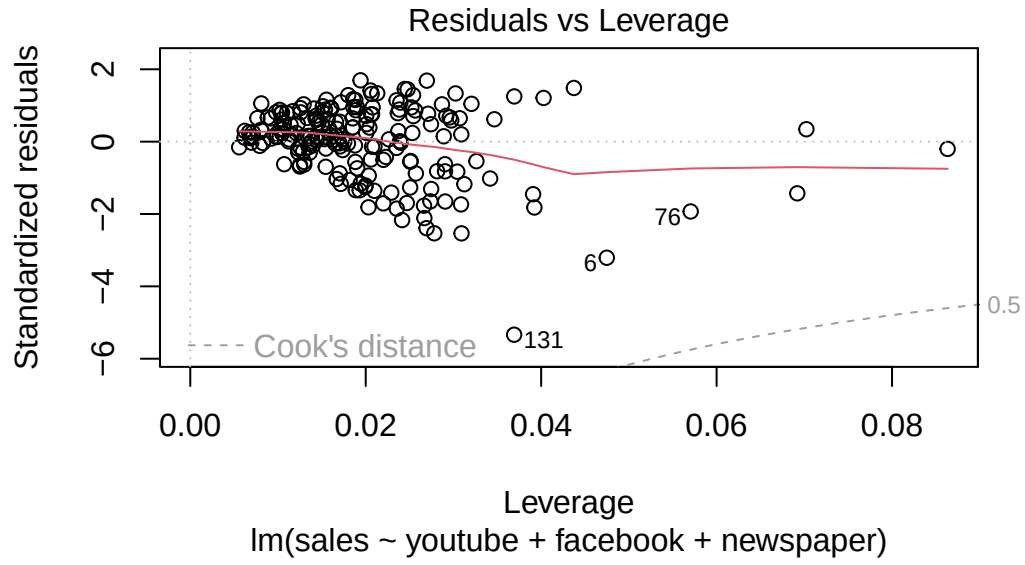
As a final note, observe that we can put the `model` object in the `plot()` function to obtain the diagnostic plots.

```
plot(full_model)
```


Residuals vs Fitted
Residuals
5
0
−5
−10
179
6
131
5
10
15
20
25
30
Fitted values
lm(sales ~ youtube + facebook + newspaper)


Q−Q Residuals
Standardized residuals
2
0
−2
−4
6
131
−3
−2
−1
0
1
2
3
Theoretical Quantiles
lm(sales ~ youtube + facebook + newspaper)





We will learn in later chapters how to check the assumptions more thoroughly and how to remedy violations of the assumptions.

3.5.3 Homework stop 5

Complete the assigned textbook problems for Chapter 4.

Exercise 3.19. List the assumptions for the normal MLR model and the MLR model. Write down how you would check each assumption.

3.6 Simple linear regression

A special case of the multiple linear regression is **simple linear regression**. A simple linear regression model is a regression model with **one explanatory variable**: $Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$.

$$y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, X = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix}, \epsilon = \begin{pmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{pmatrix}.$$

3.6.1 Estimated Coefficients

In this case, following some matrix manipulations (verify this for homework), we have

$$X^\top X = \begin{pmatrix} n & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{pmatrix}, \; X^\top y = \begin{pmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i y_i \end{pmatrix}.$$

Now, recall if

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

then

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

From MATH 1131 (or simple algebraic manipulation), we know

$$\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n x_i y_i - n\bar{x}\bar{y} = \sum_{i=1}^n x_i y_i - n^{-1} \left(\sum_{i=1}^n x_i \right) \left(\sum_{i=1}^n y_i \right)$$

$$\sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n x_i^2 - n\bar{x}^2 = \sum_{i=1}^n x_i^2 - n^{-1} \left(\sum_{i=1}^n x_i \right)^2.$$

Therefore

$$\begin{aligned}(X^\top X)^{-1} &= \frac{1}{n\sum_{i=1}^n x_i^2 - (\sum_{i=1}^n x_i)^2}\begin{pmatrix} \sum_{i=1}^n x_i^2 & -\sum_{i=1}^n x_i \\ -\sum_{i=1}^n x_i & n \end{pmatrix} \\ &= \frac{1}{n\sum_{i=1}^n (x_i-\bar{x})^2}\begin{pmatrix} \sum_{i=1}^n x_i^2 & -\sum_{i=1}^n x_i \\ -\sum_{i=1}^n x_i & n \end{pmatrix}.\end{aligned}$$

To summarize:

$$\begin{aligned}X^\top X &= \begin{bmatrix} n & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{bmatrix} \\ X^\top y &= \begin{bmatrix} \sum_{i=1}^n y_i \\ \sum_{i=1}^n x_i y_i \end{bmatrix} \\ (X^\top X)^{-1} &= \frac{1}{n\sum_{i=1}^n (x_i-\bar{x})^2}\begin{bmatrix} \sum_{i=1}^n x_i^2 & -\sum_{i=1}^n x_i \\ -\sum_{i=1}^n x_i & n \end{bmatrix}\end{aligned}$$

Now, we have that

$$\begin{aligned}\hat{\beta} &= \begin{pmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \end{pmatrix} = (X^\top X)^{-1}X^\top y \\ &= \frac{1}{n\sum_{i=1}^n (x_i-\bar{x})^2}\begin{pmatrix} \sum_{i=1}^n x_i^2 \sum_{i=1}^n y_i - \sum_{i=1}^n x_i \sum_{i=1}^n x_i y_i \\ -\sum_{i=1}^n x_i \sum_{i=1}^n y_i + n\sum_{i=1}^n x_i y_i \end{pmatrix}.\end{aligned}$$

Now,

$$\hat{\beta}_1 = \frac{1}{n\sum_{i=1}^n (x_i-\bar{x})^2}\left(-\sum_{i=1}^n x_i \sum_{i=1}^n y_i + n\sum_{i=1}^n x_i y_i\right).$$

Exercise 3.20. Let's show that

$$\hat{\beta}_1 = \frac{n\sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \sum_{i=1}^n y_i}{n\sum_{i=1}^n (x_i-\bar{x})^2} = \frac{\sum_{i=1}^n (x_i-\bar{x})(y_i-\bar{y})}{\sum_{i=1}^n (x_i-\bar{x})^2}.$$

Does this look familiar? We see that,

$$\hat{\beta}_1 = \hat{\text{cov}}(X,Y)\frac{\hat{\sigma}_y}{\hat{\sigma}_x},$$

where $\hat{\text{cov}}(X,Y)$ is the estimated correlation between X and Y. Let's interpret this:

1. If $\hat{\text{cov}}(X,Y) \approx 0$ then $\hat{\beta}_1 \approx 0$ - low correlation implies an estimated slope close to 0.
2. The estimated coefficient $\hat{\beta}_1$ is the product of the estimated correlation between X and Y and the ratio of the estimated standard deviation of Y to that of X.

Now, looking at the intercept term, we have

$$\hat{\beta}_0 = \frac{1}{n\sum_{i=1}^n (x_i - \bar{x})^2} \left(\sum_{i=1}^n x_i^2 \sum_{i=1}^n y_i - \sum_{i=1}^n x_i \sum_{i=1}^n x_i y_i \right).$$

Exercise 3.21. Show that

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

Observe that the intercept is the mean of Y minus the mean of X times the estimated slope. In essence, it tells us that the intercept ($\hat{\beta}_0$) represents the value of (X) when (X) is at its mean value (($\bar{X}$)) and that ($\bar{X}$) is adjusted by subtracting the contribution of ($\hat{\beta}_1 \bar{X}$).

This adjustment ensures that the regression line passes through the point (($\bar{X}, \bar{X}$)), which is the point of averages for the data.

3.6.2 Inference in SLR

We can also simplify the values used for inference in the SLR model. Recall that Var $\left[\hat{\beta}\right] = (X^\top X)^{-1}\sigma^2$, and so we have

$$\begin{aligned}
\text{Var}\left[\hat{\beta}_0\right] &= \frac{\sum_{i=1}^n x_i^2}{n\sum_{i=1}^n (x_i - \bar{x})^2}\sigma^2 = \frac{\sum_{i=1}^n x_i^2 - n\bar{x}^2 + n\bar{x}^2}{n\sum_{i=1}^n (x_i - \bar{x})^2}\sigma^2 \\
&= \left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2}\right]\sigma^2 \\
\text{Var}\left[\hat{\beta}_1\right] &= \frac{1}{\sum_{i=1}^n (x_i - \bar{x})^2}\sigma^2 \\
\text{cov}(\hat{\beta}_0, \hat{\beta}_1) &= -\frac{\bar{x}}{\sum_{i=1}^n (x_i - \bar{x})^2}\sigma^2.
\end{aligned}$$

We know from previous sections that a $(1-\alpha)100$ confidence interval of β_i, where $i = 0, 1$, is

$$\hat{\beta}_i \pm t_{df_E, \alpha/2}\sqrt{\widehat{var}(\hat{\beta}_i)}.$$

Similarly, let β_i^0 be a hypothesized value of β_i, for $i = 0, 1$. If we want to test whether $\beta_i = \beta_i^0$, then the observed test statistic is given by

$$\frac{\hat{\beta}_i - \beta_i^0}{\sqrt{\widehat{var}(\hat{\beta}_i)}},$$

and the corresponding p-value is obtained via the t_{dfE} distribution as usual.

> **Note**
>
> Similarly, inference for the mean response and predictions can be obtained. We can also simplify the $ANOVA$ table, R^2, etc. For instance, the R^2 is the square of the sample correlation coefficient between X and Y.

3.6.3 Inference for the correlation coefficient

If we are interested in doing a hypothesis test, or constructing confidence intervals for the correlation between two variables, say X and Y, we can use the simple linear regression model.

We have already derived the relationship between the estimated correlation coefficient and the estimated slope of the simple linear regression model. More specifically, if the estimated correlation coefficient is 0, then the estimated slope of the simple linear regression is 0. One can show that the same relationship holds at the population level: $\beta_1 = \rho\sigma_y/\sigma_x$, where $\rho = \text{corr}[X, Y]$.

Now, suppose that we want to test if $H_0 : \rho = 0$ versus $H_a : \rho \neq 0$. Using the fact that $\beta_1 = \rho\sigma_y/\sigma_x$, the above test is equivalent to the statement $H_0 : \beta_1 = 0$ versus $H_a : \beta_1 \neq 0$. Therefore, we can just test if the slope parameter in the model $Y|X = \beta_0 + \beta_1 X + \epsilon$ is 0.

Letting $\hat{\rho} = \hat{\text{corr}}(X, Y)$ The observed test statistic is then:

$$\frac{\hat{\beta}_1}{\sqrt{\widehat{var}(\hat{\beta}_1)}} = \frac{\hat{\rho}\sqrt{n-2}}{\sqrt{1-\hat{\rho}^2}},$$

and the corresponding p-value is obtained based on the t_{dfE} distribution.

However, when the hypothesized value for ρ is non-zero, the problem becomes very complicated. The exact distribution of $\hat{\rho}$ is extremely difficult to obtain under the null hypothesis. The following procedure gives an approximation of the distribution of a function of $\hat{\rho}$ under the null hypothesis. In particular, Fisher suggested the transformation for $\rho \in (0, 1)$,

$$\theta = \frac{1}{2}\log\frac{1+\rho}{1-\rho}.$$

Then

$$\hat{\theta} = \frac{1}{2}\log\frac{1+\hat{\rho}}{1-\hat{\rho}},$$

is an estimate of θ, where $\hat{\theta}$ is approximately distributed as normal with mean θ and variance $\frac{1}{n-3}$. Hence, an approximate $(1-\alpha)100$ confidence interval of θ is

$$\hat{\theta} \pm z_{\alpha/2}\sqrt{\frac{1}{n-3}},$$

and the corresponding confidence interval of ρ can be obtained by the inverse transformation. Similarly, if the hypothesized value of ρ is ρ_0, then the hypothesized value of θ is $\theta_0 = \frac{1}{2}\log\frac{1+\rho_0}{1-\rho_0}$. The observed test statistic can be obtained and the corresponding p-value can be obtained based on the standard normal distribution.

Example 3.11. In Example 3.1 test if the correlation between body fat and weight is 0. Next, test if the correlation is greater than 1/2. Construct a 95% CI for ρ.

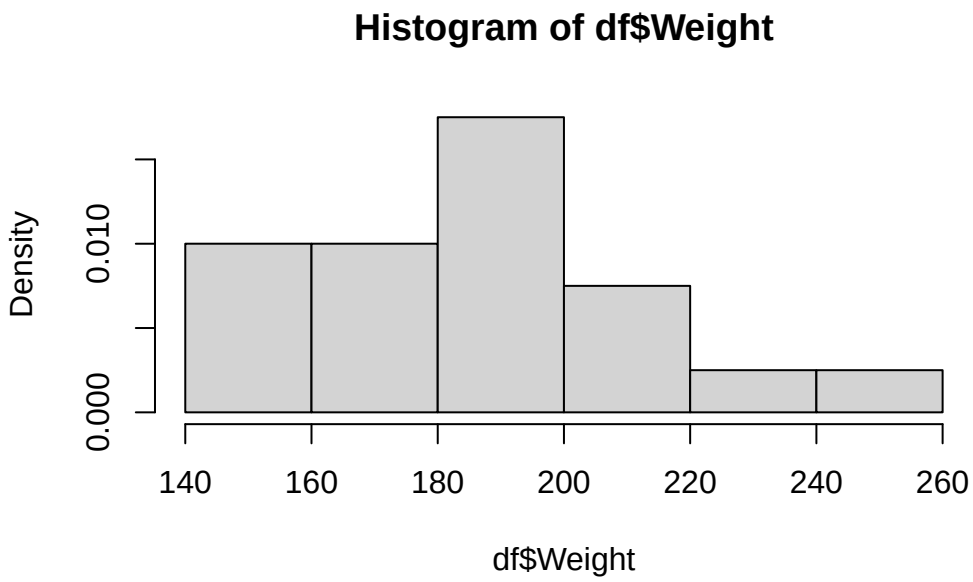
```
############################ Exploratory
Weight=c(175 , 181 , 200 , 159 , 196 , 192 , 205 ,
         173 , 187 , 188 , 188 , 240 , 175 , 168 ,
         246 , 160 , 215 , 159 , 146 , 219 )
BodyFat =c(6 , 21 , 15 , 6 , 22 , 31 , 32 ,21 , 25 ,
           30 , 10 , 20 , 22 , 9 , 38 , 10 , 27 , 12 , 10 , 28 )

df=data.frame(cbind(Weight=Weight,BodyFat=BodyFat))

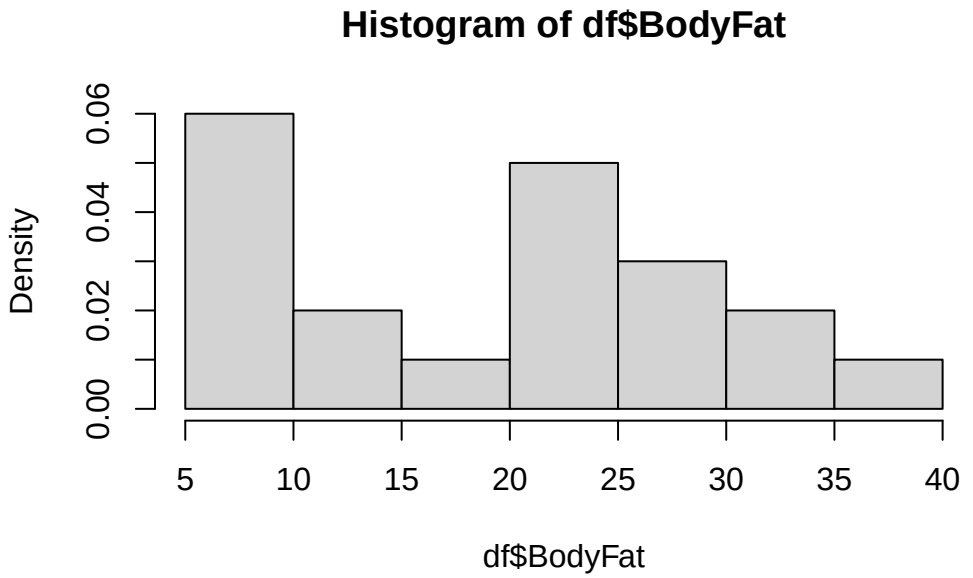

cor(df)
```

```
           Weight   BodyFat
Weight  1.0000000 0.6966328
BodyFat 0.6966328 1.0000000
```

```
hist(df$Weight,freq=F)
```




```
hist(df$BodyFat,freq=F)
```



```r
summary(df)
```

```
     Weight          BodyFat
 Min.   :146.0   Min.   : 6.00
 1st Qu.:171.8   1st Qu.:10.00
 Median :187.5   Median :21.00
 Mean   :188.6   Mean   :19.75
 3rd Qu.:201.2   3rd Qu.:27.25
 Max.   :246.0   Max.   :38.00
```

```r
cor(df)[1,2]
```

```
[1] 0.6966328
```

```r
X=cbind(rep(1,nrow(df)), df$Weight)
Y=df$BodyFat

beta_hat= solve(t(X)%*%X)%*%t(X)%*%Y
beta_hat
```

```
             [,1]
[1,] -27.3762623
[2,]   0.2498741
```

```r
model=lm(BodyFat~ Weight,df)
model
```

```

Call:
lm(formula = BodyFat ~ Weight, data = df)

Coefficients:
(Intercept)       Weight
   -27.3763       0.2499
```

```r
summary(model)
```

```
Call:
lm(formula = BodyFat ~ Weight, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-12.5935  -5.7904   0.6536   5.2731  10.4004

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -27.37626   11.54743  -2.371 0.029119 *
Weight        0.24987    0.06065   4.120 0.000643 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.049 on 18 degrees of freedom
Multiple R-squared:  0.4853,    Adjusted R-squared:  0.4567
F-statistic: 16.97 on 1 and 18 DF,  p-value: 0.0006434
```

```r
cor(df)[1,2]^2
```

```
[1] 0.4852972
```

```r
cor(df)[1,2]
```

```
[1] 0.6966328
```

```r
a=function(x){
  (exp(2*x)-1)/(exp(2*x)+1)
}
a(1.36)
```

```
[1] 0.8763931
```

3.6.4 Homework stop 6

- Complete the Chapter 2 questions in the textbook.

Exercise 3.22. For

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

$$y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, X = \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \beta = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix}, \epsilon = \begin{pmatrix} \epsilon_1 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

- Compute $\hat{\beta}$, Var $\left[\hat{\beta}_1\right]$, Var $\left[\hat{\beta}_0\right]$, cov $[(]\, \hat{\beta}_0, \hat{\beta}_1)$
- Show $\hat{\beta}_1 = r\frac{\hat{\sigma}_y}{\hat{\sigma}_x}$

3.7 Additional concepts & examples

Here we touch on a few important examples and notes about the MLR.

3.7.1 Beware scatter plots in MLR

Sometimes, scatter plots are misleading for determining the relationship between Y and a collection of p covariates. In the following example, it appears that $X1$ and Y do not have a relationship, when in fact they do. Generally, this phenomena goes away with higher sample sizes.

```
# Scatter diagram beware?
# x1=c(2,3,4,1,5,6,7,8)
# x2=c(2,3,4,1,5,6,7,8)
# x=c(2,3,4,1,5,6,7,8)

# x1=1:8
# x2=c(2,1:6,4)
# y=8-5*x1+12*x2+rnorm(8,0,2)

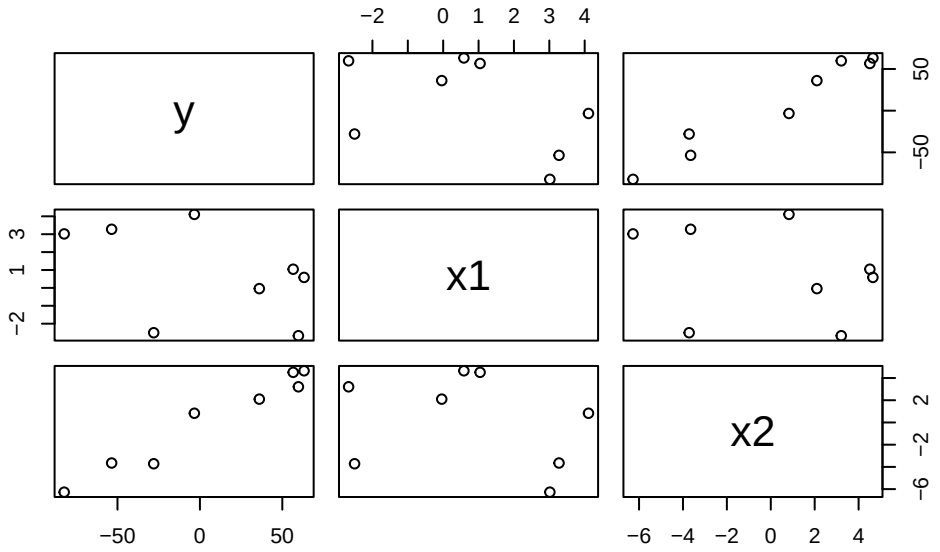
set.seed(445)



n=8
x1=rnorm(n,5,5)
x2=rnorm(n,3,5)
y=8-5*x1+12*x2+rnorm(n,0,2)

df=data.frame(cbind(y,x1,x2))
```

```
plot(df)
```



Next, we do an example from the textbook, which uses the NFL data. Specifically, we try to evaluate the relationship between number of wins and several explanatory variables.

Example 3.12. Using the following NFL data, complete 3.1-3.4, 4.1 and 4.2 in the textbook.

```
################## NFL example #################
# This gives you the data sets used in the textbook
# install.packages('MPV')
df=MPV::table.b1
# Note for more information, run ?MPV::table.b1

head(df)
```

```
   y   x1   x2   x3   x4 x5  x6   x7   x8   x9
1 10 2113 1985 38.9 64.7  4 868 59.7 2205 1917
2 11 2003 2855 38.8 61.3  3 615 55.0 2096 1575
3 11 2957 1737 40.1 60.0 14 914 65.6 1847 2175
4 13 2285 2905 41.6 45.3 -4 957 61.4 1903 2476
5 10 2971 1666 39.2 53.8 15 836 66.1 1457 1866
6 11 2309 2927 39.7 74.1  8 786 61.0 1848 2339
```

```r
# names too long
names(df)
```

```
 [1] "y"  "x1" "x2" "x3" "x4" "x5" "x6" "x7" "x8" "x9"
```

```r
# rename to make it easier
names(df)=c("Wins","RushY","PassY","PuntA","FGP","TurnD","PenY","PerR","ORY","OPY")
names(df)
```

```
 [1] "Wins"  "RushY" "PassY" "PuntA" "FGP"   "TurnD" "PenY"  "PerR"  "ORY"
[10] "OPY"
```

```r
# Wins~ beta_1+beta_2Passing yrds+beta_3per_rush+beta_4ORY+epsilon
# summary(df)
# plot(df)
# run the model
regression_model=lm(  Wins ~ PassY+PerR+ORY  ,data= df )
summary(regression_model)
```

```
Call:
lm(formula = Wins ~ PassY + PerR + ORY, data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-3.0370 -0.7129 -0.2043  1.1101  3.7049

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.808372   7.900859  -0.229 0.820899
PassY        0.003598   0.000695   5.177 2.66e-05 ***
PerR         0.193960   0.088233   2.198 0.037815 *
ORY         -0.004816   0.001277  -3.771 0.000938 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.706 on 24 degrees of freedom
Multiple R-squared:  0.7863,    Adjusted R-squared:  0.7596
F-statistic: 29.44 on 3 and 24 DF,  p-value: 3.273e-08
```

```
# model=lm(Wins~PassY+PerR+ORY,data=df)
# get the confidence intervals.
confint(regression_model)
```

```
                    2.5 %       97.5 %
(Intercept) -18.114944410 14.498200293
PassY         0.002163664  0.005032477
PerR          0.011855322  0.376065098
ORY          -0.007451027 -0.002179961
```

What conclusions can you make from this output? - All variables seem important! For isntance, we sdee that for every 1% increase in percentage rushing, there is a 0.193960 increase in number of wins, on average, holding passing yards and opponent rushing yards constant.

```
#### CI
# mean response of z'\beta , z=(2000,60,1900)'
new_data=data.frame(  matrix(c(2000,60,1900),ncol=3) )
names(new_data)
```

```
[1] "X1" "X2" "X3"
```

```
names(new_data)=c( 'PassY','PerR','ORY' )

predict(regression_model, new_data  ,   interval = 'confidence')
```

```
       fit      lwr      upr
1 7.875942 7.072672 8.679213
```

```
predict(regression_model, new_data  ,   interval = 'predict')
```

```
       fit      lwr     upr
1 7.875942 4.263986 11.4879
```

```
## ANOVA

regression_model_reduced=lm(  Wins ~ 1  ,data= df )
summary(regression_model_reduced)
```

```
Call:
lm(formula = Wins ~ 1, data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-6.9643 -2.9643 -0.4643  3.0357  6.0357

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   6.9643     0.6576   10.59 4.09e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.48 on 27 degrees of freedom
```

```
anova(regression_model_reduced,regression_model)
```

```
Analysis of Variance Table

Model 1: Wins ~ 1
Model 2: Wins ~ PassY + PerR + ORY
  Res.Df    RSS Df Sum of Sq      F    Pr(>F)
1     27 326.96
2     24  69.87  3    257.09 29.437 3.273e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# subset test

regression_model_reduced=lm(  Wins ~ PassY ,data= df )
anova(regression_model_reduced,regression_model)
```

```
Analysis of Variance Table

Model 1: Wins ~ PassY
Model 2: Wins ~ PassY + PerR + ORY
  Res.Df    RSS Df Sum of Sq      F    Pr(>F)
1     26 250.77
2     24  69.87  2     180.9 31.069 2.189e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# summary(df)

summary(regression_model)
```

```
Call:
lm(formula = Wins ~ PassY + PerR + ORY, data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-3.0370 -0.7129 -0.2043  1.1101  3.7049

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.808372   7.900859  -0.229 0.820899
PassY        0.003598   0.000695   5.177 2.66e-05 ***
PerR         0.193960   0.088233   2.198 0.037815 *
ORY         -0.004816   0.001277  -3.771 0.000938 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.706 on 24 degrees of freedom
Multiple R-squared:  0.7863,    Adjusted R-squared:  0.7596
F-statistic: 29.44 on 3 and 24 DF,  p-value: 3.273e-08
```

```r
# anova(regression_model)
summ=summary(regression_model)
summ$r.squared
```

```
[1] 0.7863069
```

```r
summ$adj.r.squared
```

```
[1] 0.7595953
```

```r
regression_model2=lm(Wins~PassY+ORY,data=df)


SSER=sum(regression_model2$residuals*regression_model2$residuals); SSER
```

```
[1] 83.9382
```

```
dfer=regression_model2$df.residual; dfer
```

```
[1] 25
```

```
SSEC=sum(regression_model$residuals*regression_model$residuals); SSEC
```

```
[1] 69.87
```

```
dfeC=regression_model$df.residual; dfeC
```

```
[1] 24
```

```
SSdrop=SSER-SSEC; SSdrop
```

```
[1] 14.06819
```

```
dfdrop=dfer-dfeC

MSdrop=SSdrop/dfdrop; MSdrop
```

```
[1] 14.06819
```

```
R_prp=SSdrop/SSER; R_prp
```

```
[1] 0.1676018
```

```
MSdrop
```

```
[1] 14.06819
```

```
1-pf(MSdrop,dfdrop,dfeC)
```

```
[1] 0.000986662
```

```r
cor(regression_model$fitted.values , df$Wins)^2
```

```
[1] 0.7863069
```

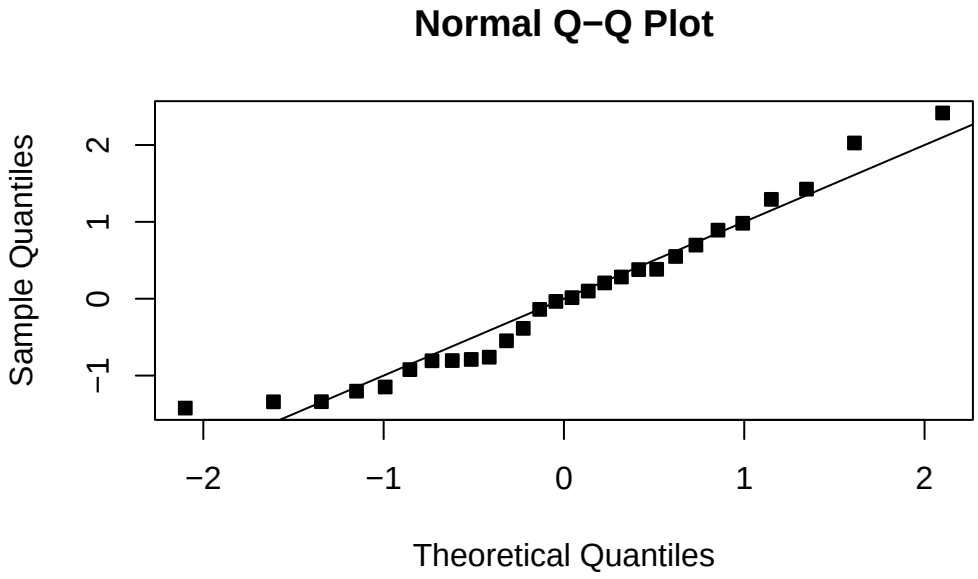
```r
confint(regression_model2)
```

```
                  2.5 %       97.5 %
(Intercept)  9.321778092 20.103571885
PassY        0.001654121  0.004568143
ORY         -0.008797465 -0.004819085
```

```r
new_data=df[1,c(3,8,9)]
new_data[1,]=c(2300 , 56 , 2100)
predict(regression_model2,new_data,interval = 'confidence')
```

```
     fit      lwr      upr
1 7.5709 6.814662 8.327138
```

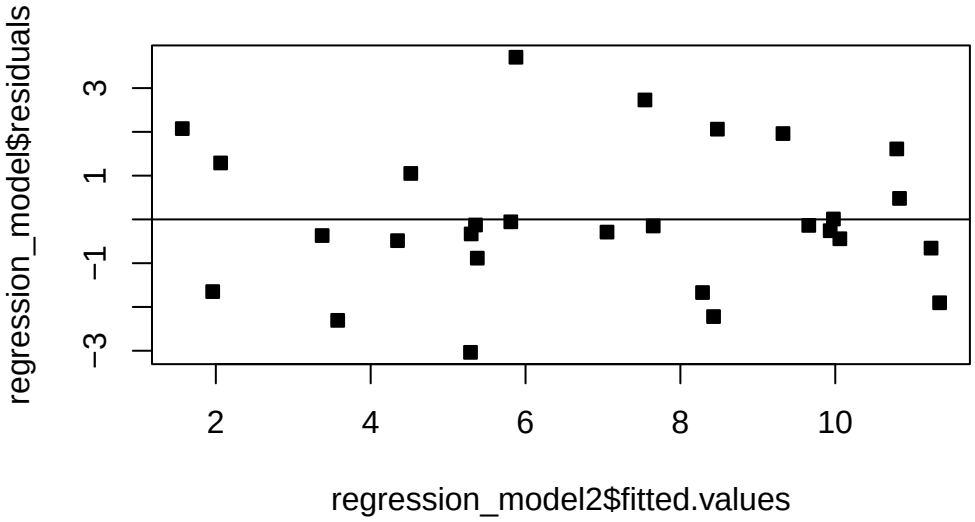
```r
#########  check the fit #######
MSE=summ$sigma^2
qqnorm(regression_model2$residuals/summ$sigma,pch=22,bg=1)
abline(0,1)
```



```
hist(regression_model2$residuals,breaks=6)
```




```
plot(regression_model2$fitted.values,regression_model$residuals,pch=22,bg=1)
abline(h=0)
```




```
n=nrow(df)
plot(1:n,regression_model2$residuals,pch=22,bg=1)
abline(h=0)

time=(1:n)
res=lm(regression_model2$residuals~time)
summary(res)
```

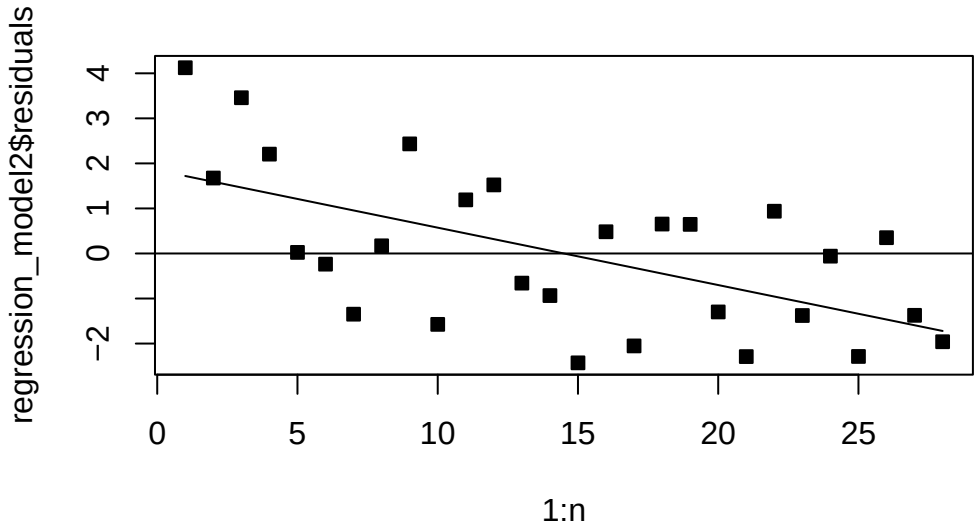
```
Call:
lm(formula = regression_model2$residuals ~ time)

Residuals:
     Min       1Q   Median       3Q      Max
-2.36425 -1.04520 -0.07845  1.16457  2.40353

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   1.8479     0.5610   3.294 0.002852 **
time         -0.1274     0.0338  -3.771 0.000848 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 1.445 on 26 degrees of freedom
Multiple R-squared:  0.3535,    Adjusted R-squared:  0.3286
F-statistic: 14.22 on 1 and 26 DF,  p-value: 0.0008481
```

```
lines(time,res$fitted.values)
```




```
regression_model3=lm(Wins~PerR+ORY,data=df)
summ3=summary(regression_model3)
summ3
```

```
Call:
lm(formula = Wins ~ PerR + ORY, data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-3.7985 -1.5166 -0.5792  1.9927  4.5248

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 17.944319   9.862484   1.819  0.08084 .
```

```
PerR         0.048371   0.119219   0.406  0.68839
ORY         -0.006537   0.001758  -3.719  0.00102 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.432 on 25 degrees of freedom
Multiple R-squared:  0.5477,    Adjusted R-squared:  0.5115
F-statistic: 15.13 on 2 and 25 DF,  p-value: 4.935e-05
```

```
summ3$r.squared
```

```
[1] 0.5476628
```

```
summ3$adj.r.squared
```

```
[1] 0.5114759
```

```
confint(regression_model)
```

```
                   2.5 %       97.5 %
(Intercept) -18.114944410 14.498200293
PassY         0.002163664  0.005032477
PerR          0.011855322  0.376065098
ORY          -0.007451027 -0.002179961
```

```
confint(regression_model3)
```

```
                  2.5 %       97.5 %
(Intercept) -2.36784828 38.256485319
PerR        -0.19716429  0.293906022
ORY         -0.01015637 -0.002916818
```

```
new_data3=df[1,c(8,9)]
new_data3[1,]=c(56 , 2100)
predict(regression_model3,new_data3,interval = 'confidence')
```

```
       fit      lwr      upr
1 6.926243 5.828643 8.023842
```

```
predict(regression_model2,new_data,interval = 'confidence')
```

```
     fit      lwr      upr
1 7.5709 6.814662 8.327138
```

Be careful about extrapolating beyond the region containing the original observations. It is very possible that a model that fits well in the region of the original data will perform poorly outside that region. It is easy to inadvertently extrapolate, since the levels of the regressors jointly define a region containing the data which is impossible to visualize in its entirety beyond 2 dimensions. Ideally, we want to make inferences which lie inside the convex hull of the regressors.

We can use the diagonal of the hat matrix $H = X(X^\top X)^{-1}X^\top$. In general, the point that has the largest value of h_{ii}, say h_{max}, will lie on the boundary of the convex hull in a region of the x-space where the density of the observations is relatively low. Points that lie in the set $\{x^\top(X^\top X)^{-1}x \leq h_{max}\}$ enclose the convex hull. Thus, for a value we are interested in predicting, say y, we can check if we are extrapolating with $y^\top(X^\top X)^{-1}y \leq h_{max}$.

A serious problem that may dramatically impact the usefulness of a regression model is multicollinearity, or near - linear dependence among the regression variables. Multicollinearity implies near - linear dependence among the regressors. The regressors are the columns of the X matrix, so clearly an exact linear dependence would result in a singular $X^\top X$. This will impact our ability to estimate β.

We can check for this dependence with the **variance inflation factor** (VIF). The variance inflation factor can be written as $(1 - R_j^2)^{-1}$, where R_j^2 is the coefficient of determination obtained from regressing X_j on the other regressor variables. If VIF is large, say > 3, then you will likely need to make some changes to your regression model.

Sometimes, you may observe that regression coefficients have the a sign that is unexpected, or contradicts nature. This is likely due to one of the following:

- The range of some of the regressors is too small – if the range of some of the regressors is too small, then the variance of $\widehat{\beta}$ is high.
- Important regressors have not been included in the model.
- Multicollinearity is present.
- Computational errors have been made.

We close this Chapter with the following statement. Recall the modelling overview from Chapter 1:

- Posit the model: What is the linear regression model – what are all the assumptions of the linear regression model?
- Estimation: How can we estimate parameters of the linear regression model?

- Inference: How can we compute confidence intervals and run hypothesis tests associated with the linear regression model?
- Fit: Does our fitted line match up with the data? What about the normality assumption? Do the errors appear normal? Do the errors seem independent? Is the variance constant? How much variability is explained by our model?
- Prediction: How can we predict a new Y? What is the error of this prediction

If you have learned the concepts of this chapter, you should be able to complete all of these steps! In the following chapters, we will discuss different problems that can arise in regression modelling and how to remedy them.

3.7.2 Homework questions

Exercise 3.23. Show Var $\left[\widehat{Y}|X\right] = \sigma^2 H$.

Exercise 3.24. Check for multicolinearity in our past examples.

Exercise 3.25. Complete the problem sets from Chapter's 2, 3 and 4!

4 Residual analysis

Recall that we want to study the normal MLR:

$$Y|X = X\beta + \epsilon,$$

where

- $\forall i \in [n]$, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ and $\epsilon_i \perp \epsilon_j$ for $i \neq j,\ i, j \in [n]$.
- $\beta \in \mathbb{R}^{p\times 1}$ is the unknown, population coefficient vector
- $X \in \mathbb{R}^{n\times p}$ is a covariate matrix

We assume that:

- The relationship is linear $Y|X = X\beta + \epsilon$,
- $\forall i \in [n]$, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$.
- $\epsilon_i \perp \epsilon_j$ for $i \neq j,\ i, j \in [n]$.

We have seen some methods for checking if these are appropriate, we will dive deeper now.

Recall that the residuals are defined as:

$$\hat{\epsilon} = Y - \hat{Y} = Y - X\hat{\beta}.$$

Given that a residual may be viewed as the deviation between the data and the fit, it is also a measure of the variability in the response variable not explained by the regression model. It is also convenient to think of the residuals as the realized or observed values of the model errors. Thus, it's reasonable to conclude that departures from the assumptions on the errors should show up in the residuals. Analysis of the residuals is an effective way to discover several types of model inadequacies.

4.1 Properties of residuals

The following are some properties of the residual vector. First, the sample mean of the residuals is zero: $\sum_{i=1}^{n} \hat{\epsilon}_i/n = \hat{\epsilon}1_n \cdot 1/n = 0$. We also have that $\mathrm{E}[\hat{\epsilon}] = 0$. Next, the sample variance of the residual vector is approximately the MSE: $\frac{1}{n-1}\sum_{i=1}^{n} \hat{\epsilon}_i^2 = \frac{n-p}{n-1}MSE$. Lastly, unlike the random error ϵ_i, the residuals **are not** independent. Sometimes we say that they are "approximately independent" if $p << n$, which we will touch on later.

4.2 Types of residuals

We will refer to $\hat{\epsilon}_i$ as simply the residuals, or ordinary residuals when we need to be extra clear.

The standardized residual is given by

$$d_i = \hat{\epsilon}_i / \sqrt{MSE}.$$

This is an approximate Z-score for the residuals, since the residuals have 0 mean, the MSE is approximately the variance of the random error and the residuals approximate the random error. We will generally prefer to use a different type of residual, which we now present.

We now introduce the hat matrix: $H = X(X^\top X)^{-1}X^\top$. Note that H is symmetric and idempotent. The hat matrix appears often in regression analysis, and you should remember this quantity. It is called the hat matrix because $\hat{Y} = HY$.

Note that the eigenvalues of H, and any idempotent matrix A are either 0 or 1:

$$\lambda x = Ax = A^2 x = A\lambda x = \lambda^2 x,$$

which implies that $\lambda \in \{0, 1\}$.

Exercise 4.1. Verify that H is symmetric and idempotent and that $\hat{Y} = HY$, where one recalls that a matrix A is idempotent if $AA = A$.

Now, note that:

$$\hat{\epsilon} = (I - H)Y = (I - H)\epsilon.$$

Exercise 4.2. Verify that $\hat{\epsilon} = (I - H)Y = (I - H)\epsilon$..

Using this identity, we have that $\text{Var}\,[\hat{\epsilon}] = (I - H)\epsilon(I - H)^\top = (I - H)\sigma^2$.

The fact that H is symmetric and idempotent implies that its diagonal elements are between 0 and 1. It follows that the elements on the diagonal of $(I - H)$ are also between 0 and 1. Therefore, the MSE overestimates the variance of the residuals: the variance of residual i is given by $(1 - h_{ii})\sigma^2 < \sigma^2 \approx MSE$. Here, h_{ii} denotes the ith diagonal element of the matrix H.

It can be shown that h_{ii} is a measure of how outlying x_i is, relative to the other observed covariate vectors. Therefore, $\text{Var}\,[\hat{\epsilon}_i]$ depends on how outlying x_i is. Covariate vectors that are central, relative to the other observed covariate vectors, have larger variance than residuals at more remote locations. What do you think about this?

Now, intuitively, some remote points in our data may differ from the population and severely impact our model. However, the variance of the ordinary residuals is lower at these points.

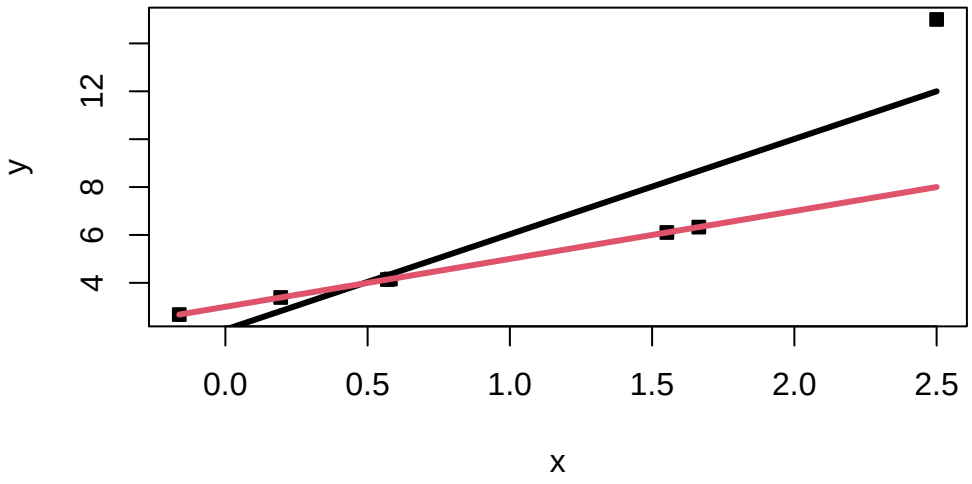
Therefore, these impacts may be hard to detect from solely the ordinary residuals because the ordinary residuals of remote points will usually be smaller.

Therefore, we will call points that are outlying in the x-space **leverage points**. We will refer to **influence points** are not only remote in terms of the specific values for the regressors, but the observed response is not consistent with the values that would be predicted based on only the other data points.

In the example below, observe that the right-most point is both a leverage and influential point.

```
##########
# Simulate some data
set.seed(330)
x=c(rnorm(6),2.5)
y=x*2+3
y[7]=y[7]+7

# Plot the data and fitted lines
plot(x,y,pch=22,bg=1)
a=lm(y~x)
curve(a$coefficients[1]+x*a$coefficients[2],add=T,lwd=3)
curve(x*2+3,add=T,col=2,lwd=3)
```



```
a$coefficients
```

```
(Intercept)           x
   2.048937    3.979977
```

We now demonstrate mathematically why leverage points can be dangerous. Let $\hat{y}_n^*$ be the estimate of Y_n based on the other data and let $\delta_n = y_n - \hat{y}_n^*$. Note that one can show that $\hat{y}_n = \hat{Y}_n^* + h_{nn}\delta_n$. Next, we know that if x_n is outlying, i.e., $||x_n||$ is large, then $h_{nn} \approx 1$. This implies that $\hat{y}_n \approx y_n$, which means that the regression line is dragged to pass through (x_n, y_n).

To detect these types of outlying points, it makes sense to then define the **studentized residuals**:

$$r_i = \frac{\hat{\epsilon}_i}{\sqrt{MSE(1-h_{ii})}}.$$

The studentized residuals in the simple linear regression model reduce to

$$r_i = \frac{\hat{\epsilon}_i}{\sqrt{MSE}\left[1 - \left(\frac{1}{n} + \frac{(X_i - \bar{X})^2}{\sum(X-\bar{X})^2}\right)\right]}.$$

Observe that as X_i grows large, we have that $\frac{(X_i-\bar{X})^2}{\sum(X-\bar{X})^2} \to 1$, which implies that $\left[1 - \left(\frac{1}{n} + \frac{(X_i-\bar{X})^2}{\sum(X-\bar{X})^2}\right)\right] \to 0$ and $r_i \to \infty$. On the other hand, as $\hat{\epsilon}_i$ grows large, we have that r_i grows large. Therefore, the studentized residual will be large for observations with large ordinary residuals, and for leverage observations.

Since the studentized residuals are on the standard normal scale, $|r_i| > 2$ for small samples $n < 100$ or $|r_i| > 3$ for larger samples may signal an outlier.

Earlier, we presented δ_i, the difference between the response of the ith observation and the predicted response based on the observations with the ith points removed. These are known as the **PRESS residuals**. This seems hard computationally, but one can show that

$$\delta_i = \frac{\hat{\epsilon}_i}{1-h_{ii}}.$$

Note that when h_{ii} is large, this indicates a highly influential point. Observe that a large PRESS residual δ_i, but small ordinary residual $\hat{\epsilon}_i$, indicates that the model fit without (X_i, Y_i) predicts Y_i poorly.

Exercise 4.3. Show that standardizing the PRESS residual, that is, dividing the PRESS residual by its standard deviation, results in $\hat{\epsilon}_i/\sqrt{\sigma^2(1-h_{ii})}$. Compare this to the studentized residual.

Lastly, if we believe that (X_i, Y_i) is outlying, then we can also leave (X_i, Y_i) out in the MSE calculation. This results in the **R-studentized residuals**:

$$\tilde{r}_i = \frac{\hat{\epsilon}_i}{\sqrt{\widetilde{MSE}_i(1-h_{ii})}},$$

where $\widetilde{MSE}_i$ is the mean squared error computed from the regression model with (X_i, Y_i) excluded:

$$\widetilde{MSE}_i = \frac{(n-p+1)MSE - \hat{\epsilon}_i^2/(1-h_{ii})}{n-p}.$$

4.3 Revisiting checking model assumptions

Recall from Checking model assumptions that we plot the residuals to check various assumptions. In this case, we can now use our upgraded residuals to make these plots. In general, any of the residuals that incorporate the values h_{ii} are acceptable. We will generally use the studentized residuals.

Recall that we may want to plot:

- QQplot of the studentized residuals
- Histogram of the studentized residuals
- Plot of studentized residuals against the fitted values
- Studentized residuals against the covariates
- Studentized residuals against covariates that are not currently in the model
- Studentized residuals against time in some contexts

4.3.1 Example

Example 4.1. Here, this data contains delivery times, the number of products in the delivery and the distance of the delivery. Perform a residual analysis on the model which regresses delivery times against the number of products in the delivery and the distance of the delivery. Compute all the different types of residuals.

```
########## Delivery Time


# Load and inspect the data
data(delivery, package="robustbase")
df=delivery
n=nrow(df)
head(df)
```

```
  n.prod distance delTime
1      7      560   16.68
2      3      220   11.50
3      3      340   12.03
4      4       80   14.88
5      6      150   13.75
6      7      330   18.11
```

```r
# Fit the model
model=lm(delTime~.,data=df)
s=summary(model); s
```

```
Call:
lm(formula = delTime ~ ., data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-5.7880 -0.6629  0.4364  1.1566  7.4197

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 2.341231   1.096730   2.135 0.044170 *
n.prod      1.615907   0.170735   9.464 3.25e-09 ***
distance    0.014385   0.003613   3.981 0.000631 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.259 on 22 degrees of freedom
Multiple R-squared:  0.9596,    Adjusted R-squared:  0.9559
F-statistic: 261.2 on 2 and 22 DF,  p-value: 4.687e-16
```

```r
# X matrix
X=model.matrix(model)

# Hat matrix
hat=X%*%solve(t(X)%*%X)%*%t(X)

# Compute h_ii
hii=diag(hat)
hii
```

```
         1          2          3          4          5          6          7
0.10180178 0.07070164 0.09873476 0.08537479 0.07501050 0.04286693 0.08179867
         8          9         10         11         12         13         14
0.06372559 0.49829216 0.19629595 0.08613260 0.11365570 0.06112463 0.07824332
        15         16         17         18         19         20         21
0.04111077 0.16594043 0.05943202 0.09626046 0.09644857 0.10168486 0.16527689
        22         23         24         25
0.39157522 0.04126005 0.12060826 0.06664345
```

```r
max(hii)
```

```
[1] 0.4982922
```

```r
# Notice 9 is large

###### ordinary residuals
regular_residuals=model$residuals
# or


# standardized residuals
stand_res=model$residuals/s$sigma

# studentized residuals
student_res=rstudent(model)


#PRESS residuals
press=model$residuals/(1-hii)

# Get the MSE_is
MSE_i=((n-2)*(s$sigma)^2-regular_residuals^2/(1-hii))/(n-3)

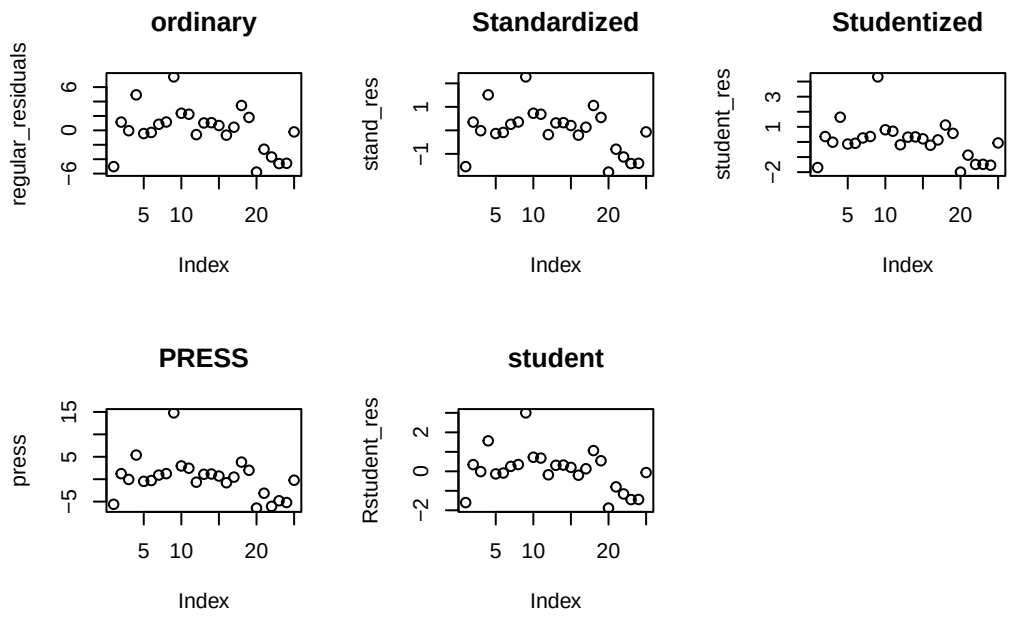
#r studentized residuals
Rstudent_res=model$residuals/sqrt(MSE_i)

# Plot them all and compare
par(mfrow=c(2,3))
plot(regular_residuals,main="ordinary")
plot(stand_res,main="Standardized")
plot(student_res,main="Studentized")
plot(press,main="PRESS")
```

```
plot(Rstudent_res,main="student")

# Notice 9 is much more outlying in the last 3 graphs.

# Reset plotting
par(mfrow=c(1,1))
```




```
# 9 is largest
which.max(student_res)
```

```
9
9
```

```
# Notice the standardized is half as large as the studentized.
student_res[9]
```

```
      9
4.31078
```

```
stand_res[9]
```

```
       9
2.276351
```

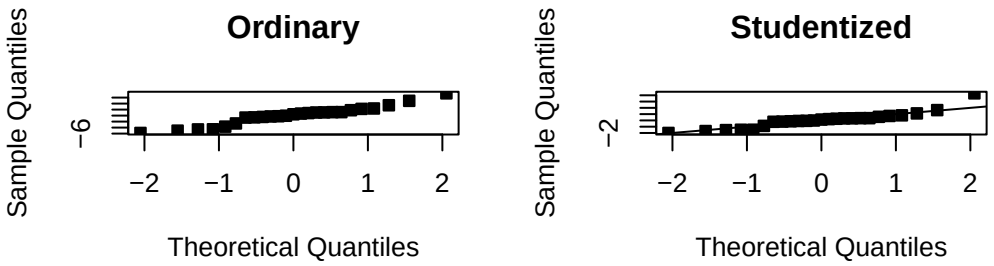
```
par(mfrow=c(2,2))

# Notice the difference !!!
qqnorm(regular_residuals,pch=22,bg=1,main="Ordinary")

qqnorm(student_res,pch=22,bg=1,main="Studentized")
abline(0,1)

# Compare all

par(mfrow=c(2,2))
```



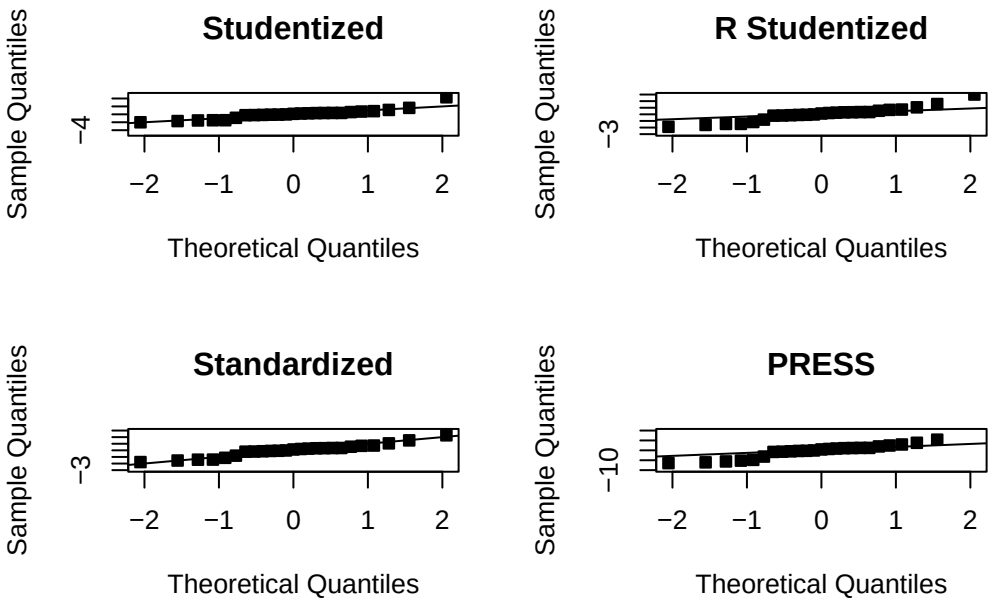

```
qqnorm(student_res,pch=22,bg=1,ylim=c(-5,5),main="Studentized")
abline(0,1)
# hist(student_res)
```

```
qqnorm(Rstudent_res,pch=22,bg=1,ylim=c(-3,3),main="R Studentized")
qqline(Rstudent_res,pch=22,bg=1,ylim=c(-10,10))
# abline(0,1)

qqnorm(stand_res,pch=22,bg=1,ylim=c(-3,3),main="Standardized")
abline(0,1)

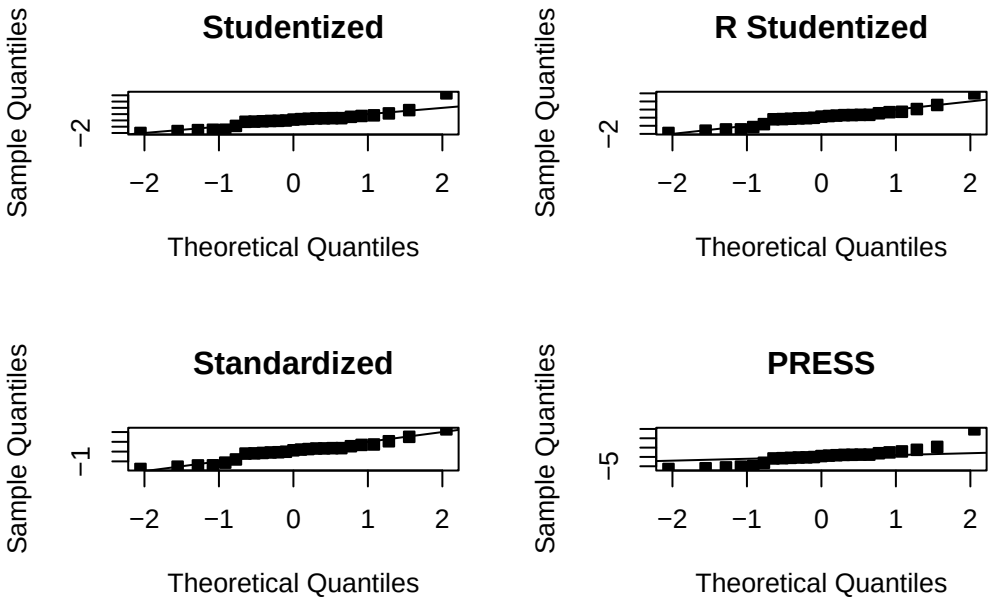
qqnorm(press,pch=22,bg=1,ylim=c(-10,10),main="PRESS")
qqline(press,pch=22,bg=1,ylim=c(-10,10))
#careful of the scale!

par(mfrow=c(3,2))
qqline(model$residuals,pch=22,bg=1,main="Ordinary")
```

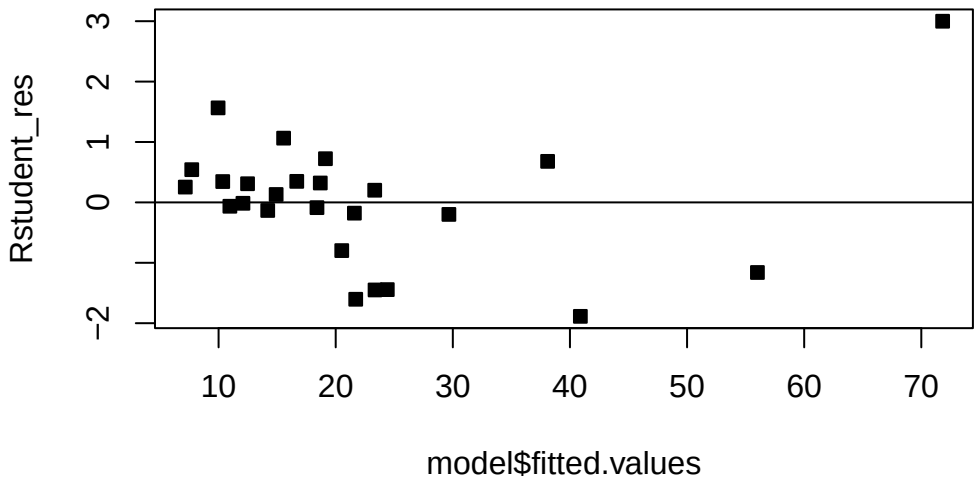



```
par(mfrow=c(2,2))
qqnorm(student_res,pch=22,bg=1,main="Studentized")
abline(0,1)
qqnorm(Rstudent_res,pch=22,bg=1,main="R Studentized")
abline(0,1)
qqnorm(stand_res,pch=22,bg=1,main="Standardized")
abline(0,1)
qqnorm(press,pch=22,bg=1,main="PRESS")
```

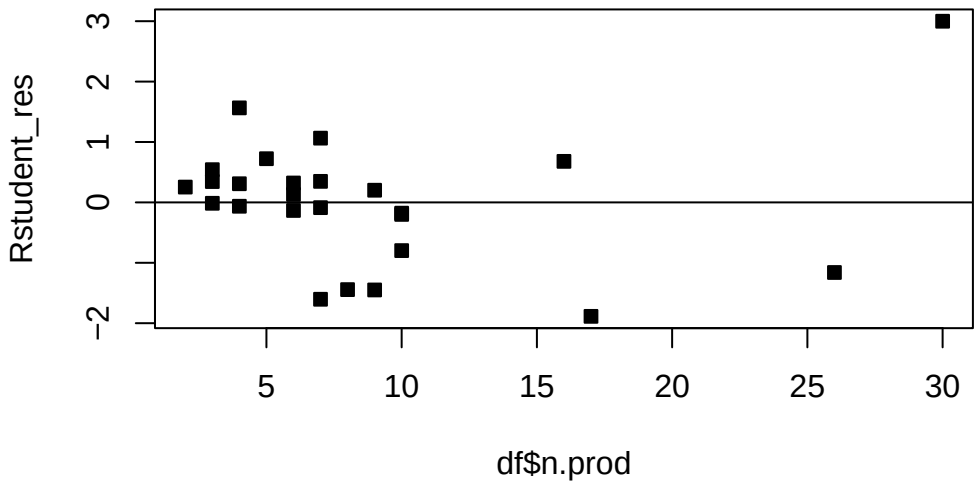
```
abline(0,1)
```



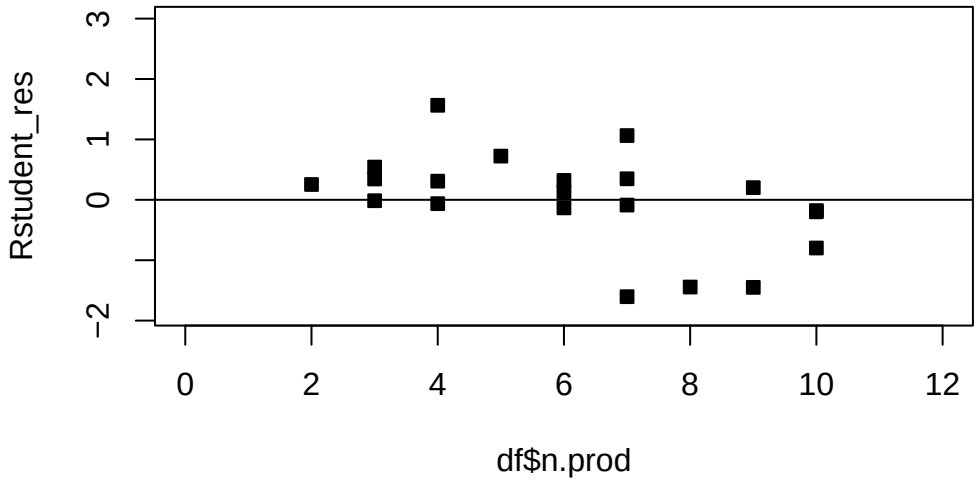

```
# Now we plot the fitted values against the R studentized residuals
par(mfrow=c(1,1),pch=22)
plot(model$fitted.values,Rstudent_res,bg=1)
abline(h=0)
```



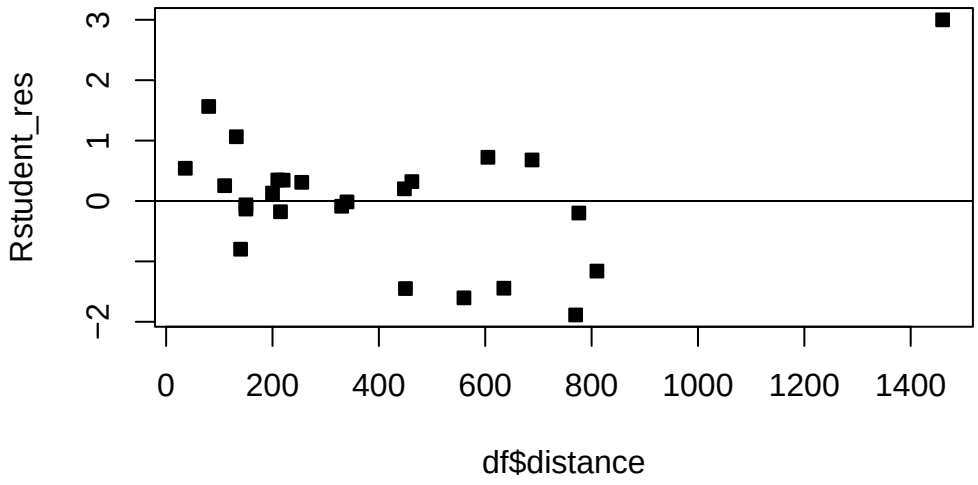

```
# Now we plot the number of products against the R studentized residuals
# There is one moderately large delivery!
plot(df$n.prod,Rstudent_res,bg=1)
abline(h=0)
```



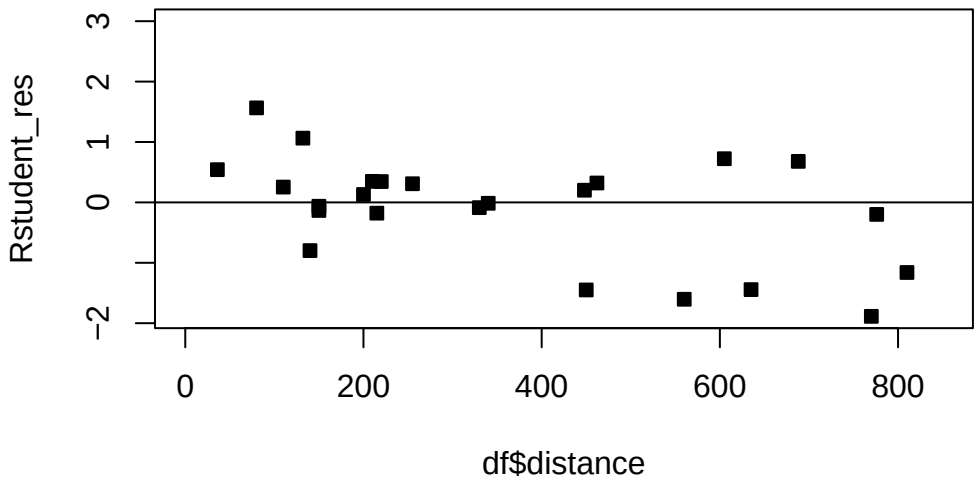

```
# Care for the scale
plot(df$n.prod,Rstudent_res,bg=1,xlim=c(0,12))
abline(h=0)
```



```r
# There is one very far delivery!
plot(df$distance,Rstudent_res,bg=1)
abline(h=0)
```




```r
# Care for the scale
plot(df$distance,Rstudent_res,bg=1,xlim=c(0,850))
abline(h=0)
```




```
# What happens to the model when we remove this outlying observation (the far distance delive
df2=df[-which.max(df$distance),]


# refit the model
model=lm(delTime~.,data=df2)
s=summary(model); s
```

```
Call:
lm(formula = delTime ~ ., data = df2)

Residuals:
    Min      1Q  Median      3Q     Max
-4.0325 -1.2331  0.0199  1.4730  4.8167

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 4.447238   0.952469   4.669 0.000131 ***
n.prod      1.497691   0.130207  11.502 1.58e-10 ***
distance    0.010324   0.002854   3.618 0.001614 **
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.43 on 21 degrees of freedom
Multiple R-squared:  0.9487,    Adjusted R-squared:  0.9438
F-statistic: 194.2 on 2 and 21 DF,  p-value: 2.859e-14
```

```r
# X matrix
X=model.matrix(model)

# Hat matrix
hat=X%*%solve(t(X)%*%X)%*%t(X)

# Compute h_ii
hii=diag(hat)
hii
```

```
         1          2          3          4          5          6          7
0.11083391 0.07741039 0.09998709 0.10097319 0.08066357 0.04290146 0.10024969
         8         10         11         12         13         14         15
0.06537738 0.20438000 0.14675966 0.11367920 0.06437975 0.08033747 0.04661503
        16         17         18         19         20         21         22
0.21115081 0.06254612 0.10128434 0.11992977 0.18537865 0.16642759 0.55671434
        23         24         25
0.04687996 0.13894064 0.07620000
```

```r
max(hii)
```

```
[1] 0.5567143
```

```r
###### ordinary residuals
regular_residuals=model$residuals
# or


# standardized residuals
stand_res=model$residuals/s$sigma

# studentized residuals
student_res=rstudent(model)
```

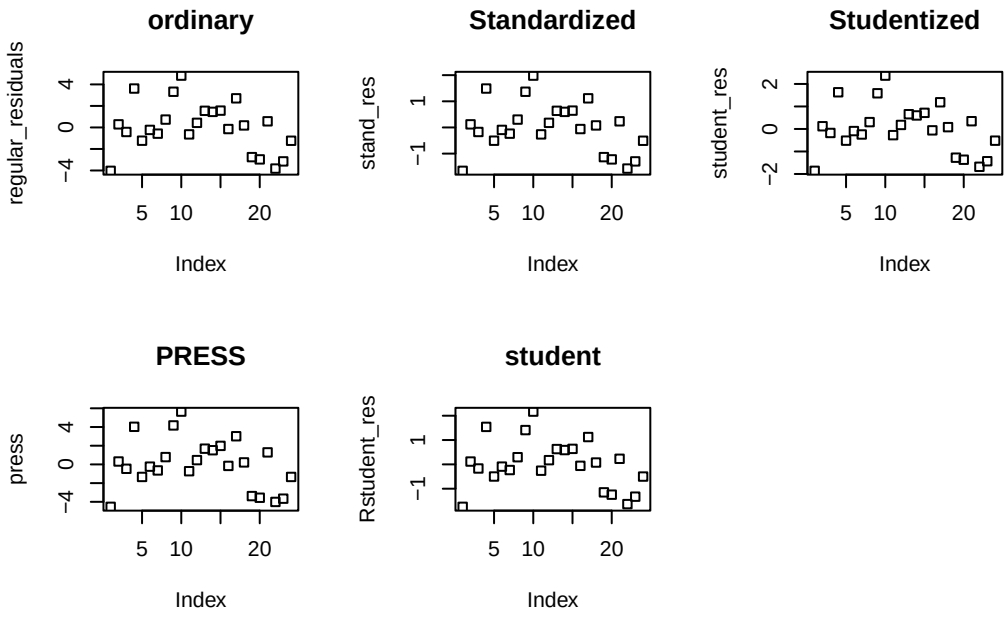
```r
#PRESS residuals
press=model$residuals/(1-hii)

# Get the MSE_is
MSE_i=((n-2)*(s$sigma)^2-regular_residuals^2/(1-hii))/(n-3)

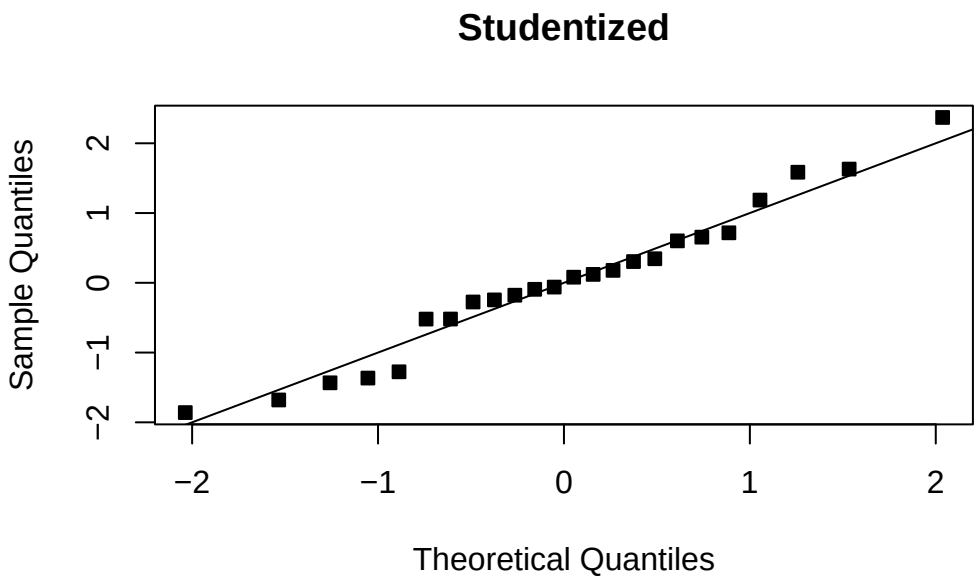
#r studentized residuals
Rstudent_res=model$residuals/sqrt(MSE_i)

# Plot them all and compare - much better
par(mfrow=c(2,3))
plot(regular_residuals,main="ordinary")
plot(stand_res,main="Standardized")
plot(student_res,main="Studentized")
plot(press,main="PRESS")
plot(Rstudent_res,main="student")

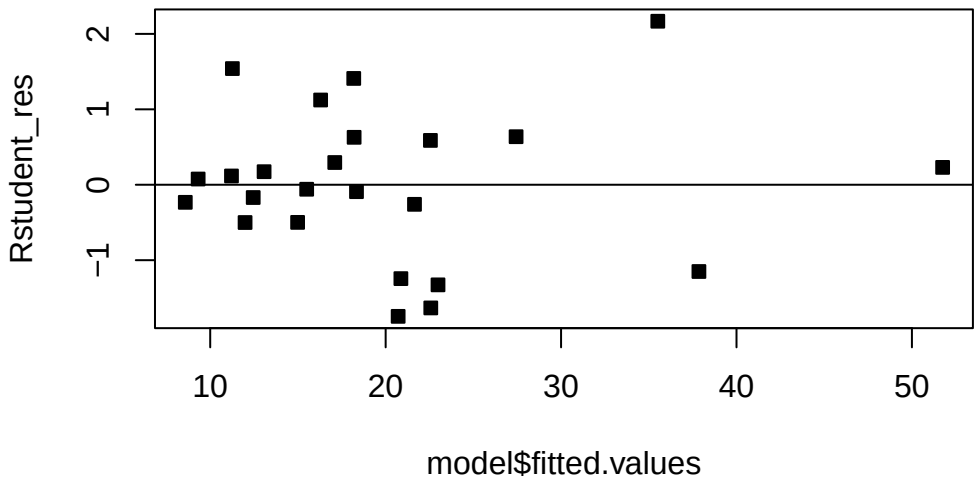

# Notice that these graphs are fine now...
par(mfrow=c(1,1))
```



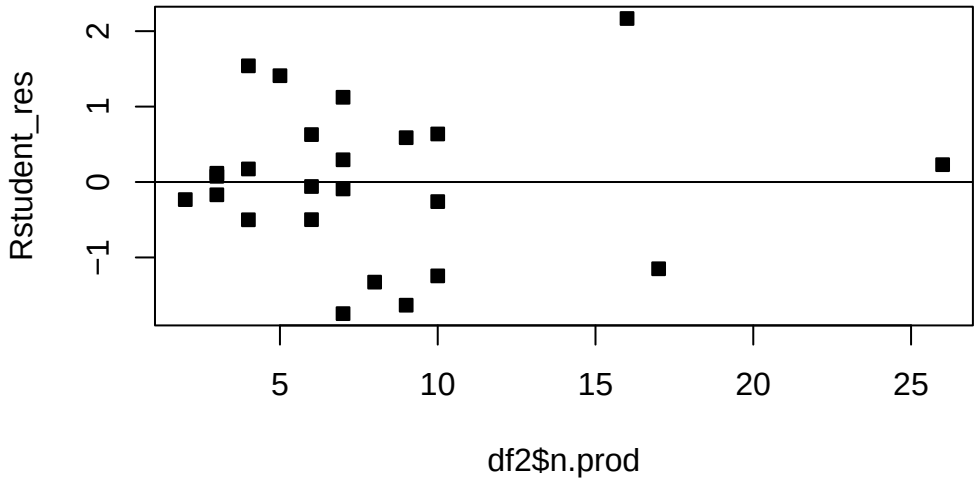
```
qqnorm(student_res,pch=22,bg=1,main="Studentized")
abline(0,1)
```



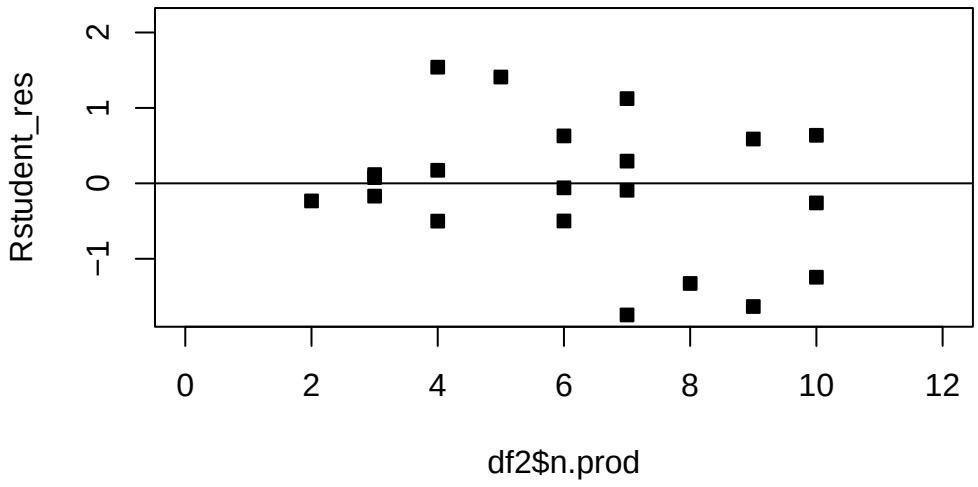

```
# Now we plot the fitted values against the R studentized residuals
par(mfrow=c(1,1),pch=22)
plot(model$fitted.values,Rstudent_res,bg=1)
abline(h=0)
```



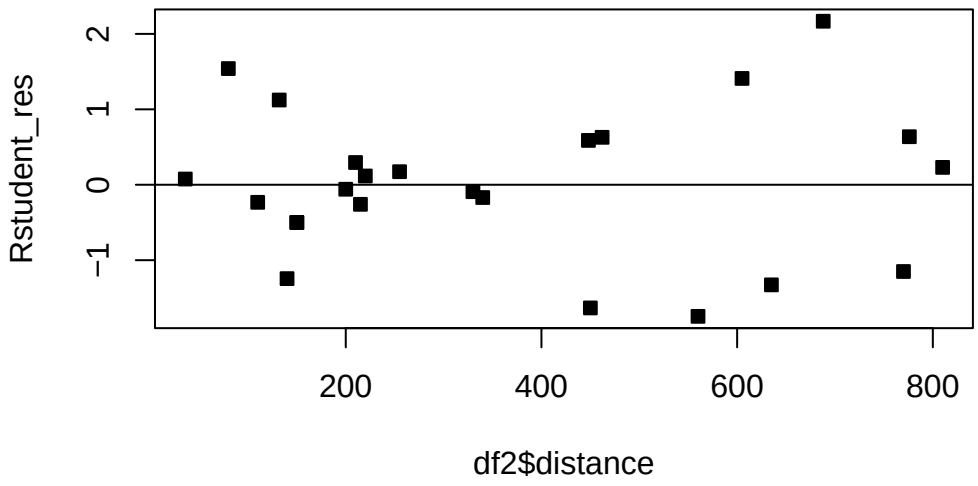

```
# Now we plot the number of products against the R studentized residuals
plot(df2$n.prod,Rstudent_res,bg=1)
abline(h=0)
```



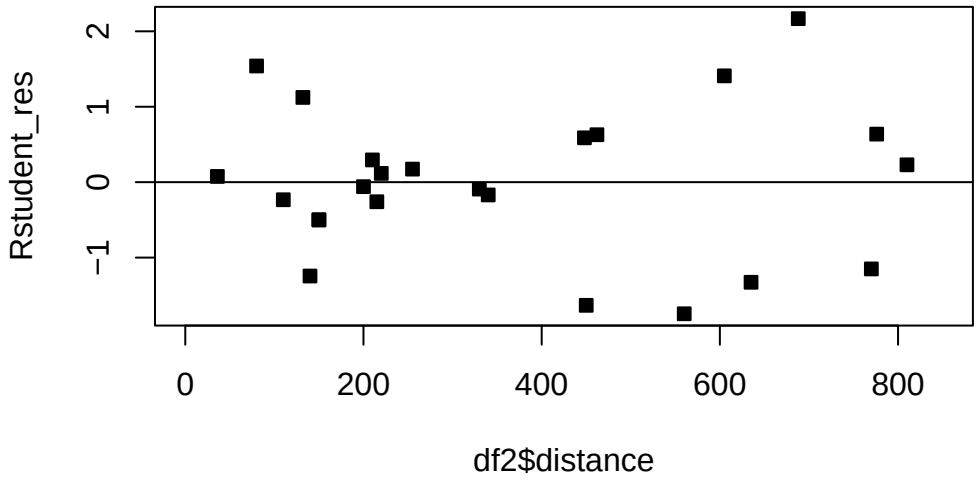
```
# Care for the scale
plot(df2$n.prod,Rstudent_res,bg=1,xlim=c(0,12))
abline(h=0)
```




```
plot(df2$distance,Rstudent_res,bg=1)
abline(h=0)
```




```
# Care for the scale
plot(df2$distance,Rstudent_res,bg=1,xlim=c(0,850))
abline(h=0)
```



Now, we have introduced different types of residuals and the appropriate graphs to examine when checking for violations of the assumptions. When we observe violations of the assumptions - what do we do? That will be the topic of the next section.

Some of these remedies include: - Transformations of the response - Transformations of certain regressors - Robust methods/outlier removal - Inclusion of new regressors

4.4 Homework stop

Do the Chapter 4 questions from the textbook.

Exercise 4.4. In the context of a regression model, do you think a point whose covariates are outlying is more problematic than a point whose response, given the covariates, is outlying?

Exercise 4.5. Make a table describing the differences between each type of residual.

Exercise 4.6. Perform a residual analysis on the marketing data from Example Example 3.7.

Exercise 4.7. Perform a residual analysis on the data from Example Example 3.8.

5 Transformations

5.1 Variance-stabilizing transformations

Recall that we assume that $\forall i \in [n]$, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. A common reason for a violation of this assumption is for Y to have a distribution in which the variance is related to its mean. For example, if the response Y is a Poisson random variable, i.e.,

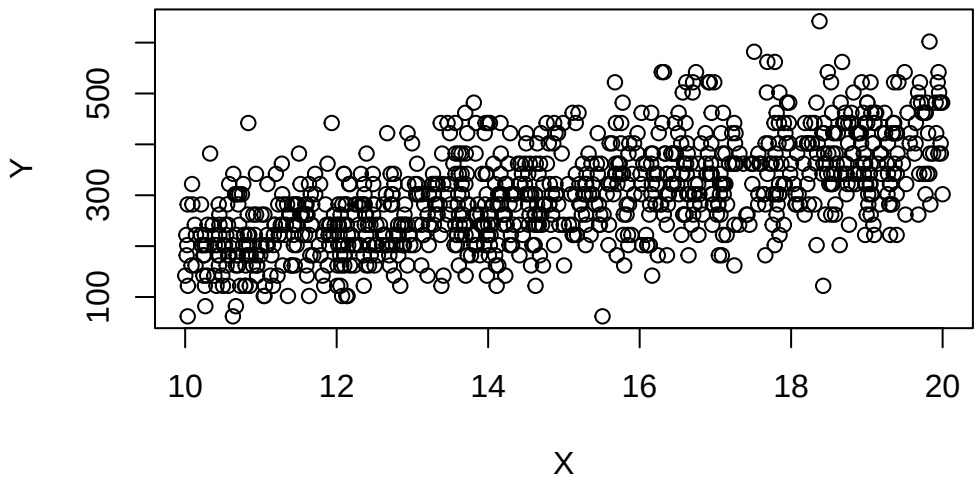
$$Y|X \sim Pois(X\beta),$$

then we have that $\mathrm{E}[Y] = X\beta$, then $\mathrm{E}[Y] = \mathrm{Var}[Y] = X\beta$. In this case, the simple linear regression assumptions are violated. In particular, the variance is not the same for each observation. Here, it happens that taking the response to be roughly $\sqrt{Y}$ fixes the problem. That is, performing the regression analysis with $\sqrt{Y}$ as the response variable instead of Y, ensures that the regression assumptions are (approximately) satisfied. This example gives rise to the idea of **transformations**. If our data do not satisfy the assumptions for the MLR or the normal MLR, we might ask if there is some transformation of either the response, some of the covariates, or both that make the data suitable for a MLR analysis. Note that the assumptions are important. For instance, if the variance is not homogeneous, the OLS estimator will still be unbiased, but they will no longer have BLUE property. That means that some other estimator will work better for such data!

Which transformation should we choose? Sometimes, we can use prior experience or theoretical considerations to guide us in selecting an appropriate transformation. Other times, we must choose it empirically, i.e., based on the data. Often, the square root and the logarithm are popular choices. If you response is between 0 and 1, and the data appear to be "football shaped", then you may like to take the $\arcsin(\sqrt{Y})$.

We now demonstrate what one of these relationships looks like in simple linear regression. We now simulate a dataset where $\sigma^2 \propto \mathrm{E}[Y|X]$, and plot X against Y. We use the Poisson example discussed previously.

```
set.seed(2352)
n=1000
X=runif(n,5,10)*2
Y=20*rpois(n,X)+2
plot(X,Y)
```




```
# Performing a regression analysis yields:
model=lm(Y~X)
# Notice the intercept is poorly estimated!
summary(model)
```

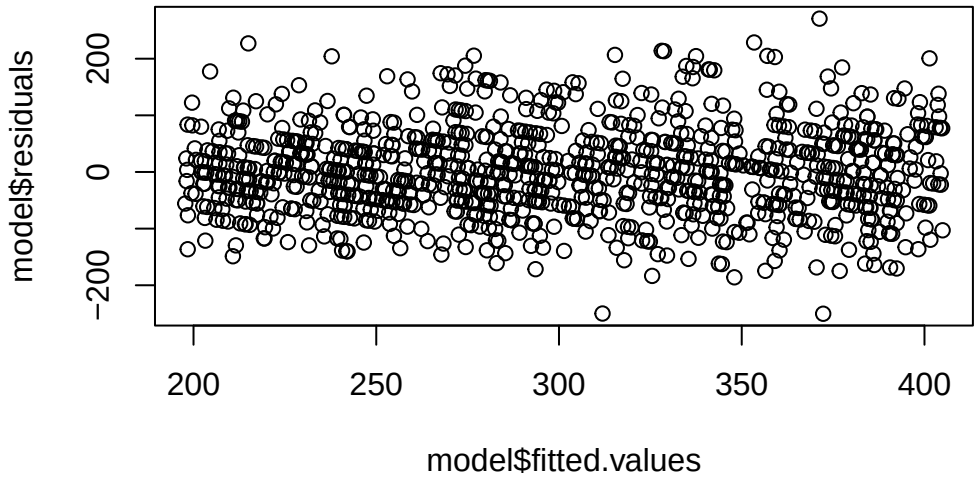
```
Call:
lm(formula = Y ~ X)

Residuals:
     Min       1Q   Median       3Q      Max
-250.289  -53.392   -2.127   48.795  270.716

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -9.5199    13.0735  -0.728    0.467
X            20.7241     0.8599  24.101   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 77.61 on 998 degrees of freedom
Multiple R-squared:  0.3679,    Adjusted R-squared:  0.3673
F-statistic: 580.9 on 1 and 998 DF,  p-value: < 2.2e-16
```

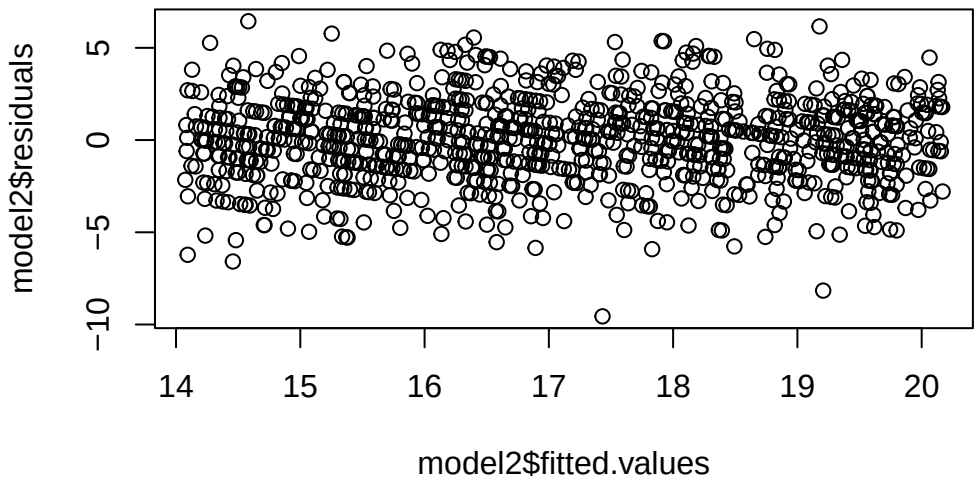
```r
# Notice the fan shape in the residuals against the fitted values?
plot(model$fitted.values,model$residuals)
```




```r
# Let's perform the transformations

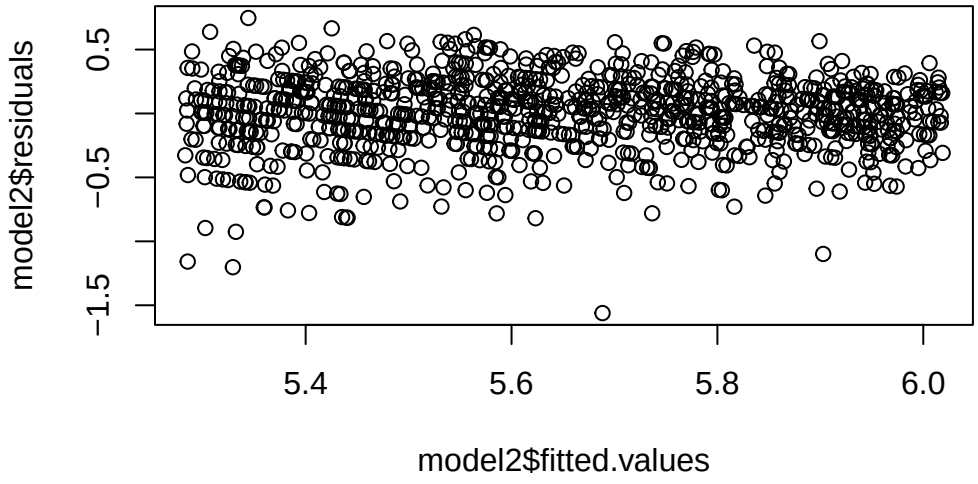
model2=lm(sqrt(Y)~X)

plot(model2$fitted.values,model2$residuals)
```




```
model2=lm(log(Y)~X)

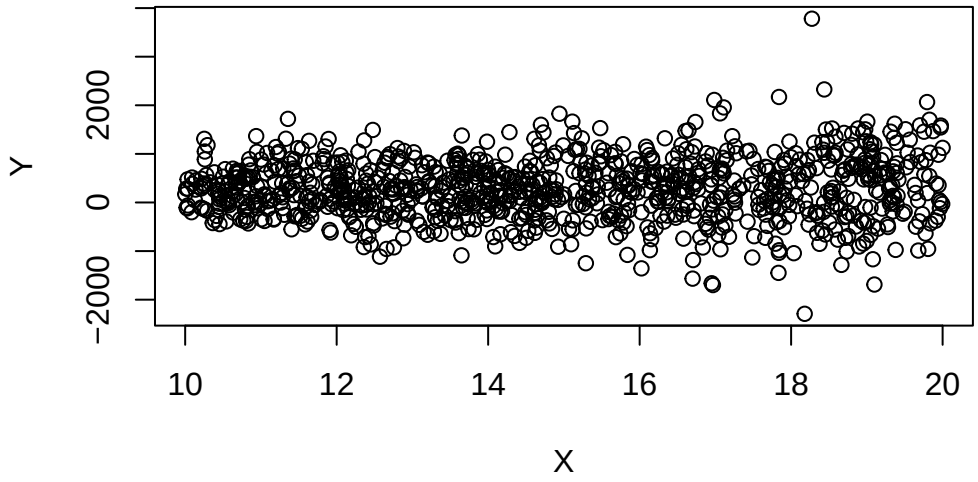
plot(model2$fitted.values,model2$residuals)
```



However, these transformations do not always work. Suppose we have that $Y \sim \mathcal{N}(X, 4 * X^2)$. We then have that $\sigma = 2 * X = 2 * \mathrm{E}[Y|X]$. Notice how the spread of the points is increasing with X? This is a symptom of non-homogeneous variance. However, the proposed transformations do not work.

```
set.seed(2352)
# \sigma^2\propto \E{Y}
Y=20*rnorm(n,X,X*2)+2

plot(X,Y)
```




```
# Performing a regression analysis yields:
model=lm(Y~X)

# notice the intercept is poorly estimated.
summary(model)
```

```
Call:
lm(formula = Y ~ X)

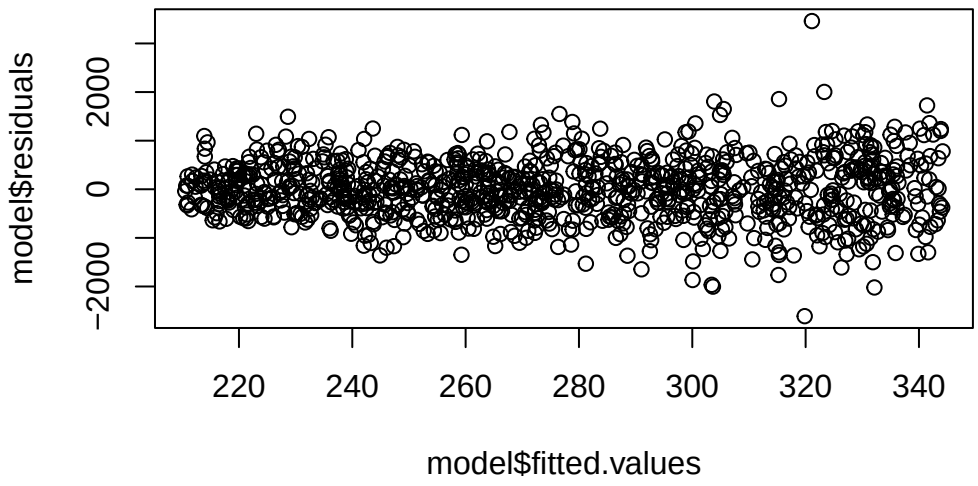
Residuals:
```

```
    Min      1Q  Median      3Q     Max
-2610.8  -375.3    -3.5   384.8  3460.5

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   76.844    103.345   0.744   0.4573
X             13.367      6.797   1.966   0.0495 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 613.5 on 998 degrees of freedom
Multiple R-squared:  0.00386,   Adjusted R-squared:  0.002861
F-statistic: 3.867 on 1 and 998 DF,  p-value: 0.04953
```

```r
# Notice the fan shape in the residuals against the fitted values?
plot(model$fitted.values,model$residuals)
```

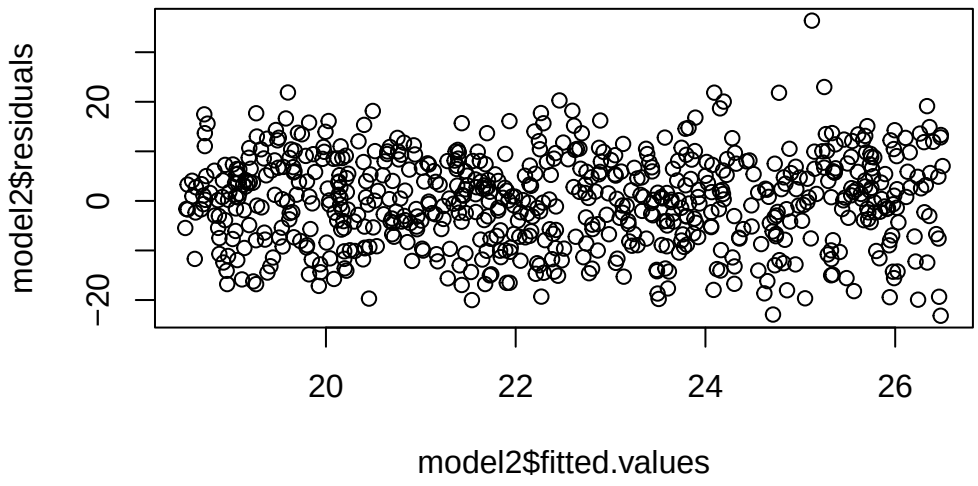



```r
# Let's perform the transformation

# Performing a regression analysis yields:
model2=lm(sqrt(Y)~X)
```

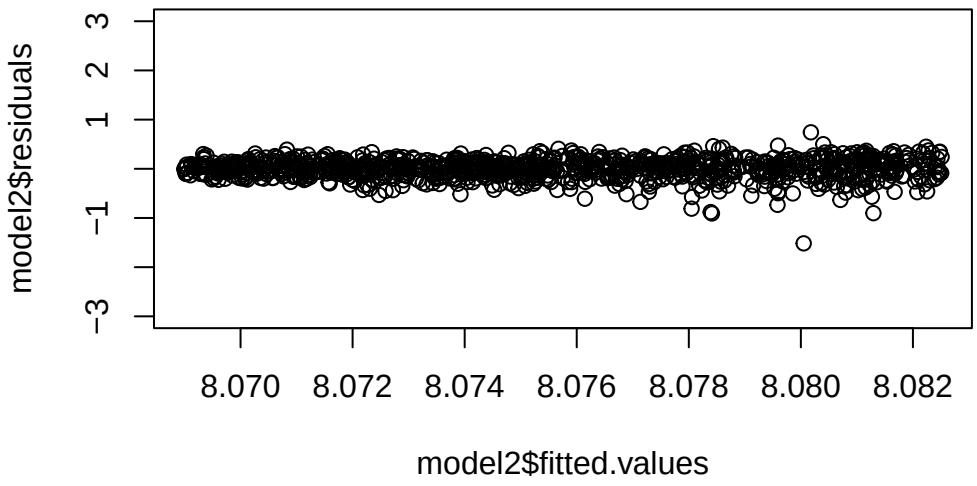
```
Warning in sqrt(Y): NaNs produced
```

```r
# Notice the fan shape in the residuals against the fitted values?
plot(model2$fitted.values,model2$residuals)
```




```r
# Performing a regression analysis yields:
model2=lm(log(Y+3000)~X)

# Notice the fan shape in the residuals against the fitted values?
plot(model2$fitted.values,model2$residuals,ylim=c(-3,3))
```



In general, a good transformation to correct violated assumptions can improve estimates and test accuracy.

> **Caution**
>
> It is often necessary to convert any predicted values back to the original units. Applying the inverse transformation to predicted values gives an estimate of the median of the distribution of the (untransformed) response – instead of the mean. This implies that predictions are generally biased. Prediction and confidence intervals do not suffer this illness. They can be converted back to the original units via the inverse transformation and the interpretation will remain the same.

5.1.1 Inverse transformations

Let's expand on this. It is a good time to recall that in general, for a real function f, we have that $\mathrm{E}\left[f(X)\right] \neq f(\mathrm{E}\left[X\right])$. For instance, for many random variables Z, we would have that $\mathrm{E}\left[Z^2\right] \neq \mathrm{E}\left[Z\right]^2$, $\mathrm{E}\left[\log Z\right] \neq \log \mathrm{E}\left[Z\right]$ etc. .

In a transformed regression model, we fit the following model:

$$f(Y) = X\beta + \epsilon.$$

If we are interested in predicting the value of Y given z, then it seems natural to take the predictions for $f(Y)$ given z, which are given by $\beta^\top z$ and apply the inverse transformation f^{-1}.

For instance, to predict $Y|Z=z$, we may compute: $f^{-1}(\beta^\top z)$. It turns out, this prediction is biased, and we should use a different method instead.

To see why it's biased, observe that the predictions from the model $f(Y)=X\beta+\epsilon$ for a new set of covariates z are given by $\widehat{f}(Y)=\widehat{\beta}^\top z \approx \mathrm{E}\left[f(Y)|Z=z\right]$. Now, we have that

$$f^{-1}(\widehat{\beta}^\top z) \approx f^{-1}(\mathrm{E}\left[f(Y)|Z=z\right]) \neq \mathrm{E}\left[f^{-1}(f(Y))|Z=z\right] = \mathrm{E}\left[\widehat{f}(Y)|Z=z\right].$$

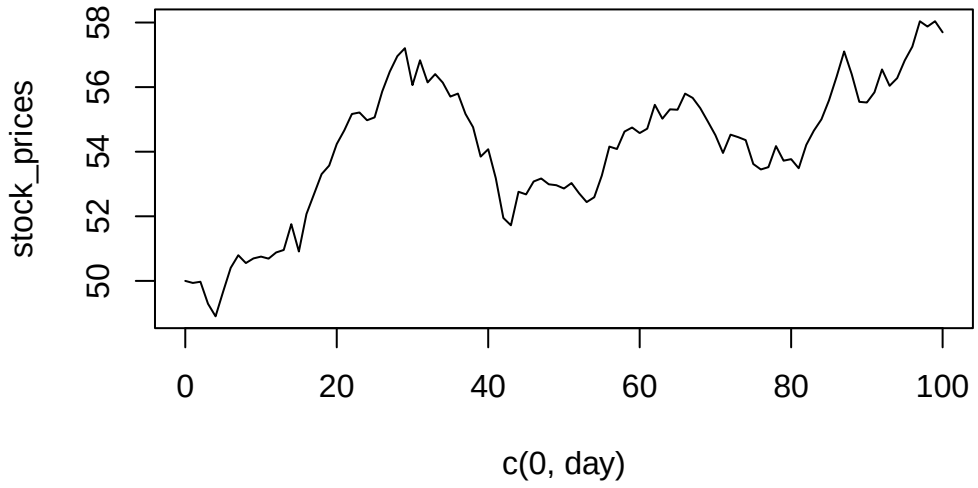
The solution to this problem is to adjust for the bias. For the log transform, we can multiply the resulting inverse transformed predictions by $\exp(\widehat{\sigma}^2/2)$. For the square root transformation, we add $\widehat{\sigma}^2$ to the resulting inverse transformed predictions. See (Miller 1984) for more information.

One can also use confidence and prediction intervals to predict the value of Y given z. Confidence or prediction intervals may be directly converted from one metric to another – such interval estimates are percentiles of a distribution which are unaffected by the transformation. They can be converted back to the original units via the inverse transformation and the interpretation will remain the same. Optimal intervals are intervals with the shortest average interval length for a given confidence level, under a given set of assumptions. However, it may be that the resulting intervals may not be "optimal". One way to get a prediction in the original units, is to apply the inverse transformation to the prediction interval computed from the transformed model and take the midpoint of that interval. This does not always work well - and should be checked against the original data.

Example 5.1. Let's simulate what happens when, given the day $t \in [100]$, we try to estimate the mean stock price P_t for some stock (maybe ?Gamestop?) in a model which regresses the logged rate of return against the day. Note that the logged returns at time t are given by: $L=\log\left(\frac{P_t}{P_{t-1}}\right)$.

```
set.seed(2352)
# Simulate the stock prices
n=100
day=seq(1:n)
log_return=rnorm(n,0.000001+0.000005*day,0.01)
# log_return=rnorm(n,0.000001+0.000005*day,0.0005)
stock_prices=c(50,50*exp(cumsum(log_return)))
# exp(log_return)[1:10]

plot(c(0,day), stock_prices,type='l')
```

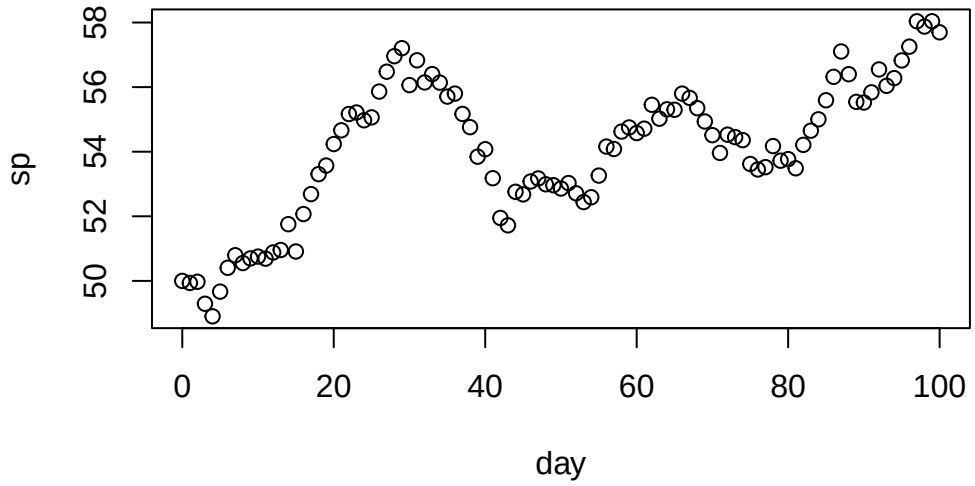



```
df=data.frame(cbind("day"=c(0,day),"sp"=stock_prices))


# Now, suppose this is our starting dataset
head(df)
```

```
  day       sp
1   0 50.00000
2   1 49.93624
3   2 49.97369
4   3 49.29229
5   4 48.90210
6   5 49.66743
```

```
# Notice that the pattern is not great... but we can regress on the transformed response
plot(df)
```




```r
# Fitting the model

# Compute the log returns
df$lr=NA
df$lr[2:(n+1)]=log(df$sp[-1]/df$sp[-(n+1)])

# Sanity Check
# log_return[1:5]
# df$lr[2:6]

model=lm(lr~day,df)
summary(model)
```

```
Call:
lm(formula = lr ~ day, data = df)

Residuals:
       Min          1Q      Median          3Q         Max
-0.0249054 -0.0064851  0.0000518  0.0075839  0.0208044

Coefficients:
```

```
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.824e-03  1.926e-03   0.947    0.346
day         -7.771e-06  3.312e-05  -0.235    0.815

Residual standard error: 0.00956 on 98 degrees of freedom
  (1 observation deleted due to missingness)
Multiple R-squared:  0.0005615, Adjusted R-squared:  -0.009637
F-statistic: 0.05506 on 1 and 98 DF,  p-value: 0.815
```

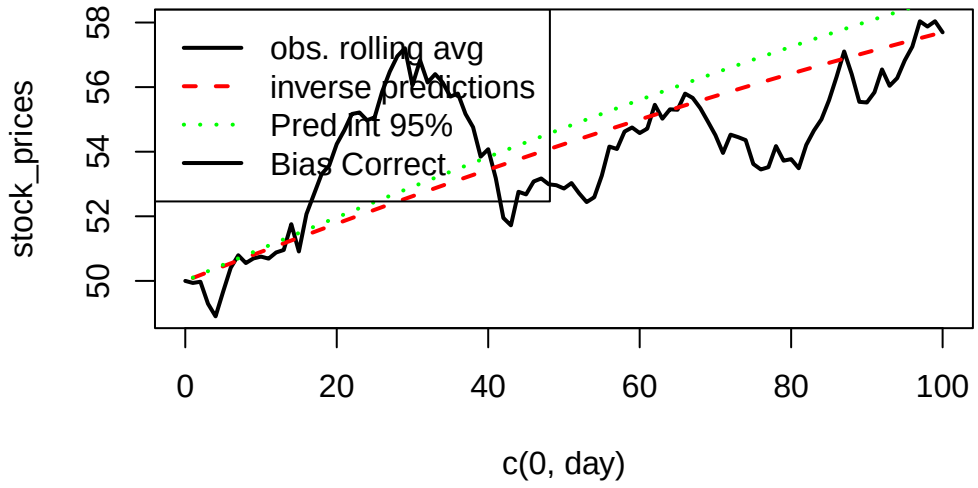
```r
plot(c(0,day), stock_prices,type='l',lwd=2)
lines(50*exp(cumsum(fitted.values(model))),col='red',lty=2,lwd=2)

# Intervals for the mean at each time point
intervals=predict(model,interval ='prediction')[,2:3]
```

```
Warning in predict.lm(model, interval = "prediction"): predictions on current data refer to
```

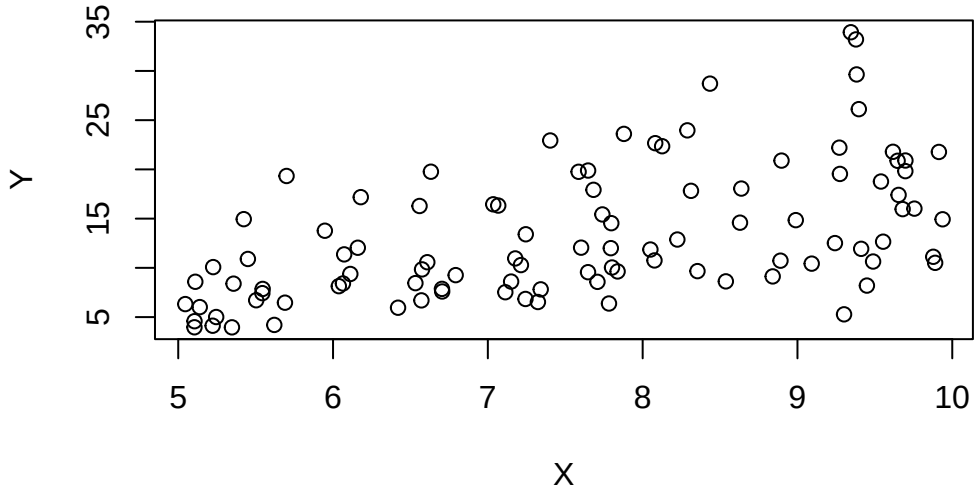
```r
midpoint=ret=rep(0,n)
for(i in 1:n){
  if(i==1){
    midpoint[i]=50*exp(intervals[i,1])/2+50*exp(intervals[i,2])/2
  }
  else
    midpoint[i]=midpoint[i-1]*(exp(intervals[i,1])+exp(intervals[i,2]))/2
}
lines(midpoint,col="green",lty=3,lwd=2)

legend("topleft",legend=c("obs. rolling avg","inverse predictions","Pred int 95%","Bias Corre
```



A second example...

```
set.seed(2352)
# Simulate data
n=100
X=runif(n,5,10)
logs=rnorm(n,1+0.2*X,0.5)
Y=exp(logs)
plot(X,Y)
```




```r
df=data.frame(cbind("X"=X,"Y"=Y))
df=df[order(X),]

# Fitting the model
model=lm(log(Y)~X,data=df)
summary(model)
```

```
Call:
lm(formula = log(Y) ~ X, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-1.13484 -0.31721 -0.02877  0.29765  0.85929

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.00060    0.21048   4.754 6.85e-06 ***
X            0.19333    0.02721   7.106 1.94e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.4113 on 98 degrees of freedom
Multiple R-squared:  0.3401,    Adjusted R-squared:  0.3333
F-statistic:  50.5 on 1 and 98 DF,  p-value: 1.937e-10
```

```
s=summary(model)$sigma
# Rolling average



zb=zoo::zoo(x=df$Y,df$X)
rm=zoo::rollmean(zb,25)

plot(attributes(rm)$index,rm,lty=1,lwd=3,type='l')
# plot(X,Y)

zb=zoo::zoo(x=exp(fitted.values(model)),df$X)
rm=zoo::rollmean(zb,25)
lines(attributes(rm)$index,rm,col=2,lty=2,lwd=3)

lines(attributes(rm)$index,rm*exp(s^2/2),col=6,lty=2,lwd=3)

# Intervals for the mean at each time point
nd=data.frame("X"=df$X)
ivs=predict(model, newdata = nd,interval = 'prediction')[,2:3]
intervals=rowMeans(exp(ivs))
zb=zoo::zoo(x=intervals,df$X)
rm=zoo::rollmean(zb,25)
lines(attributes(rm)$index,rm,col=3,lty=3,lwd=4)

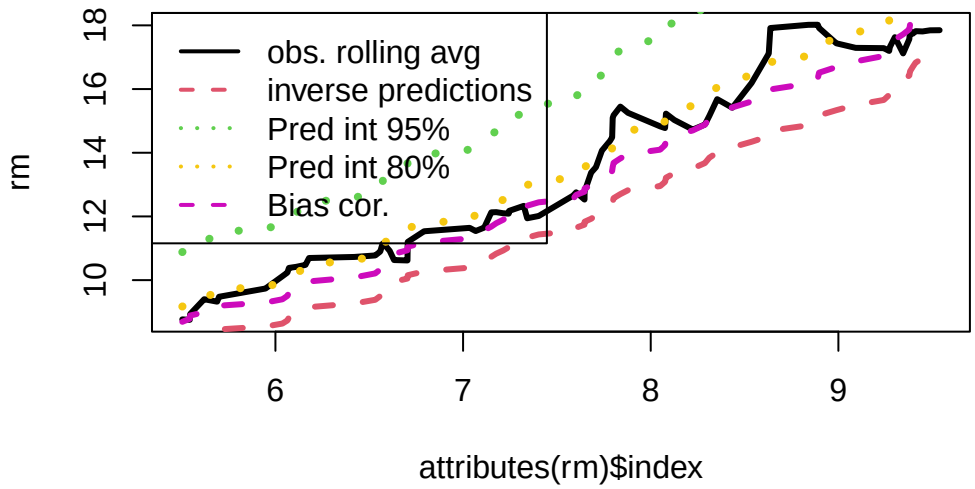

# Intervals for the mean at each time point - notice when we lower the level the performance
nd=data.frame("X"=df$X)
ivs=predict(model, newdata = nd,interval = 'prediction', level = 0.8)[,2:3]
intervals=rowMeans(exp(ivs))
zb=zoo::zoo(x=intervals,df$X)
rm=zoo::rollmean(zb,25)
lines(attributes(rm)$index,rm,col=7,lty=3,lwd=4)




# Intervals for the mean at each time point using confidence intervals
```

```
# ivs=predict(model, newdata = nd,interval = 'confidence', level = 0.8)[,2:3]
# intervals=rowMeans(exp(ivs))
# zb=zoo::zoo(x=intervals,df$X)
# rm=zoo::rollmean(zb,25)
# lines(attributes(rm)$index,rm,col=6,lty=3,lwd=3)

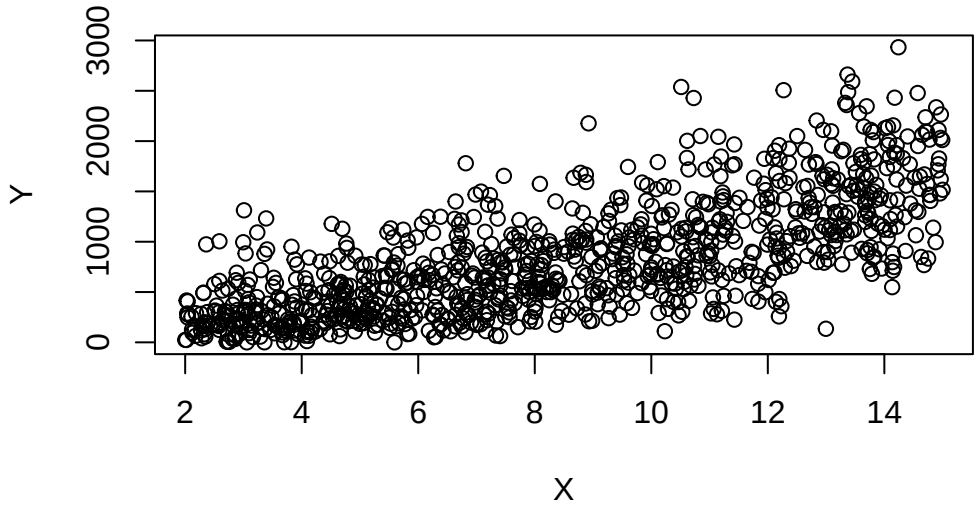
legend("topleft",legend=c("obs. rolling avg","inverse predictions","Pred int 95%","Pred int 8
```




```
set.seed(2352)

# Simulate data

n=1000
X=runif(n,2,15)
sq=rnorm(n,10+2*X,7)
Y=sq^2
plot(X,Y)
```




```
df=data.frame(cbind("X"=X,"Y"=Y))
df=df[order(X),]

# Fitting the model
model=lm(sqrt(Y)~X,data=df)
summary(model)
```

```
Call:
lm(formula = sqrt(Y) ~ X, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-24.2975  -4.5612  -0.0315   4.7083  20.2670

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  9.94439    0.54223   18.34   <2e-16 ***
X            1.99810    0.05897   33.88   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 6.919 on 998 degrees of freedom
Multiple R-squared:  0.535, Adjusted R-squared:  0.5345
F-statistic:  1148 on 1 and 998 DF,  p-value: < 2.2e-16
```

```
s=summary(model)$sigma
# plot(X,Y)
# Rolling average



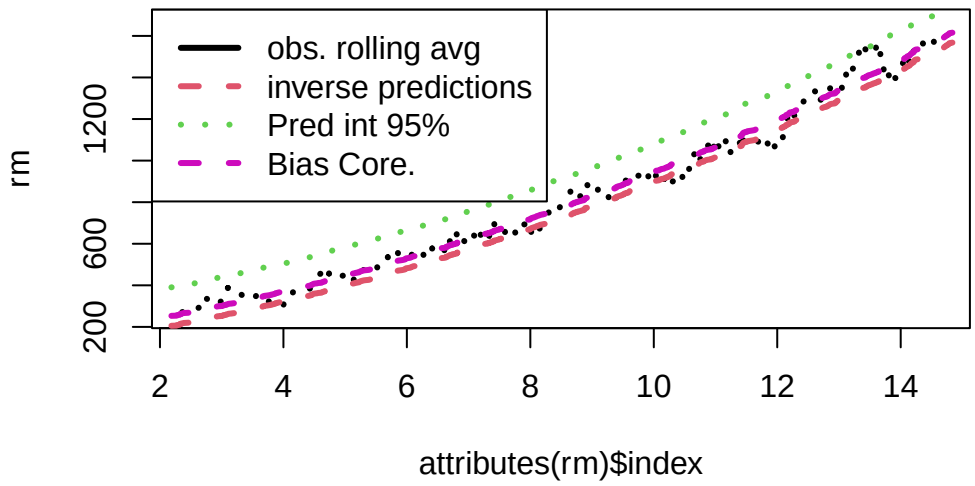
zb=zoo::zoo(x=df$Y,df$X)
rm=zoo::rollmean(zb,50)

plot(attributes(rm)$index,rm,col=1,lty=3,lwd=3,type='l')
zb=zoo::zoo(fitted.values(model)^2,df$X)
rm=zoo::rollmean(zb,25)
lines(attributes(rm)$index,rm,col=2,lty=2,lwd=3)
lines(attributes(rm)$index,rm+s^2,col=6,lty=2,lwd=3)


# Intervals for the mean at each time point
intervals=rowMeans(predict.lm(model,interval = 'prediction')[,2:3]^2)
```

```
Warning in predict.lm(model, interval = "prediction"): predictions on current data refer to
```

```
zb=zoo::zoo(intervals,df$X)
rm=zoo::rollmean(zb,25)
lines(attributes(rm)$index,rm,col=3,lty=3,lwd=3)
legend("topleft",legend=c("obs. rolling avg","inverse predictions","Pred int 95%","Bias Core
```



We see that the bias correction is the best performing method. However, this involves working out the bias for each transformation. For a complicated transformation, this may be quite difficult. For common transformations, this has already been completed for us.

Let's do an example with some real data. The following example is taken from the textbook:

Example 5.2. An electric utility is interested in developing a model relating peak - hour demand Y to total energy usage during the month X. This is an important planning problem because while most customers pay directly for energy usage (in kilowatt - hours), the generation system must be large enough to meet the maximum demand imposed. Data for 53 residential customers for the month of August is given below.

```
# Electric Utility Data

df<- data.frame(
    Customer = c(1:53),
    x_kWh = c(679, 292, 1012, 493, 582, 1156, 997, 2189, 1097, 2078, 1818, 1700, 747, 2030, 1
    y_kW = c(0.79, 0.44, 0.56, 0.79, 2.70, 3.64, 4.73, 9.50, 5.34, 6.85, 5.84, 5.21, 3.25, 4
  )


df
```

```
   Customer x_kWh  y_kW
```

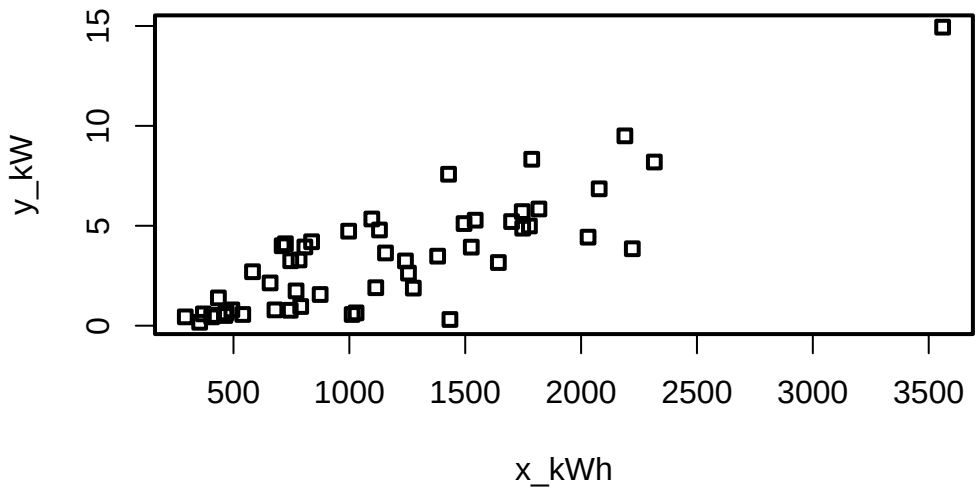
1	1	679	0.79
2	2	292	0.44
3	3	1012	0.56
4	4	493	0.79
5	5	582	2.70
6	6	1156	3.64
7	7	997	4.73
8	8	2189	9.50
9	9	1097	5.34
10	10	2078	6.85
11	11	1818	5.84
12	12	1700	5.21
13	13	747	3.25
14	14	2030	4.43
15	15	1643	3.16
16	16	414	0.50
17	17	354	0.17
18	18	1276	1.88
19	19	745	0.77
20	20	435	1.39
21	21	540	0.56
22	22	874	1.56
23	23	1543	5.28
24	24	1029	0.64
25	25	710	4.00
26	26	1434	0.31
27	27	837	4.20
28	28	1748	4.88
29	29	1381	3.48
30	30	1428	7.58
31	31	1255	2.63
32	32	1777	4.99
33	33	370	0.59
34	34	2316	8.19
35	35	1130	4.79
36	36	463	0.51
37	37	770	1.74
38	38	724	4.10
39	39	808	3.94
40	40	790	0.96
41	41	783	3.29
42	42	406	0.44
43	43	1242	3.24

```
44       44   658  2.14
45       45  1746  5.71
46       46   468  0.64
47       47  1114  1.90
48       48   413  0.51
49       49  1787  8.33
50       50  3560 14.94
51       51  1495  5.11
52       52  2221  3.85
53       53  1526  3.93
```

```r
#  changing the plot aesthetics
par(pch=22,lwd=2)


# Explore

plot(df[,2:3])
```

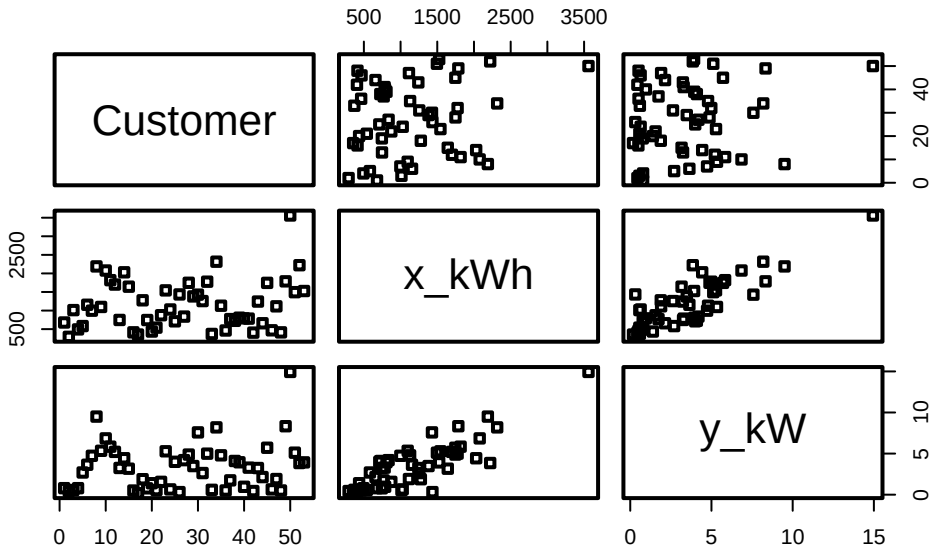



```r
summary(df)
```

```
    Customer       x_kWh           y_kW
```

```
 Min.   : 1   Min.   : 292   Min.   : 0.170
 1st Qu.:14   1st Qu.: 679   1st Qu.: 0.790
 Median :27   Median :1029   Median : 3.250
 Mean   :27   Mean   :1153   Mean   : 3.413
 3rd Qu.:40   3rd Qu.:1543   3rd Qu.: 4.880
 Max.   :53   Max.   :3560   Max.   :14.940
```

```
plot(df)
```




```
# Model

model=lm(y_kW~x_kWh, df); model
```

```
Call:
lm(formula = y_kW ~ x_kWh, data = df)

Coefficients:
(Intercept)        x_kWh
  -0.831304     0.003683
```

```r
summ=summary(model); summ
```

```
Call:
lm(formula = y_kW ~ x_kWh, data = df)

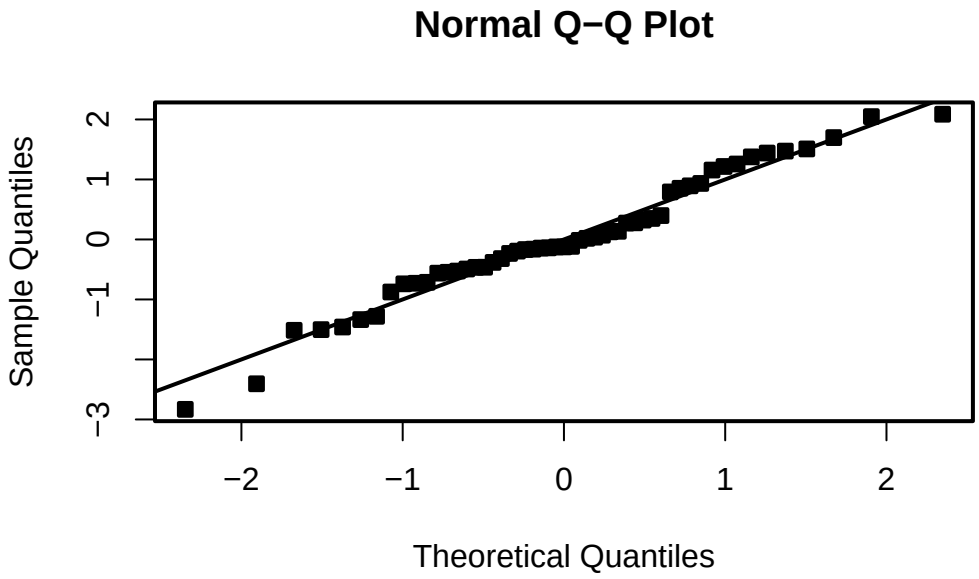
Residuals:
    Min      1Q  Median      3Q     Max
-4.1399 -0.8275 -0.1934  1.2376  3.1522

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.8313037  0.4416121  -1.882   0.0655 .
x_kWh        0.0036828  0.0003339  11.030 4.11e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

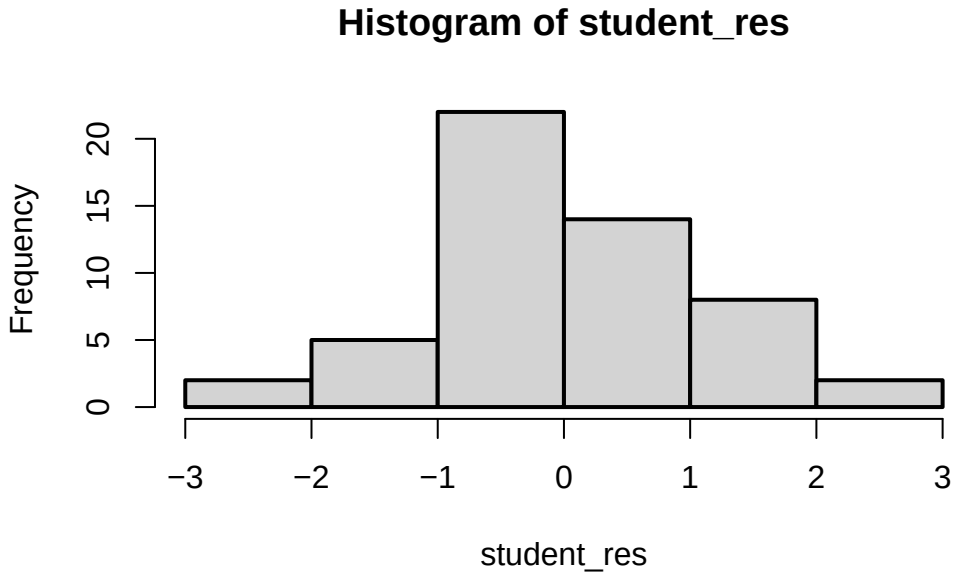
Residual standard error: 1.577 on 51 degrees of freedom
Multiple R-squared:  0.7046,    Adjusted R-squared:  0.6988
F-statistic: 121.7 on 1 and 51 DF,  p-value: 4.106e-15
```

```r
# Now do the residual analysis
# Studentized residuals
student_res=rstudent(model)
MSE=summ$sigma^2

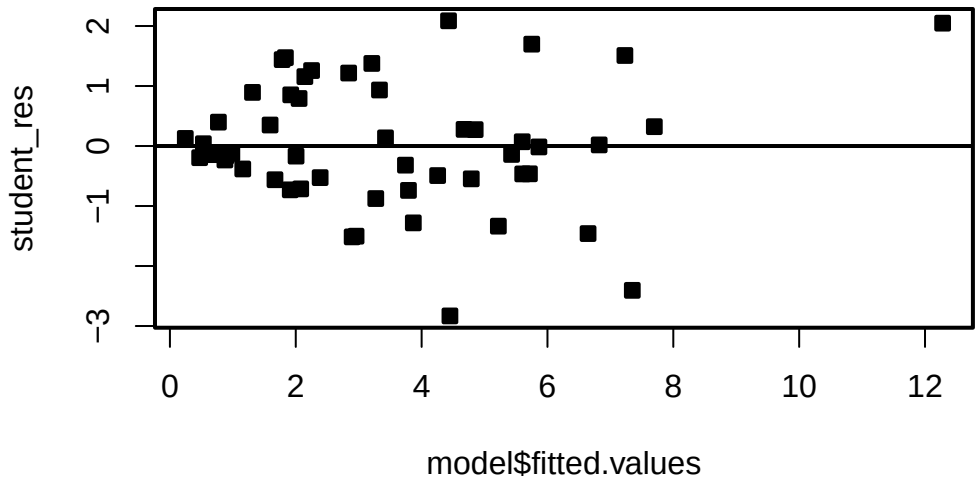
qqnorm(student_res,pch=22,bg=1)
abline(0,1)
```




```
hist(student_res,breaks=6)
```



```
plot(model$fitted.values,student_res,pch=22,bg=1)
abline(h=0)
```



We see that the residual variance increases with the mean of Y. This is easily seen by the fan shape of the residuals in the plot of the residuals against the fitted values.

```
######## Let's try the sqrt transformation

model2=lm(sqrt(y_kW)~x_kWh, df)
model2
```

```
Call:
lm(formula = sqrt(y_kW) ~ x_kWh, data = df)

Coefficients:
(Intercept)        x_kWh
  0.5822259    0.0009529
```

```
summ2=summary(model2); summ2
```

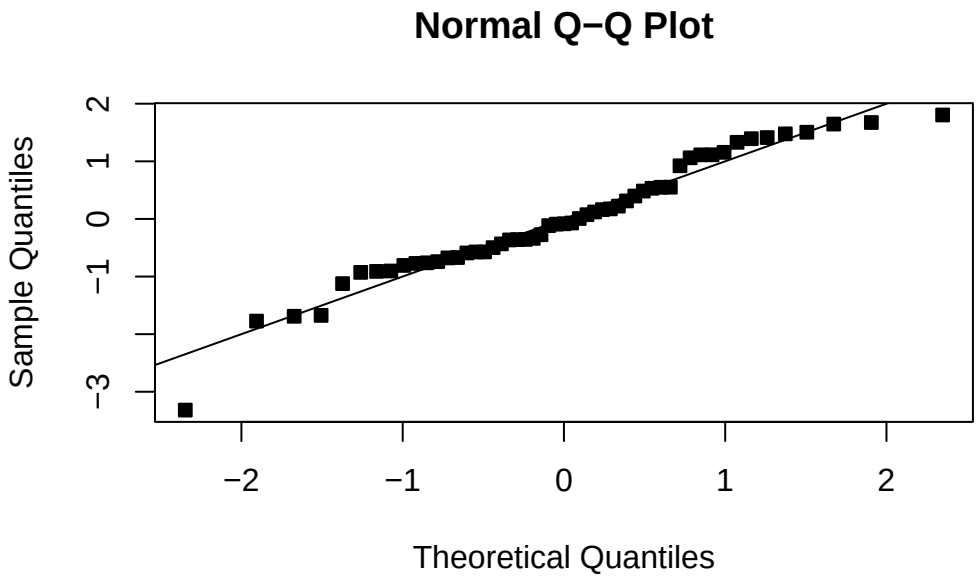
```
Call:
lm(formula = sqrt(y_kW) ~ x_kWh, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-1.39185 -0.30576 -0.03875  0.25378  0.81027

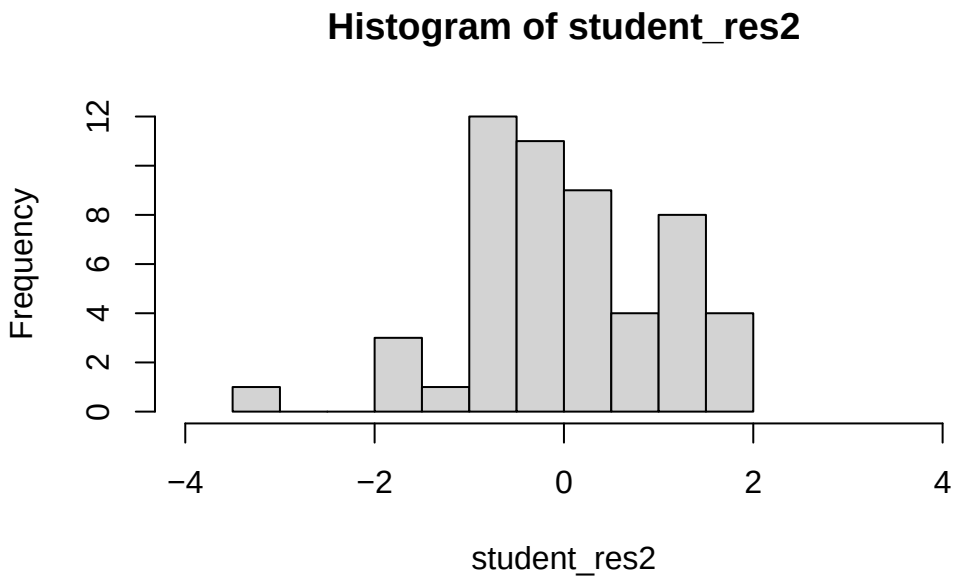
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 5.822e-01  1.299e-01   4.481 4.22e-05 ***
x_kWh       9.529e-04  9.824e-05   9.699 3.61e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.464 on 51 degrees of freedom
Multiple R-squared:  0.6485,    Adjusted R-squared:  0.6416
F-statistic: 94.08 on 1 and 51 DF,  p-value: 3.614e-13
```

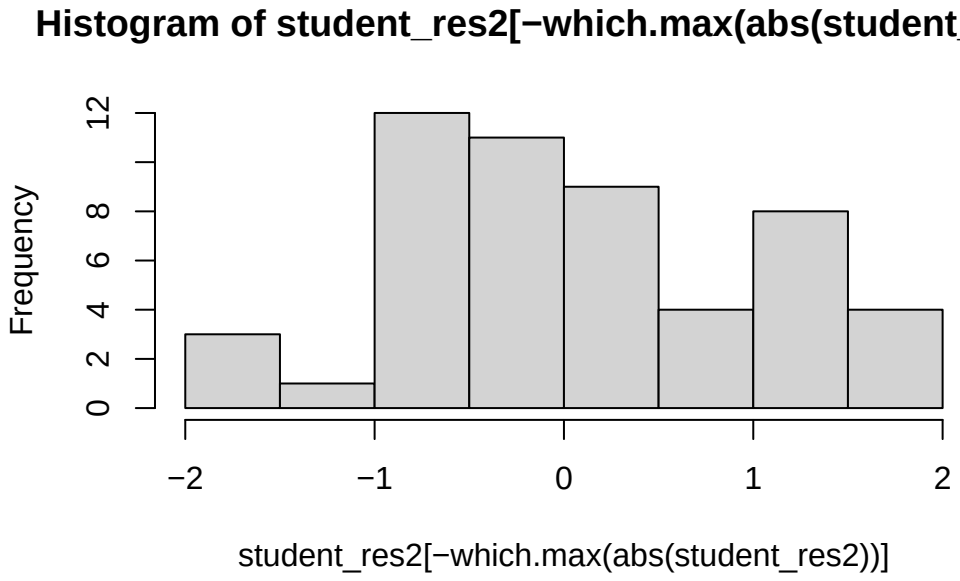
```r
student_res2=rstudent(model2)
MSE2=summ2$sigma^2
qqnorm(student_res2,pch=22,bg=1)
abline(0,1)
```



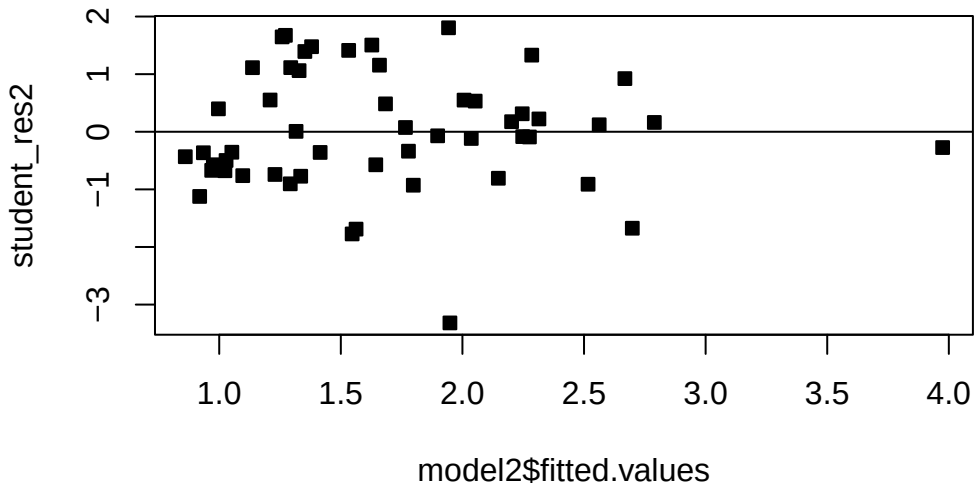
```
hist(student_res2,breaks=10,xlim=c(-4,4))
```



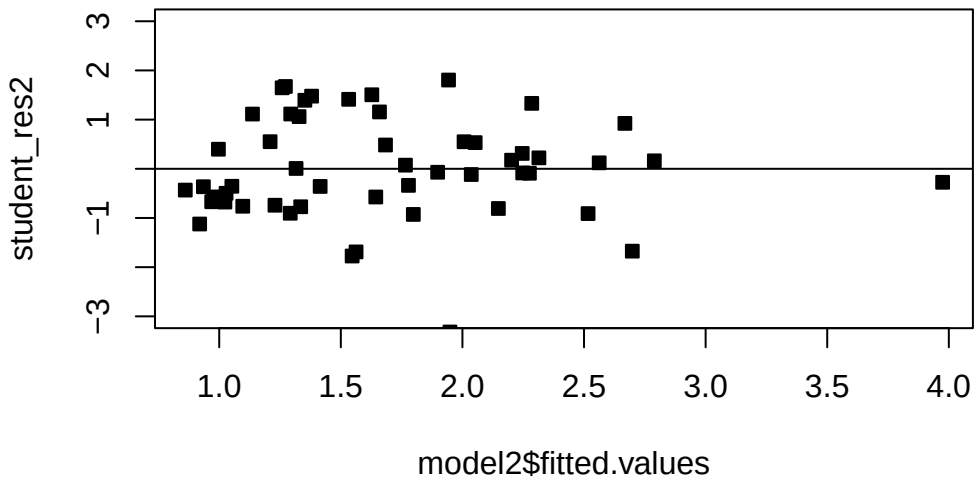

```
hist(student_res2[-which.max(abs(student_res2))],breaks=10,xlim=c(-2,2))
```



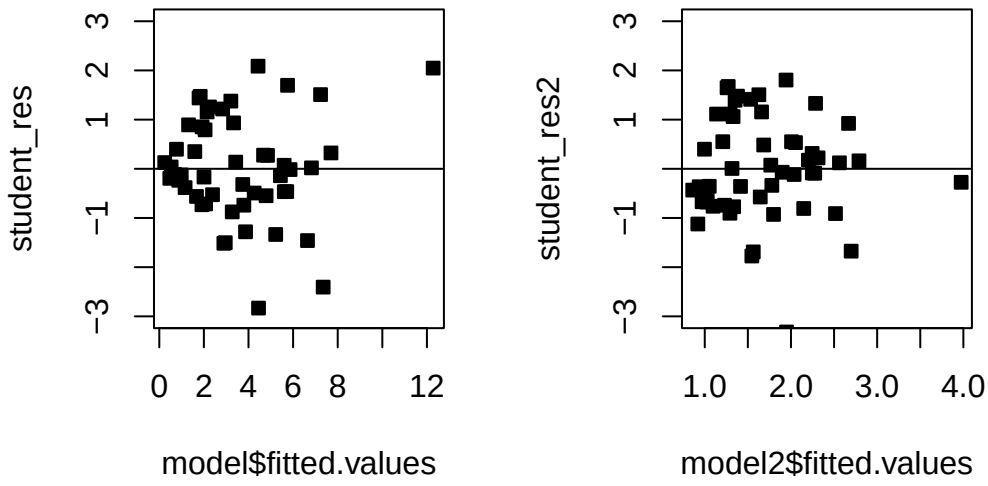
```
plot(model2$fitted.values,student_res2,pch=22,bg=1)
abline(h=0)
```




```
# There is one large outlier skewing the previous plot. Let's rescale and remove.
plot(model2$fitted.values,student_res2,pch=22,bg=1,ylim=c(-3,3))
abline(h=0)
```




```
# Compare!
par(mfrow=c(1,2))
plot(model$fitted.values,student_res,pch=22,bg=1,ylim=c(-3,3))
abline(h=0)
plot(model2$fitted.values,student_res2,pch=22,bg=1,ylim=c(-3,3))
abline(h=0)
```



We see that the transformation has solved the problem. Note that sometimes, even though the square-root transformation may be more suitable, the analyst may opt for the logarithm transform. This is because the log transformation gives a nicer interpretation to the coefficients. In this case, that is not working well, see below:

```
######## Let's try the log transformation
par(mfrow=c(1,1))


model3=lm(log(y_kW)~x_kWh, df)
model3
```

```
Call:
lm(formula = log(y_kW) ~ x_kWh, data = df)

Coefficients:
(Intercept)        x_kWh
  -0.558713     0.001172
```

```
summ3=summary(model3); summ3
```

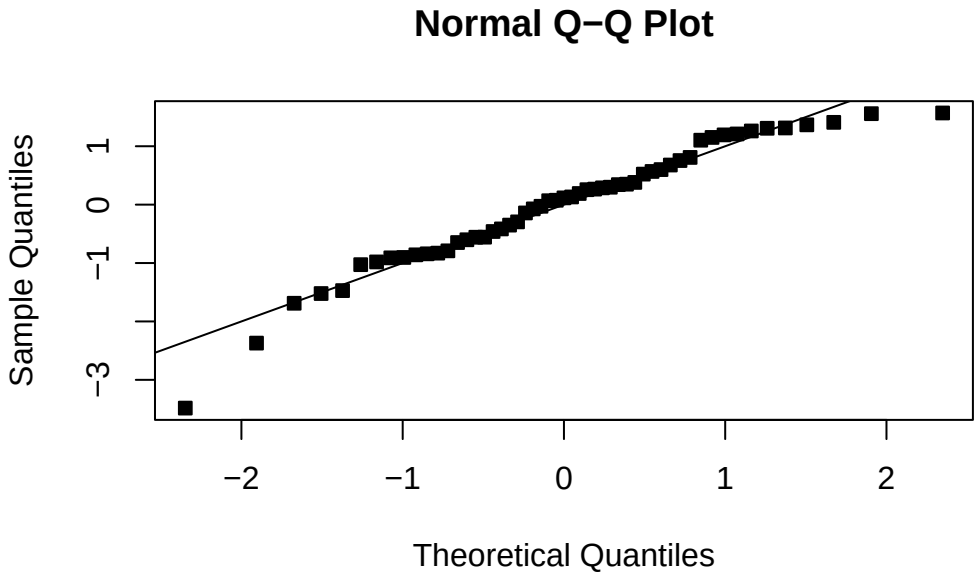
```
Call:
lm(formula = log(y_kW) ~ x_kWh, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-2.29261 -0.47256  0.08414  0.49628  1.12143

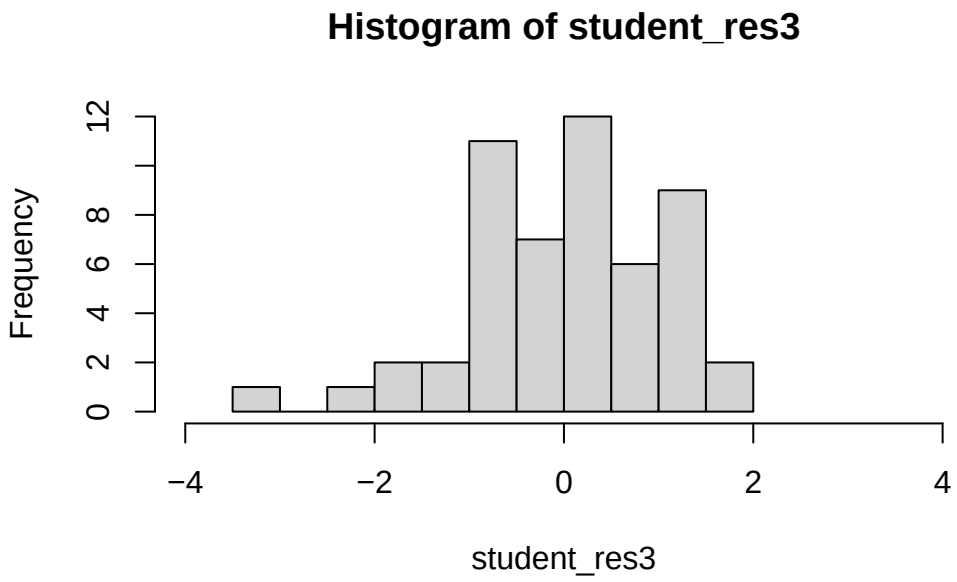
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.5587131  0.2057201  -2.716    0.009 **
x_kWh        0.0011716  0.0001555   7.533 7.86e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.7347 on 51 degrees of freedom
Multiple R-squared:  0.5266,    Adjusted R-squared:  0.5174
F-statistic: 56.74 on 1 and 51 DF,  p-value: 7.862e-10
```

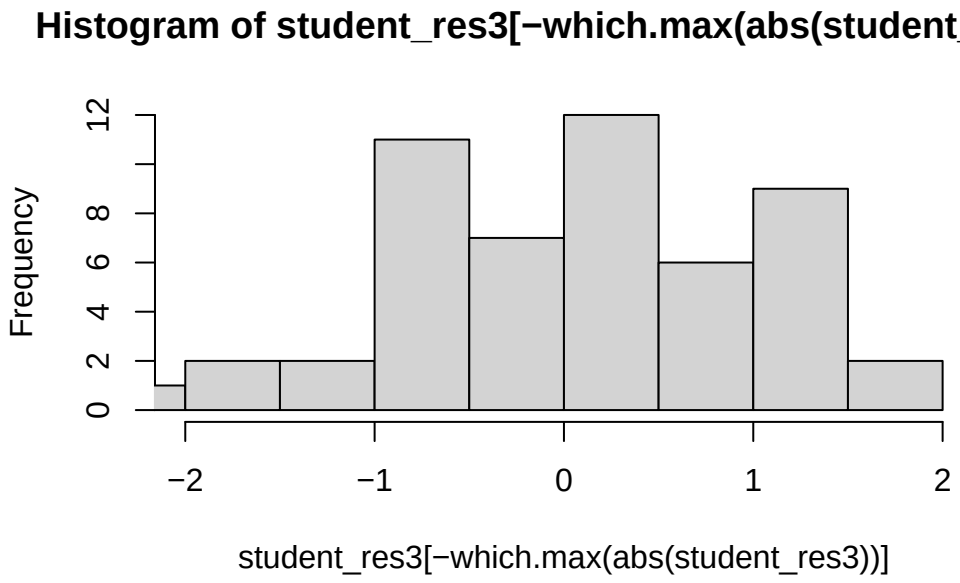
```r
student_res3=rstudent(model3)
MSE3=summ3$sigma^2
qqnorm(student_res3,pch=22,bg=1)
abline(0,1)
```



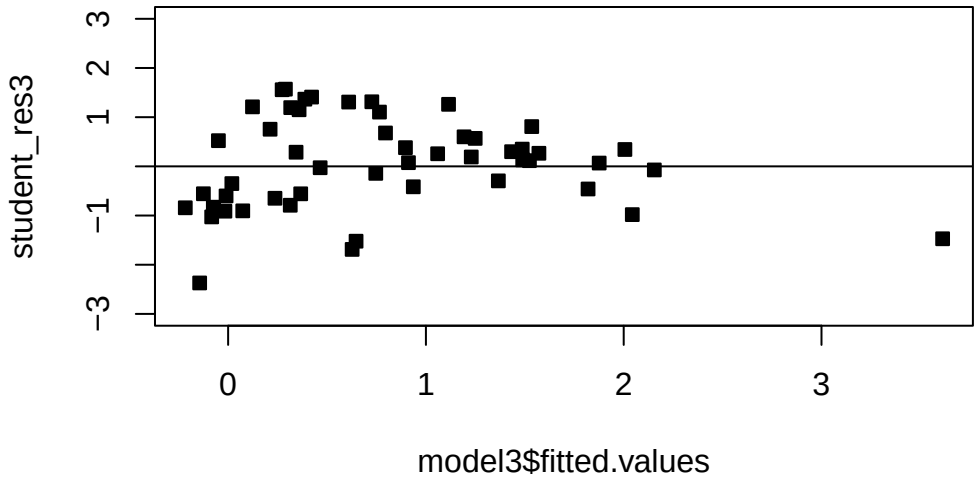
```
hist(student_res3,breaks=10,xlim=c(-4,4))
```




```
hist(student_res3[-which.max(abs(student_res3))],breaks=10,xlim=c(-2,2))
```



```r
plot(model3$fitted.values,student_res3,pch=22,bg=1,ylim=c(-3,3))
abline(h=0)
```



5.2 Linearizing the model

Moving on, we may suspect that the relationship between the regressors and the response is nonlinear, either through empirical evidence or theoretical justification. In some cases a nonlinear function can be linearized by using a suitable transformation. Such nonlinear models are called intrinsically linear. For example, consider the model $Y = \beta_0 e^{\beta_1 X} \epsilon$. Taking the log of both sides yields:

$$\log(Y) = \log(\beta_0) + \beta_1 X + \log(\epsilon).$$

Reparameterizing with $Z = \log(Y)$, $\alpha_0 = \log(\beta_0)$ and $\eta = \log(\epsilon)$, we have that

$$Z = \alpha_0 + \beta_1 X + \eta.$$

If we are willing to assume that η are symmetric about 0 with a constant variance, then we can run a linear regression with the model given above. To get estimates for the original units Y, we can transform back as previously discussed. A model is linearizable if there exists some reparameterization which places the model in the form of the MLR.

Exercise 5.1. Show the following models are linearizable - that is, find the linear reparameterization of the following models: 1. $Y = \beta_0 X_1^{\beta}$ 2. $Y = \beta_0 X^{\beta_1 X}$ 3. $Y = \beta_0 + \log X$ 4. $Y = X/(\beta_0 X - \beta_1)$

Example 5.3. A research engineer is investigating the use of a windmill to generate electricity. He has collected data on the DC output from his windmill and the corresponding wind velocity. See below. Find a well-fitting regression model for this data.

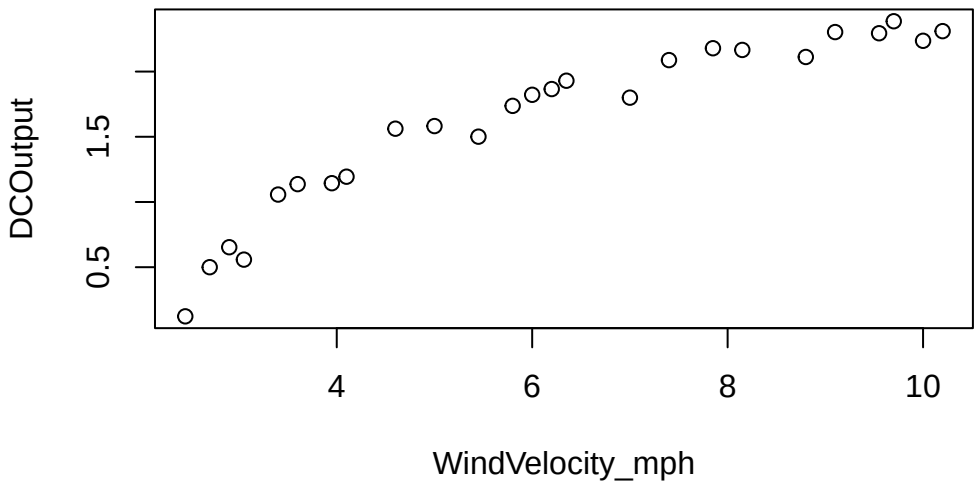
```
############ Windmill data




# Create the data frame
df_wind <- data.frame(
  WindVelocity_mph = c(5.00, 6.00, 3.40, 2.70, 10.00, 9.70, 9.55, 3.05, 8.15, 6.20,
                       2.90, 6.35, 4.60, 5.80, 7.40, 3.60, 7.85, 8.80, 7.00, 5.45,
                       9.10, 10.20, 4.10, 3.95, 2.45),
  DCOutput = c(1.582, 1.822, 1.057, 0.500, 2.236, 2.386, 2.294, 0.558, 2.166, 1.866,
               0.653, 1.930, 1.562, 1.737, 2.088, 1.137, 2.179, 2.112, 1.800, 1.501,
               2.303, 2.310, 1.194, 1.144, 0.123)
)

###############

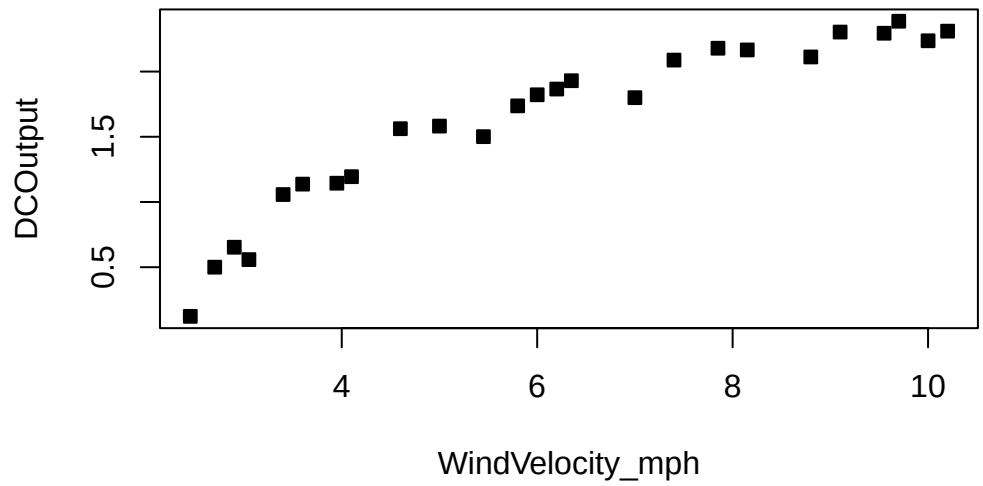

par(mfrow=c(1,1))
plot(df_wind)
```




```
summary(df_wind)
```

```
 WindVelocity_mph    DCOutput
 Min.   : 2.450   Min.   :0.123
 1st Qu.: 3.950   1st Qu.:1.144
 Median : 6.000   Median :1.800
 Mean   : 6.132   Mean   :1.610
 3rd Qu.: 8.150   3rd Qu.:2.166
 Max.   :10.200   Max.   :2.386
```

```
plot(df_wind,pch=22,bg=1)
```




```
model=lm(DCOutput~WindVelocity_mph, df_wind)
model
```

```

Call:
lm(formula = DCOutput ~ WindVelocity_mph, data = df_wind)

Coefficients:
     (Intercept)  WindVelocity_mph
          0.1309            0.2411
```

```
summ=summary(model); summ
```

```

Call:
lm(formula = DCOutput ~ WindVelocity_mph, data = df_wind)

Residuals:
     Min       1Q   Median       3Q      Max
-0.59869 -0.14099  0.06059  0.17262  0.32184

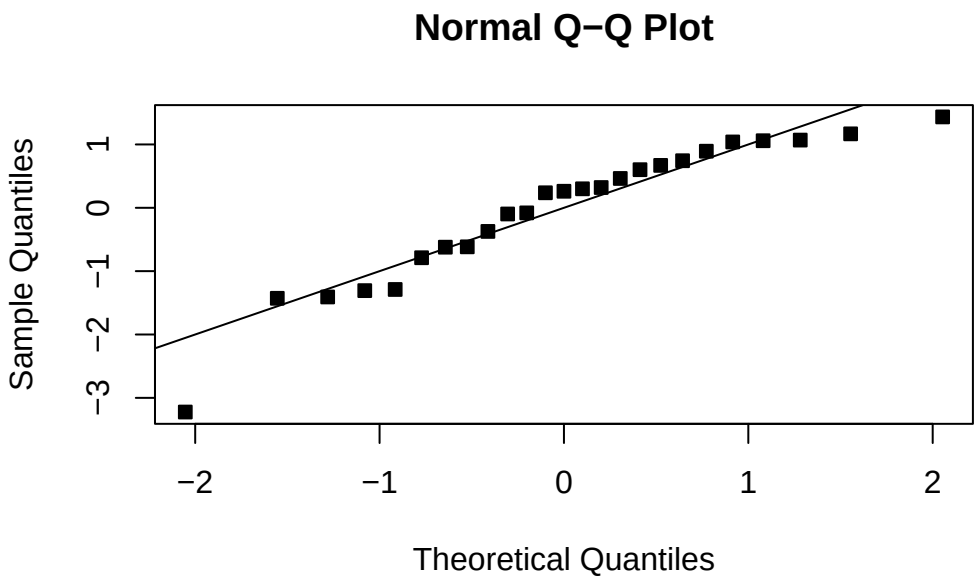
Coefficients:
```

```
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)       0.13088    0.12599   1.039     0.31
WindVelocity_mph  0.24115    0.01905  12.659 7.55e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

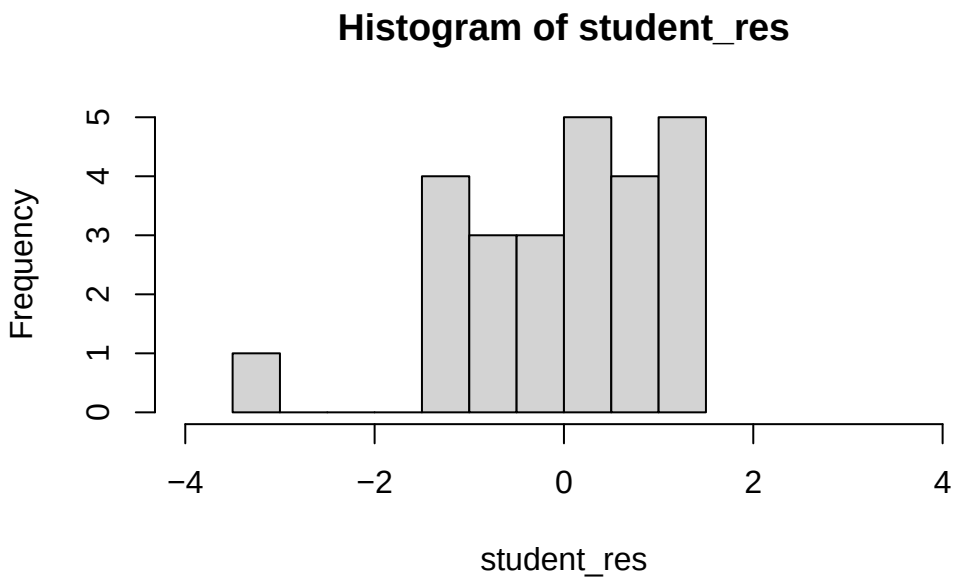
Residual standard error: 0.2361 on 23 degrees of freedom
Multiple R-squared:  0.8745,    Adjusted R-squared:  0.869
F-statistic: 160.3 on 1 and 23 DF,  p-value: 7.546e-12
```

```r
student_res=rstudent(model)

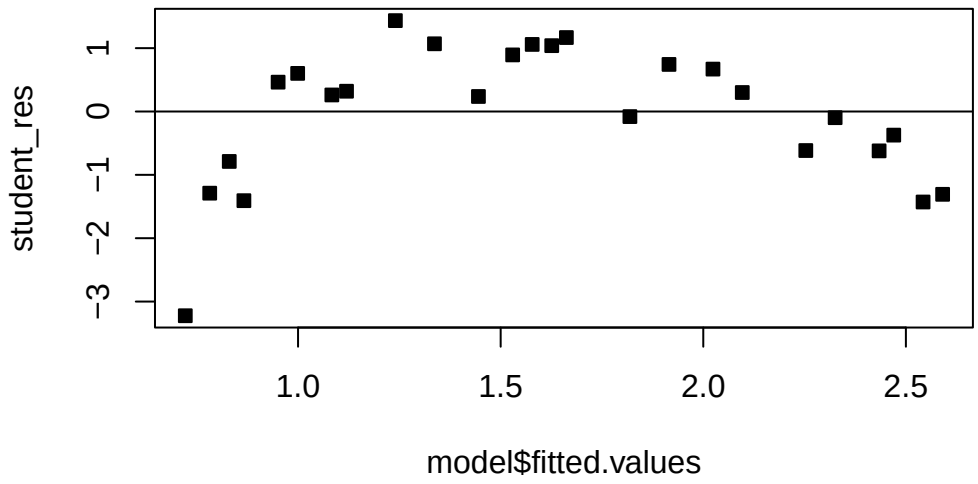
MSE=summ$sigma^2
qqnorm(student_res,pch=22,bg=1)
abline(0,1)
```




```r
hist(student_res,breaks=10,xlim=c(-4,4))
```

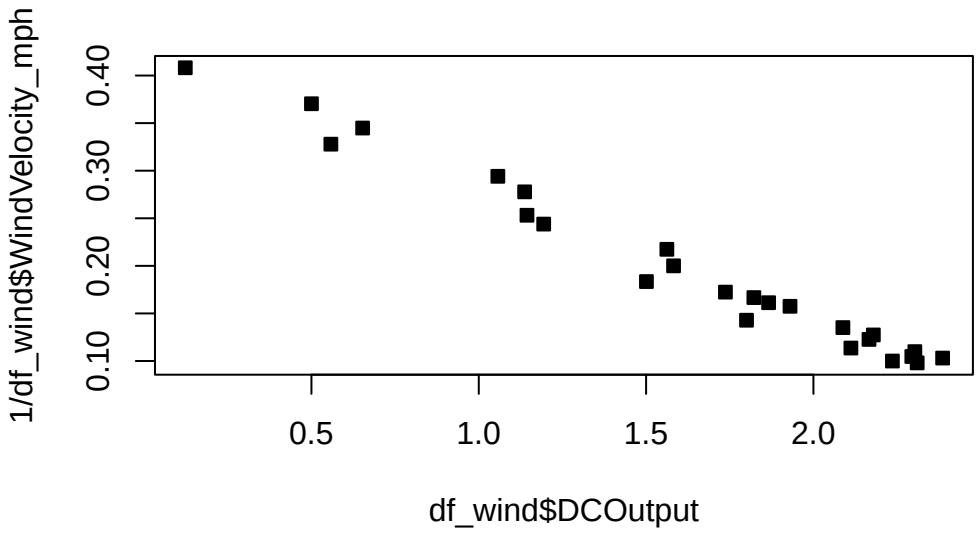



```
plot(model$fitted.values,student_res,pch=22,bg=1)
abline(h=0)
```



The fit is not good. Looking at the scatterplot, we might initially consider using a quadratic model to account for the pictured curvature. However, the scatterplot suggests that as wind speed increases, DC output approaches an upper limit of approximately 2.5. This is also consistent with the theory of windmill operation. Since the quadratic model will eventually bend downward as wind speed increases, it would not be appropriate for these data. A more reasonable model for the windmill data that incorporates an upper asymptote would be based on $1/X$.

```
plot(df_wind$DCOutput,1/df_wind$WindVelocity_mph,pch=22,bg=1)
```




```
# plot(df$DCOutput,log(df$WindVelocity_mph))
df_wind$WindVelocity_mph_inv=1/df_wind$WindVelocity_mph
model2=lm(DCOutput~WindVelocity_mph_inv, df_wind)
model2
```

```
Call:
lm(formula = DCOutput ~ WindVelocity_mph_inv, data = df_wind)

Coefficients:
         (Intercept)  WindVelocity_mph_inv
               2.979                -6.935
```

```r
summ=summary(model2); summ
```

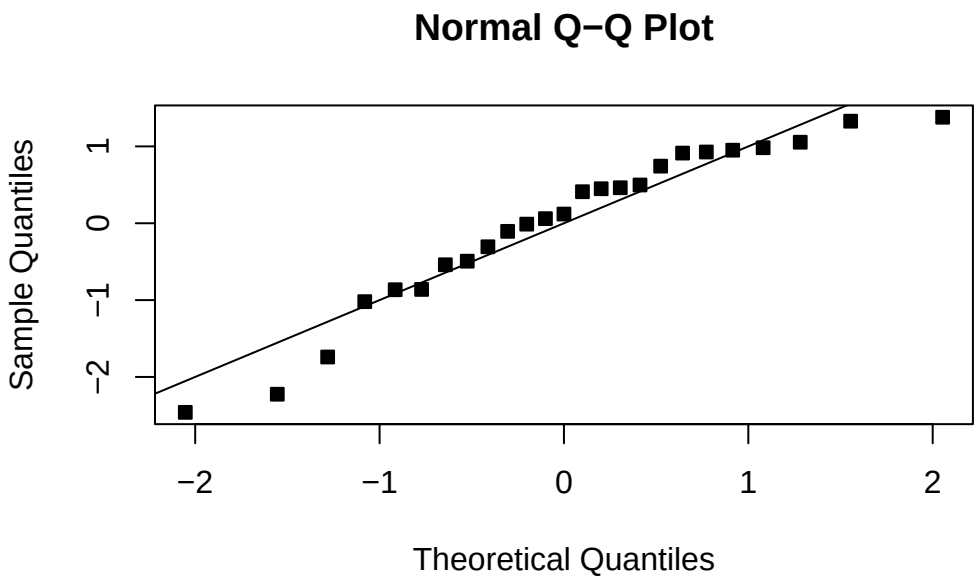
```
Call:
lm(formula = DCOutput ~ WindVelocity_mph_inv, data = df_wind)

Residuals:
     Min       1Q   Median       3Q      Max
-0.20547 -0.04940  0.01100  0.08352  0.12204

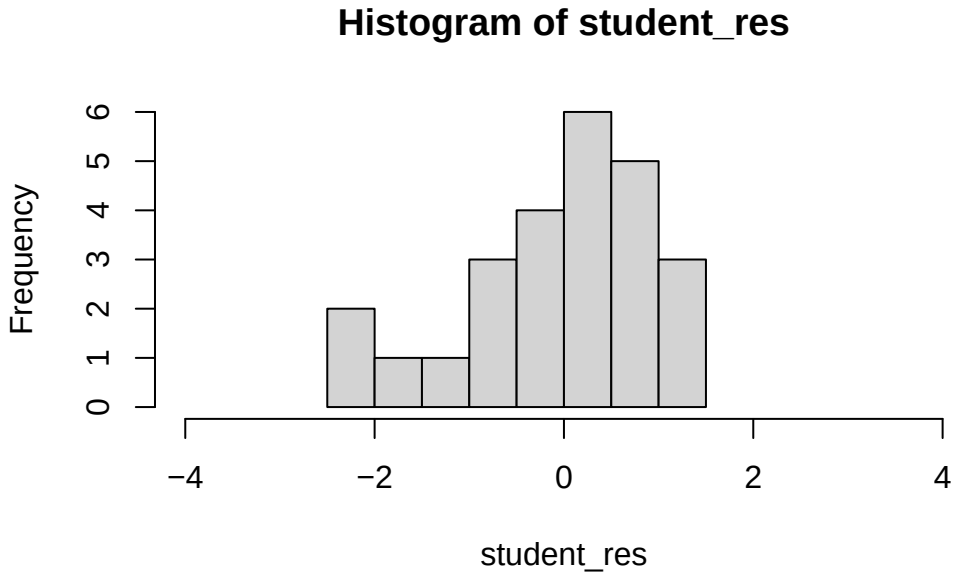
Coefficients:
                     Estimate Std. Error t value Pr(>|t|)
(Intercept)            2.9789     0.0449   66.34   <2e-16 ***
WindVelocity_mph_inv  -6.9345     0.2064  -33.59   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.09417 on 23 degrees of freedom
Multiple R-squared:   0.98, Adjusted R-squared:  0.9792
F-statistic:  1128 on 1 and 23 DF,  p-value: < 2.2e-16
```

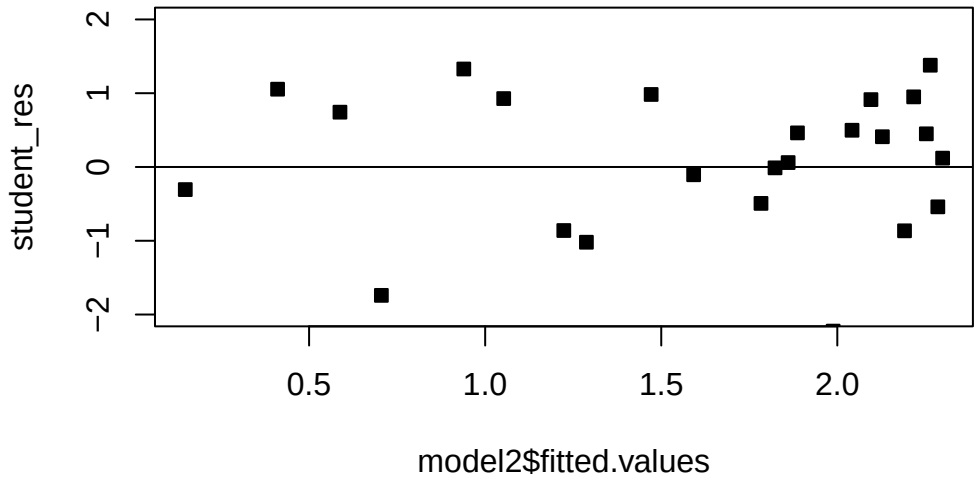
```r
student_res=rstudent(model2)
MSE=summ$sigma^2
qqnorm(student_res,pch=22,bg=1)
abline(0,1)
```




```
hist(student_res,xlim=c(-4,4))
```



```r
plot(model2$fitted.values,student_res,pch=22,bg=1,ylim=c(-2,2))
abline(h=0)
```



5.3 Box Cox Transformations

One technique is to use the data to estimate which transformation is best, a popular instance is the Box-Cox transformation. Consider the class of transformations: $\{y^\lambda : \lambda \in \mathbb{R}\}$. The regression model and λ can be estimated simultaneously using the method of maximum likelihood. Recall that we used the method of least squares to estimate the model parameters - maximum likelihood is an alternative estimation strategy. Think of λ like an extra model parameter, on top of β and σ that we need to estimate.

Let

$$\tilde{y} = \log^{-1}(1/n \sum_{i=1}^{n} \log y_i)$$

$$y_\lambda = \begin{cases} \dfrac{y^\lambda - 1}{\lambda \tilde{y}^{\lambda-1}} & \lambda \neq 0 \\ \tilde{y} \log y & \lambda = 0 \end{cases}.$$

We then fit the following model

$$y_\lambda = X\beta + \epsilon.$$

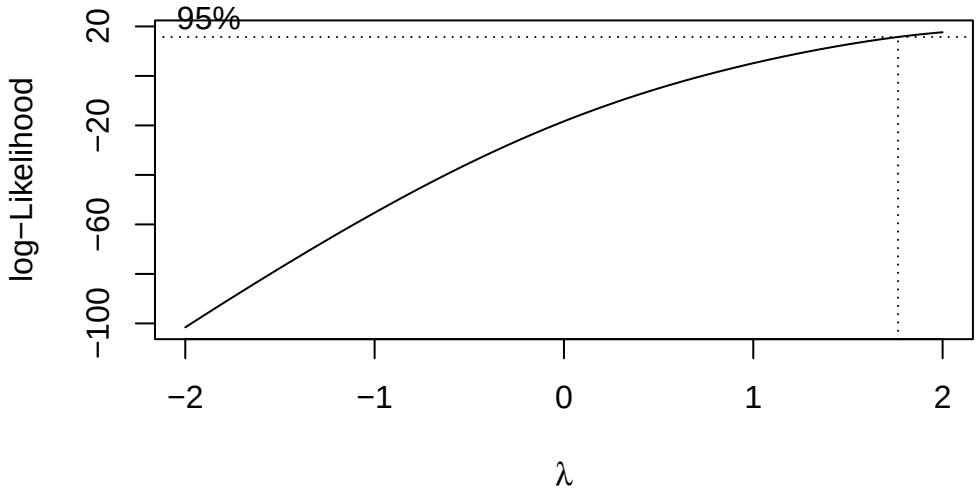
Even though $y^\lambda \neq y_\lambda$, we use y^λ (or $\log y$ if $\lambda = 0$) as the final response - as it is more interpretable. It is entirely acceptable to use y_λ as the response for the final model - this model will have a scale difference and an origin shift in comparison to the model using y^λ (or $\log y$). Usually the final λ used in the model is rounded to a nice number for interpretation. A computational procedure is used for estimating λ, which we will not cover here. In general, we can compute a confidence interval for λ, and if it contains 1 then we may not need to transform.

Example 5.4. Let's apply the Box-Cox transformation to the two previous examples.

The R function `boxcox` from the `MASS` package can be used to execute the Box-Cox transformation. It requires you to specify a grid of points for λ, given below by `seq(-2, 2, 1/10)`. We can set the `plotit` parameter to `TRUE` in order to see if this grid is big enough. We should see a peak or mode in the log-likelihood function that is plotted. If we don't, we can expand the grid on the side which has the largest value of the log-likelihood. Observe below that we need to include points higher than 2 on the grid, as the function is still increasing for at $\lambda = 2$:

```
# You need the MASS package.
# install.packages('MASS')

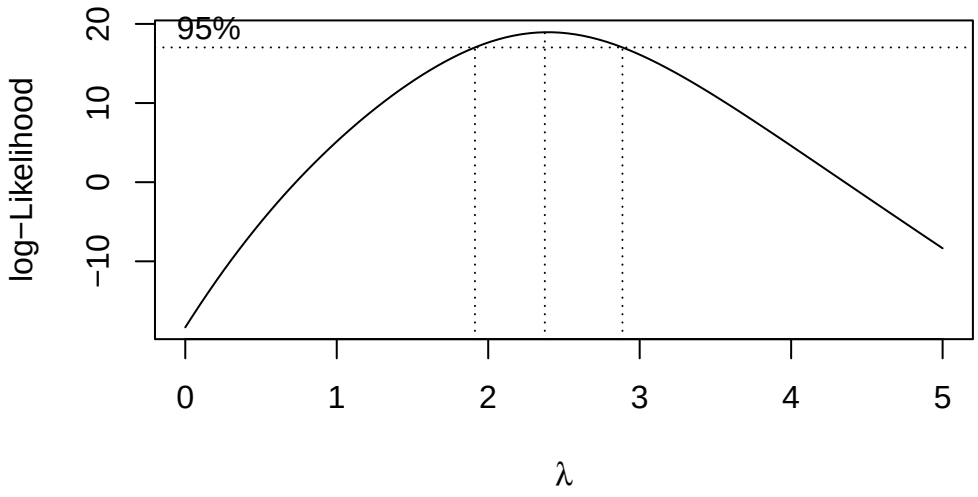
bc=MASS::boxcox(DCOutput~WindVelocity_mph,data=df_wind,
       lambda = seq(-2, 2, 1/10),
       plotit = TRUE,
       eps = 1/50,
       xlab = expression(lambda),
       ylab = "log-Likelihood")
```




```
# bc

#Observe that

bc=MASS::boxcox(DCOutput~WindVelocity_mph,data=df_wind,
                lambda = seq(0, 5, 1/10),
                plotit = TRUE,
                eps = 1/50,
                xlab = expression(lambda),
                ylab = "log-Likelihood")
```




```
# bc

#Seems like we should try lambda=2
```

The confidence interval goes from just below 2 to just below 3. Let's pick a round number, and try the transformation $\lambda = 2$.

```
# plot(df$DCOutput,log(df$WindVelocity_mph))
model3=lm(DCOutput^2~WindVelocity_mph, df_wind)
model3
```

```
Call:
lm(formula = DCOutput^2 ~ WindVelocity_mph, data = df_wind)

Coefficients:
     (Intercept)  WindVelocity_mph
         -1.3585            0.7107
```

```
summ3=summary(model3); summ3
```

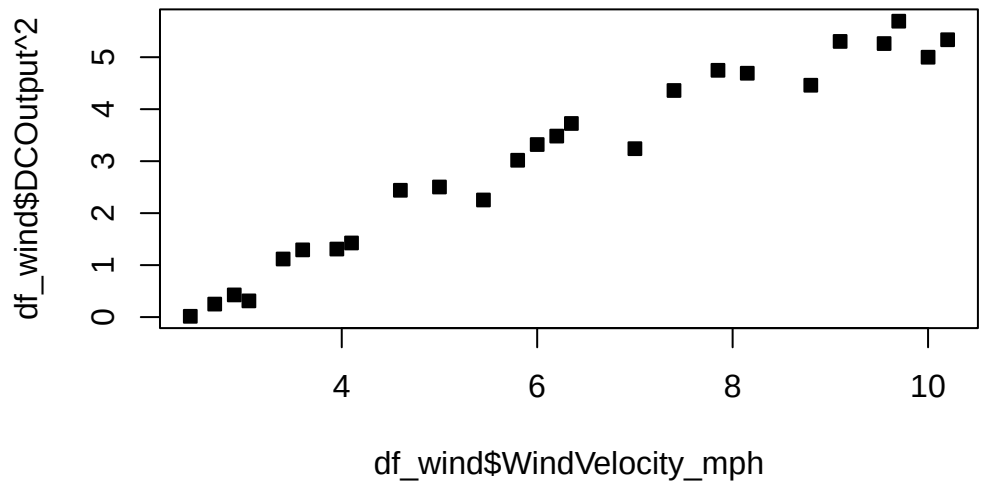
```
Call:
lm(formula = DCOutput^2 ~ WindVelocity_mph, data = df_wind)

Residuals:
     Min       1Q   Median       3Q      Max
-0.74840 -0.31027  0.05951  0.30793  0.57072

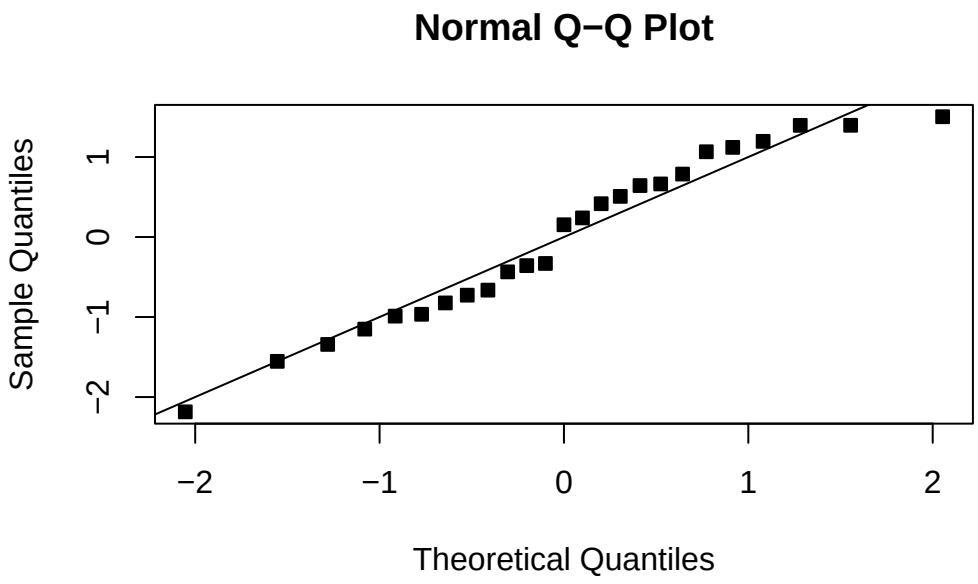
Coefficients:
                 Estimate Std. Error t value Pr(>|t|)
(Intercept)      -1.35851    0.21239  -6.396 1.58e-06 ***
WindVelocity_mph  0.71066    0.03211  22.130  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3979 on 23 degrees of freedom
Multiple R-squared:  0.9551,    Adjusted R-squared:  0.9532
F-statistic: 489.7 on 1 and 23 DF,  p-value: < 2.2e-16
```

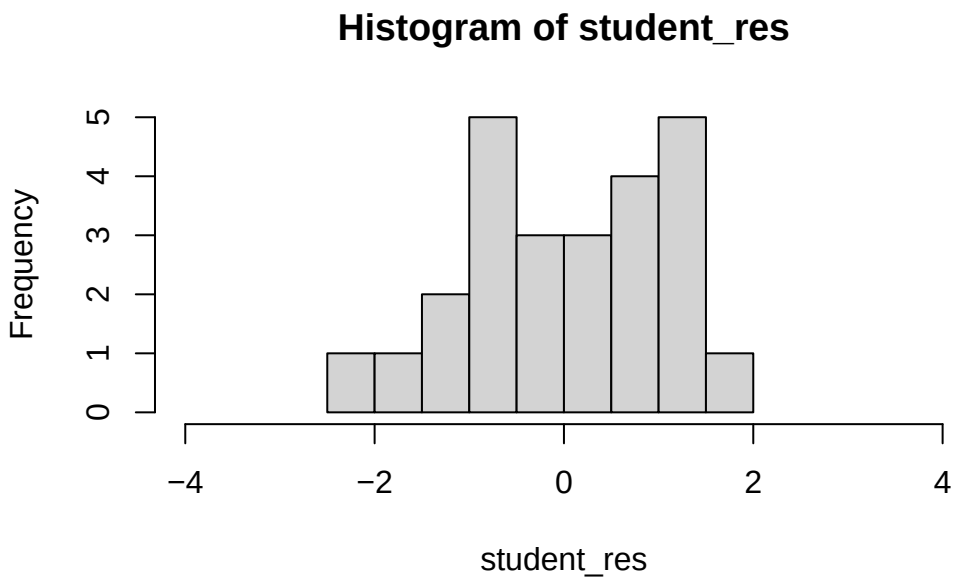
```
plot(df_wind$WindVelocity_mph,df_wind$DCOutput^2,pch=22,bg=1)
```



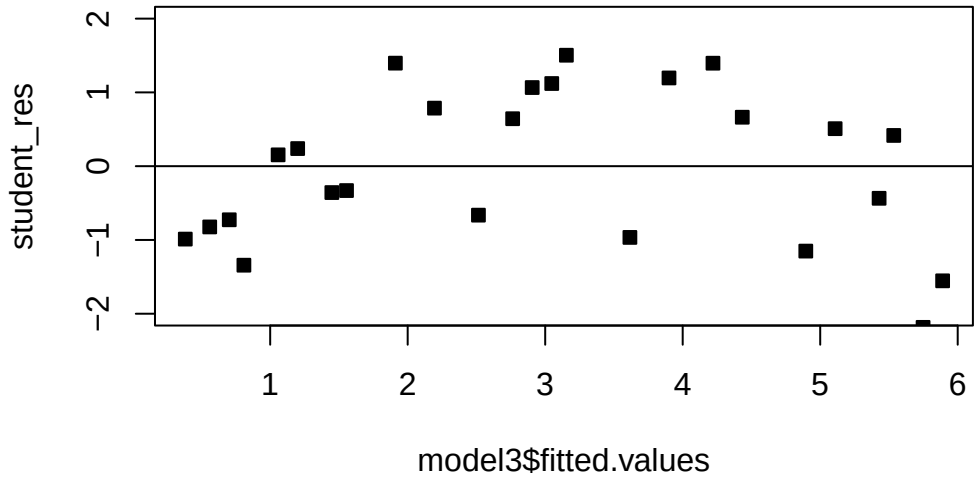
```r
student_res=rstudent(model3)
MSE=summ3$sigma^2
qqnorm(student_res,pch=22,bg=1)
abline(0,1)
```




```r
hist(student_res,xlim=c(-4,4))
```




```
plot(model3$fitted.values,student_res,pch=22,bg=1,ylim=c(-2,2))
abline(h=0)
```



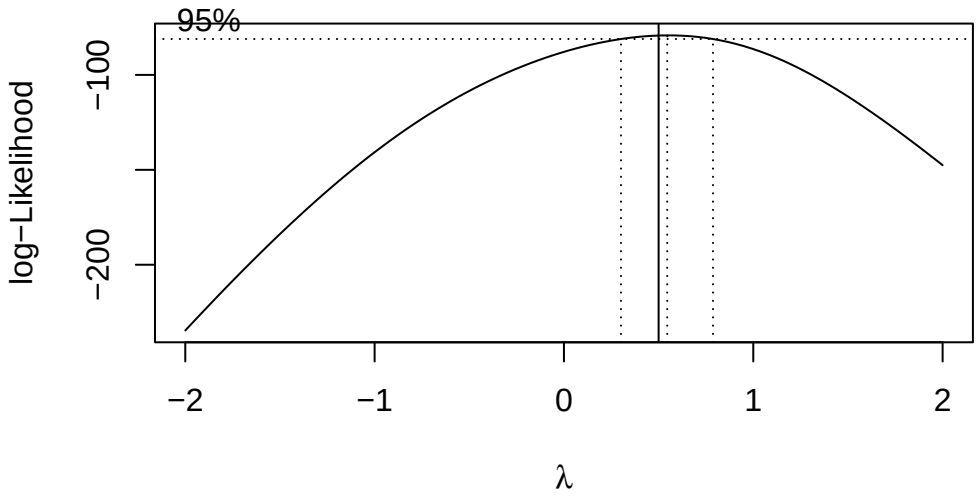
The fit is not bad. The R^2 is very high. There is a pattern in the QQplot and a slight pattern in the residuals plot. For knowing nothing about wind velocity, it is not bad.

The electricity data clearly points to the square root transformation - matching the analysis we did previously.

```
## Electricity

model=lm(y_kW~x_kWh, df)

bc=MASS::boxcox(y_kW~x_kWh,data=df,
                lambda = seq(-2, 2, 1/10),
                plotit = TRUE,
                eps = 1/50,
                xlab = expression(lambda),
                ylab = "log-Likelihood")
abline(v=0.5)
```




```
# bc
```

5.4 Homework stop

Complete the assigned Chapter 5 questions and complete assignment 2.

6 Indicator Variables

Often we will have regressors that are categorical. We now discuss how to include those in a regression model. In general, categorical variables can be included in a regression model via **indicator variables**.

6.1 What are indicator variables?

If a regressor has two categories A and B, that regressor can be included in the model as

$$z = \begin{cases} 0 & \text{if the observation is type A} \\ 1 & \text{if the observation is type B} \end{cases}$$

Sometimes people choose

$$z = \begin{cases} -1 & \text{if the observation is type A} \\ 1 & \text{if the observation is type B} \end{cases}.$$

The variable z is an indicator variable. Indicator variables are in numeric form, and can therefore be included in the design matrix X in the usual way we do for continuous regressors.

Example 6.1. Let's recall Example 3.1 and suppose we have some new data as follows:

It is difficult to accurately determine a person's body fat percentage without immersing them in water. However, we can easily obtain the weight of a person. A researcher would like to know if weight and body fat percentage are related? They also suspect that sex plays a role in the prediction. This researcher collected the following data:

Individual	1	2	3	4	5	6	7	8	9	10
Weight (lb)	175	181	200	159	196	192	205	173	187	188
Body Fat (%)	6	21	15	6	22	31	32	21	25	30
Sex	F	M	F	F	M	F	F	M	M	F
Individual	11	12	13	14	15	16	17	18	19	20

Individual	1	2	3	4	5	6	7	8	9	10
Weight (lb)	188	240	175	168	246	160	215	159	146	219
Body Fat (%)	10	20	22	9	38	10	27	12	10	28
Sex	F	F	M	M	F	F	M	F	F	M

Write out the appropriate indicator variable for Sex. Interpret the resulting regression equation for regressing Body fat against weight and sex. Interpret the coefficient that corresponds to the Sex variable.

We have that

$$X_{2i} = \begin{cases} 0 & \text{if the subject } i \text{ is male} \\ 1 & \text{if the subject } i \text{ is female} \end{cases}.$$

The regression equation is then

$$Y_i|X = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \epsilon_i,$$

where X_{i1} is the weight of individual i.

When $X_{i2} = 0$, then the regression equation for males is given by $Y_i|X = \beta_0 + \beta_1 X_{i1} + \epsilon_i$. Therefore, the expected body fat percentage for males is $\mathrm{E}[Y_i|X] = \beta_0 + \beta_1 X_{i1}$. Additionally, when $X_{i2} = 1$, then the regression equation for females is given by $Y_i|X = \beta_0 + \beta_1 X_{i1} + \beta_2 + \epsilon_i$. It follows that the expected body fat percentage for females is $\mathrm{E}[Y_i|X] = \beta_0 + \beta_1 X_{i1} + \beta_2$. Thus, we have that the expected body fat percentage for females is β_2 higher than for males, holding weight constant. This is the interpretation of the coefficient for the dummy variable in this case. Observe that for males and females, the regression lines are parallel. The model says that sex accounts for a constant shift in your expected body fat, but the slope (the coefficient for weight) of the regression line remains the same.

Let's observe.

```
# Make the data frame
Weight=c(175 , 181 , 200 , 159 , 196 , 192 , 205 , 173 , 187 , 188 ,
         188 , 240 , 175 , 168 , 246 , 160 , 215 , 159 , 146 , 219 )
BodyFat =c(6 , 21 , 15 , 6 , 22 , 31 , 32 ,21 , 25 , 30 ,
           10 , 20 , 22 , 9 , 38 , 10 , 27 , 12 , 10 , 28 )
Sex=c("F","M","F","F","M","F","F","M","M","F","F","F","M","M","F","F","M","F","F","M")

df=data.frame(Weight=Weight,BodyFat=BodyFat,Sex=Sex,stringsAsFactors = T)

df$Sex=relevel(df$Sex,"M")
```

```r
mod=lm(BodyFat~Weight+Sex,data=df)
summary(mod)
```

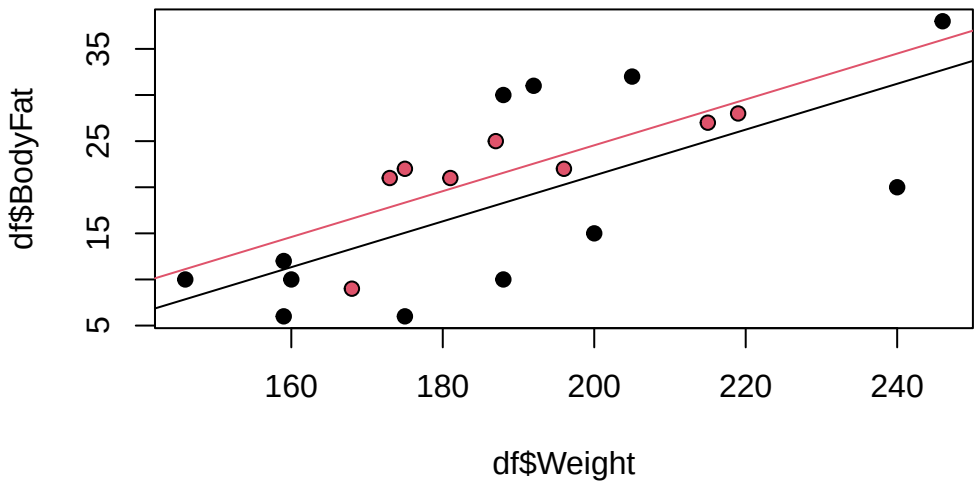
```
Call:
lm(formula = BodyFat ~ Weight + Sex, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-11.2198  -5.3804  -0.1767   3.6719  11.7136

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -25.17526   11.73681  -2.145 0.046695 *
Weight        0.24861    0.06061   4.102 0.000743 ***
SexF         -3.27233    3.21493  -1.018 0.323014
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.042 on 17 degrees of freedom
Multiple R-squared:  0.5149,    Adjusted R-squared:  0.4578
F-statistic: 9.021 on 2 and 17 DF,  p-value: 0.002137
```

```r
plot(df$Weight,df$BodyFat,bg=((df$Sex=="M")+1),pch=21)
abline(coef(mod)[1],coef(mod)[2],col=2)
abline(coef(mod)[1]+coef(mod)[3],coef(mod)[2],col=1)
```



We can generalize this idea out of this example. In the case of two categories, the interpretation of the coefficient for the dummy variable is given as follows: Holding other regressors constant, on average, the change in response attributed to the case where the dummy variable is 1, relative to the case where the dummy variable is 0, is given by the coefficient for the dummy variable.

Moving on, a regressor that has k categories can be represented by $k-1$ indicator variables:

x_1	x_2	...	x_{k-1}	Category
0	0	...	0	1
1	0	...	0	2
0	1	...	0	3
⋮	⋮	...	⋮	⋮
0	0	...	1	k

In this case, the category, or level, where all dummy variables are equal to 0 is the **reference category**. The reference category is the baseline we will compare all other categories to. You may want to choose this carefully. In this case, the interpretation of each of the $k-1$ coefficients is going to be as follows: Holding other regressors constant, on average, the change in response attributed to the case where the dummy variable corresponding to the coefficient is 1, relative to the reference category, is given by the coefficient for the dummy variable. Note that the reference category has no variable associated with it.

Example 6.2. When evaluating factors that affect the price of real estate, we may wish to consider location, while adjusting for lot size, year built and finished square feet. The data set `clean_data.csv` contains the prices of various types of real estate, as well as several important regressors. Regress the sale price on location, lot size, year built and finished square feet. Interpret the coefficient related to District 14. According to the model, holding other variables constant, what district has the highest priced properties? the lowest? Observe that District 7 has a non significant coefficient. In this case, what does it mean for District 7 to have a coefficient of 0?

```
####### Packages needed  ###############

library(lubridate)
```

```
Warning: package 'lubridate' was built under R version 4.5.3
```

```
Attaching package: 'lubridate'
```

```
The following objects are masked from 'package:base':

    date, intersect, setdiff, union
```

```
# Example: We would like to see how sale price of a home is related to
# various factors

####### Loading data  ###############

df_clean2=read.csv('clean_data.csv',stringsAsFactors = T)
df_clean2$District=as.factor(df_clean2$District)

# The first level is the reference category
attributes(df_clean2$District)$levels[1]
```

```
[1] "1"
```

```
####### Fitting the model  ###############
model=lm(Sale_price~District+Fin_sqft+Lotsize+Year_Built,df_clean2)

summ=summary(model); summ
```

```
Call:
lm(formula = Sale_price ~ District + Fin_sqft + Lotsize + Year_Built,
    data = df_clean2)

Residuals:
    Min      1Q  Median      3Q     Max
-399923  -25360     426   23383 1580056

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -9.170e+05  4.175e+04 -21.963  < 2e-16 ***
District2    1.335e+04  2.312e+03   5.772 7.92e-09 ***
District3    1.815e+05  2.462e+03  73.723  < 2e-16 ***
District4   -3.525e+04  4.777e+03  -7.378 1.66e-13 ***
District5    5.552e+04  1.988e+03  27.923  < 2e-16 ***
District6    1.202e+04  2.849e+03   4.218 2.47e-05 ***
District7   -4.300e+03  2.482e+03  -1.732  0.08327 .
District8    1.732e+04  2.708e+03   6.396 1.62e-10 ***
District9    2.810e+04  2.474e+03  11.361  < 2e-16 ***
District10   6.363e+04  2.073e+03  30.699  < 2e-16 ***
District11   7.032e+04  1.990e+03  35.333  < 2e-16 ***
District12   1.014e+04  3.240e+03   3.129  0.00175 **
District13   6.877e+04  2.057e+03  33.430  < 2e-16 ***
District14   1.026e+05  2.086e+03  49.212  < 2e-16 ***
District15  -3.375e+04  3.050e+03 -11.066  < 2e-16 ***
Fin_sqft     6.519e+01  6.525e-01  99.899  < 2e-16 ***
Lotsize      3.906e+00  1.228e-01  31.812  < 2e-16 ***
Year_Built   4.506e+02  2.153e+01  20.930  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 55680 on 24604 degrees of freedom
Multiple R-squared:  0.5889,    Adjusted R-squared:  0.5886
F-statistic:  2073 on 17 and 24604 DF,  p-value: < 2.2e-16
```

```
coef(model)[which.max(coef(model))]
```

```
District3
 181495.9
```

```
coef(model)[order(coef(model))[1:3]]
```

```
(Intercept)   District4  District15
 -916960.88   -35245.83   -33748.17
```

1. Interpret the coefficient for District 14: Holding lot size, finished square feet and year built constant, on average, the change in the price of a property in District 14, relative to District 1, is 102 600.
2. According to the model, holding other variables constant, what district has the highest priced properties on average? the lowest? The highest coefficient is District 3, and it is positive, so,holding other variables constant, on average District 3 has the highest priced properties. The lowest coefficient is District 4, and it is negative, so, holding other variables constant, on average District 4 has the lowest priced properties.
3. Observe that District 7 has a non significant coefficient. In this case, what does it mean for District 7 to have a coefficient of 0? This means that there is not enough evidence to show that District 1 and District 7 have different prices, holding other variables constant, on average.

> **Note**
>
> If all coefficients for the dummy variables are positive, then the reference category has the lowest average value of the response. Analogously, if all coefficients for the dummy variables are negative, then the reference category has the highest average value of the response. Why? Use the interpretation of the coefficients to answer this question.

> **Note**
>
> ANOVA is Regression!- In one-way ANOVA, recall that we test for a difference in group means for a continuous response. We can represent the treatment groups with dummy variables and view this as a regression problem. That is, regressing the outcome on the dummy variables. It turns out that the regression ANOVA, that is, the overall F-test, applied to these dummy variables is equivalent to the one-way ANOVA (see Section 8.3 of the textbook.)

6.2 Interaction effects

An interaction effect occurs when the effect of one regressor on the response depends on the value of another regressor. In other words, the combined effect of two variables is not simply additive; the value of one variable modifies the impact of the other. In linear regression, this means that the coefficient of one regressor depends on the other.

We now give a simple example:

Suppose we are studying the effect of hours studied (X_1) and attendance (X_2) on exam scores (Y). An interaction effect between X_1 and X_2 would imply that the effect of studying on exam scores is different depending on the level of attendance. This can be modeled by including an interaction term ($X_1 \times X_2$) in the regression equation:

$$Y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 (X_{1i} \times X_{2i}) + \epsilon_i.$$

The coefficient β_3 represents the interaction effect between X1 and X2. For instance, if $\beta_3 > 0$, then the more the student has attended the course, the more beneficial the student's hours studied will be.

Some examples of how interaction effects are applied in real life are given by:

- **Psychology**: Studying how different treatments and demographic factors interact to influence behavior.
- **Marketing**: Analyzing how different marketing strategies and customer demographics interact to affect sales.
- **Medicine**: Investigating how different drugs and patient characteristics interact to affect health outcomes.

Example 6.3. Let's recall Example 6.1 and suppose the researcher would like you to include the interaction effect between Weight and Sex. Explain how the regression line changes with a non-zero interaction effect. Interpret the estimated interaction effect. Is it significant?

The regression equation is

$$Y_i|X = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \beta_3 X_{i1} X_{i2} + \epsilon_i.$$

If $\beta_3 \neq 0$ then we proceed as follows. When $X_{i2} = 0$, then the regression equation for males is still given by $Y_i|X = \beta_0 + \beta_1 X_{i1} + \epsilon_i$. Therefore, the expected body fat percentage for males is $\mathrm{E}[Y_i|X] = \beta_0 + \beta_1 X_{i1}$. Additionally, when $X_{i2} = 1$, then the regression equation for females is given by

$$Y_i|X = \beta_0 + \beta_1 X_{i1} + \beta_2 + \beta_3 X_{i1} + \epsilon_i = \beta_0 + (\beta_1 + \beta_3) X_{i1} + \beta_2 + \epsilon_i.$$

It follows that the expected body fat percentage for females is $\mathrm{E}[Y_i|X] = \beta_0 + (\beta_1 + \beta_3) X_{i1} + \beta_2$. Thus, adding an interaction effect allows the model to generate a completely different regression line, that is a different slope **and** intercept for females. The expected body fat percentage for females is then $\beta_2 + \beta_3 X_{i1}$ higher than for males, holding weight constant. Adding an interaction effect allows the slope to also vary, depending on whether the subject is male or female.

Let's observe.

```r
# Make the data frame
Weight=c(175 , 181 , 200 , 159 , 196 , 192 , 205 , 173 , 187 , 188 ,
         188 , 240 , 175 , 168 , 246 , 160 , 215 , 159 , 146 , 219 )
BodyFat =c(6 , 21 , 15 , 6 , 22 , 31 , 32 ,21 , 25 , 30 ,
           10 , 20 , 22 , 9 , 38 , 10 , 27 , 12 , 10 , 28 )
Sex=c("F","M","F","F","M","F","F","M","M","F","F","F","M","M","F","F","M","F","F","M")

df=data.frame(Weight=Weight,BodyFat=BodyFat,Sex=Sex,stringsAsFactors = T)

df$Sex=relevel(df$Sex,"M")

mod=lm(BodyFat~Weight+Sex+Weight*Sex,data=df)
summary(mod)
```

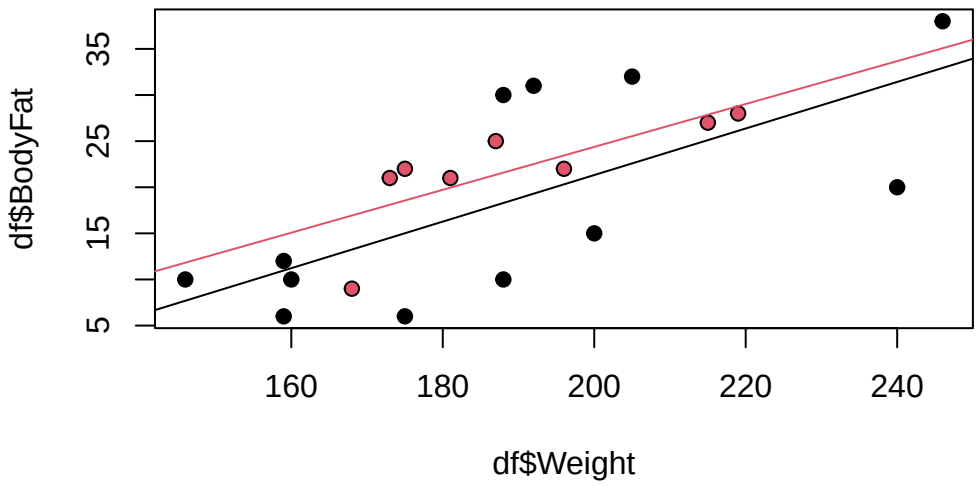
```
Call:
lm(formula = BodyFat ~ Weight + Sex + Weight * Sex, data = df)

Residuals:
     Min       1Q   Median       3Q      Max
-11.4171  -5.3084   0.1178   3.4912  11.7087

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -22.13450   27.12486  -0.816    0.426
Weight        0.23255    0.14269   1.630    0.123
SexF         -7.02921   30.18090  -0.233    0.819
Weight:SexF   0.01987    0.15869   0.125    0.902

Residual standard error: 7.255 on 16 degrees of freedom
Multiple R-squared:  0.5153,    Adjusted R-squared:  0.4245
F-statistic: 5.671 on 3 and 16 DF,  p-value: 0.007662
```

```r
plot(df$Weight,df$BodyFat,bg=((df$Sex=="M")+1),pch=21)
abline(coef(mod)[1],coef(mod)[2],col=2)
abline(coef(mod)[1]+coef(mod)[3],coef(mod)[2]+coef(mod)[4],col=1)
```



Notice how the slopes of the regression lines differ! Now, the estimated interaction effect is `0.01987`. Let's interpret it. We have that, on average, for every one lb increase in weight, the body fat percentage of a female increases by 0.01987 more than that of a male.

(In this case, the term is not significant, so we would probably drop it.)

> **Note**
>
> We can include interaction effects in the regression model in R by adding `variable_1*variable_2` to the right-hand side of the formula equation.

Example 6.4. When evaluating factors that affect the price of real estate, we may wish to consider location, while adjusting for lot size, year built and finished square feet. The data set `clean_data.csv` contains the prices of various types of real estate, as well as several important regressors. Regress the sale price on location, lot size, year built and finished square feet. Add an interaction term between year built and location. Interpret the interaction term for District 14.

```
####### Fitting the model  ###############
model=lm(Sale_price~District+Fin_sqft+Lotsize+Year_Built+Year_Built*District,df_clean2)

summ=summary(model); summ
```

```
Call:
lm(formula = Sale_price ~ District + Fin_sqft + Lotsize + Year_Built +
    Year_Built * District, data = df_clean2)

Residuals:
    Min      1Q  Median      3Q     Max
-400250  -24589     441   23005 1569420

Coefficients:
                       Estimate Std. Error t value Pr(>|t|)
(Intercept)          -1.567e+06  2.215e+05  -7.077 1.52e-12 ***
District2             9.848e+05  3.594e+05   2.740 0.006148 **
District3            -2.929e+05  2.723e+05  -1.076 0.282044
District4             6.838e+05  3.573e+05   1.914 0.055676 .
District5             1.551e+06  2.634e+05   5.888 3.97e-09 ***
District6             5.445e+05  2.683e+05   2.029 0.042446 *
District7            -7.242e+05  3.234e+05  -2.239 0.025139 *
District8             8.463e+05  3.000e+05   2.821 0.004790 **
District9            -5.339e+04  3.041e+05  -0.176 0.860634
District10            8.690e+05  2.602e+05   3.339 0.000842 ***
District11            1.899e+05  2.660e+05   0.714 0.475313
District12            1.282e+06  2.994e+05   4.283 1.85e-05 ***
District13            6.296e+05  2.557e+05   2.463 0.013799 *
District14            1.502e+06  2.392e+05   6.278 3.49e-10 ***
District15            5.815e+04  2.610e+05   0.223 0.823727
Fin_sqft              6.497e+01  6.659e-01  97.556  < 2e-16 ***
Lotsize               3.989e+00  1.237e-01  32.256  < 2e-16 ***
Year_Built            7.850e+02  1.139e+02   6.891 5.68e-12 ***
District2:Year_Built -4.986e+02  1.841e+02  -2.708 0.006770 **
District3:Year_Built  2.547e+02  1.409e+02   1.807 0.070772 .
District4:Year_Built -3.703e+02  1.858e+02  -1.993 0.046287 *
District5:Year_Built -7.666e+02  1.352e+02  -5.668 1.46e-08 ***
District6:Year_Built -2.727e+02  1.388e+02  -1.965 0.049443 *
District7:Year_Built  3.734e+02  1.667e+02   2.240 0.025118 *
District8:Year_Built -4.278e+02  1.555e+02  -2.751 0.005938 **
District9:Year_Built  3.721e+01  1.555e+02   0.239 0.810891
District10:Year_Built -4.146e+02  1.340e+02  -3.093 0.001985 **
District11:Year_Built -6.285e+01  1.366e+02  -0.460 0.645462
District12:Year_Built -6.608e+02  1.555e+02  -4.251 2.14e-05 ***
District13:Year_Built -2.887e+02  1.314e+02  -2.198 0.027981 *
District14:Year_Built -7.231e+02  1.232e+02  -5.869 4.43e-09 ***
District15:Year_Built -4.403e+01  1.347e+02  -0.327 0.743706
```

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 55410 on 24590 degrees of freedom
Multiple R-squared:  0.5931,    Adjusted R-squared:  0.5925
F-statistic:  1156 on 31 and 24590 DF,  p-value: < 2.2e-16
```

Observe that the interaction term between year built and District 14 is -723.1$. In addition, note that year built has a positive coefficient of 785$. We can interpret the interaction effect as follows: Holding finished square feet and lot size constant, a one year increase in year built for a home in District 14 results in an increase in price that is 723.1 lower than that of District 1. We can also reword this to make it a little more clear - Holding finished square feet and lot size constant, a one year increase in year built for a home in District 1 results in an increase in price that is 723.1 higher than that of District 14.

To see this observe that for a one unit increase in year built in District 14, we have that the price goes up by 785.0-723.1=61.9 on average, holding other variables constant. On the other hand, in District 1, the price goes up by 785.0 on average, holding other variables constant. Therefore, in general, newer homes are much more valuable in District 1.

Exercise 6.1. Interpret the main effects and the interaction effect with year built for Districts 2-4. (The main effects are the coefficients for Districts 2-4.)

> ⚠ Warning
>
> Including interaction effects changes the interpretation of the main effects. When a variable is involved in an interaction term, its main effect no longer represents its overall marginal association with the response. Instead, the main effect must be interpreted at the level where the interacting variable is equal to 0 (i.e., at the reference or baseline value used in the model).

Note: Including interaction effects changes how we interpret the main effects.
When a variable is involved in an interaction term, its main effect no longer represents its overall marginal association with the response. Instead, the main effect is interpreted **at the level where the interacting variable equals 0** (the baseline level used in the model).

Suppose we fit the model

$$\text{ExamScore} = \beta_0 + \beta_1 \text{StudyHours} + \beta_2 \text{ProgramStats} + \beta_3 (\text{StudyHours} \times \text{ProgramStats}) + \varepsilon,$$

where `ProgramStats = 1` for Stats students and `0` for the baseline group (Math students).

This model includes an interaction between **StudyHours** and **Program**.

- **Interpretation of β_1** (main effect of StudyHours):
  β_1 is *not* the overall effect of increasing StudyHours.
  It is the effect **only for Math students**, because Math is coded as 0 for `ProgramStats`.
  In other words, β_1 is the slope of StudyHours **when ProgramStats = 0**.
- **Interpretation of β_2** (main effect of ProgramStats):
  β_2 is *not* the overall difference between Stats and Math students.
  It compares Stats vs. Math **when StudyHours = 0**, because the interaction term equals 0 at StudyHours = 0.

This example shows that once an interaction is included, the main effects must be interpreted **at the point where the other interacting variable is 0**.

> **Warning**
>
> The interpretation for interaction effects is difficult and nuanced. Make sure you study this topic carefully.

6.3 Increasing codes and quantitative regressors via dummy variables

Another approach to the treatment of a qualitative variable in regression is to measure the levels of the variable by an allocated code. Suppose we model the effect of the number of bedrooms on real estate price by its numerical value, instead of categorical value. Let's see what happens to the regression equation. In general, ordinal variables may be better represented by dummy variables/indicators - however, dummy variables increase the complexity of the model, which we may not have enough data to support, and could lead to overfitting.

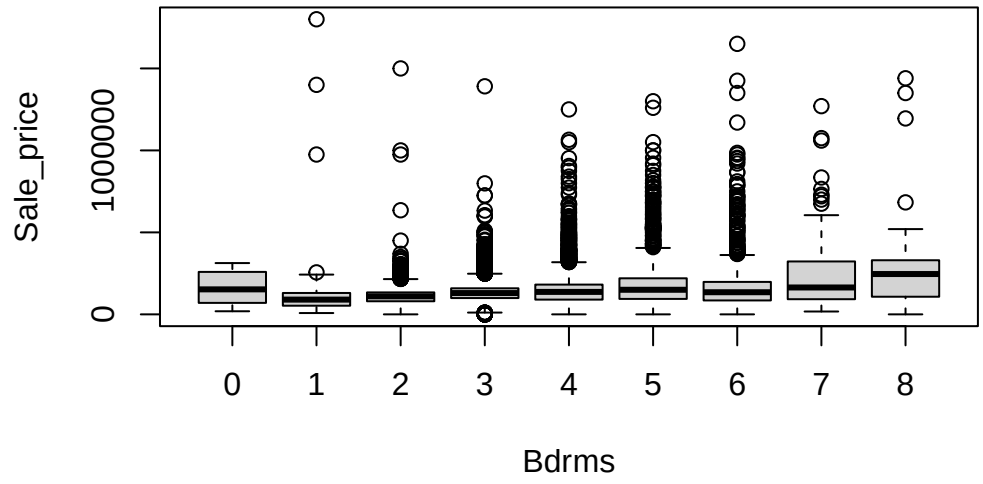
Example 6.5. When evaluating factors that affect the price of real estate, we may wish to consider the unadjusted effect of the number of bedrooms. Regress the sale price on number of bedrooms treating number of bedrooms as a continuous variable. Then, regress the sale price on number of bedrooms treating number of bedrooms as a categorical variable. Compare and contrast the two models, and the resulting fits.

```
####### Fitting the model  ##############
unique(df_clean2$Bdrms)
```

```
 [1] >8 2  0  4  7  3  1  6  8  5
Levels: >8 0 1 2 3 4 5 6 7 8
```

```
# Drop these rows
df_clean3=df_clean2[df_clean2$Bdrms!='>8',]
df_clean3$Bdrms=droplevels(df_clean3$Bdrms)
df_clean3$Bdrms=relevel(df_clean3$Bdrms,"0")
# Add new continuous variable
df_clean3$Bdrms2=as.numeric(df_clean3$Bdrms)-1

boxplot(Sale_price~Bdrms,df_clean3)
```




```
model_ca=lm(Sale_price~Bdrms,df_clean3)
model_co=lm(Sale_price~Bdrms2,df_clean3)

summ=summary(model_ca); summ
```

```
Call:
lm(formula = Sale_price ~ Bdrms, data = df_clean3)

Residuals:
```

```
    Min      1Q  Median      3Q     Max
-271312  -43342   -6342   26658 1672084

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   161825      29764   5.437 5.47e-08 ***
Bdrms1        -33909      30818  -1.100 0.271216
Bdrms2        -53113      29800  -1.782 0.074712 .
Bdrms3        -28483      29774  -0.957 0.338758
Bdrms4        -15157      29785  -0.509 0.610830
Bdrms5         24156      29852   0.809 0.418414
Bdrms6          6826      29861   0.229 0.819189
Bdrms7         81735      30717   2.661 0.007798 **
Bdrms8        109487      31332   3.494 0.000476 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 84190 on 24586 degrees of freedom
Multiple R-squared:  0.05675,   Adjusted R-squared:  0.05644
F-statistic: 184.9 on 8 and 24586 DF,  p-value: < 2.2e-16
```

```
summ=summary(model_co); summ
```

```
Call:
lm(formula = Sale_price ~ Bdrms2, data = df_clean3)

Residuals:
    Min      1Q  Median      3Q     Max
-224144  -43153   -6651   26849 1705343

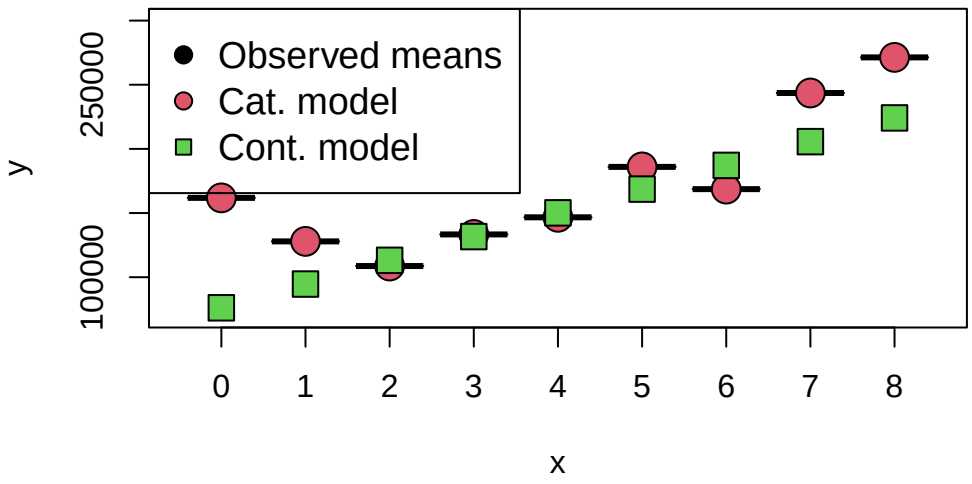
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  76159.0     1849.3   41.18   <2e-16 ***
Bdrms2       18498.1      522.4   35.41   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 84540 on 24593 degrees of freedom
Multiple R-squared:  0.04851,   Adjusted R-squared:  0.04847
F-statistic:  1254 on 1 and 24593 DF,  p-value: < 2.2e-16
```

Observe that the continuous model says that for every additional bedroom, on average the price increases by 18 498$. On the other hand, the categorical model says that the change in price depends on the number of bedrooms. For example, going from 0 bedrooms to 1 bedroom, we actually see a reduction in price of -33 909$. Let's graph the expected price for each number of bedrooms from both models:

```
####### Fitting the model  ##############
new_dat=data.frame('Bdrms'=sort(unique(df_clean3$Bdrms)))
new_dat2=data.frame('Bdrms2'=0:8)
# predict(model_ca,new_dat)
# predict(model_co,new_dat2)

observed_means=aggregate(Sale_price~Bdrms,data=df_clean3, FUN = "mean")
plot(observed_means[,1],observed_means[,2],pch=25,bg=1,cex=3,ylim=c( 70000,300000))
points(x=observed_means[,1],predict(model_ca,new_dat),pch=21,bg=2,cex=2)
points(x=observed_means[,1],predict(model_co,new_dat2),pch=22,bg=3,cex=2)
legend("topleft",legend=c("Observed means","Cat. model","Cont. model"),pch=c(21,21,22),pt.bg=
```



Observe that the continuous model does not match the data at all, while the categorical model is able to model the **non-linear** relationship between the number of bedrooms and the sale price! Why is this the case? Treating a regressor as continuous implies that there is a linear relationship between that regressor and the response. On the other hand, modelling the variable

with indicators does not place any assumption on the relationship between the regressor and the response. The drawback, is that we need 7 more parameters in the model.

> **Warning**
>
> When deciding to treat continuous or ordinal variables as continuous, it is critical that you evaluate whether it is acceptable to assume a linear relationship between the regression and the response. If you cannot verify this assumption, or it seems invalid, it is best to treat the regressor as categorical.

Quantitative regressors can also be represented by indicator variables. Sometimes this is necessary because it is difficult to collect accurate information on the quantitative regressor, or the exact values are obscured for privacy reasons. Treating a quantitative factor as a qualitative one increases the complexity of the model. This approach also reduces the degrees of freedom for error. However, the indicator variable approach does not require the analyst to make any prior assumptions about the functional form of the relationship between the response and the regressor variable, as previously discussed.

6.4 A larger scale example

It is a good time to stop introducing new material and do a larger scale example.

Example 6.6. Explore the pricing data, and evaluate what factors influence the price of a property. Be sure to assess the fit of the model and check assumptions.

```
####### Packages needed  ###############

library(lubridate)

# Example: We would like to see how sale price
# of a home is related to
# various factors

####### Loading data  ###############

df_clean2=read.csv('C:/Users/12RAM/OneDrive - York University/Teaching/Courses/Math 3330 Regr

####### Analyzing the data via EDA  ###############

names(df_clean2)
```

```
 [1] "District"   "Extwall"    "Stories"    "Year_Built" "Fin_sqft"
 [6] "Units"      "Bdrms"      "Fbath"      "Lotsize"    "Sale_date"
[11] "Sale_price"
```

```
df_clean2$District=as.factor(df_clean2$District)

df_clean2=df_clean2[,c('Sale_price','Fin_sqft',
                       'District','Sale_date','Year_Built','Lotsize')]

# Sale price, as a fn of sq ft, Distict, Sale_date, Year_Built, Lotsize

model=lm(Sale_price~Fin_sqft+District+Sale_date+ Year_Built+ Lotsize,data=df_clean2)
summary(model)
```

```
Call:
lm(formula = Sale_price ~ Fin_sqft + District + Sale_date + Year_Built +
    Lotsize, data = df_clean2)

Residuals:
    Min      1Q  Median      3Q     Max
-396057  -25416     299   23393 1582073

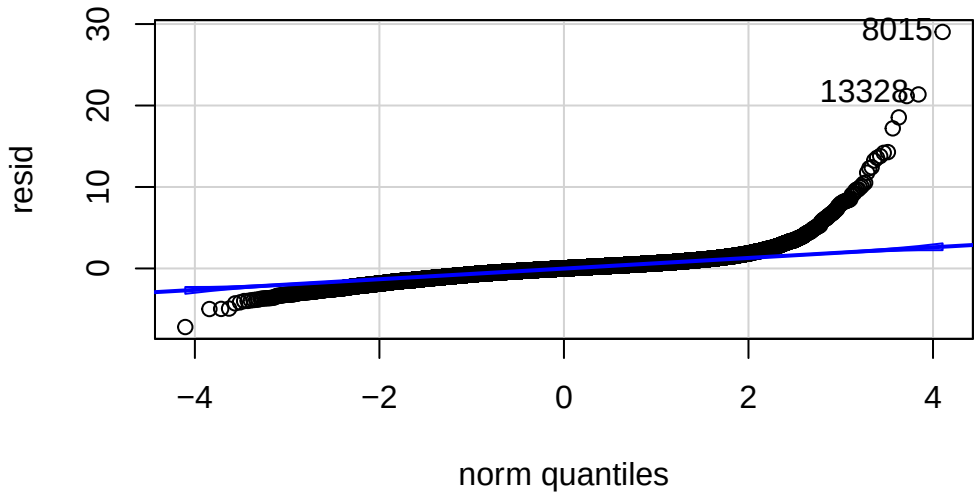
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -9.956e+05  4.193e+04 -23.744  < 2e-16 ***
Fin_sqft     6.495e+01  6.500e-01  99.928  < 2e-16 ***
District2    1.372e+04  2.303e+03   5.959 2.58e-09 ***
District3    1.818e+05  2.452e+03  74.145  < 2e-16 ***
District4   -3.407e+04  4.758e+03  -7.161 8.23e-13 ***
District5    5.587e+04  1.980e+03  28.219  < 2e-16 ***
District6    1.231e+04  2.837e+03   4.340 1.43e-05 ***
District7   -4.386e+03  2.472e+03  -1.774  0.07603 .
District8    1.804e+04  2.697e+03   6.691 2.27e-11 ***
District9    2.796e+04  2.463e+03  11.352  < 2e-16 ***
District10   6.393e+04  2.064e+03  30.973  < 2e-16 ***
District11   7.048e+04  1.982e+03  35.560  < 2e-16 ***
District12   9.987e+03  3.226e+03   3.096  0.00197 **
District13   6.867e+04  2.049e+03  33.519  < 2e-16 ***
District14   1.031e+05  2.077e+03  49.614  < 2e-16 ***
District15  -3.352e+04  3.037e+03 -11.037  < 2e-16 ***
Sale_date    4.705e+00  3.256e-01  14.449  < 2e-16 ***
```

```
Year_Built   4.514e+02  2.144e+01  21.056  < 2e-16 ***
Lotsize      3.906e+00  1.223e-01  31.953  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 55440 on 24603 degrees of freedom
Multiple R-squared:  0.5923,    Adjusted R-squared:  0.592
F-statistic:  1986 on 18 and 24603 DF,  p-value: < 2.2e-16
```

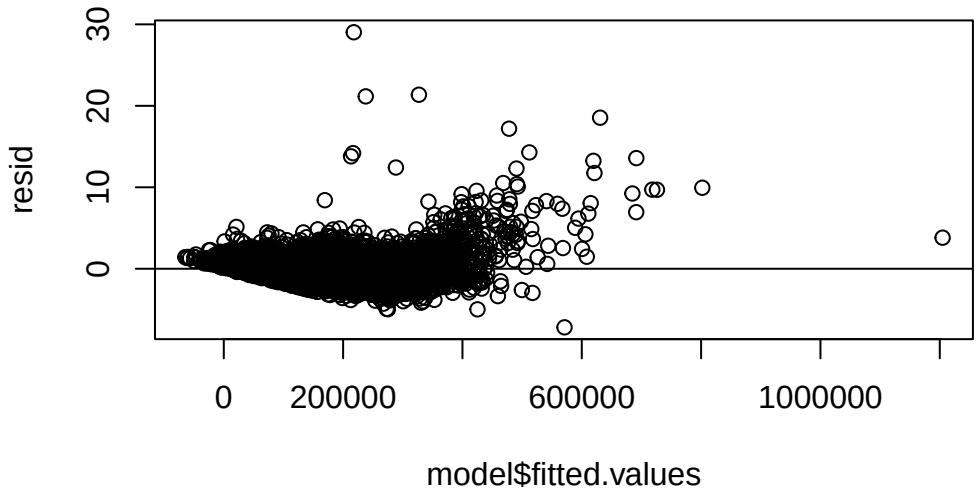
```r
resid=rstudent(model)

car::qqPlot(resid)
```

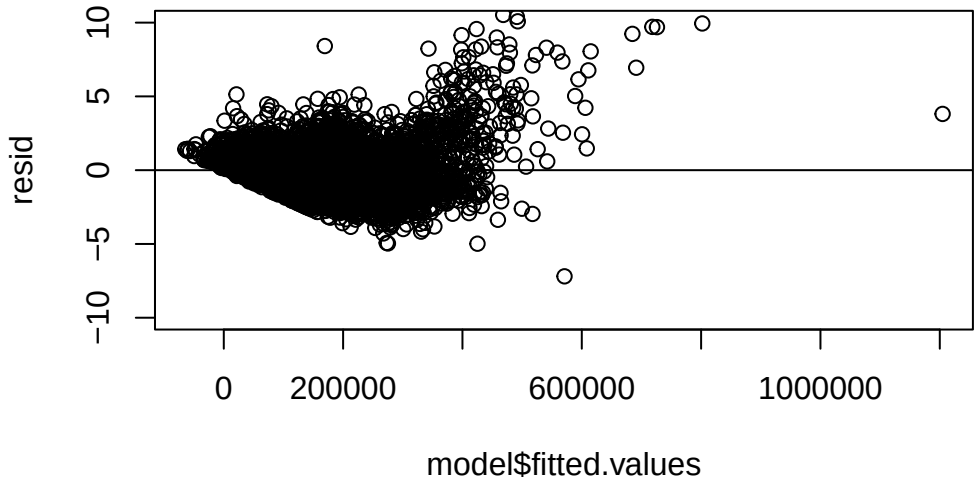



```
[1]  8015 13328
```

```r
# roblm
plot(model$fitted.values,resid )
abline(h=0)
```



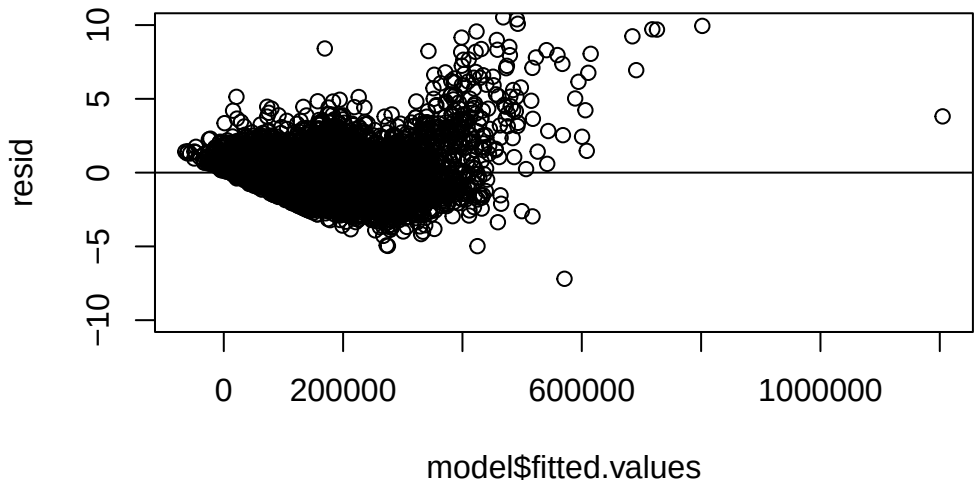

```
plot(model$fitted.values,resid,ylim=c(-10,10) )
abline(h=0)
```



```
df_clean2[which.max(resid),]
```

```
     Sale_price Fin_sqft District Sale_date Year_Built Lotsize
8015    1800000     1330        3     15949       1928       0
```

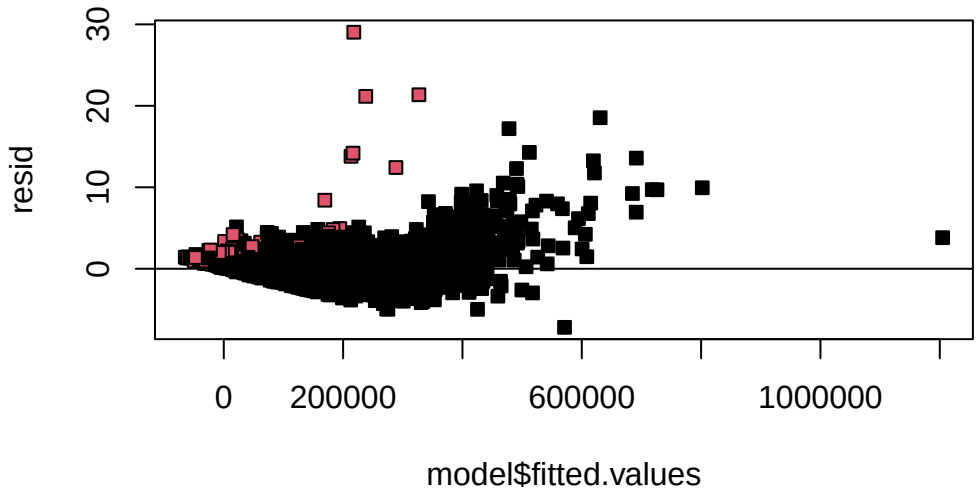
```
plot(model$fitted.values,resid,ylim=c(-10,10) )
abline(h=0)
```




```
sum(df_clean2$Lotsize==0)
```

```
[1] 146
```

```
par(pch=22,bg='white')
plot(model$fitted.values,resid,bg=as.numeric(df_clean2$Lotsize==0)+1 )
abline(h=0)
```




```
# remove those with 0 lot size
df3=df_clean2[df_clean2$Lotsize>0,]
model=lm(Sale_price~Fin_sqft+District+Sale_date+ Year_Built+ Lotsize,data=df3)
summary(model)
```

```
Call:
lm(formula = Sale_price ~ Fin_sqft + District + Sale_date + Year_Built +
    Lotsize, data = df3)

Residuals:
    Min      1Q  Median      3Q     Max
-406081  -25101     616   23636 1017694

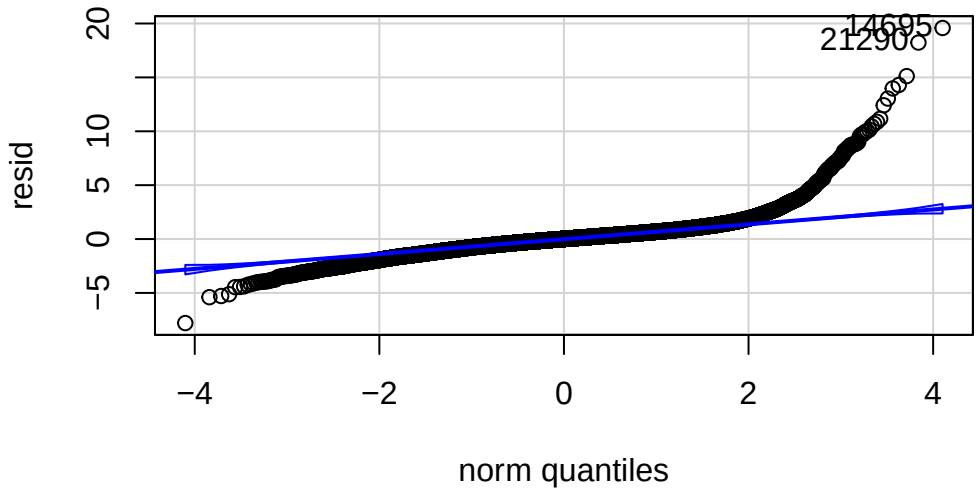
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.011e+06  3.987e+04 -25.350  < 2e-16 ***
Fin_sqft     6.575e+01  6.185e-01 106.305  < 2e-16 ***
District2    1.351e+04  2.179e+03   6.200 5.73e-10 ***
District3    1.771e+05  2.328e+03  76.060  < 2e-16 ***
District4   -3.517e+04  4.516e+03  -7.788 7.05e-15 ***
District5    5.554e+04  1.873e+03  29.646  < 2e-16 ***
```

```
District6    9.986e+03  2.703e+03   3.695 0.000221 ***
District7   -4.296e+03  2.340e+03  -1.836 0.066382 .
District8    1.699e+04  2.572e+03   6.607 3.99e-11 ***
District9    2.675e+04  2.332e+03  11.474  < 2e-16 ***
District10   6.401e+04  1.954e+03  32.766  < 2e-16 ***
District11   7.027e+04  1.875e+03  37.472  < 2e-16 ***
District12   6.017e+03  3.114e+03   1.932 0.053354 .
District13   6.814e+04  1.939e+03  35.145  < 2e-16 ***
District14   1.030e+05  1.966e+03  52.371  < 2e-16 ***
District15  -3.472e+04  2.890e+03 -12.011  < 2e-16 ***
Sale_date    4.611e+00  3.089e-01  14.928  < 2e-16 ***
Year_Built   4.585e+02  2.038e+01  22.492  < 2e-16 ***
Lotsize      4.210e+00  1.162e-01  36.239  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 52450 on 24457 degrees of freedom
Multiple R-squared:  0.617, Adjusted R-squared:  0.6167
F-statistic:  2189 on 18 and 24457 DF,  p-value: < 2.2e-16
```

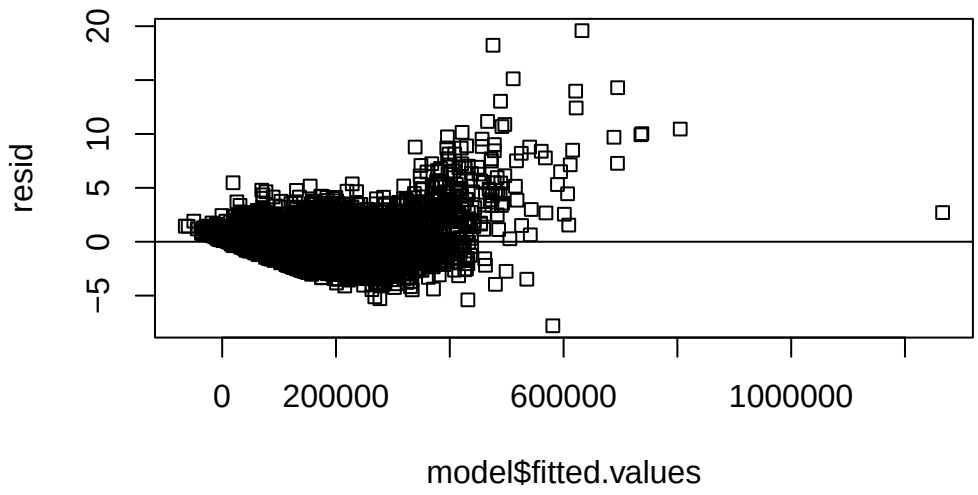
```r
resid=rstudent(model)

car::qqPlot(resid)
```

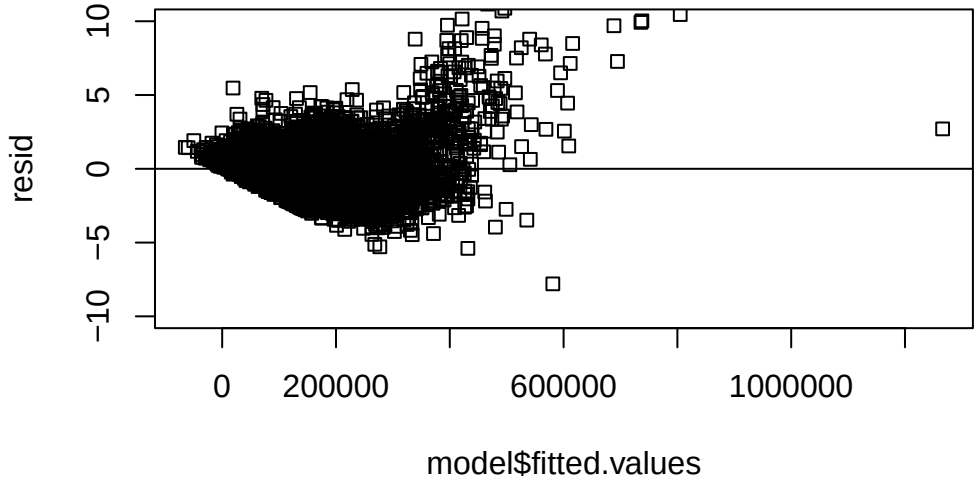


```
14695 21290
14619 21161
```

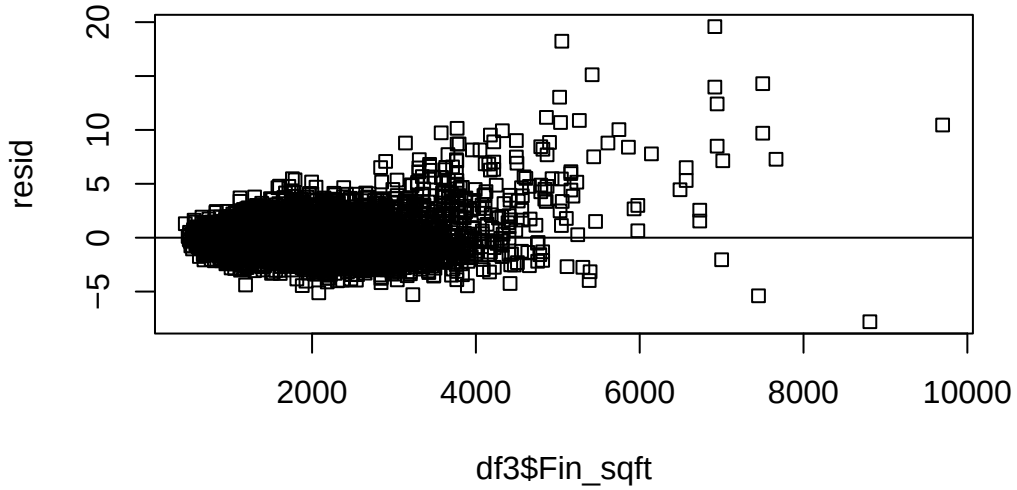
```
# roblm
plot(model$fitted.values,resid )
abline(h=0)
```




```
plot(model$fitted.values,resid,ylim=c(-10,10) )
abline(h=0)
```

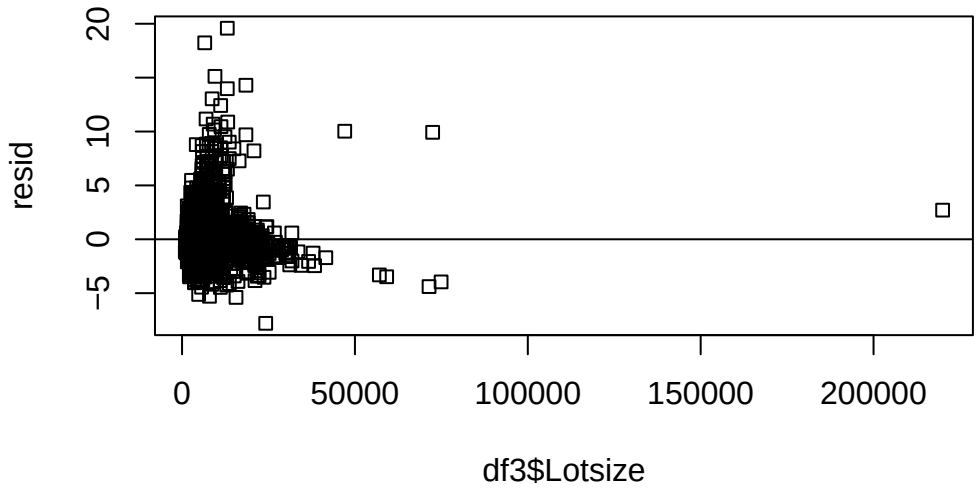



```
plot(df3$Fin_sqft,resid )
abline(h=0)
```

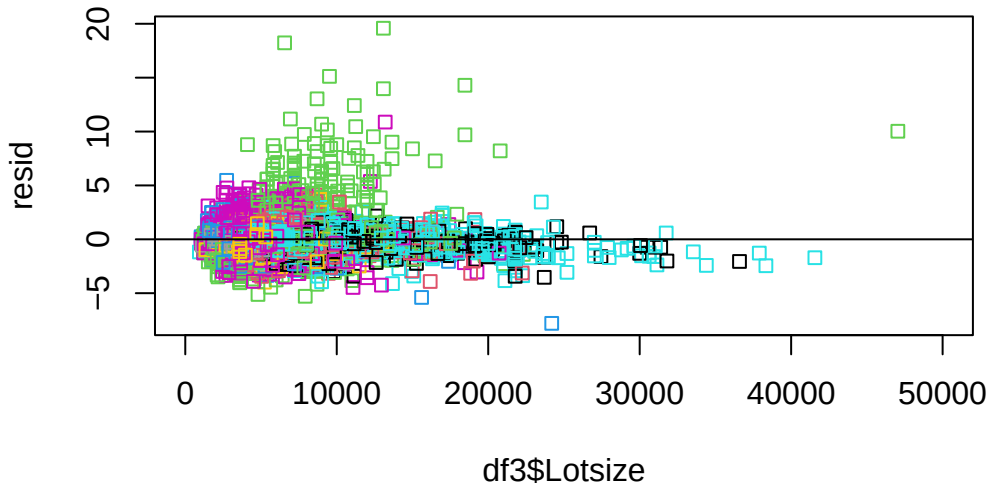


```r
# just for comparison, we look at the
# plot(df3$Fin_sqft,model$residuals )
# abline(h=0)

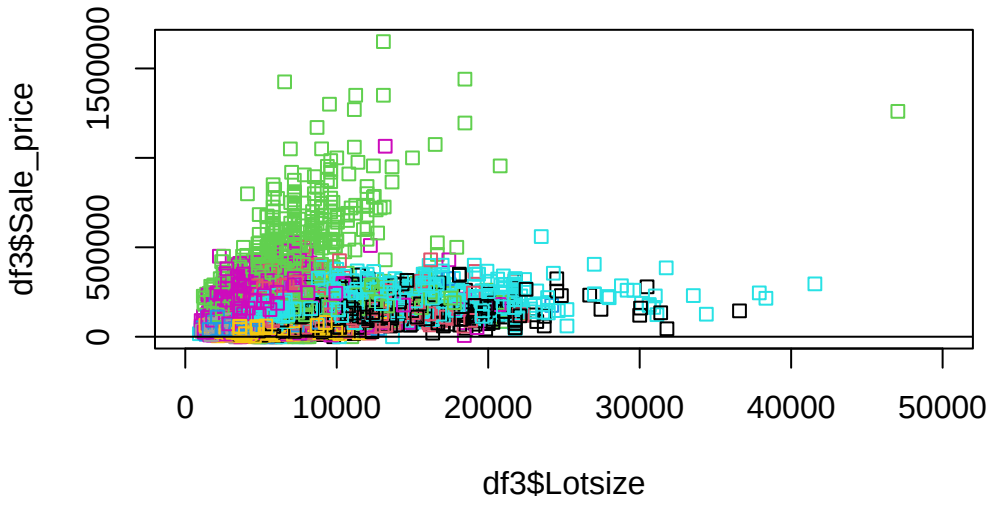

plot(df3$Lotsize,resid )
abline(h=0)
```




```r
# remove largest lot size
plot(df3$Lotsize,resid ,xlim=c(0,50000),col=df3$District)
abline(h=0)
```

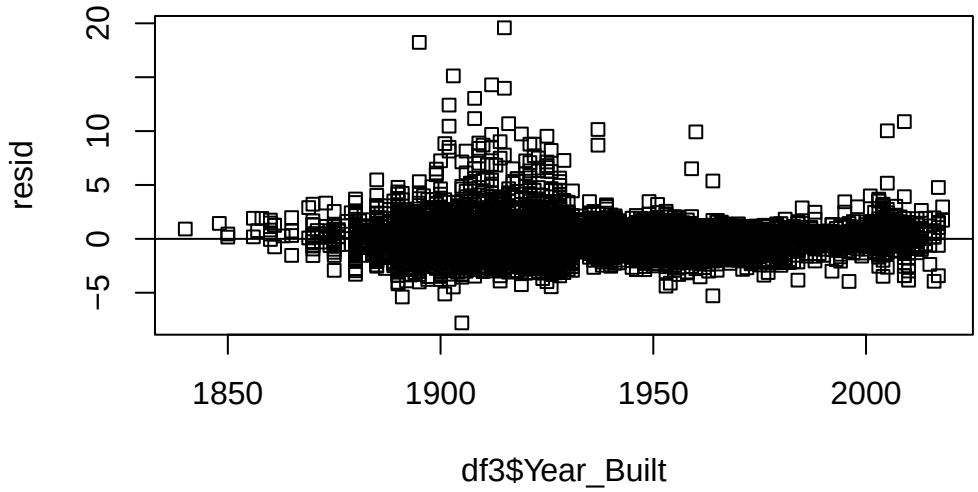



```
plot(df3$Lotsize,df3$Sale_price ,xlim=c(0,50000),col=df3$District)
abline(h=0)
```

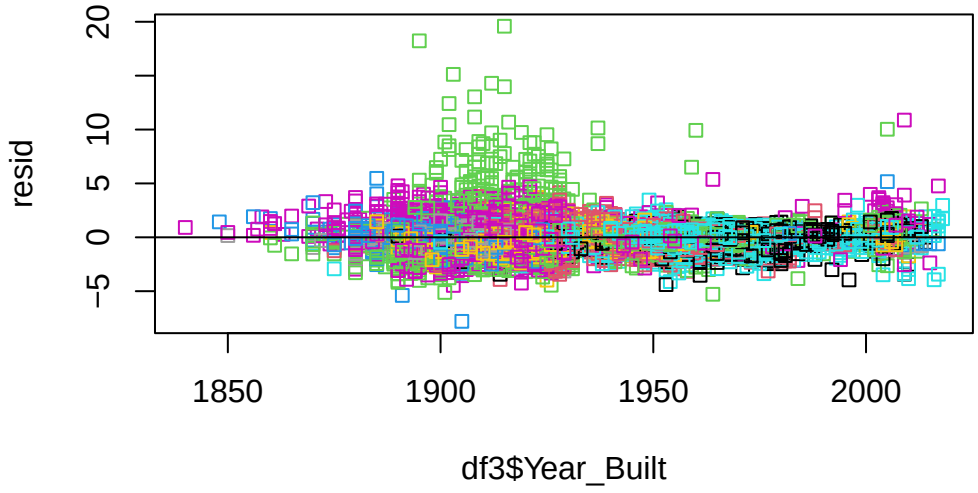


```
# non constant variance

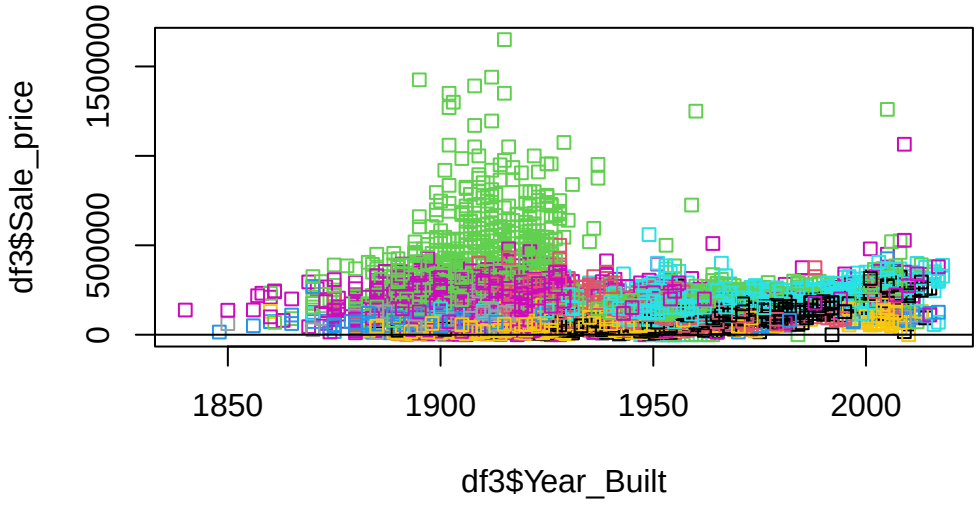

plot(df3$Year_Built,resid )
abline(h=0)
```




```
plot(df3$Year_Built,resid ,col=df3$District)
abline(h=0)
```

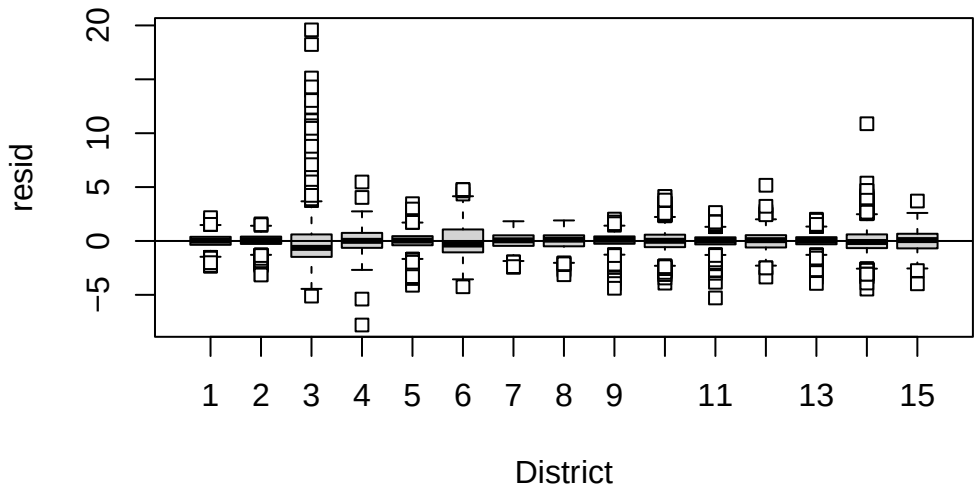



```
plot(df3$Year_Built,df3$Sale_price ,col=df3$District)
abline(h=0)
```



```r
# non constant variance

rdf=df3
rdf=cbind(df3,resid)
boxplot(resid~District,rdf )
abline(h=0)
```




```r
df4=df3[df3$Lotsize<150000,]
model=lm(Sale_price~Fin_sqft+District+Sale_date+ Year_Built+ Lotsize+District*Lotsize,data=d
summary(model)
```

```
Call:
lm(formula = Sale_price ~ Fin_sqft + District + Sale_date + Year_Built +
    Lotsize + District * Lotsize, data = df4)

Residuals:
     Min       1Q   Median       3Q      Max
-1132285   -24249      -82    22444   929557

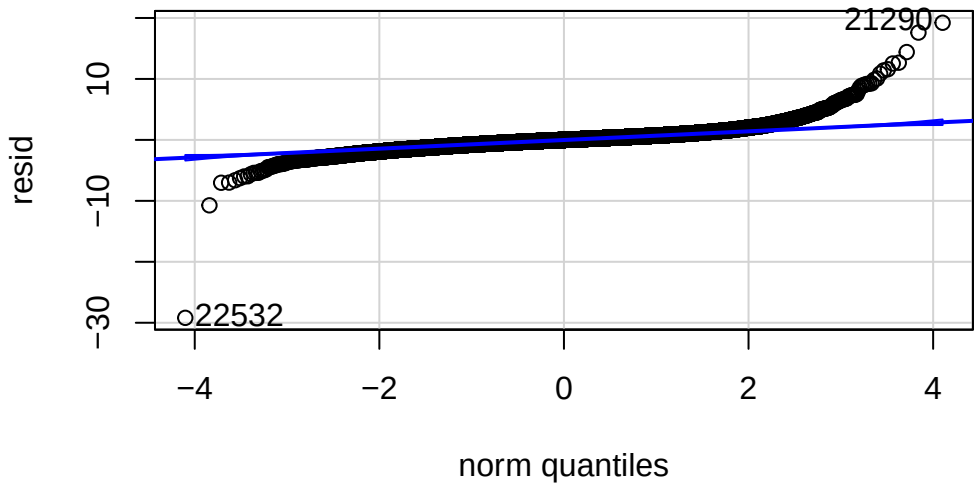
Coefficients:
                    Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept)        -9.614e+05  3.784e+04 -25.410  < 2e-16 ***
Fin_sqft            5.921e+01  5.943e-01  99.633  < 2e-16 ***
District2           2.503e+04  5.435e+03   4.605 4.15e-06 ***
District3           5.552e+04  4.593e+03  12.088  < 2e-16 ***
District4           2.608e+04  9.017e+03   2.892 0.003826 **
District5           7.135e+04  4.185e+03  17.049  < 2e-16 ***
District6           3.378e+04  6.030e+03   5.602 2.15e-08 ***
District7          -9.708e+03  7.684e+03  -1.263 0.206444
District8           2.851e+04  7.687e+03   3.709 0.000209 ***
District9           4.723e+04  5.124e+03   9.218  < 2e-16 ***
District10          4.150e+04  5.273e+03   7.870 3.69e-15 ***
District11          7.638e+04  4.802e+03  15.906  < 2e-16 ***
District12          3.550e+04  8.201e+03   4.329 1.50e-05 ***
District13          7.622e+04  4.412e+03  17.276  < 2e-16 ***
District14          1.107e+05  4.783e+03  23.143  < 2e-16 ***
District15         -4.977e+04  7.708e+03  -6.457 1.09e-10 ***
Sale_date           4.737e+00  2.873e-01  16.489  < 2e-16 ***
Year_Built          4.379e+02  1.923e+01  22.768  < 2e-16 ***
Lotsize             3.766e+00  5.439e-01   6.923 4.52e-12 ***
District2:Lotsize  -1.691e+00  7.689e-01  -2.199 0.027894 *
District3:Lotsize   2.507e+01  6.915e-01  36.252  < 2e-16 ***
District4:Lotsize  -1.053e+01  1.478e+00  -7.126 1.06e-12 ***
District5:Lotsize  -2.097e+00  5.835e-01  -3.593 0.000327 ***
District6:Lotsize  -4.761e+00  1.060e+00  -4.490 7.16e-06 ***
District7:Lotsize   1.248e+00  1.359e+00   0.918 0.358648
District8:Lotsize  -2.561e+00  1.621e+00  -1.579 0.114313
District9:Lotsize  -1.867e+00  6.278e-01  -2.973 0.002947 **
District10:Lotsize  4.395e+00  8.530e-01   5.152 2.59e-07 ***
District11:Lotsize -8.301e-01  6.770e-01  -1.226 0.220133
District12:Lotsize -7.806e+00  1.906e+00  -4.096 4.22e-05 ***
District13:Lotsize -1.021e+00  6.088e-01  -1.676 0.093669 .
District14:Lotsize -1.625e+00  7.793e-01  -2.085 0.037118 *
District15:Lotsize  3.720e+00  1.380e+00   2.696 0.007012 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 48770 on 24442 degrees of freedom
Multiple R-squared:  0.6662,    Adjusted R-squared:  0.6657
F-statistic:  1524 on 32 and 24442 DF,  p-value: < 2.2e-16
```

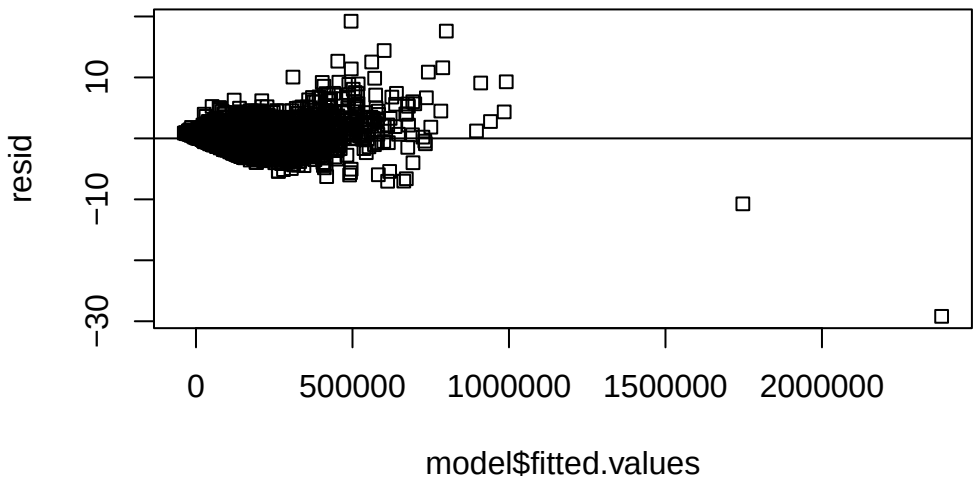
```r
resid=rstudent(model)
```

```
car::qqPlot(resid)
```

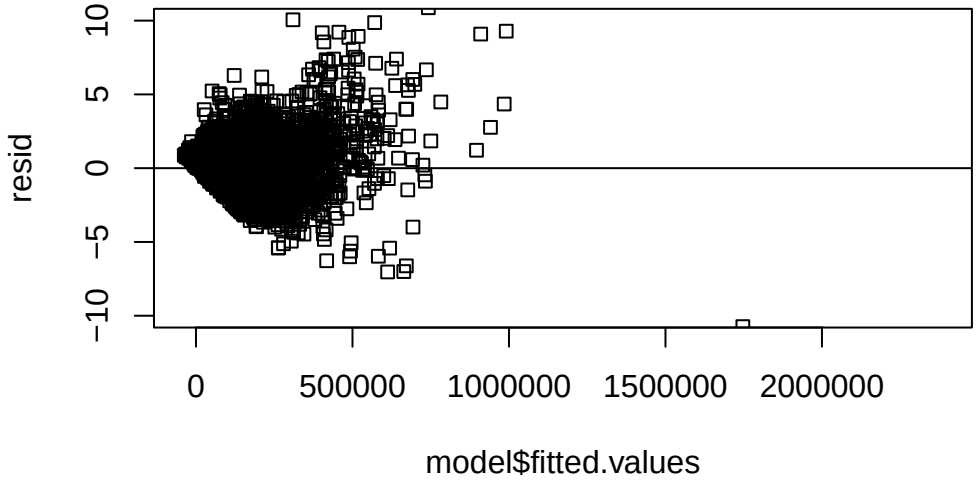



```
22532 21290
22398 21160
```

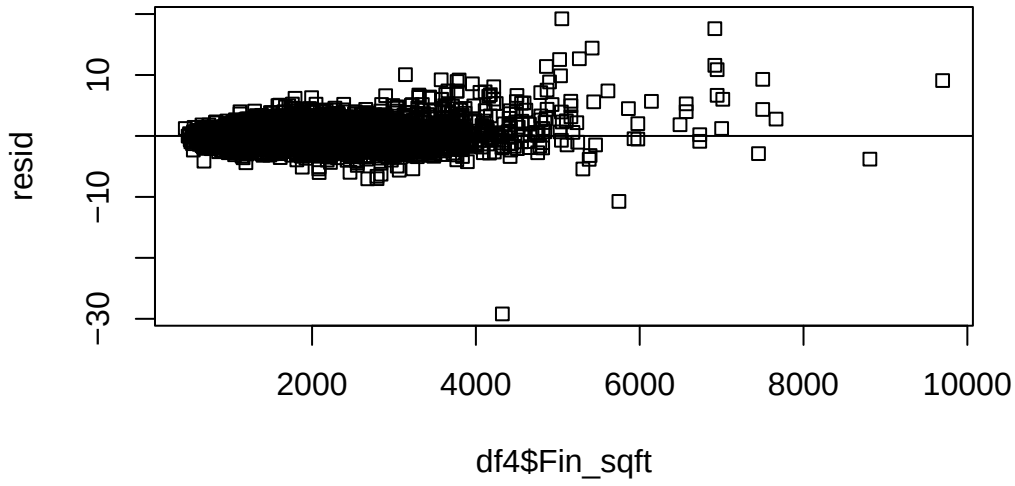
```
# roblm
plot(model$fitted.values,resid )
abline(h=0)
```




```
plot(model$fitted.values,resid,ylim=c(-10,10) )
abline(h=0)
```

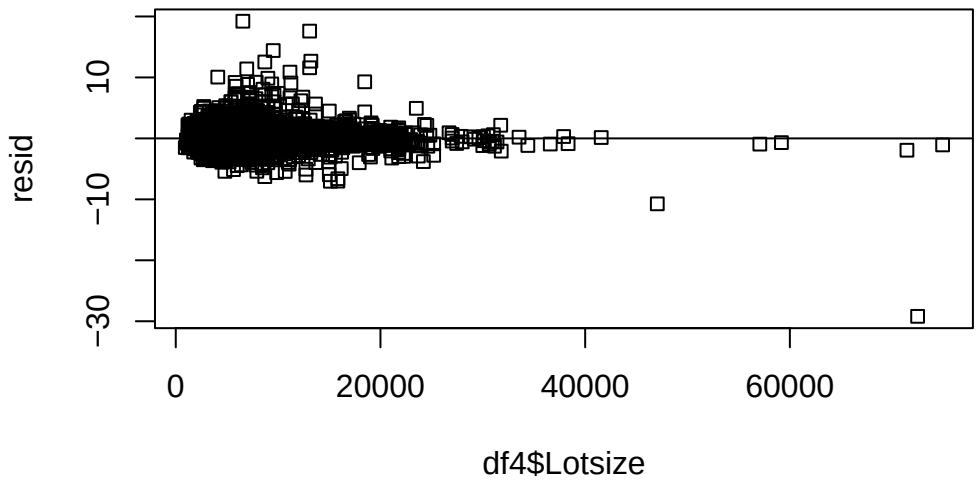


```r
plot(df4$Fin_sqft,resid )
abline(h=0)
```



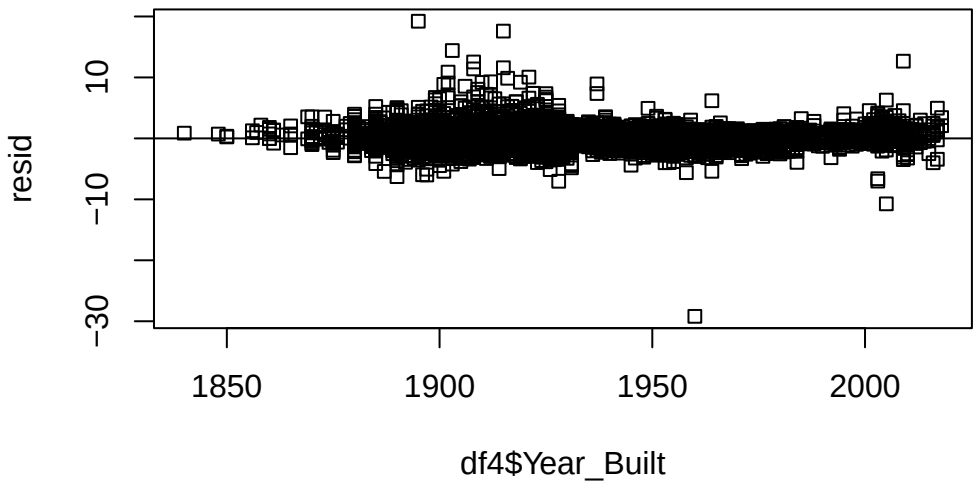

```r
# just for comparison, we look at the
# plot(df3$Fin_sqft,model$residuals )
# abline(h=0)


plot(df4$Lotsize,resid )
abline(h=0)
```

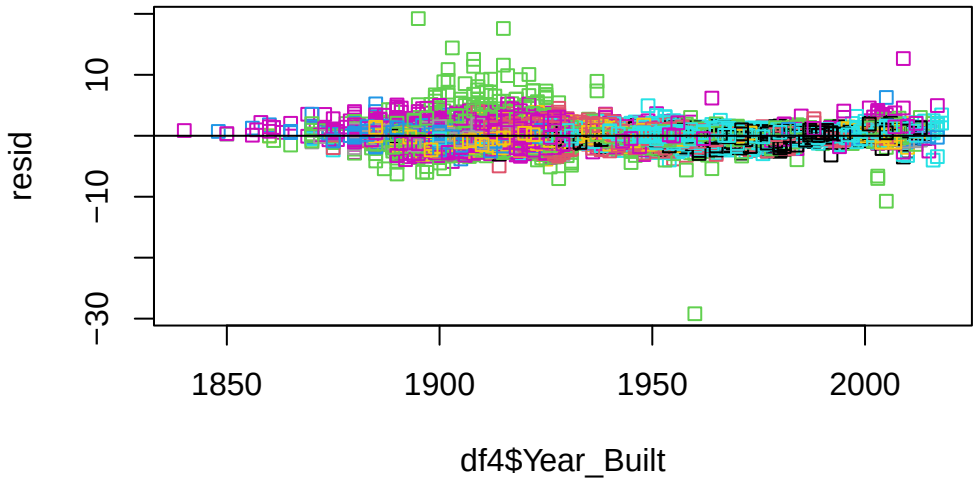



```
# non constant variance

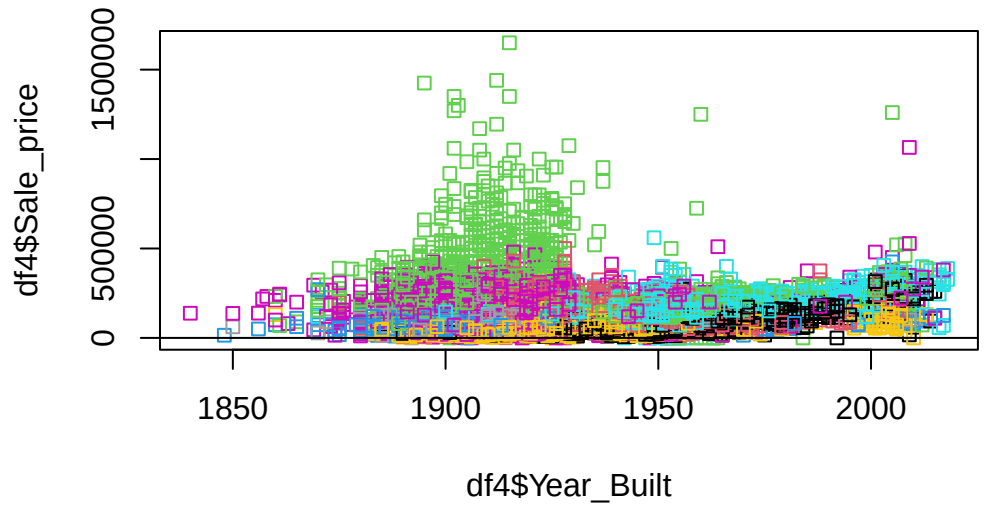

plot(df4$Year_Built,resid )
abline(h=0)
```




```
plot(df4$Year_Built,resid ,col=df4$District)
abline(h=0)
```

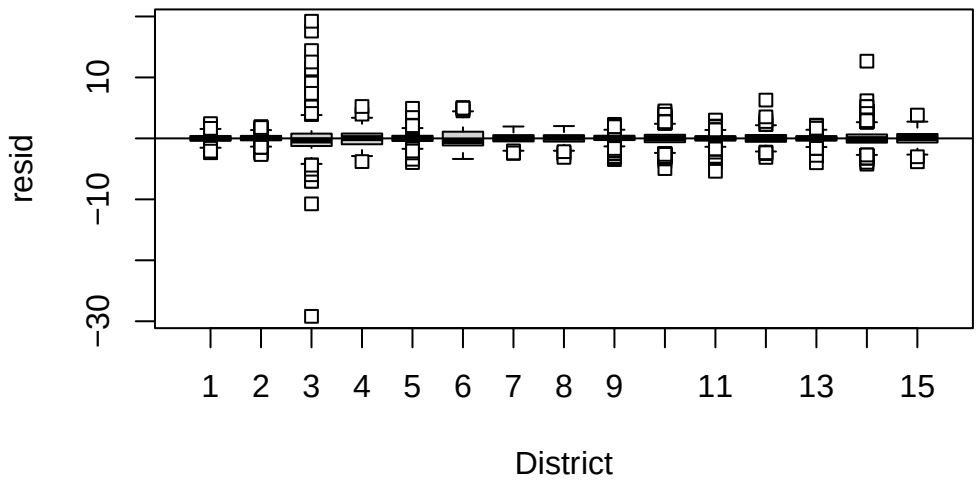


```
plot(df4$Year_Built,df4$Sale_price ,col=df4$District)
abline(h=0)
```




```
# non constant variance

rdf=df4
rdf=cbind(df4,resid)
boxplot(resid~District,rdf )
abline(h=0)
```




```
df5=df4[df4$District!=3,]
df5$District=droplevels(df5$District)
model=lm(Sale_price~Fin_sqft+District+Sale_date+ Year_Built+ Lotsize+District*Lotsize,data=d
summary(model)
```

```
Call:
lm(formula = Sale_price ~ Fin_sqft + District + Sale_date + Year_Built +
    Lotsize + District * Lotsize, data = df5)

Residuals:
    Min      1Q  Median      3Q     Max
-236186  -23227   -1087   20835  661626

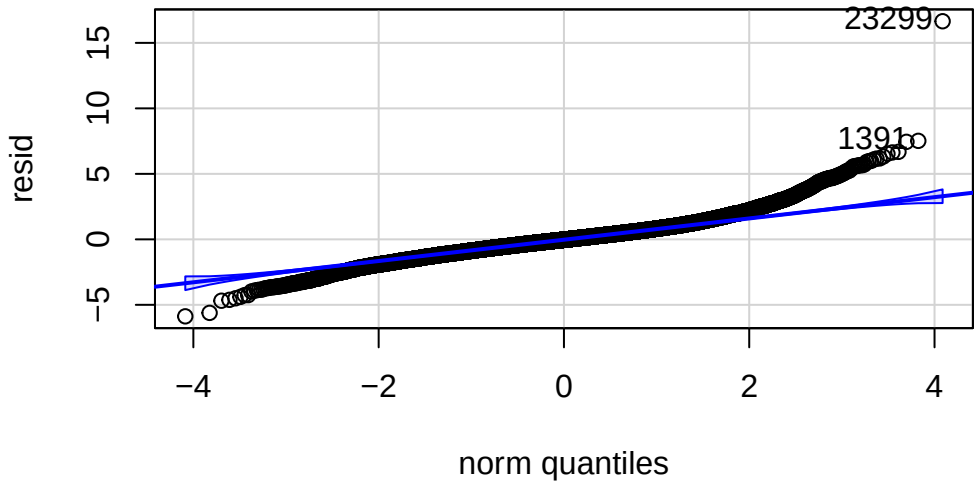
Coefficients:
                    Estimate Std. Error t value Pr(>|t|)
(Intercept)       -1.030e+06  3.219e+04 -32.007  < 2e-16 ***
Fin_sqft           4.467e+01  5.442e-01  82.089  < 2e-16 ***
District2          2.255e+04  4.483e+03   5.029 4.96e-07 ***
District4          2.893e+04  7.438e+03   3.889 0.000101 ***
District5          6.801e+04  3.452e+03  19.702  < 2e-16 ***
District6          3.885e+04  4.976e+03   7.807 6.12e-15 ***
```

```
District7          -2.871e+03  6.338e+03  -0.453 0.650602
District8           2.776e+04  6.342e+03   4.377 1.21e-05 ***
District9           4.474e+04  4.227e+03  10.584  < 2e-16 ***
District10          4.320e+04  4.349e+03   9.933  < 2e-16 ***
District11          7.275e+04  3.961e+03  18.367  < 2e-16 ***
District12          3.594e+04  6.767e+03   5.311 1.10e-07 ***
District13          7.372e+04  3.639e+03  20.259  < 2e-16 ***
District14          1.137e+05  3.947e+03  28.808  < 2e-16 ***
District15         -4.184e+04  6.360e+03  -6.579 4.85e-11 ***
Sale_date           4.626e+00  2.437e-01  18.986  < 2e-16 ***
Year_Built          4.837e+02  1.637e+01  29.545  < 2e-16 ***
Lotsize             3.917e+00  4.486e-01   8.731  < 2e-16 ***
District2:Lotsize  -1.468e+00  6.342e-01  -2.315 0.020617 *
District4:Lotsize  -7.717e+00  1.220e+00  -6.326 2.56e-10 ***
District5:Lotsize  -1.621e+00  4.813e-01  -3.369 0.000756 ***
District6:Lotsize  -3.913e+00  8.747e-01  -4.474 7.71e-06 ***
District7:Lotsize   8.121e-01  1.121e+00   0.724 0.468874
District8:Lotsize  -6.929e-01  1.338e+00  -0.518 0.604459
District9:Lotsize  -1.688e+00  5.178e-01  -3.259 0.001118 **
District10:Lotsize  4.870e+00  7.035e-01   6.922 4.58e-12 ***
District11:Lotsize -3.962e-01  5.584e-01  -0.710 0.477954
District12:Lotsize -5.923e+00  1.572e+00  -3.767 0.000165 ***
District13:Lotsize -7.282e-01  5.021e-01  -1.450 0.147014
District14:Lotsize -1.560e+00  6.428e-01  -2.427 0.015237 *
District15:Lotsize  4.381e+00  1.138e+00   3.849 0.000119 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 40220 on 22888 degrees of freedom
Multiple R-squared:  0.5189,    Adjusted R-squared:  0.5183
F-statistic: 822.8 on 30 and 22888 DF,  p-value: < 2.2e-16
```

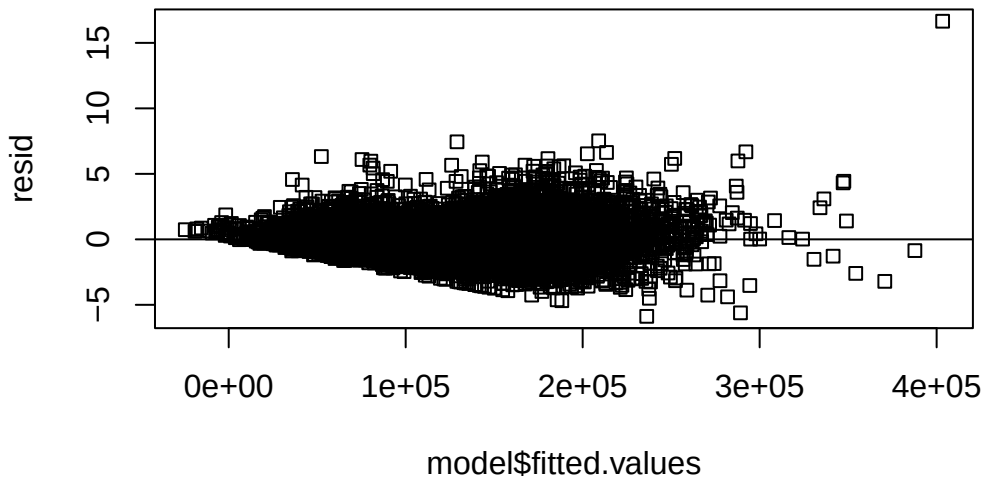
```r
resid=rstudent(model)

car::qqPlot(resid)
```

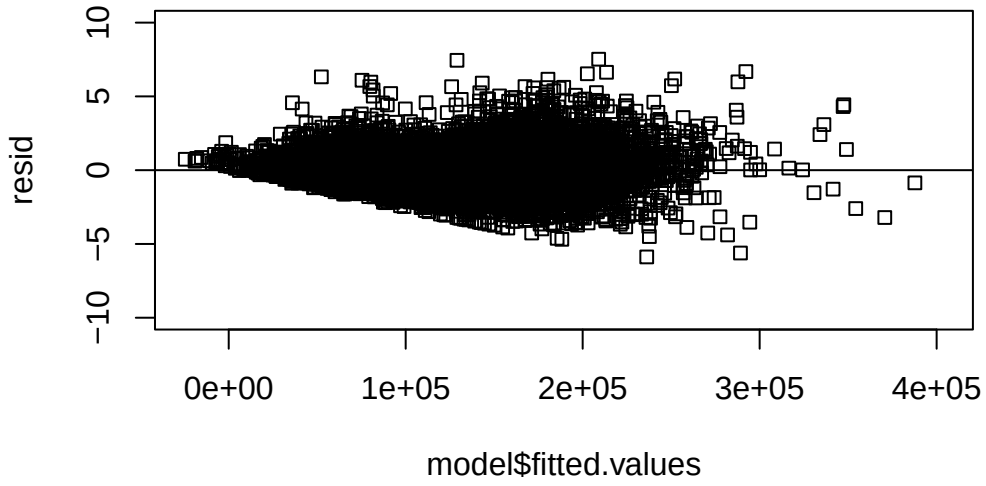



```
23299   1391
21669   1327
```

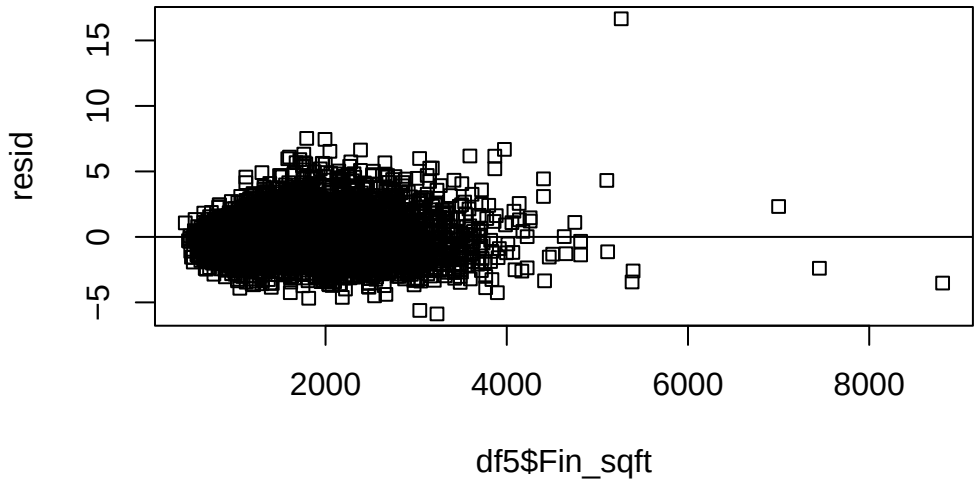
```
# roblm
plot(model$fitted.values,resid )
abline(h=0)
```




```
plot(model$fitted.values,resid,ylim=c(-10,10) )
abline(h=0)
```

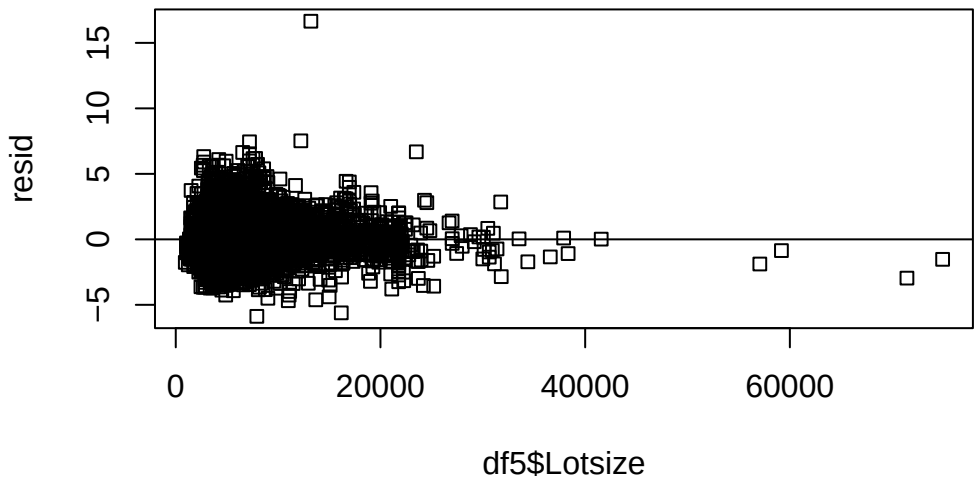


```r
plot(df5$Fin_sqft,resid )
abline(h=0)
```



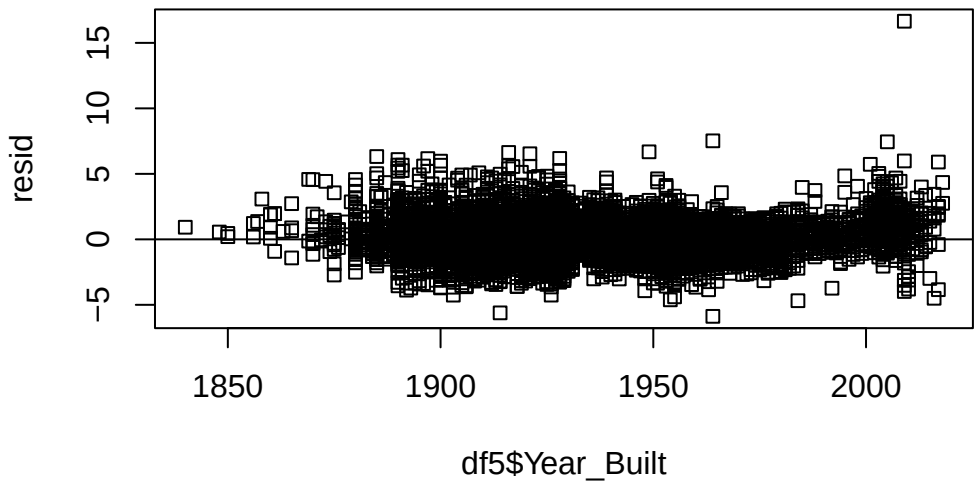

```r
# just for comparison, we look at the
# plot(df3$Fin_sqft,model$residuals )
# abline(h=0)


plot(df5$Lotsize,resid )
abline(h=0)
```

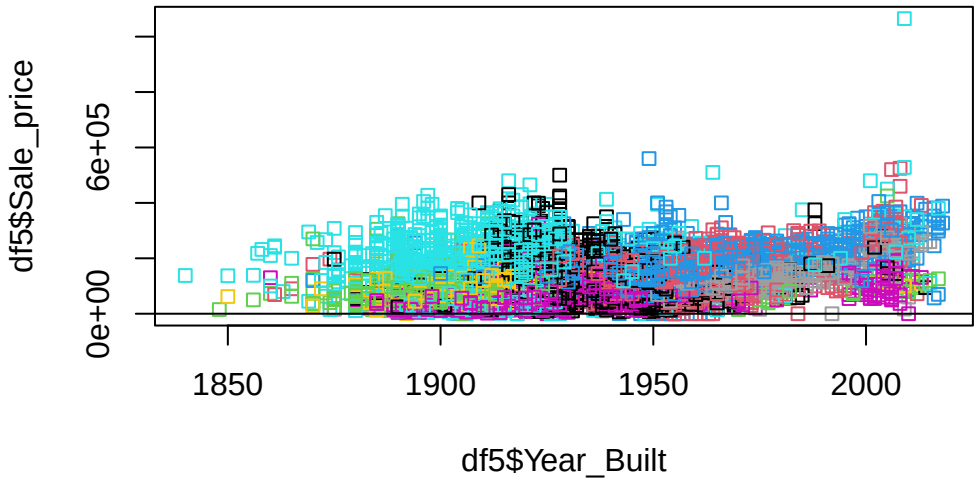



```
# non constant variance


plot(df5$Year_Built,resid )
abline(h=0)
```

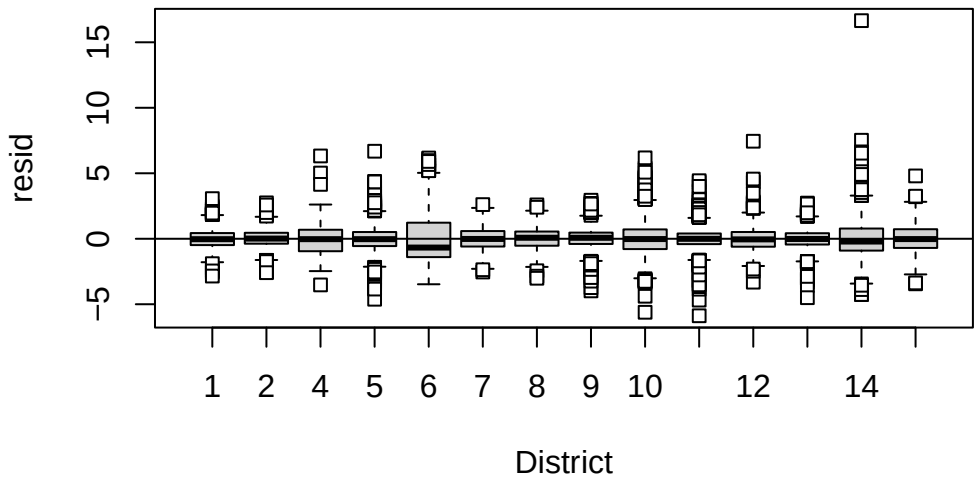



```
plot(df5$Year_Built,df5$Sale_price ,col=df5$District)
abline(h=0)
```



```r
# non constant variance

rdf=df5
rdf=cbind(df5,resid)
boxplot(resid~District,rdf )
abline(h=0)
```




```r
###################

model=lm(sqrt(Sale_price)~Fin_sqft+District+Sale_date+ Year_Built+ Lotsize,data=df5)
summary(model)
```

```
Call:
lm(formula = sqrt(Sale_price) ~ Fin_sqft + District + Sale_date +
    Year_Built + Lotsize, data = df5)

Residuals:
    Min      1Q  Median      3Q     Max
-493.55  -31.58    1.65   32.78  338.94

Coefficients:
```

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.325e+03  4.517e+01 -29.335  < 2e-16 ***
Fin_sqft     5.738e-02  7.698e-04  74.538  < 2e-16 ***
District2    2.832e+01  2.384e+00  11.879  < 2e-16 ***
District4   -1.029e+01  4.958e+00  -2.075    0.038 *
District5    9.542e+01  2.049e+00  46.558  < 2e-16 ***
District6    1.722e+01  2.968e+00   5.802 6.63e-09 ***
District7    2.445e+00  2.562e+00   0.954    0.340
District8    4.274e+01  2.824e+00  15.137  < 2e-16 ***
District9    5.978e+01  2.560e+00  23.348  < 2e-16 ***
District10   1.084e+02  2.140e+00  50.640  < 2e-16 ***
District11   1.140e+02  2.051e+00  55.606  < 2e-16 ***
District12   1.822e+01  3.417e+00   5.333 9.75e-08 ***
District13   1.125e+02  2.121e+00  53.040  < 2e-16 ***
District14   1.555e+02  2.155e+00  72.134  < 2e-16 ***
District15  -3.837e+01  3.174e+00 -12.087  < 2e-16 ***
Sale_date    6.408e-03  3.473e-04  18.449  < 2e-16 ***
Year_Built   7.087e-01  2.312e-02  30.660  < 2e-16 ***
Lotsize      3.560e-03  1.486e-04  23.953  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 57.35 on 22901 degrees of freedom
Multiple R-squared:  0.5259,    Adjusted R-squared:  0.5256
F-statistic:  1494 on 17 and 22901 DF,  p-value: < 2.2e-16
```

```
model=lm(sqrt(Sale_price)~Fin_sqft+District+Sale_date+ Year_Built+ Lotsize+District*Lotsize,
summary(model)
```

```
Call:
lm(formula = sqrt(Sale_price) ~ Fin_sqft + District + Sale_date +
    Year_Built + Lotsize + District * Lotsize, data = df5)

Residuals:
    Min      1Q  Median      3Q     Max
-494.63  -31.21    1.80   32.67  325.84

Coefficients:
                     Estimate Std. Error t value Pr(>|t|)
(Intercept)        -1.332e+03  4.561e+01 -29.200  < 2e-16 ***
Fin_sqft            5.776e-02  7.710e-04  74.919  < 2e-16 ***
```

```
District2           5.082e+01  6.350e+00   8.002 1.28e-15 ***
District4           4.973e+01  1.054e+01   4.719 2.38e-06 ***
District5           1.290e+02  4.890e+00  26.382  < 2e-16 ***
District6           5.545e+01  7.050e+00   7.865 3.84e-15 ***
District7          -1.012e+01  8.979e+00  -1.127 0.259839
District8           4.534e+01  8.985e+00   5.046 4.54e-07 ***
District9           9.039e+01  5.989e+00  15.094  < 2e-16 ***
District10          9.443e+01  6.162e+00  15.326  < 2e-16 ***
District11          1.350e+02  5.611e+00  24.060  < 2e-16 ***
District12          5.489e+01  9.586e+00   5.726 1.04e-08 ***
District13          1.351e+02  5.155e+00  26.212  < 2e-16 ***
District14          1.855e+02  5.592e+00  33.180  < 2e-16 ***
District15         -8.533e+01  9.010e+00  -9.471  < 2e-16 ***
Sale_date           6.393e-03  3.452e-04  18.520  < 2e-16 ***
Year_Built          6.996e-01  2.319e-02  30.166  < 2e-16 ***
Lotsize             7.269e-03  6.355e-04  11.438  < 2e-16 ***
District2:Lotsize  -3.485e-03  8.984e-04  -3.879 0.000105 ***
District4:Lotsize  -1.073e-02  1.728e-03  -6.209 5.42e-10 ***
District5:Lotsize  -5.059e-03  6.819e-04  -7.420 1.22e-13 ***
District6:Lotsize  -6.919e-03  1.239e-03  -5.583 2.39e-08 ***
District7:Lotsize   3.263e-03  1.588e-03   2.054 0.039982 *
District8:Lotsize   1.260e-03  1.895e-03   0.665 0.505954
District9:Lotsize  -4.381e-03  7.336e-04  -5.973 2.37e-09 ***
District10:Lotsize  3.349e-03  9.967e-04   3.360 0.000781 ***
District11:Lotsize -3.274e-03  7.911e-04  -4.139 3.51e-05 ***
District12:Lotsize -7.268e-03  2.227e-03  -3.263 0.001103 **
District13:Lotsize -3.528e-03  7.113e-04  -4.960 7.12e-07 ***
District14:Lotsize -5.048e-03  9.107e-04  -5.543 3.01e-08 ***
District15:Lotsize  1.043e-02  1.612e-03   6.471 9.95e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 56.98 on 22888 degrees of freedom
Multiple R-squared:  0.5323,    Adjusted R-squared:  0.5317
F-statistic: 868.3 on 30 and 22888 DF,  p-value: < 2.2e-16
```

```
model=lm(sqrt(Sale_price)~Fin_sqft+District+Sale_date+ Year_Built,data=df5)
summary(model)
```

```
Call:
lm(formula = sqrt(Sale_price) ~ Fin_sqft + District + Sale_date +
```

```
    Year_Built, data = df5)

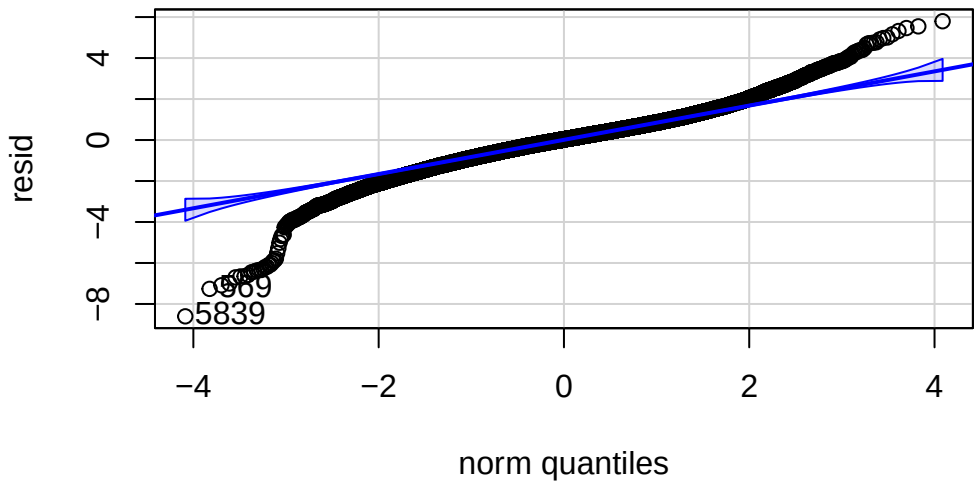
Residuals:
    Min      1Q  Median      3Q     Max
-498.58  -32.16    1.67   33.23  335.25

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.531e+03  4.490e+01 -34.094  < 2e-16 ***
Fin_sqft     6.123e-02  7.622e-04  80.335  < 2e-16 ***
District2    2.779e+01  2.413e+00  11.515  < 2e-16 ***
District4   -1.469e+01  5.016e+00  -2.930 0.003398 **
District5    9.645e+01  2.075e+00  46.493  < 2e-16 ***
District6    1.261e+01  2.999e+00   4.205 2.62e-05 ***
District7   -2.070e+00  2.587e+00  -0.800 0.423680
District8    3.695e+01  2.848e+00  12.974  < 2e-16 ***
District9    6.851e+01  2.566e+00  26.702  < 2e-16 ***
District10   1.047e+02  2.161e+00  48.438  < 2e-16 ***
District11   1.147e+02  2.076e+00  55.245  < 2e-16 ***
District12   1.169e+01  3.449e+00   3.389 0.000703 ***
District13   1.146e+02  2.145e+00  53.425  < 2e-16 ***
District14   1.510e+02  2.174e+00  69.468  < 2e-16 ***
District15  -4.390e+01  3.205e+00 -13.697  < 2e-16 ***
Sale_date    6.402e-03  3.516e-04  18.206  < 2e-16 ***
Year_Built   8.237e-01  2.289e-02  35.984  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 58.06 on 22902 degrees of freedom
Multiple R-squared:  0.5141,    Adjusted R-squared:  0.5137
F-statistic:  1514 on 16 and 22902 DF,  p-value: < 2.2e-16
```

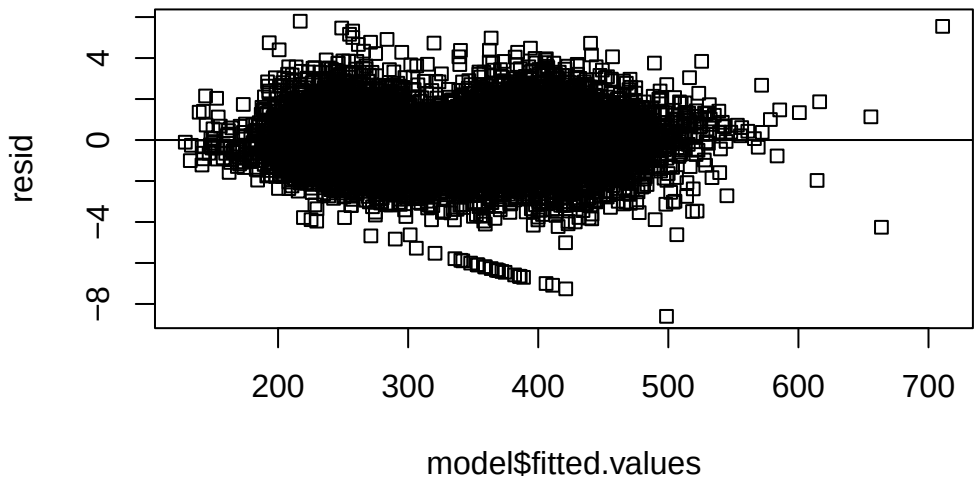
```r
resid=rstudent(model)

car::qqPlot(resid)
```

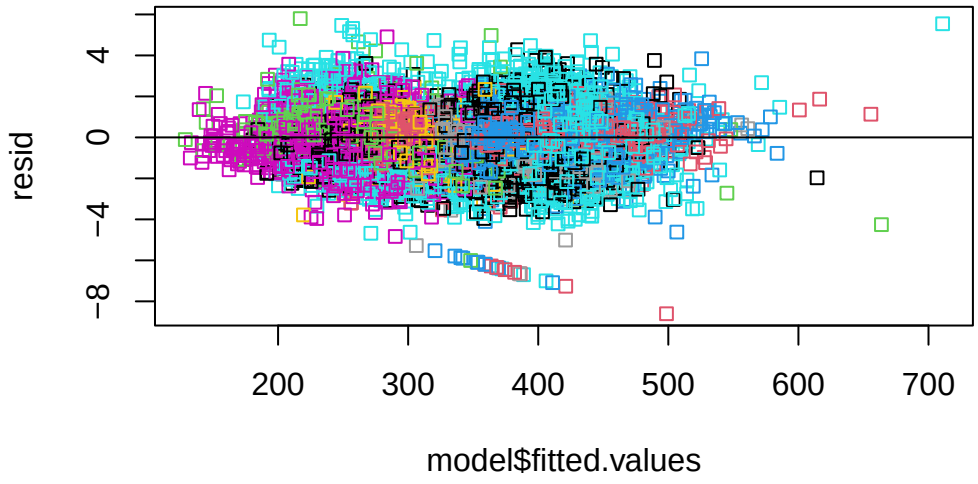



```
5839  569
5501  529
```

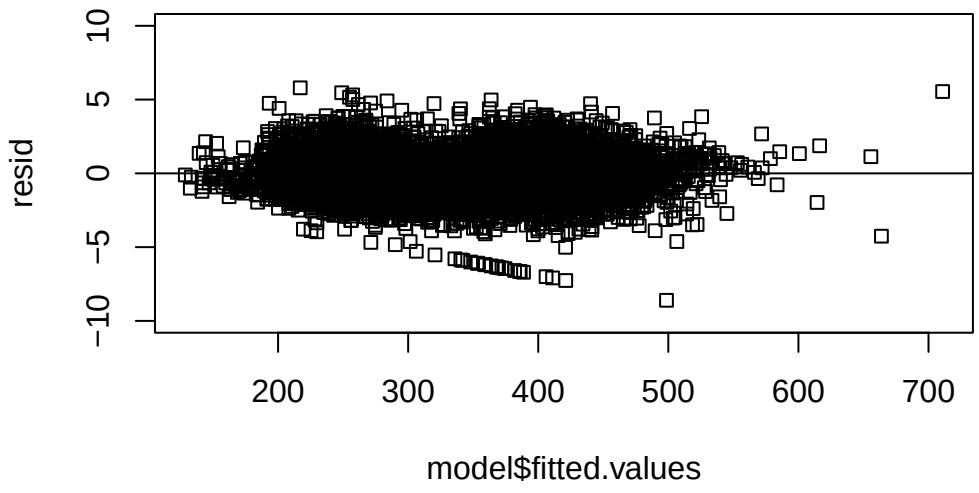
```
# roblm
plot(model$fitted.values,resid )
abline(h=0)
```



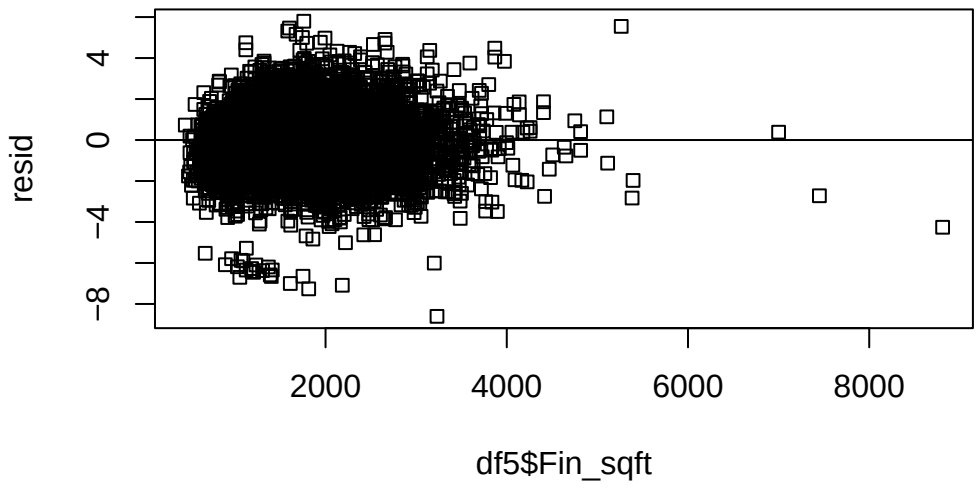

```
plot(model$fitted.values,resid,col=df5$District )
abline(h=0)
```



```
plot(model$fitted.values,resid,ylim=c(-10,10) )
abline(h=0)
```

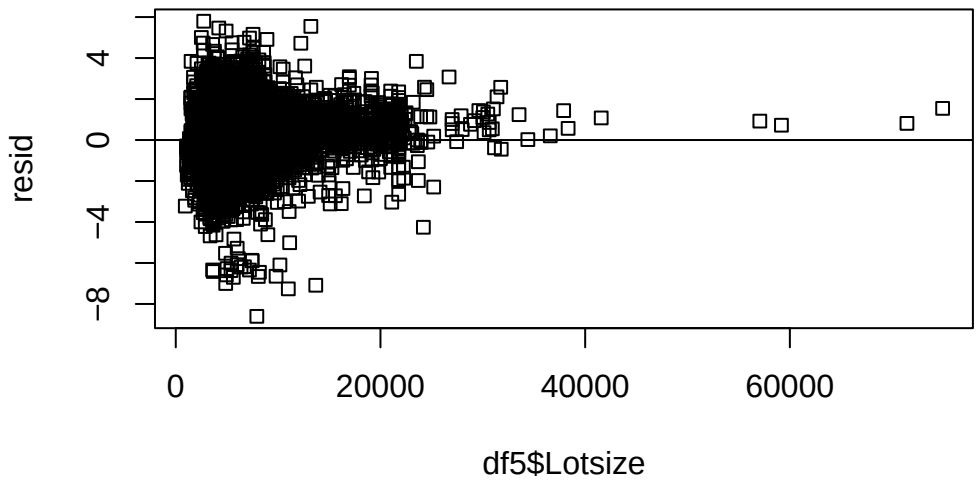



```
plot(df5$Fin_sqft,resid )
abline(h=0)
```



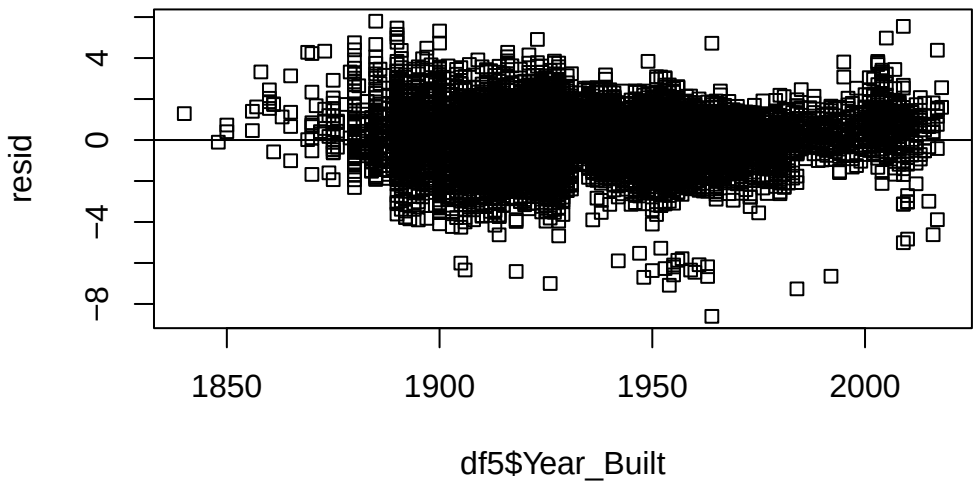

```
# just for comparison, we look at the
# plot(df3$Fin_sqft,model$residuals )
# abline(h=0)


plot(df5$Lotsize,resid )
abline(h=0)
```

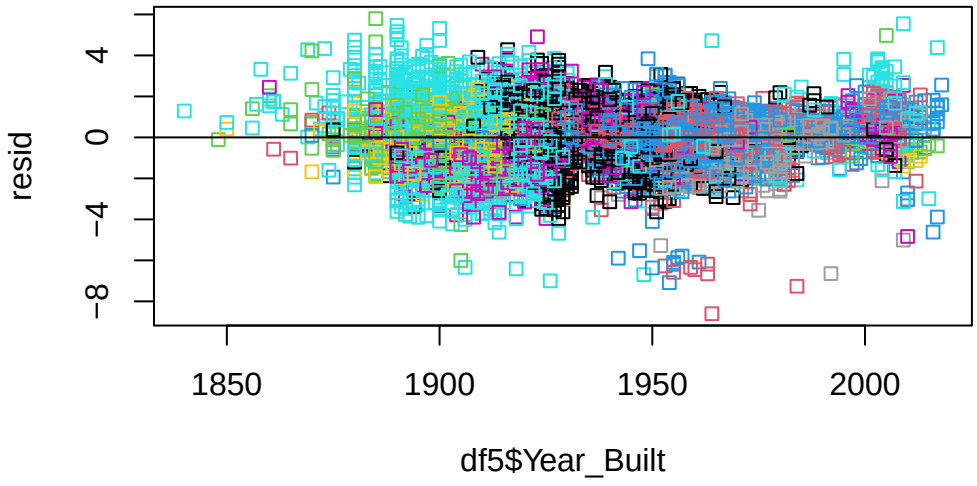



```
# non constant variance

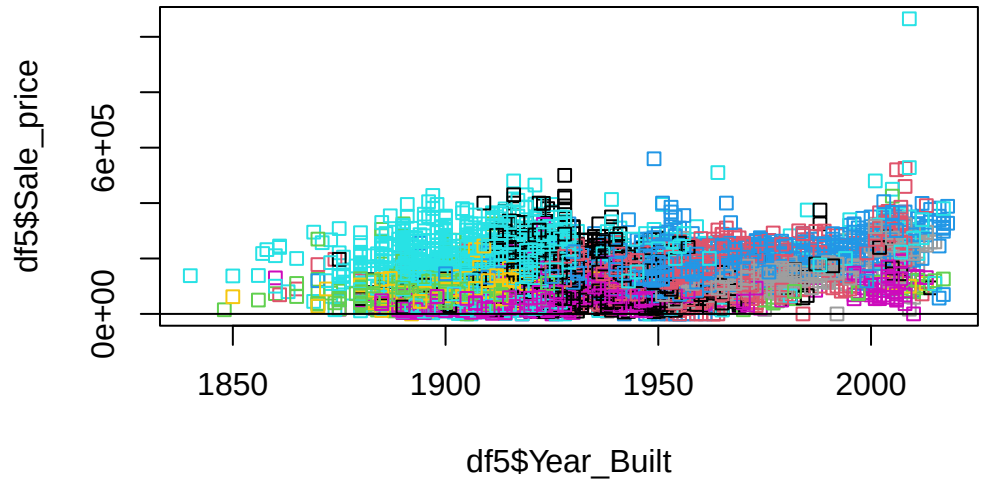

plot(df5$Year_Built,resid )
abline(h=0)
```




```
plot(df5$Year_Built,resid ,col=df5$District)
abline(h=0)
```

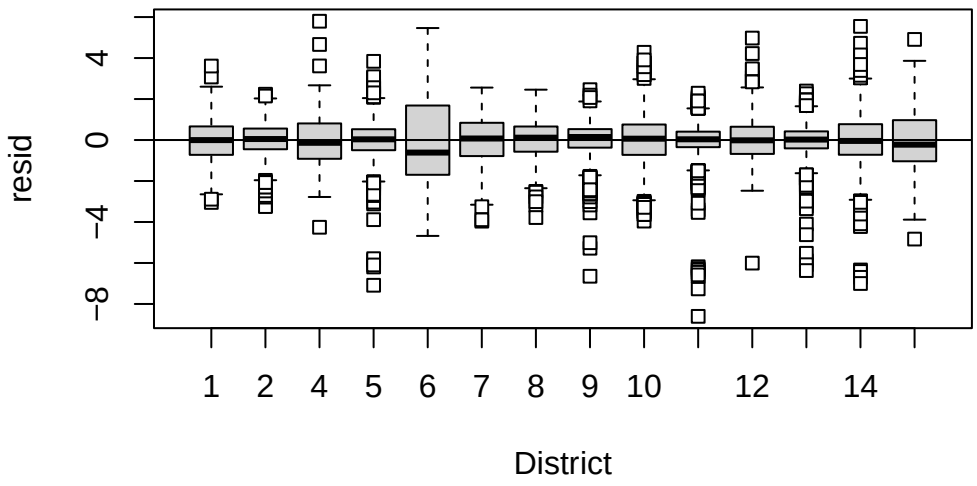


```
plot(df5$Year_Built,df5$Sale_price ,col=df5$District)
abline(h=0)
```




```
# non constant variance

rdf=df5
rdf=cbind(df5,resid)
boxplot(resid~District,rdf )
abline(h=0)
```



6.5 Homework questions

Exercise 6.2. Consider a ML regression model for bread quality against bake time and 3 types of yeast (A, B and C). Write out the dummy variables for the variable yeast type. What is the interpretation of the coefficient of each of the dummy variables, in this context?

Exercise 6.3. Suppose we regress real estate price against number of bathrooms. What is the difference in interpretation between representing number of bedrooms with dummy variables versus a continuous variable?

Exercise 6.4. Consider a ML regression model for bread quality against bake time and 3 types of yeast (A, B and C). Write out the regression equation that includes an interaction between yeast and bake time. What is the interpretation of the coefficient for each of the interaction effects? Compare and contrast the regression model in Exercise 6.2 to this one.

Complete the Chapter 8 questions.

7 Leverage and Influence

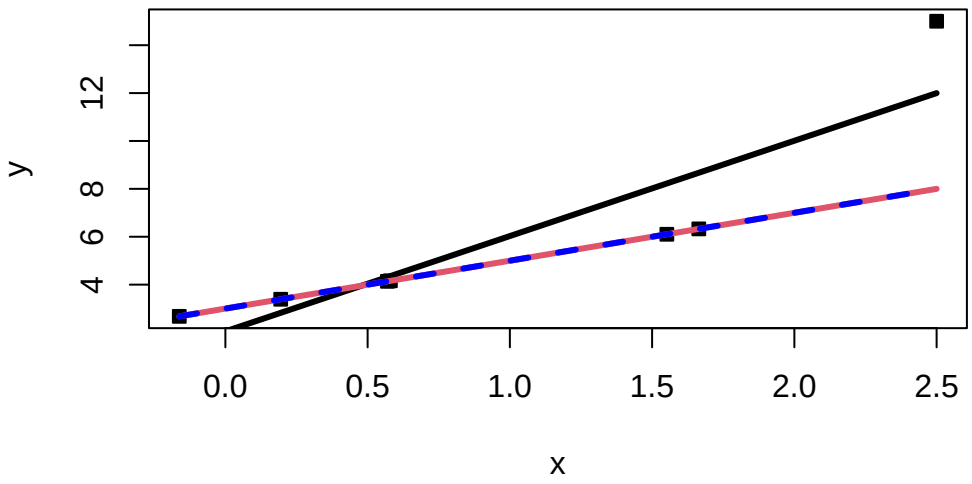
7.1 Influential observations and leverage

Recall that violations of model assumptions are more likely at remote points, and these violations may be hard to detect from inspection of the ordinary residuals because their residuals will usually be smaller. Points that are outlying in the x-direction are known as **leverage points**. **Influential points** are not only remote in terms of the specific values for the regressors, but the observed response is not consistent with the values that would be predicted based on only the other data points. It is important to find these influential points and assess their impact on the model.

Below gives an example of an influential point. The seventh point in the data set is outlying in the x-direction, and it's response value is not consistent with the regression line based on the other six observations:

```
set.seed(330)
x=c(rnorm(6),2.5)
y=x*2+3
y[7]=y[7]+7
plot(x,y,pch=22,bg=1)
a=lm(y~x)
curve(a$coefficients[1]+x*a$coefficients[2],add=T,lwd=3)
curve(x*2+3,add=T,col=2,lwd=3)

a2=lm(y[-7]~x[-7])
curve(a2$coefficients[1]+x*a2$coefficients[2],add=T,lwd=3,col='blue',lty=2)
```




```
a$coefficients
```

```
(Intercept)           x
   2.048937    3.979977
```

Sometimes we find that a regression coefficient may have a sign that does not make engineering or scientific sense, a regressor known to be important may be statistically insignificant, or a model that fits the data well and that is logical from an application – environment perspective may produce poor predictions. These situations may be the result of one or, perhaps, a few influential observations.

Recall the hat matrix $H = X(X^\top X)^{-1}X^\top$, as well as that it holds that $\text{Var}\left[\widehat{\epsilon}\right] = \sigma^2(I - H)$ and $\text{Var}\left[\widehat{Y}\right] = \sigma^2 H$. Note that h_{ij} can be interpreted as the amount of leverage exerted by the ith observation y_i on the jth fitted value $\widehat{y}_j$. We usually focus attention on the diagonal elements h_{ii} of the hat matrix H, which may be written as

$$h_{ii} = x_i^\top (X^\top X)^{-1} x_i,$$

where $X_i^\top$ is the ith row of X. The hat matrix diagonal is a standardized measure of the distance of the ith observation from the center (or centroid) of the x-space. Therefore, large values of h_{ii} implies that x_i is potentially influential. Furthermore, note that $rank(H) = p$ since the trace of an idempotent matrix equals its rank, which means that $\bar{h} = p/n$. It follows that values well above p/n, say $h_{ii} > 2p/n$, can be called leverage points.

```r
X=as.matrix(cbind(rep(1,length(x)),x))
# or

X=model.matrix(a)
hat=X%*%solve(t(X)%*%X)%*%t(X)

diag(hat)
```

```
        1         2         3         4         5         6         7
0.2027453 0.2288737 0.2596869 0.1751432 0.1735495 0.3887329 0.5712686
```

```r
p=2
n=7
diag(hat)>2*p/n
```

```
    1     2     3     4     5     6     7
FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

7.2 Cook's Distance

Cook's Distance is one way to incorporate both the X and Y values into an outlyingness measure:

$$D_i\left(X^\top X, p, MSE\right) \equiv D_i = \frac{\left(\hat{\beta}_{(i)} - \hat{\beta}\right)^\top X^\top X \left(\hat{\beta}_{(i)} - \hat{\beta}\right)}{pMSE}, \; i \in [n],$$

where $\hat{\beta}_{(i)}$ is the OLS estimator with the ith point removed.Large values of Cook's distance signal a leverage point.

What do we mean by a large value? We can compare D_i to the 50th percentile of the $F_{p,n-p}$ distribution. This gives the interpretation that deleting the ith point moves the estimate to the boundary of a 50% confidence interval. $F_{p,n-p} \approx 1$, and so usually take $D_i \geq 1$ to be large.

Observe that

$$D_i = \frac{r_i^2}{p} \frac{\operatorname{Var}\left(\hat{Y}_i\right)}{\operatorname{Var}\left(\hat{\epsilon}_i\right)} = \frac{r_i^2}{p} \frac{h_{ii}}{1 - h_{ii}}, \quad i = 1, 2, \ldots, n,$$

where it is important to recall that r_i is the studentized residual. Now, the quantity $\frac{h_{ii}}{1-h_{ii}}$ can be shown to be the distance from the vector x_i to the centroid of the remaining data. Therefore, D_i is the product of outlyingness in both the X and Y directions. We may also write D_i as

$$D_i = \frac{\left\|\hat{y}_{(i)} - \hat{y}\right\|^2}{pMSE},$$

which allows for the interpretation: The Cook's distance of the ith point is the normalized distance between the fitted value with and without point i.

```
#cut off
cooks.distance(a)
```

```
           1            2            3            4            5            6
0.1708029420 0.2516095165 0.0180669722 0.0009569213 0.0011772793 0.2002829110
           7
3.3311562309
```

```
cooks.distance(a)>1
```

```
    1     2     3     4     5     6     7
FALSE FALSE FALSE FALSE FALSE FALSE  TRUE
```

```
df=data.frame(cbind(y,x))
df[cooks.distance(a)>1,]
```

```
   y   x
7 15 2.5
```

7.3 Data depth functions

A more modern approach and nonparametric approach to outlier detection is through data depth. A data depth function gives meaning to centrality, order and outlyingness in spaces beyond $\mathbb{R}$. A data depth function is a function which takes a sample and a point, and returns how central the point is, with respect to the sample. Depth functions can be written as $\mathrm{D}\colon \mathbb{R}^d \times \text{Sample} \to \mathbb{R}^+$. There are different definitions of depth, so I will give a few.

Let $S^{d-1} = \{x \in \mathbb{R}^d \colon \|x\| = 1\}$ be the set of unit vectors in $\mathbb{R}^d$, let $\mathbb{X}_n = \{(Y_1, X_{1,1}, \ldots, X_{1,p-1}), \ldots, (Y_n, X_{n,1}, \ldots, X_{n,p-1})\}$ be the sample, let $\mathbb{X}_n^\top u$ be $\mathbb{X}_n$ projected onto $u \in S^{d-1}$ and let $\hat{F}_u$ be the empirical CDF with respect to $\mathbb{X}_n^\top u$.

The halfspace depth D_H of a point $x \in \mathbb{R}^d$ with respect to a distribution F over $\mathbb{R}^d$ is

$$\mathrm{D}_H(x; F) = \inf_{u \in S^{d-1}} \hat{F}_u(x^\top u) \wedge (1 - F_u(x^\top u)) = \inf_{u \in S^{d-1}} F_u(x^\top u).$$

Given a translation and scale equivariant location estimate μ and a translation and scale invariant scale estimate σ, the outlyingness at $x \in \mathbb{R}^d$ is defined as

$$O(x) = \sup_{u \in S^{d-1}} \frac{\left| x^\top u - \mu(\mathbb{X}_n^\top u) \right|}{\sigma(\mathbb{X}_n^\top u)}.$$

Define projection depth as

$$\mathrm{D}_p(x) = (1 + O(x))^{-1}.$$

In order to detect outliers, we look for observations that have low depth. See, continuing our toy example:

```
# install.packages('ddalpha')
depths=ddalpha::depth.projection(cbind(x,y),cbind(x,y))
depths
```

```
[1] 0.2764090105 0.2552720744 0.5000000000 0.9737543285 0.9730469270
[6] 0.3389544152 0.0001124812
```

```
depths<0.015
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE  TRUE
```

Example 7.1. Recall example Example 6.6. Check for leverage and influential points in the proposed models. Compute all three measures of leverage/influence/outlyingness introduced in this lesson. What do you find?

I will load in the data below:

We can now analyze the data:

```
####### Loading data  ###############

df_clean2=read.csv('C:/Users/12RAM/OneDrive - York University/Teaching/Courses/Math 3330 Regr

####### Analyzing the data via EDA  ###############

names(df_clean2)
```

```
 [1] "District"   "Extwall"    "Stories"    "Year_Built" "Fin_sqft"
 [6] "Units"      "Bdrms"      "Fbath"      "Lotsize"    "Sale_date"
[11] "Sale_price"
```

```r
# Convert to factors
df_clean2$District=as.factor(df_clean2$District)
df_clean2$Extwall=as.factor(df_clean2$Extwall)
df_clean2$Stories=as.factor(df_clean2$Stories)
df_clean2$Fbath=as.factor(df_clean2$Fbath)
df_clean2$Bdrms=as.factor(df_clean2$Bdrms)
df_clean2$Units=as.factor(df_clean2$Units)
# df_clean2=df_clean2[,c('Sale_price','Fin_sqft',
#                        'District','Sale_date','Year_Built','Lotsize')]

# We are not going to remove the outliers that we found earlier , and see if these diagnostic
# remove those with 0 lot size
# df3=df_clean2[df_clean2$Lotsize>0,]
# df4=df3[df3$Lotsize<150000,]
df5=df_clean2[df_clean2$District!=3,]
df5$District=droplevels(df5$District)

# Old model
model=lm(sqrt(Sale_price)~Fin_sqft+District+Sale_date+ Year_Built,data=df5)
summary(model)
```

```
Call:
lm(formula = sqrt(Sale_price) ~ Fin_sqft + District + Sale_date +
    Year_Built, data = df5)

Residuals:
    Min      1Q  Median      3Q     Max
-497.28  -32.37    1.59   33.15  376.67

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.501e+03  4.491e+01 -33.426  < 2e-16 ***
Fin_sqft     6.069e-02  7.617e-04  79.682  < 2e-16 ***
District2    2.803e+01  2.427e+00  11.550  < 2e-16 ***
District4   -1.394e+01  5.029e+00  -2.772  0.00558 **
District5    9.666e+01  2.086e+00  46.328  < 2e-16 ***
District6    1.509e+01  2.995e+00   5.038 4.73e-07 ***
```

```
District7   -1.979e+00  2.601e+00  -0.761  0.44683
District8    3.787e+01  2.841e+00  13.331  < 2e-16 ***
District9    6.892e+01  2.580e+00  26.712  < 2e-16 ***
District10   1.047e+02  2.173e+00  48.196  < 2e-16 ***
District11   1.149e+02  2.088e+00  55.000  < 2e-16 ***
District12   1.583e+01  3.399e+00   4.656 3.23e-06 ***
District13   1.149e+02  2.157e+00  53.271  < 2e-16 ***
District14   1.513e+02  2.185e+00  69.223  < 2e-16 ***
District15  -4.360e+01  3.204e+00 -13.610  < 2e-16 ***
Sale_date    6.514e-03  3.528e-04  18.461  < 2e-16 ***
Year_Built   8.079e-01  2.290e-02  35.284  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 58.4 on 23020 degrees of freedom
Multiple R-squared:  0.5116,    Adjusted R-squared:  0.5112
F-statistic:  1507 on 16 and 23020 DF,  p-value: < 2.2e-16
```

```r
# All variables - singular
# model=lm(sqrt(Sale_price)~.,data=df5)
s=summary(model)

# Compute residuals
student_res2=rstudent(model)
summ2=summary(model); summ2
```

```
Call:
lm(formula = sqrt(Sale_price) ~ Fin_sqft + District + Sale_date +
    Year_Built, data = df5)

Residuals:
    Min      1Q  Median      3Q     Max
-497.28  -32.37    1.59   33.15  376.67

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.501e+03  4.491e+01 -33.426  < 2e-16 ***
Fin_sqft     6.069e-02  7.617e-04  79.682  < 2e-16 ***
District2    2.803e+01  2.427e+00  11.550  < 2e-16 ***
District4   -1.394e+01  5.029e+00  -2.772  0.00558 **
District5    9.666e+01  2.086e+00  46.328  < 2e-16 ***
```

```
District6    1.509e+01  2.995e+00   5.038 4.73e-07 ***
District7   -1.979e+00  2.601e+00  -0.761  0.44683
District8    3.787e+01  2.841e+00  13.331  < 2e-16 ***
District9    6.892e+01  2.580e+00  26.712  < 2e-16 ***
District10   1.047e+02  2.173e+00  48.196  < 2e-16 ***
District11   1.149e+02  2.088e+00  55.000  < 2e-16 ***
District12   1.583e+01  3.399e+00   4.656 3.23e-06 ***
District13   1.149e+02  2.157e+00  53.271  < 2e-16 ***
District14   1.513e+02  2.185e+00  69.223  < 2e-16 ***
District15  -4.360e+01  3.204e+00 -13.610  < 2e-16 ***
Sale_date    6.514e-03  3.528e-04  18.461  < 2e-16 ***
Year_Built   8.079e-01  2.290e-02  35.284  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

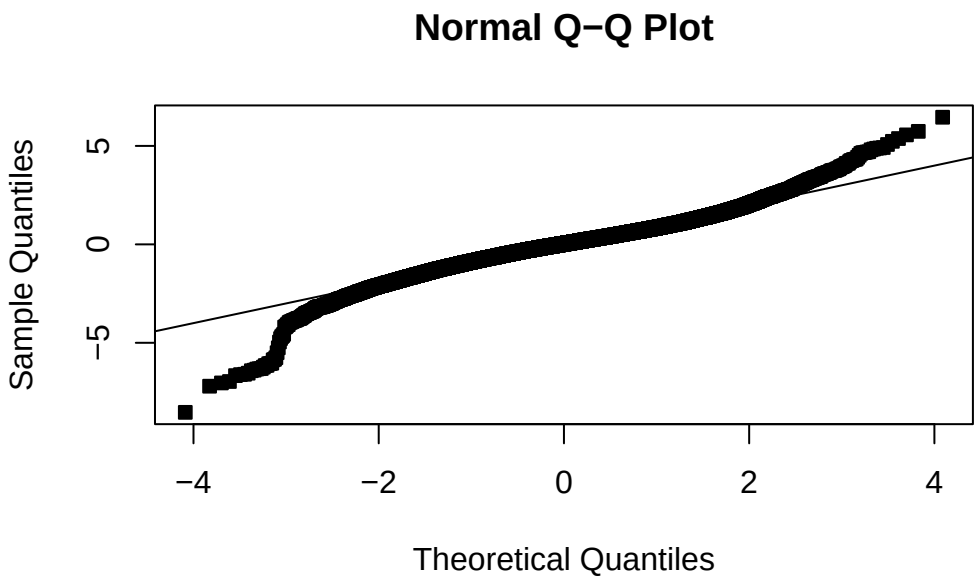
Residual standard error: 58.4 on 23020 degrees of freedom
Multiple R-squared:  0.5116,    Adjusted R-squared:  0.5112
F-statistic:  1507 on 16 and 23020 DF,  p-value: < 2.2e-16
```

```
summ2$adj.r.squared
```

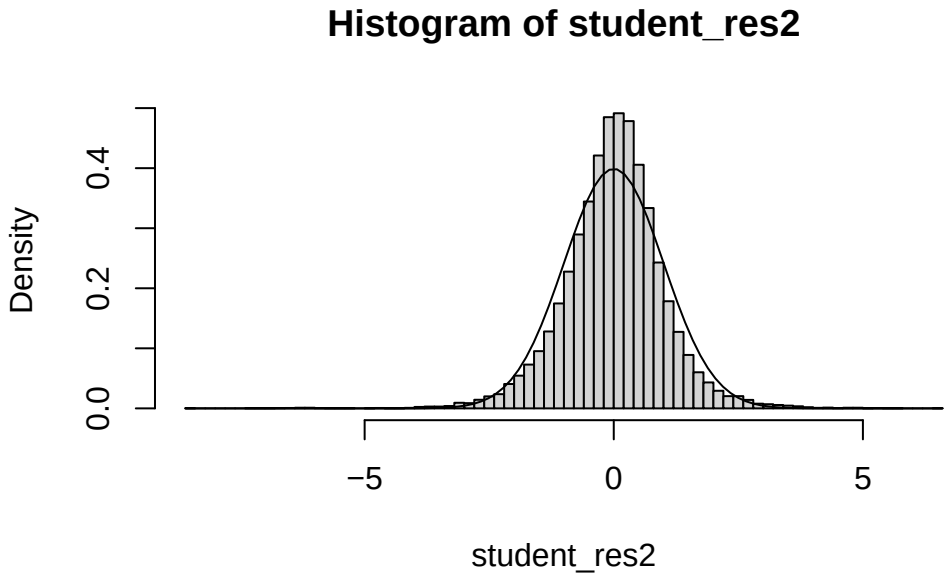
```
[1] 0.5112154
```

```
# Compute residual analysis

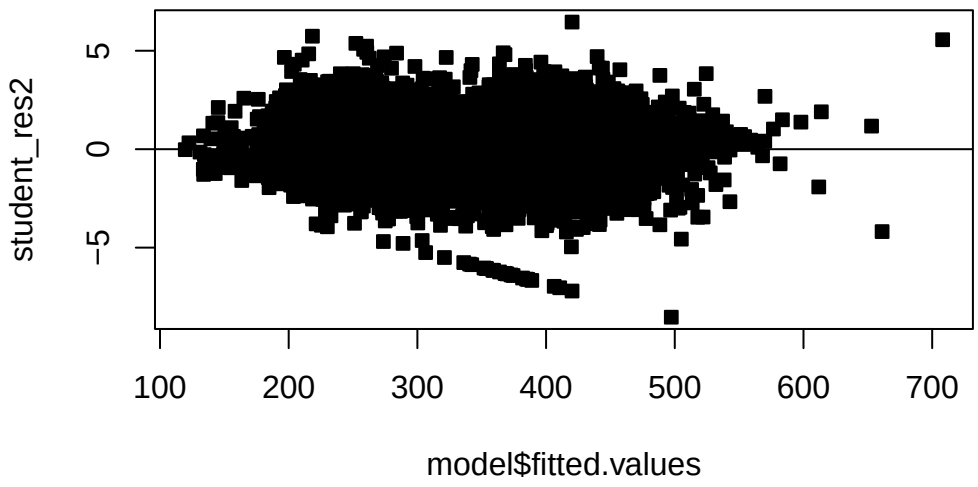

MSE2=summ2$sigma^2
qqnorm(student_res2,pch=22,bg=1)
abline(0,1)
```




```
hist(student_res2,freq=F,breaks=100)
curve(dnorm(x,0,1),add=T)
```



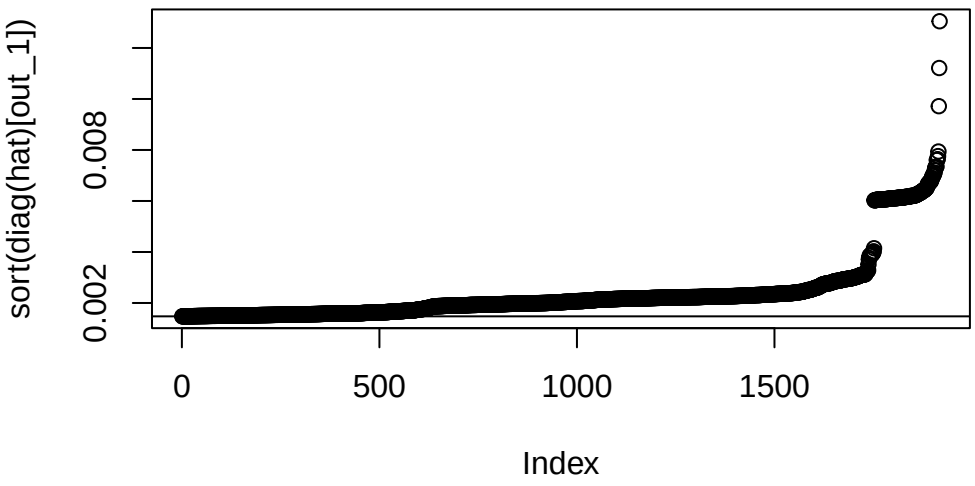
```r
# hist(student_res2,freq=F,breaks=100)
# curve(dnorm(x,0,1),add=T)
plot(model$fitted.values,student_res2,pch=22,bg=1)
abline(h=0)
```




```r
# First measure

X=model.matrix(model)
hat=X%*%solve(t(X)%*%X)%*%t(X)

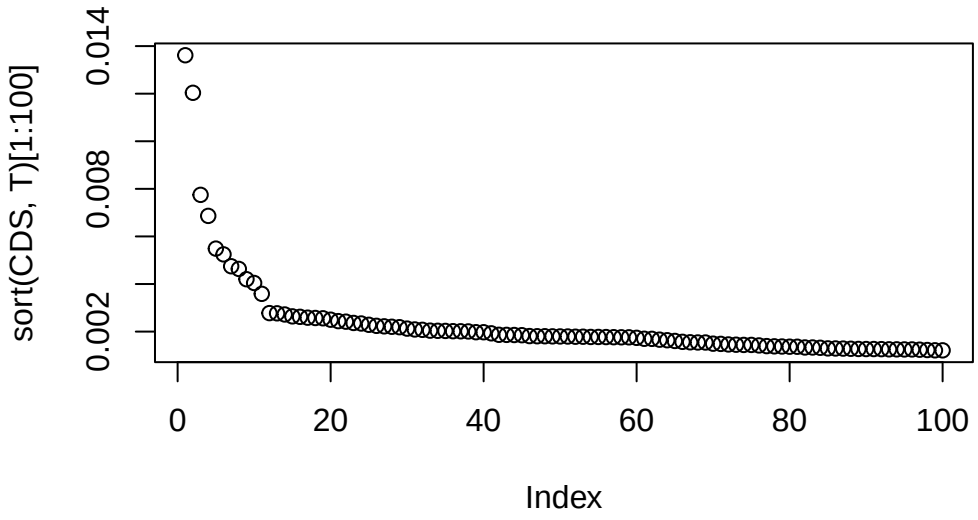
# diag(hat)
p=ncol(X)
n=nrow(X)
out_1=which(diag(hat)>2*p/n)
plot(sort(diag(hat)[out_1]))
abline(h=2*p/n)
```




```
# I would still look at those after the elbow
# There is a large break


# Cooks distances

CDS=cooks.distance(model)
plot(sort(CDS,T)[1:100])
```




```
which(CDS>1)
```

```
named integer(0)
```

```
max(CDS)
```

```
[1] 0.01361538
```

```
df[CDS>1,]
```

```
[1] y x
<0 rows> (or 0-length row.names)
```

```
# I would still look at those two values that are
# far from the other distances
# I would still look at those before the elbow



# We may only look at numeric values for depth functions - so we can either
```

```
numer=NULL

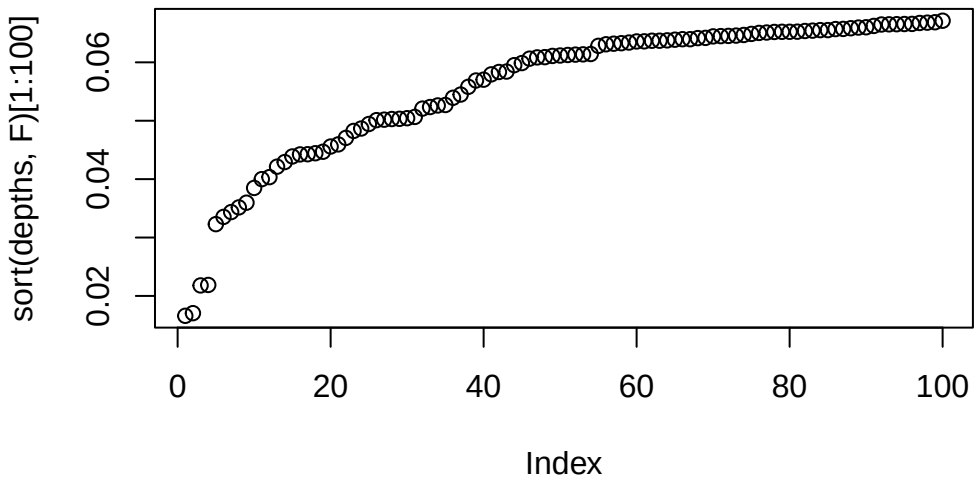
for(i in names(df5)){
  if(!is.factor(df5[1,i])){
    numer=c(numer,i)
  }
}
numer
```

```
[1] "Year_Built" "Fin_sqft"   "Lotsize"    "Sale_date"  "Sale_price"
```

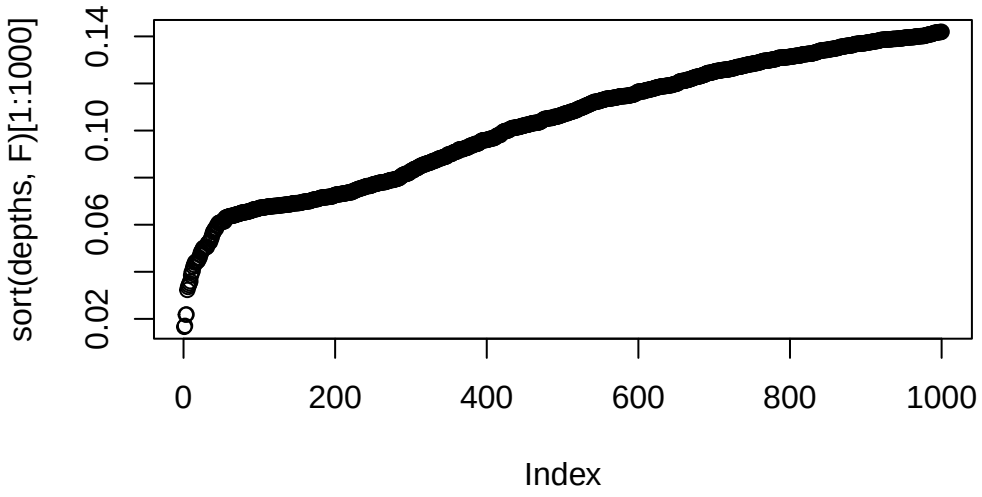
```
df_mat=as.matrix(df5[,numer])
depths=ddalpha::depth.projection(df_mat,df_mat)
which(depths<0.1)[1:10]
```

```
 [1]   4 178 555 599 629 691 694 705 780 889
```

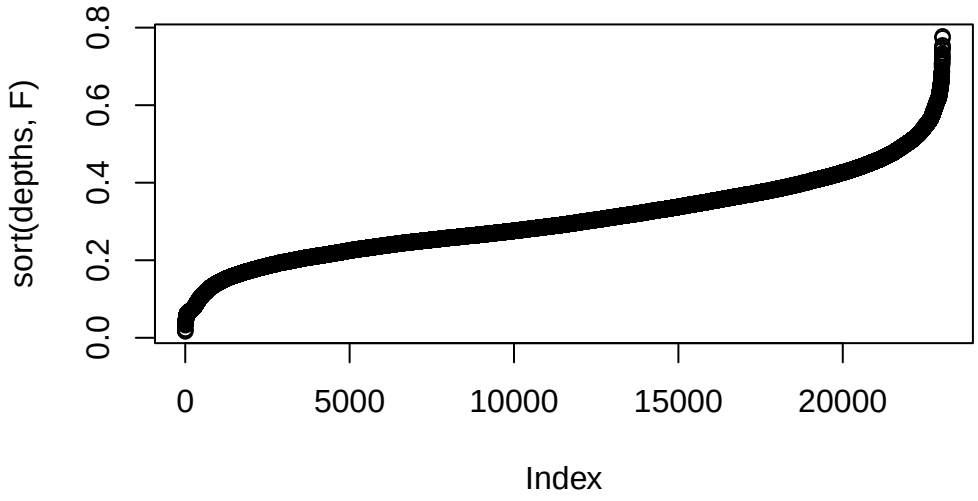
```
plot(sort(depths,F)[1:100])
```



```r
# Notice there is a crack around 0.035, I would look at those observations
plot(sort(depths,F)[1:1000])
```



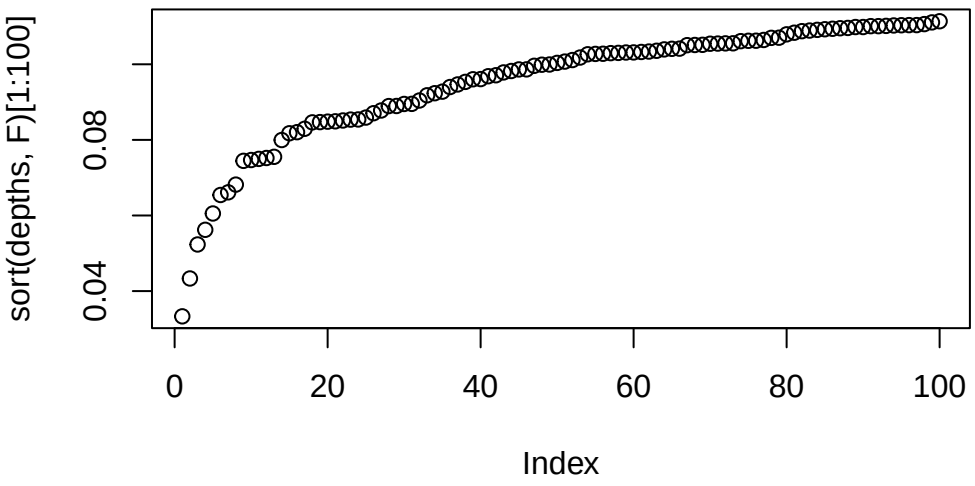

```r
plot(sort(depths,F))
```




```
# OR
depths=ddalpha::depth.projection(cbind(X,df5$Sale_price),cbind(X,df5$Sale_price))
which(depths<0.1)[1:10]
```

```
 [1]    4  178 1063 1330 1460 2035 3004 3238 4095 4576
```

```
plot(sort(depths,F)[1:100])
```




```
which.max(diag(hat))
```

```
6764
6377
```

```
which.max(CDS)
```

```
6764
6377
```

```
which.min(depths)
```

```
[1] 21774
```

```
# Hugely expensive home!
df5[which.min(depths),]
```

```
      District         Extwall Stories Year_Built Fin_sqft Units Bdrms Fbath
23299       14 Masonry / Frame       2       2009     5263     2     4     4
      Lotsize Sale_date Sale_price
23299   13200     17744    1065000
```

```
# These have many rooms etc
df5[which.max(CDS),]
```

```
     District Extwall Stories Year_Built Fin_sqft Units Bdrms Fbath Lotsize
6764        4   Brick      >2       1905     8810     1    >8    >4   24192
     Sale_date Sale_price
6764     15737     175000
```

```
df5[which.max(diag(hat)),]
```

```
     District Extwall Stories Year_Built Fin_sqft Units Bdrms Fbath Lotsize
6764        4   Brick      >2       1905     8810     1    >8    >4   24192
     Sale_date Sale_price
6764     15737     175000
```

```
# what happens when we remove some variables?
model2=lm(sqrt(Sale_price)~Fin_sqft+District+Sale_date+Year_Built,data=df5[-order(CDS,decreas
summary(model)
```

```
Call:
lm(formula = sqrt(Sale_price) ~ Fin_sqft + District + Sale_date +
    Year_Built, data = df5)

Residuals:
    Min      1Q  Median      3Q     Max
-497.28  -32.37    1.59   33.15  376.67

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.501e+03  4.491e+01 -33.426  < 2e-16 ***
Fin_sqft     6.069e-02  7.617e-04  79.682  < 2e-16 ***
District2    2.803e+01  2.427e+00  11.550  < 2e-16 ***
District4   -1.394e+01  5.029e+00  -2.772  0.00558 **
District5    9.666e+01  2.086e+00  46.328  < 2e-16 ***
District6    1.509e+01  2.995e+00   5.038 4.73e-07 ***
District7   -1.979e+00  2.601e+00  -0.761  0.44683
District8    3.787e+01  2.841e+00  13.331  < 2e-16 ***
District9    6.892e+01  2.580e+00  26.712  < 2e-16 ***
District10   1.047e+02  2.173e+00  48.196  < 2e-16 ***
```

```
District11   1.149e+02  2.088e+00  55.000  < 2e-16 ***
District12   1.583e+01  3.399e+00   4.656 3.23e-06 ***
District13   1.149e+02  2.157e+00  53.271  < 2e-16 ***
District14   1.513e+02  2.185e+00  69.223  < 2e-16 ***
District15  -4.360e+01  3.204e+00 -13.610  < 2e-16 ***
Sale_date    6.514e-03  3.528e-04  18.461  < 2e-16 ***
Year_Built   8.079e-01  2.290e-02  35.284  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 58.4 on 23020 degrees of freedom
Multiple R-squared:  0.5116,    Adjusted R-squared:  0.5112
F-statistic:  1507 on 16 and 23020 DF,  p-value: < 2.2e-16
```

```
# Notice that some of the coefficients moved several standard errors!! This is a huge change
sort(abs(model2$coefficients-model$coefficients)/s$coefficients[,2],T)
```

```
  District6   Sale_date (Intercept)  Year_Built   District4  District12
 4.45849467  1.31275625  1.14789619  0.99472519  0.95484626  0.64271066
  District9    Fin_sqft  District15   District8  District11  District10
 0.54351424  0.33354506  0.31519160  0.30775318  0.21754140  0.14803702
 District14  District13   District2   District7   District5
 0.14756268  0.13380072  0.10805161  0.09331690  0.07037254
```

How should we treat influential observations? The easiest course of action is removal. If there are many influential observations, then you might want to try robust model fitting methods, which automatically account for outliers and influential observations.

7.4 Homework questions

Complete the Chapter 6 textbook questions.

Exercise 7.1. What are the three methods we have learned for detecting influential/leverage points?

Exercise 7.2. Compute the hat values, Cook's distances and the depth values for the body weight example. Are there any influential/leverage points/outliers?

Exercise 7.3. Compute the hat values, Cook's distances and the depth values for the cars example. Are there any outliers/influential/leverage points?

Exercise 7.4. Fit a model without location to the real estate data of your choosing. Compute the hat values, Cook's distances and the depth values for the cars example. Are there any influential/leverage points/outliers? Print out the influential/leverage points/outliers. Why do you think they are outlying? Should we remove them?

8 Multicollinearity

8.1 Multicollinearity and the problems it creates

A serious problem that may dramatically impact the usefulness of a regression model is multicollinearity, or near-linear dependence among the regression variables. That is multicollinearity refers to near-linear dependence among the regressors. The regressors are the columns of the X matrix, so clearly an exact linear dependence among the regressors would result in a singular $X^\top X$. This will impact our ability to estimate β.

To elaborate, assume that the regressor variables and the response have been centered and scaled to unit length. The matrix $X^\top X$ is then a $p \times p$ correlation matrix (of the vector of regressors) and $X^\top Y$ is the vector of correlations between the regressors and response. Recall that a set of vectors $v_1, \ldots, v_n$ are linearly dependent if there exists $c \neq 0$ such that $\sum_{i=1}^n c_i v_i = 0$. If the columns of X are linearly dependent, then $X^\top X$ is not invertible! We say there is multicollinearity if there exists c such that $||c|| > k$ and that $||\sum_{i=1}^p c_i x_i|| < \epsilon$ for some small ϵ and some large enough k. Here x_i are the columns of the X matrix.

Multicollinearity results in large variances and covariances for the least - squares estimators of the regression coefficients. Let $A = (X^\top X)^{-1}$, where the regressors have been centered and scaled to unit length. That is, columns $2, \ldots, p$ of X have their mean subtracted and are divided by their respective norms. Then

$$A_{jj} = (1 - R_j^2)^{-1},$$

where R_j^2 is the coefficient of multiple determination from the regression of X_j on the remaining $p-1$ regressor variables. Now, recall that $\text{Var}\left[\hat{\beta}_j\right] = A_{jj}\sigma^2$. What happens when the correlation is approximately 1 between X_j and another regressor? It is easy to see that the the variance of coefficient j goes to ∞ as $\text{corr}\left[X_j, X_i\right] \to 1$.

This huge variance results in large magnitude of the least squares estimators of the regression coefficients. We have that $\text{E}\left[\left\|\hat{\beta} - \beta\right\|^2\right] = \sum_{i=1}^p \text{Var}\left[\hat{\beta}_i\right] = \sigma^2 \,\text{trace}(X^\top X)^{-1}$. Recall! $\text{trace}(A)$ is the sum of its eigenvalues. If $X^\top X$ has near linearly dependent columns, then some of the eigenvalues $\lambda_1, \ldots, \lambda_p$ will be near 0 (why?). Thus,

$$\text{E}\left[\left\|\hat{\beta} - \beta\right\|^2\right] = \sigma^2 \,\text{trace}(X^\top X)^{-1} = \sigma^2 \sum_{i=1}^p \frac{1}{\lambda_i}.$$

We can also show that

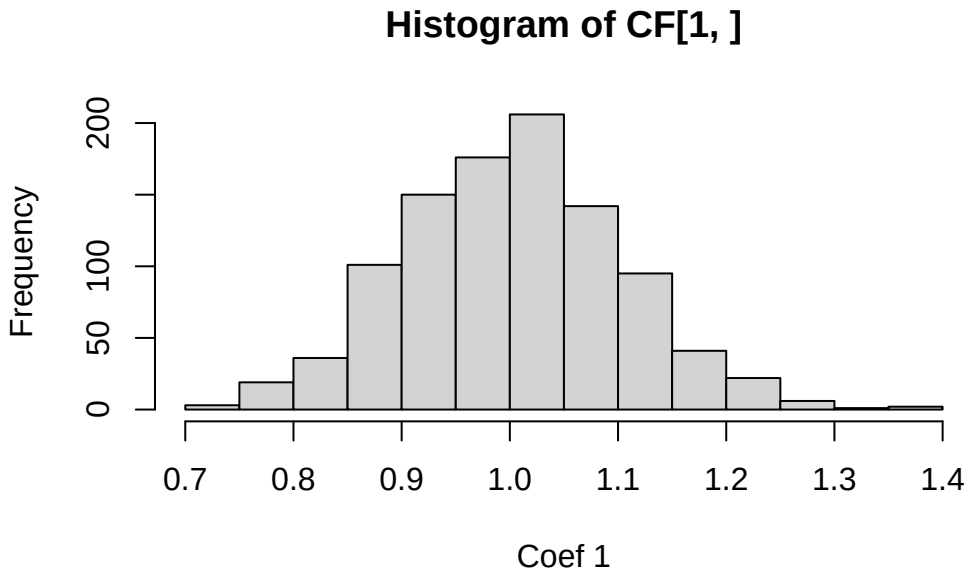
$$\mathrm{E}\left[\hat{\beta}^\top \hat{\beta}\right] = \beta^\top \beta + \sigma^2 \operatorname{Tr}\left(X^\top X\right)^{-1},$$

which gives the same interpretation.

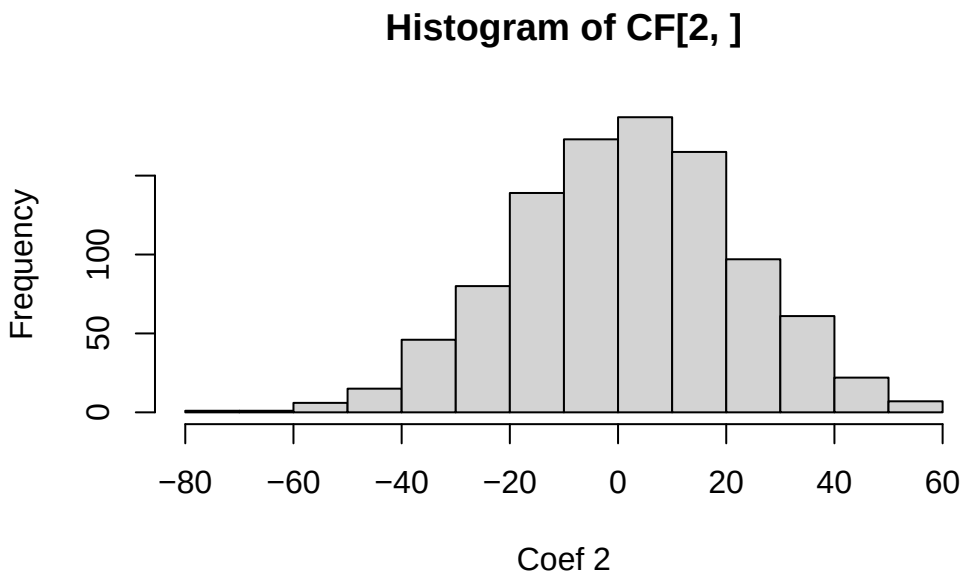
We can also observe this empirically.

```
# Let's simulate data from a regression model with highly correlated regressors.
simulate_coef=function(){
  n=100
  X=rnorm(n)
  X2=2*X+rnorm(n,0,0.01)
  Y=1+2*X+X2*2+rnorm(n)
  return(coef(lm(Y~X+X2)))
}

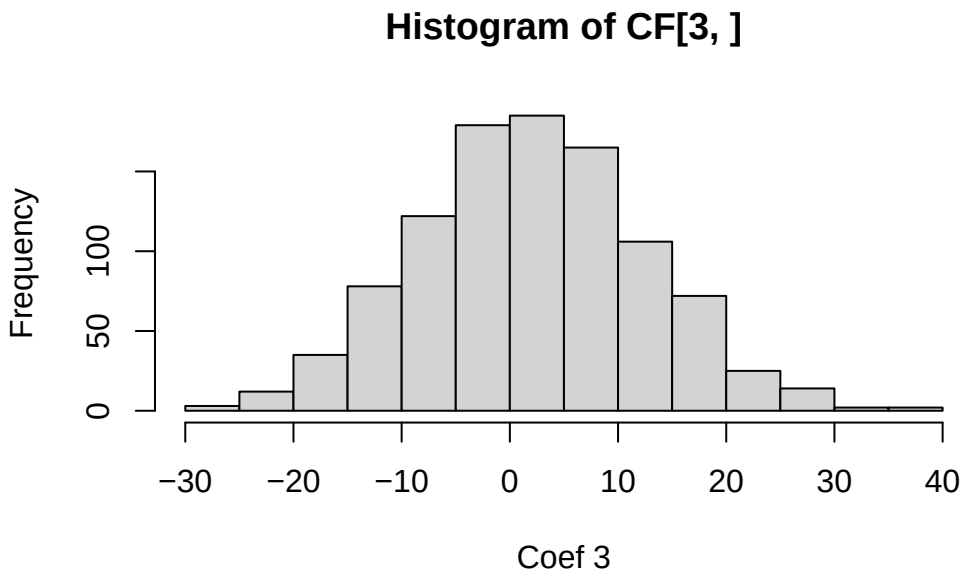
# See the HUGE variance in the estimated coefficients? 0 this is from MCL!
CF=replicate(1000,simulate_coef())
hist(CF[1,],xlab='Coef 1')
```




```
hist(CF[2,],xlab='Coef 2')
```




```
hist(CF[3,],xlab='Coef 3')
```



It is easy to see from this analysis that multicollinearity is a serious problem, and we should check for it in regression modelling.

8.2 Multicollinearity Diagnostics

8.2.1 VIF

We need some diagnostics to detect multicollinearity. The first of which is the **correlation matrix**, which is good for detecting pairwise correlations, but not so much for more complicated dependencies! To correct this, a popular diagnostic is the **variance inflation factor** (VIF): these are the diagonals of $(X^\top X)^{-1}$. A VIF that exceeds 3, 5 or 10 is an indication that the associated regression coefficients are poorly estimated because of multicollinearity. The variance inflation factor can be written as $(1 - R_j^2)^{-1}$, where R_j^2 is the coefficient of determination obtained from regressing x_j on the other regressor variables. For categorical variables, we may look at their VIF together, instead of for the individual dummy variables. This is done via the generalized VIF (GVIF), which was developed by our very own Georges Monette and John Fox (Fox and Monette 1992). We can consider the $(GVIF^{(1/(2\text{number of dummy variables}))})$. However, we want to compare these to the square root of our rules of thumb, so $\sqrt{3}, \sqrt{5}, \sqrt{10}$. The values $(GVIF^{(1/(2\text{number of dummy variables}))})$ are computed automatically in R. When your model has interaction effects, or polynomial terms, it is best to exclude those and compute the VIFs. In summary:

- Compare the VIF of continuous variables to 3, 5 or 10, above those values signals multicollinearity
- Compare the $(GVIF^{(1/(2\text{number of dummy variables}))})$ of categorical variables to $\sqrt{3}, \sqrt{5},$ or $\sqrt{10}$, above those values signals multicollinearity
- When computing VIF and GVIF, it is best to exclude interaction effects and or polynomial terms
- The given thresholds are rules of thumb, and we should not discard evidence of multicollinearity of we are very close to a threshold, e.g., 9.9.

8.2.2 Condition number

One can also look at the eigenvalues of $X^\top X$, where the regressors are centered and normalized to unit length. If the eigenvalues are small, this indicates multicollinearity. One metric computed from the eigenvalues is the **condition number** of $X^\top X$: $\kappa = \max \lambda_j / \min \lambda_j$, where the regressors are centered and normalized to unit length. Condition numbers between 100 and 1000 imply moderate to strong multicollinearity, and if κ exceeds 1000, severe multicollinearity is indicated. Diagonalizing via $X^\top X = \Lambda D \Lambda^\top$ yields the eigenvectors, which help us determine the exact dependence between variables is. You can check the eigenvectors associated with the small eigenvalues. Components that are large in the eigenvector indicate that that variable is contributing to the multicollinearity. Again, for this computation, it is best to exclude interaction effects and or polynomial terms.

> **Note**
>
> While the method of least squares will generally produce poor estimates of the individual model parameters when strong multicollinearity is present, this does not necessarily imply that the fitted model is a poor predictor.
>
> 1. If predictions are confined to regions of the X-space where the multicollinearity holds approximately, the fitted model often produces satisfactory predictions.
> 2. The linear combinations $X\beta$ may be estimated well, even if β is not.

Example 8.1. Recall example Example 6.6. Check for multicollinearity in the proposed models.

I will load in the data below:

```
####### Analyzing the data via EDA  ###############

names(df_clean2)
```

```
 [1] "District"   "Extwall"    "Stories"    "Year_Built" "Fin_sqft"
 [6] "Units"      "Bdrms"      "Fbath"      "Lotsize"    "Sale_date"
[11] "Sale_price"
```

```
# Convert to factors
df_clean2$District=as.factor(df_clean2$District)
df_clean2$Extwall=as.factor(df_clean2$Extwall)
df_clean2$Stories=as.factor(df_clean2$Stories)
df_clean2$Fbath=as.factor(df_clean2$Fbath)
df_clean2$Bdrms=as.factor(df_clean2$Bdrms)
df_clean2$Units=as.factor(df_clean2$Units)
# df_clean2=df_clean2[,c('Sale_price','Fin_sqft',
#                        'District','Sale_date','Year_Built','Lotsize')]

# We are not going to remove the outliers that we found earlier , and see if these diagnostic
# remove those with 0 lot size
df3=df_clean2[df_clean2$Lotsize>0,]
df4=df3[df3$Lotsize<150000,]
df5=df_clean2[df_clean2$District!=3,]
df5$District=droplevels(df5$District)
df=df5
# Old model
```

```r
# model2=lm(sqrt(Sale_price)~Fin_sqft+District+Sale_date+ Year_Built,data=df5)
# summary(model2)
# More variables, which might be related
model2=lm(sqrt(Sale_price)~Fin_sqft+District+Sale_date+ Year_Built+Units+Bdrms+Fbath+Extwall

s=summary(model2)
s
```

```
Call:
lm(formula = sqrt(Sale_price) ~ Fin_sqft + District + Sale_date +
    Year_Built + Units + Bdrms + Fbath + Extwall, data = df5)

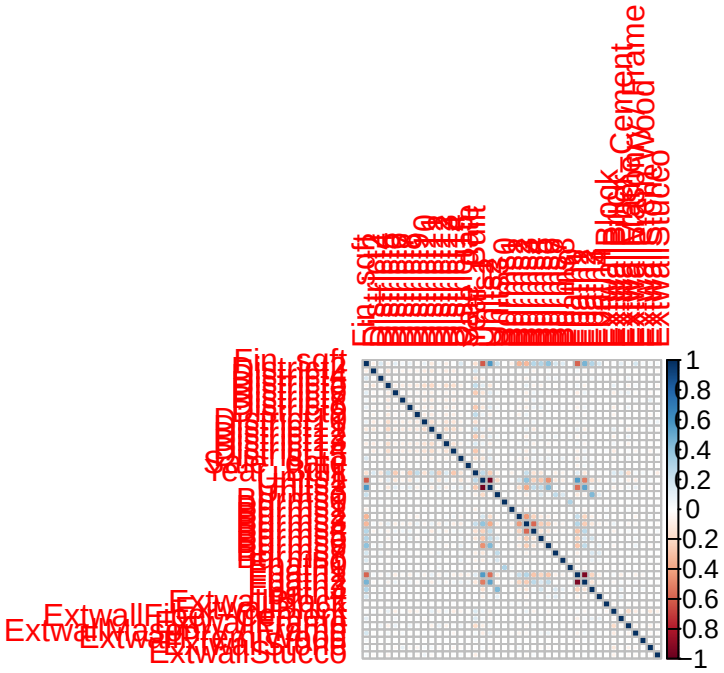
Residuals:
    Min      1Q  Median      3Q     Max
-471.88  -28.67    2.10   30.51  366.97

Coefficients:
                       Estimate Std. Error t value Pr(>|t|)
(Intercept)          -1.181e+03  4.847e+01 -24.374  < 2e-16 ***
Fin_sqft              8.565e-02  1.164e-03  73.587  < 2e-16 ***
District2             3.085e+01  2.193e+00  14.066  < 2e-16 ***
District4            -2.670e+01  4.595e+00  -5.811 6.30e-09 ***
District5             8.931e+01  1.887e+00  47.324  < 2e-16 ***
District6             1.171e+01  2.722e+00   4.302 1.70e-05 ***
District7            -7.729e+00  2.350e+00  -3.289 0.001005 **
District8             3.756e+01  2.570e+00  14.616  < 2e-16 ***
District9             6.685e+01  2.340e+00  28.570  < 2e-16 ***
District10            9.604e+01  1.965e+00  48.870  < 2e-16 ***
District11            1.114e+02  1.888e+00  59.013  < 2e-16 ***
District12            2.154e+01  3.081e+00   6.991 2.81e-12 ***
District13            1.102e+02  1.950e+00  56.496  < 2e-16 ***
District14            1.461e+02  1.980e+00  73.803  < 2e-16 ***
District15           -4.363e+01  2.901e+00 -15.043  < 2e-16 ***
Sale_date             6.867e-03  3.203e-04  21.441  < 2e-16 ***
Year_Built            5.165e-01  2.158e-02  23.929  < 2e-16 ***
Units1                2.362e+01  9.744e+00   2.424 0.015359 *
Units2               -4.792e+01  9.730e+00  -4.925 8.48e-07 ***
Units3               -7.944e+01  1.052e+01  -7.550 4.50e-14 ***
Bdrms0                1.186e+02  2.301e+01   5.155 2.56e-07 ***
Bdrms1                8.658e+01  1.371e+01   6.315 2.76e-10 ***
Bdrms2                9.214e+01  1.260e+01   7.312 2.72e-13 ***
```

```
Bdrms3                  1.033e+02  1.253e+01   8.240  < 2e-16 ***
Bdrms4                  9.367e+01  1.249e+01   7.502 6.53e-14 ***
Bdrms5                  8.904e+01  1.249e+01   7.127 1.06e-12 ***
Bdrms6                  6.985e+01  1.251e+01   5.584 2.38e-08 ***
Bdrms7                  4.501e+01  1.352e+01   3.328 0.000876 ***
Bdrms8                  2.732e+01  1.504e+01   1.816 0.069317 .
Fbath0                  1.075e+02  2.368e+01   4.540 5.64e-06 ***
Fbath1                  8.894e+01  2.059e+01   4.319 1.57e-05 ***
Fbath2                  1.118e+02  2.055e+01   5.440 5.39e-08 ***
Fbath3                  1.223e+02  2.050e+01   5.965 2.48e-09 ***
Fbath4                  1.066e+02  2.144e+01   4.973 6.65e-07 ***
ExtwallBlock           -4.583e+00  4.512e+00  -1.016 0.309733
ExtwallBrick            7.014e+00  8.879e-01   7.899 2.93e-15 ***
ExtwallFiber-Cement     5.247e+01  4.798e+00  10.936  < 2e-16 ***
ExtwallFrame           -3.386e+00  1.242e+00  -2.727 0.006394 **
ExtwallMasonry / Frame  9.203e+00  2.165e+00   4.250 2.15e-05 ***
ExtwallPrem Wood        2.286e+01  6.930e+00   3.299 0.000971 ***
ExtwallStone            1.942e+01  1.867e+00  10.404  < 2e-16 ***
ExtwallStucco           8.629e+00  2.908e+00   2.968 0.003005 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 52.61 on 22995 degrees of freedom
Multiple R-squared:  0.6041,    Adjusted R-squared:  0.6034
F-statistic: 855.7 on 41 and 22995 DF,  p-value: < 2.2e-16
```

```r
# Correlations
X=model.matrix(model2)
corrplot::corrplot(cor(X[,-1]))
```




```
# VIFS
# predictor removes the interactions
# car::vif(model2,type = 'predictor')
car::vif(model2)
```

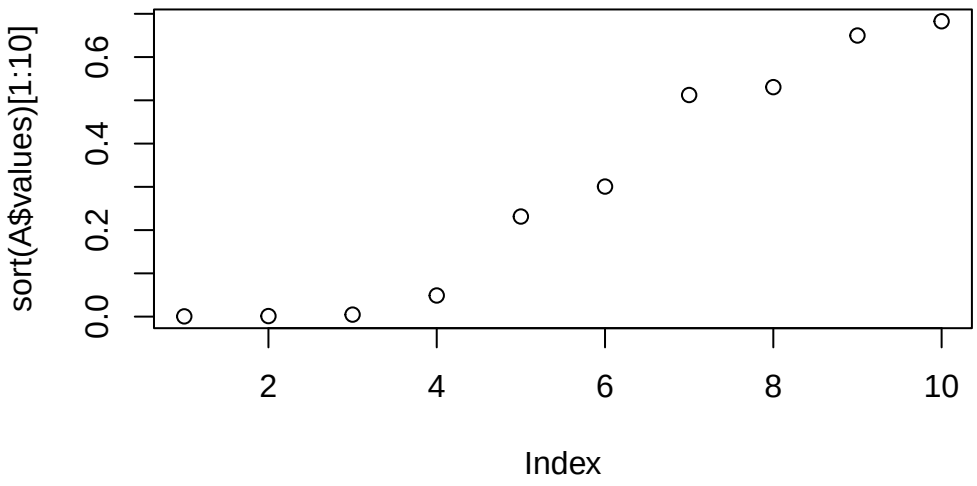
```
              GVIF Df GVIF^(1/(2*Df))
Fin_sqft   3.230637  1        1.797397
District   2.567955 13        1.036939
Sale_date  1.018171  1        1.009045
Year_Built 2.086281  1        1.444396
Units      3.383703  3        1.225271
Bdrms      3.972579  9        1.079647
Fbath      2.886231  5        1.111817
Extwall    1.435525  8        1.022853
```

```
# Correlations
# X=X[,-1]
X2=apply(X,2,function(x){(x-mean(x))/sqrt(sum((x-mean(x))^2))}); dim(X2)
```

```
[1] 23037    42
```

```
X2[,1]=X[,1]
X=X2
A=eigen(t(X)%*%X)

plot(sort(A$values)[1:10])
```




```
which.min(A$values)
```

```
[1] 42
```

```
dim(A$vectors)
```

```
[1] 42 42
```

```
#Condition number
max(A$values)/min(A$values)
```

```
[1] 41193554
```

```r
rownames(A$vectors)=names(model2$coefficients)

# Which components are important here

A$vectors[,which.min(A$values)]
```

```
          (Intercept)              Fin_sqft             District2
         0.000000e+00          5.491915e-03         -8.098975e-05
            District4             District5             District6
         3.813210e-04         -3.033597e-04         -3.076636e-05
            District7             District8             District9
        -3.568751e-04          9.509119e-05         -3.373675e-04
           District10            District11            District12
        -4.972892e-04         -3.424186e-05         -6.326595e-05
           District13            District14            District15
        -1.667758e-04         -2.711893e-04         -7.357368e-06
            Sale_date            Year_Built                 Units1
        -2.642462e-04          2.794659e-04          2.411565e-02
                Units2                Units3                 Bdrms0
         2.234835e-02          5.572644e-03         -4.237411e-03
                Bdrms1                Bdrms2                 Bdrms3
        -1.479639e-02         -8.098271e-02         -1.185489e-01
                Bdrms4                Bdrms5                 Bdrms6
        -1.002262e-01         -5.147474e-02         -4.994474e-02
                Bdrms7                Bdrms8                 Fbath0
        -1.304716e-02         -7.847541e-03          4.286290e-02
                Fbath1                Fbath2                 Fbath3
         6.787509e-01          6.656176e-01          2.278607e-01
                Fbath4           ExtwallBlock          ExtwallBrick
         7.126118e-02          3.336024e-05         -4.642255e-04
  ExtwallFiber-Cement          ExtwallFrame ExtwallMasonry / Frame
         3.683410e-04         -2.689623e-04         -8.592603e-04
     ExtwallPrem Wood           ExtwallStone          ExtwallStucco
        -2.336143e-04         -1.897784e-04         -1.970389e-04
```

```r
# round(A$vectors[,which.min(A$values)],1)
round(A$vectors[,which.min(A$values)][abs(round(A$vectors[,which.min(A$values)],1))>0],1)
```

```
Bdrms2 Bdrms3 Bdrms4 Bdrms5 Fbath1 Fbath2 Fbath3 Fbath4
  -0.1   -0.1   -0.1   -0.1    0.7    0.7    0.2    0.1
```

```
min_4=order(A$values)[1:4]
A$vectors[,min_4]
```

	[,1]	[,2]	[,3]	[,4]
(Intercept)	0.000000e+00	0.000000e+00	0.000000e+00	0.0000000000
Fin_sqft	5.491915e-03	-9.224010e-03	1.188401e-02	0.0270844981
District2	-8.098975e-05	-7.171369e-04	-1.364305e-05	-0.2573525470
District4	3.813210e-04	-1.859440e-03	7.085189e-03	-0.0963354134
District5	-3.033597e-04	5.655736e-05	-6.190680e-04	-0.3970814791
District6	-3.076636e-05	1.020858e-03	-6.159903e-04	-0.1914688618
District7	-3.568751e-04	9.361818e-05	1.316327e-04	-0.2253975020
District8	9.509119e-05	5.218008e-04	2.282725e-04	-0.2069464707
District9	-3.373675e-04	-3.686650e-04	-1.918909e-04	-0.2340610079
District10	-4.972892e-04	1.014931e-03	-2.130468e-03	-0.3532344163
District11	-3.424186e-05	-2.199029e-04	-2.207701e-04	-0.3917050101
District12	-6.326595e-05	4.419550e-04	2.435144e-03	-0.1562487436
District13	-1.667758e-04	2.422701e-04	7.871931e-06	-0.3494070580
District14	-2.711893e-04	6.587700e-04	-1.882882e-03	-0.3573976462
District15	-7.357368e-06	-2.576823e-04	-1.104466e-03	-0.1676543070
Sale_date	-2.642462e-04	-2.658862e-04	-6.529073e-03	-0.0023254038
Year_Built	2.794659e-04	6.532106e-04	-1.943358e-03	-0.0118159033
Units1	2.411565e-02	8.847288e-02	6.999435e-01	0.0036367321
Units2	2.234835e-02	8.723723e-02	6.844527e-01	-0.0054547343
Units3	5.572644e-03	1.530008e-02	1.563099e-01	-0.0002375389
Bdrms0	-4.237411e-03	-2.129747e-02	1.193029e-02	0.0003635227
Bdrms1	-1.479639e-02	-7.811393e-02	1.190583e-02	0.0020197732
Bdrms2	-8.098271e-02	-4.213334e-01	5.737300e-02	0.0006544043
Bdrms3	-1.185489e-01	-6.081989e-01	8.083051e-02	0.0071750605
Bdrms4	-1.002262e-01	-5.081361e-01	6.617640e-02	-0.0070335013
Bdrms5	-5.147474e-02	-2.585338e-01	3.209195e-02	-0.0035521769
Bdrms6	-4.994474e-02	-2.478403e-01	3.152195e-02	-0.0038765656
Bdrms7	-1.304716e-02	-6.745185e-02	1.265355e-02	-0.0008841890
Bdrms8	-7.847541e-03	-4.416656e-02	1.083602e-02	-0.0013029795
Fbath0	4.286290e-02	-8.782624e-03	-3.455349e-03	0.0011804643
Fbath1	6.787509e-01	-1.339772e-01	-3.588576e-03	-0.0030582423
Fbath2	6.656176e-01	-1.302243e-01	-8.053043e-03	0.0018361163
Fbath3	2.278607e-01	-4.623718e-02	-3.239153e-03	-0.0000608401
Fbath4	7.126118e-02	-1.258407e-02	1.799572e-03	-0.0010827629
ExtwallBlock	3.336024e-05	3.801567e-04	1.892410e-03	-0.0024858229
ExtwallBrick	-4.642255e-04	9.304769e-04	-6.023996e-04	-0.0034608384
ExtwallFiber-Cement	3.683410e-04	7.593387e-04	-9.593496e-04	0.0002017397
ExtwallFrame	-2.689623e-04	-3.419082e-04	3.999955e-03	-0.0052695599

```
ExtwallMasonry / Frame -8.592603e-04  2.786953e-04 -1.443152e-03 -0.0053898525
ExtwallPrem Wood       -2.336143e-04  5.150836e-04 -4.476341e-04 -0.0030783990
ExtwallStone           -1.897784e-04  1.001401e-03 -7.036645e-04 -0.0056127691
ExtwallStucco          -1.970389e-04  4.980607e-04  1.245502e-05 -0.0019203399
```

```
# round(A$vectors[,which.min(A$values)],1)
for(i in min_4)
  print(round(A$vectors[,i][abs(round(A$vectors[,i],1))>0],1))
```

```
Bdrms2 Bdrms3 Bdrms4 Bdrms5 Fbath1 Fbath2 Fbath3 Fbath4
  -0.1   -0.1   -0.1   -0.1    0.7    0.7    0.2    0.1
Units1 Units2 Bdrms1 Bdrms2 Bdrms3 Bdrms4 Bdrms5 Bdrms6 Bdrms7 Fbath1 Fbath2
   0.1    0.1   -0.1   -0.4   -0.6   -0.5   -0.3   -0.2   -0.1   -0.1   -0.1
Units1 Units2 Units3 Bdrms2 Bdrms3 Bdrms4
   0.7    0.7    0.2    0.1    0.1    0.1
 District2  District4  District5  District6  District7  District8  District9
      -0.3       -0.1       -0.4       -0.2       -0.2       -0.2       -0.2
District10 District11 District12 District13 District14 District15
      -0.4       -0.4       -0.2       -0.3       -0.4       -0.2
```

```
# Everything seems good!
```

Example 8.2. Example 9.1 from the textbook - The Acetylene Data. Below presents data concerning the percentage of conversion of n - heptane to acetylene and three explanatory variables. These are typical chemical process data for which a full quadratic response surface in all three regressors is often considered to be an appropriate tentative model. Let's build the model and see how the extrapolation performs.

```
############ Example 2




df <- data.frame(
  Conversion_of_n_Heptane_to_Acetylene = c(49.0, 50.2, 50.5, 48.5, 47.5, 44.5, 28.0, 31.5, 34
  Reactor_Temperature_deg_C = c(1300, 1300, 1300, 1300, 1300, 1300, 1200, 1200, 1200, 1200, 1
  Ratio_of_H2_to_n_Heptane_mole_ratio = c(7.5, 9.0, 11.0, 13.5, 17.0, 23.0, 5.3, 7.5, 11.0, 1
  Contact_Time_sec = c(0.0120, 0.0120, 0.0115, 0.0130, 0.0135, 0.0120, 0.0400, 0.0380, 0.032
)

# Printing the dataframe
head(df)
```

```
  Conversion_of_n_Heptane_to_Acetylene Reactor_Temperature_deg_C
1                                 49.0                      1300
2                                 50.2                      1300
3                                 50.5                      1300
4                                 48.5                      1300
5                                 47.5                      1300
6                                 44.5                      1300
  Ratio_of_H2_to_n_Heptane_mole_ratio Contact_Time_sec
1                                 7.5           0.0120
2                                 9.0           0.0120
3                                11.0           0.0115
4                                13.5           0.0130
5                                17.0           0.0135
6                                23.0           0.0120
```

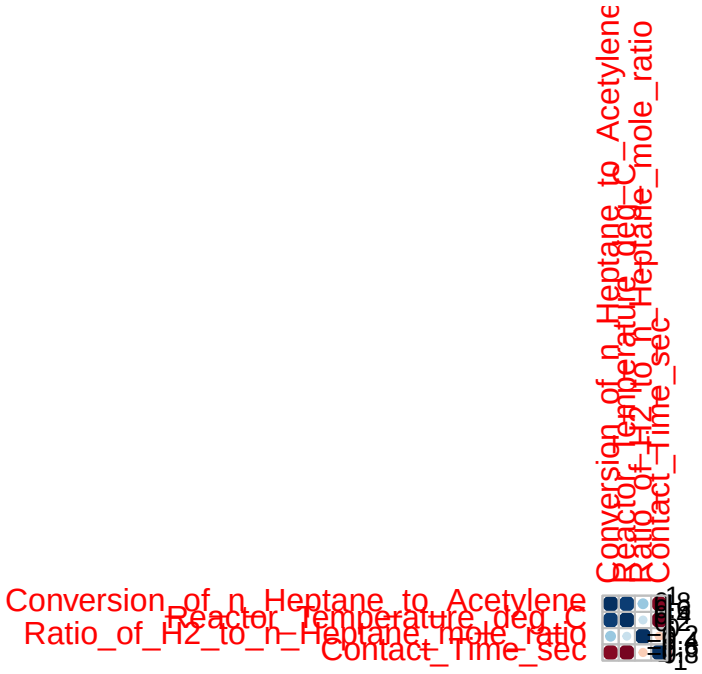
```r
# For standardizing via Z scores
unit_norm=function(x){
  x=x-mean(x)
  return(sqrt(length(x)-1)*x/sqrt(sum(x^2)))
}

df2=df

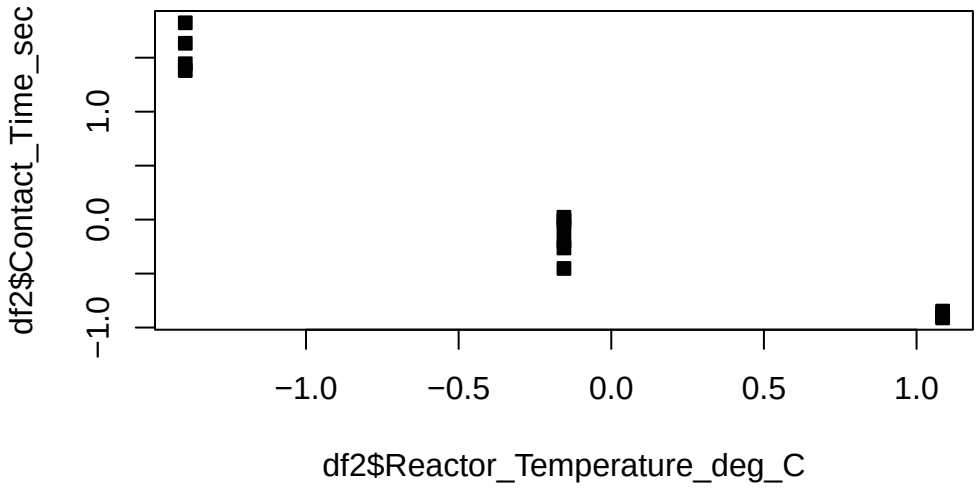
#standardizing the regressors
df2[,2:4]=apply(df[,2:4],2,unit_norm)

df2=data.frame(df2)
corrplot::corrplot(cor(df2))
```

```
Warning in corrplot::corrplot(cor(df2)): Not been able to calculate text
margin, please try again with a clean new empty window using {plot.new();
dev.off()} or reduce tl.cex
```




```
# Observe the high correlations between Contact time and Reactor temperature
plot(df2$Reactor_Temperature_deg_C,df2$Contact_Time_sec,pch=22,bg=1)
```



```r
orig=names(df2)
names(df2)=c('P','t','H','C')

df2$t2=df2$t^2
df2$H2=df2$H^2
df2$C2=df2$C^2

RS=lm(P~t+H+C+t*H+t*C+C*H+t2+H2+C2  ,df2)
summary(RS)
```

```
Call:
lm(formula = P ~ t + H + C + t * H + t * C + C * H + t2 + H2 +
    C2, data = df2)

Residuals:
    Min      1Q  Median      3Q     Max
-1.3499 -0.3411  0.1297  0.5011  0.6720

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  35.8958     1.0916  32.884 5.26e-08 ***
t             4.0038     4.5087   0.888 0.408719
H             2.7783     0.3071   9.048 0.000102 ***
C            -8.0423     6.0707  -1.325 0.233461
t2          -12.5236    12.3238  -1.016 0.348741
H2           -0.9727     0.3746  -2.597 0.040844 *
C2          -11.5932     7.7063  -1.504 0.183182
t:H          -6.4568     1.4660  -4.404 0.004547 **
t:C         -26.9804    21.0213  -1.283 0.246663
H:C          -3.7681     1.6553  -2.276 0.063116 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9014 on 6 degrees of freedom
Multiple R-squared:  0.9977,    Adjusted R-squared:  0.9943
F-statistic: 289.7 on 9 and 6 DF,  p-value: 3.225e-07
```

```r
# Crazy high VIF
car::vif(RS)
```

```
there are higher-order terms (interactions) in this model
```

```
consider setting type = 'predictor'; see ?vif
```

```
          t           H           C          t2          H2          C2
 375.247759    1.740631  680.280039 1762.575365    3.164318 1156.766284
        t:H         t:C         H:C
  31.037059 6563.345193   35.611286
```

```
car::vif(RS,type='predictor')
```

```
GVIFs computed for predictors
```

```
           GVIF Df GVIF^(1/(2*Df)) Interacts With Other Predictors
t  51654.740516  6        2.470354           H, C       t2, H2, C2
H  51654.740516  6        2.470354           t, C       t2, H2, C2
C  51654.740516  6        2.470354           t, H       t2, H2, C2
t2  1762.575365  1       41.983037           --    t, H, C, H2, C2
H2     3.164318  1        1.778853           --    t, H, C, t2, C2
C2  1156.766284  1       34.011267           --    t, H, C, t2, H2
```

```
# hidden extrapolation - be careful

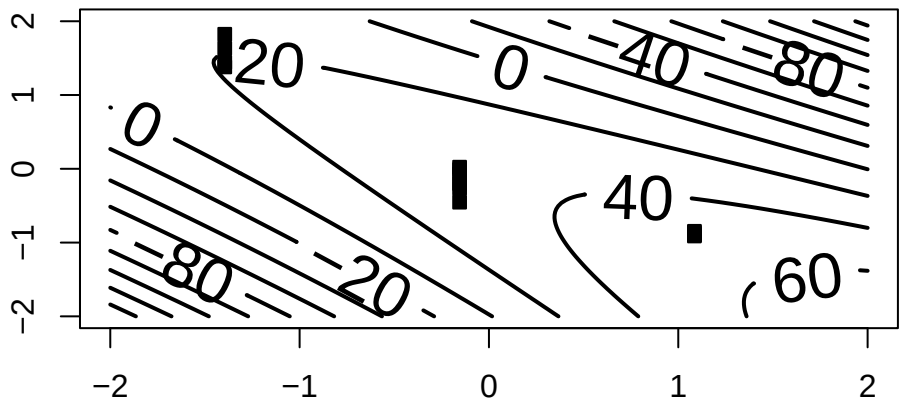
# Create a grid to extrapolate over
c_grid=seq(-2,2,l=100)
t_grid=seq(-2,2,l=100)
g=expand.grid(t_grid,c_grid)

# Adding a fixed value of H=-0.6082179
g=cbind(g,rep(-0.6082179,nrow(g)))
# Adding H^2
g=cbind(g,g^2)

# Making new dataframe for predictions
new_dat=data.frame(g)
names(new_dat)=c("t","C","H","t2","C2","H2")

# Predicting the values on the grid
Z=predict.lm(RS,new_dat)

contour(c_grid,t_grid,matrix(Z, ncol = length(t_grid)),lwd=2,labcex=2,ylim=c(-1,1)*2,xlim=c(
points(df2$t,df2$C,pch=22,bg=1)
```




```
## Notice that as soon as we leave the region with data, the response becomes negative.
# Recall that it is a percentage and can't be negative
## This shows that mild extrapolation dangerous!
```

Example 8.3. Use the NFL data from the textbook - regress the number of wins against all variables. Check for multicollinearity. Propose a method to resolve the multicollinearity.

```
#########################################################
#
#
#
#           NFL DATA EXAMPLE
#
#
#########################################################

df=MPV::table.b1
# Note

df
```

```
    y   x1   x2   x3   x4  x5   x6   x7   x8   x9
```

```
1  10 2113 1985 38.9 64.7   4  868 59.7 2205 1917
2  11 2003 2855 38.8 61.3   3  615 55.0 2096 1575
3  11 2957 1737 40.1 60.0  14  914 65.6 1847 2175
4  13 2285 2905 41.6 45.3  -4  957 61.4 1903 2476
5  10 2971 1666 39.2 53.8  15  836 66.1 1457 1866
6  11 2309 2927 39.7 74.1   8  786 61.0 1848 2339
7  10 2528 2341 38.1 65.4  12  754 66.1 1564 2092
8  11 2147 2737 37.0 78.3  -1  761 58.0 1821 1909
9   4 1689 1414 42.1 47.6  -3  714 57.0 2577 2001
10  2 2566 1838 42.3 54.2  -1  797 58.9 2476 2254
11  7 2363 1480 37.3 48.0  19  984 67.5 1984 2217
12 10 2109 2191 39.5 51.9   6  700 57.2 1917 1758
13  9 2295 2229 37.4 53.6  -5 1037 58.8 1761 2032
14  9 1932 2204 35.1 71.4   3  986 58.6 1709 2025
15  6 2213 2140 38.8 58.3   6  819 59.2 1901 1686
16  5 1722 1730 36.6 52.6 -19  791 54.4 2288 1835
17  5 1498 2072 35.3 59.3  -5  776 49.6 2072 1914
18  5 1873 2929 41.1 55.3  10  789 54.3 2861 2496
19  6 2118 2268 38.2 69.6   6  582 58.7 2411 2670
20  4 1775 1983 39.3 78.3   7  901 51.7 2289 2202
21  3 1904 1792 39.7 38.1  -9  734 61.9 2203 1988
22  3 1929 1606 39.7 68.8 -21  627 52.7 2592 2324
23  4 2080 1492 35.5 68.8  -8  722 57.8 2053 2550
24 10 2301 2835 35.3 74.1   2  683 59.7 1979 2110
25  6 2040 2416 38.7 50.0   0  576 54.9 2048 2628
26  8 2447 1638 39.9 57.1  -8  848 65.3 1786 1776
27  2 1416 2649 37.4 56.3 -22  684 43.8 2876 2524
28  0 1503 1503 39.3 47.0  -9  875 53.5 2560 2241
```

```
summary(df)
```

```
       y                x1             x2             x3              x4       
 Min.   : 0.000   Min.   :1416   Min.   :1414   Min.   :35.10   Min.   :38.10  
 1st Qu.: 4.000   1st Qu.:1896   1st Qu.:1714   1st Qu.:37.38   1st Qu.:52.42  
 Median : 6.500   Median :2111   Median :2106   Median :38.85   Median :57.70  
 Mean   : 6.964   Mean   :2110   Mean   :2127   Mean   :38.64   Mean   :59.40  
 3rd Qu.:10.000   3rd Qu.:2303   3rd Qu.:2474   3rd Qu.:39.70   3rd Qu.:68.80  
 Max.   :13.000   Max.   :2971   Max.   :2929   Max.   :42.30   Max.   :78.30  
       x5               x6              x7              x8       
 Min.   :-22.00   Min.   : 576.0   Min.   :43.80   Min.   :1457  
 1st Qu.: -5.75   1st Qu.: 710.5   1st Qu.:54.77   1st Qu.:1848  
 Median :  1.00   Median : 787.5   Median :58.65   Median :2050  
```

```
 Mean   :  0.00   Mean   : 789.9   Mean   :58.16   Mean   :2110
 3rd Qu.:  6.25   3rd Qu.: 869.8   3rd Qu.:61.10   3rd Qu.:2320
 Max.   : 19.00   Max.   :1037.0   Max.   :67.50   Max.   :2876
       x9
 Min.   :1575
 1st Qu.:1913
 Median :2101
 Mean   :2128
 3rd Qu.:2328
 Max.   :2670
```

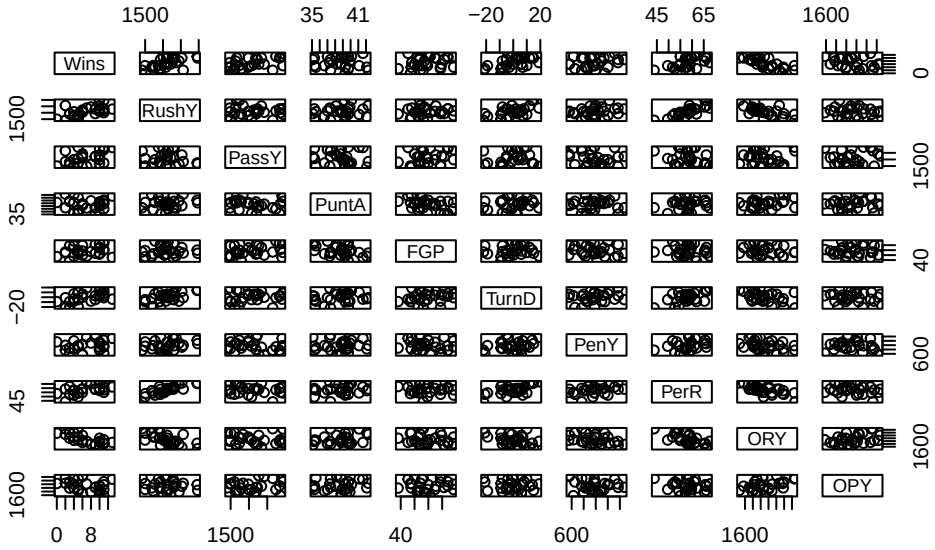
```
names(df)
```

```
 [1] "y"  "x1" "x2" "x3" "x4" "x5" "x6" "x7" "x8" "x9"
```

```
names(df)=c("Wins","RushY","PassY",
            "PuntA","FGP","TurnD",
            "PenY","PerR","ORY","OPY")
summary(df)
```

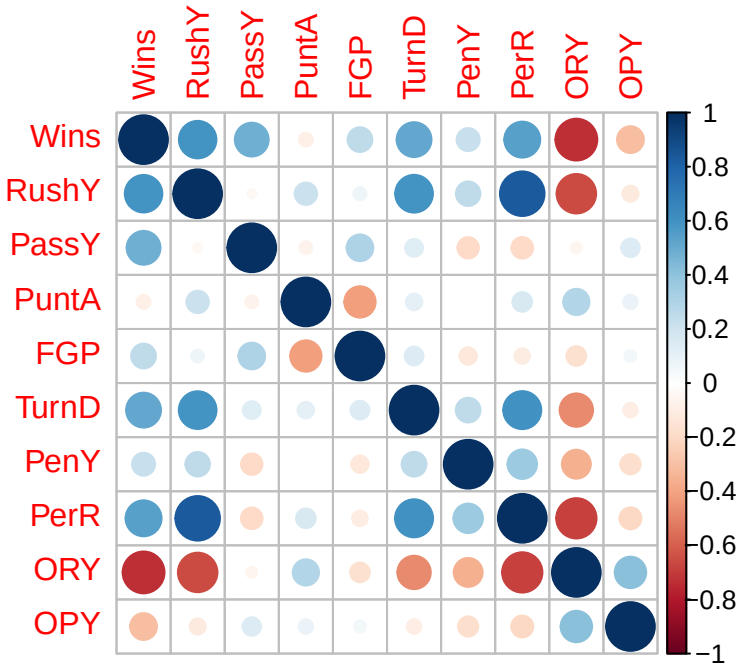
```
      Wins            RushY          PassY          PuntA            FGP
 Min.   : 0.000   Min.   :1416   Min.   :1414   Min.   :35.10   Min.   :38.10
 1st Qu.: 4.000   1st Qu.:1896   1st Qu.:1714   1st Qu.:37.38   1st Qu.:52.42
 Median : 6.500   Median :2111   Median :2106   Median :38.85   Median :57.70
 Mean   : 6.964   Mean   :2110   Mean   :2127   Mean   :38.64   Mean   :59.40
 3rd Qu.:10.000   3rd Qu.:2303   3rd Qu.:2474   3rd Qu.:39.70   3rd Qu.:68.80
 Max.   :13.000   Max.   :2971   Max.   :2929   Max.   :42.30   Max.   :78.30
     TurnD             PenY            PerR            ORY
 Min.   :-22.00   Min.   : 576.0   Min.   :43.80   Min.   :1457
 1st Qu.: -5.75   1st Qu.: 710.5   1st Qu.:54.77   1st Qu.:1848
 Median :  1.00   Median : 787.5   Median :58.65   Median :2050
 Mean   :  0.00   Mean   : 789.9   Mean   :58.16   Mean   :2110
 3rd Qu.:  6.25   3rd Qu.: 869.8   3rd Qu.:61.10   3rd Qu.:2320
 Max.   : 19.00   Max.   :1037.0   Max.   :67.50   Max.   :2876
      OPY
 Min.   :1575
 1st Qu.:1913
 Median :2101
 Mean   :2128
 3rd Qu.:2328
 Max.   :2670
```

```
plot(df)
```




```
model=lm(Wins~.,data=df)
################### Multicollinearity

###### Corr plot
corrplot::corrplot(cor(df))
```




```
###### VIF
car::vif(model)
```

```
   RushY    PassY    PuntA      FGP    TurnD     PenY     PerR      ORY
4.827645 1.420161 2.126597 1.566107 1.924035 1.275979 5.414572 4.535643
     OPY
1.423390
```

```
# recalculating via the X'X matrix
X=model.matrix(model)
dim(X)
```

```
[1] 28 10
```

```
#standardizing
X2=apply(X,2,function(x){(x-mean(x))/sqrt(sum((x-mean(x))^2))}); dim(X2)
```

```
[1] 28 10
```

```r
#replacing with column of ones again
X2[,1]=X[,1]
X=X2
diag(solve(t(X)%*%X))
```

```
(Intercept)       RushY       PassY       PuntA         FGP       TurnD
 0.03571429  4.82764538  1.42016105  2.12659726  1.56610698  1.92403474
       PenY        PerR         ORY         OPY
 1.27597850  5.41457162  4.53564335  1.42338989
```

```r
# observe that
model2=lm(RushY~.,data=df[,-1])
s=summary(model2)
(1-s$r.squared)^(-1)
```

```
[1] 4.827645
```

```r
###### Condition Number / Eigenvalue / Eigenvector
X=model.matrix(model)
dim(X)
```

```
[1] 28 10
```

```r
#standardizing
X2=apply(X,2,function(x){(x-mean(x))/sqrt(sum((x-mean(x))^2))}); dim(X2)
```

```
[1] 28 10
```

```r
X2[,1]=X[,1]
X=X2


ev=eigen(t(X)%*%X)
xev=ev$values
condition_number=max(xev)/min(xev)
condition_number
```

```
[1] 233.7211
```

```
sort(xev)
```

```
 [1]  0.1198009  0.1304874  0.4141227  0.5381718  0.7373652  0.8232571
 [7]  1.3144026  1.6989984  3.2233940 28.0000000
```

```
#index of minimum eigenvalue
minn=which.min(xev)
ev$vectors[,minn]
```

```
 [1]  6.411542e-17  7.011800e-01 -4.340761e-02 -2.023407e-01 -1.758354e-01
 [6]  8.685752e-02  5.157060e-02 -6.211992e-01  1.784152e-01 -8.172072e-02
```

```
rownames(ev$vectors)=names(model$coefficients)
ev$vectors
```

```
                    [,1]          [,2]          [,3]          [,4]
(Intercept)  1.000000e+00  3.518928e-17 -5.957417e-17  1.278524e-16
RushY       -4.202100e-17  4.860110e-01  2.451343e-02 -2.455854e-01
PassY       -1.925274e-17 -5.329696e-02 -4.452387e-01 -4.271630e-01
PuntA        1.730021e-16  3.402877e-02  5.385783e-01 -4.700037e-01
FGP          6.535596e-18  1.901803e-02 -6.336115e-01 -8.166060e-02
TurnD        3.688423e-19  4.092389e-01 -1.005237e-01 -3.161027e-01
PenY        -4.790976e-17  2.794600e-01  1.564396e-01  2.830480e-01
PerR        -4.979902e-17  5.088988e-01  1.310716e-01 -7.932060e-02
ORY         -9.388117e-17 -4.644587e-01  2.405543e-01 -2.073852e-01
OPY          1.355701e-16 -1.978859e-01 -2.649024e-03 -5.480057e-01
                    [,5]          [,6]          [,7]          [,8]
(Intercept)  1.782040e-17 -2.201849e-17 -1.424289e-18 -4.578376e-17
RushY        6.132520e-03  2.463905e-01  9.603526e-02  3.317999e-01
PassY        3.347492e-01 -5.954589e-01  2.990445e-01  1.261423e-01
PuntA        2.806944e-01 -1.156065e-01 -3.606368e-01  3.469809e-01
FGP         -1.667771e-01  2.190348e-01 -6.039025e-01  3.465457e-01
TurnD       -1.494452e-02 -1.216896e-01 -3.373137e-01 -7.555812e-01
PenY        -5.278556e-01 -6.731568e-01 -1.866976e-01  2.100622e-01
PerR        -4.210781e-02  2.029121e-01  1.700649e-01  2.906163e-02
ORY         -9.651613e-02 -1.380667e-02 -3.481051e-01 -1.326596e-01
OPY         -7.009694e-01  1.186045e-01  3.284156e-01 -5.920606e-03
                    [,9]         [,10]
(Intercept) -1.895581e-16  6.411542e-17
RushY       -1.765417e-01  7.011800e-01
```

```
PassY       -2.064018e-01 -4.340761e-02
PuntA        3.229883e-01 -2.023407e-01
FGP          2.303085e-03 -1.758354e-01
TurnD        1.234429e-01  8.685752e-02
PenY        -6.244479e-02  5.157060e-02
PerR        -5.088703e-01 -6.211992e-01
ORY         -7.094290e-01  1.784152e-01
OPY          2.013172e-01 -8.172072e-02
```

```
# names(model$coefficients)[abs(round(ev$vectors[,minn],1))>0]
round(ev$vectors[,minn],1)
```

```
(Intercept)       RushY       PassY       PuntA         FGP       TurnD
        0.0         0.7         0.0        -0.2        -0.2         0.1
       PenY        PerR         ORY         OPY
        0.1        -0.6         0.2        -0.1
```

```
# What should we do?
# Try dropping one or create a new variable

# New variable
# we can try rushing yards adjusted by the percentage of rushing plays
# This would be yards per percentage rushing,
df$RushPP=df$RushY/df$PerR

# paste(names(df),collapse='+')

model=lm(Wins~PassY+PuntA+FGP+TurnD+PenY+OPY+RushPP,data=df)
summary(model)
```

```
Call:
lm(formula = Wins ~ PassY + PuntA + FGP + TurnD + PenY + OPY +
    RushPP, data = df)

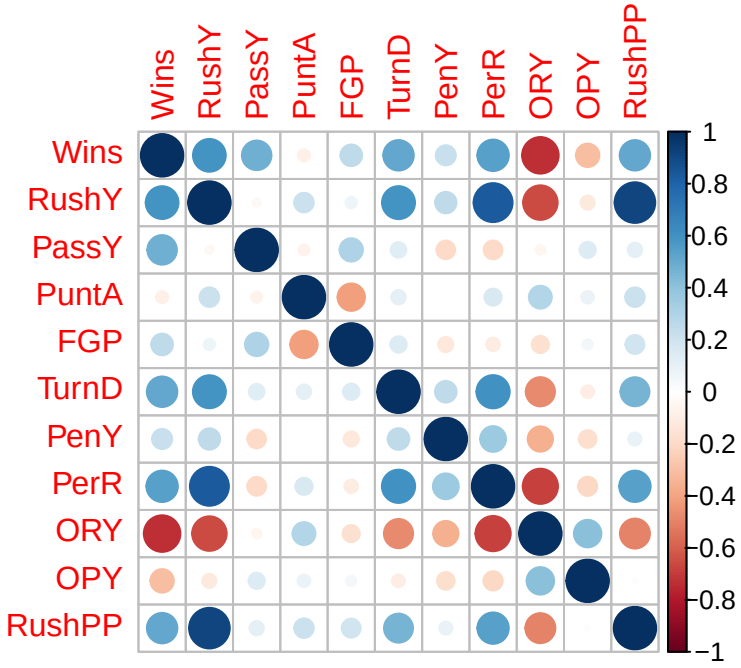
Residuals:
    Min      1Q  Median      3Q     Max
-4.9688 -0.7444  0.5091  1.0653  4.4616

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
```

```
(Intercept)  1.1913914 12.5388398   0.095  0.92525
PassY        0.0033599  0.0009656   3.480  0.00236 **
PuntA       -0.2151444  0.2657707  -0.810  0.42775
FGP         -0.0007235  0.0515376  -0.014  0.98894
TurnD        0.0744762  0.0507446   1.468  0.15775
PenY         0.0048413  0.0039483   1.226  0.23437
OPY         -0.0036345  0.0015513  -2.343  0.02959 *
RushPP       0.3018399  0.1274578   2.368  0.02806 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.312 on 20 degrees of freedom
Multiple R-squared:  0.673, Adjusted R-squared:  0.5586
F-statistic: 5.881 on 7 and 20 DF,  p-value: 0.0008225
```

```
corrplot::corrplot(cor(df))
```




```
car::vif(model)
```

```
   PassY    PuntA      FGP    TurnD     PenY      OPY   RushPP
1.173227 1.403668 1.505937 1.415294 1.181830 1.068924 1.412990
```

```r
# Dropping

# paste(names(df),collapse='+')

model=lm(Wins~PassY+PuntA+FGP+TurnD+PenY+OPY+RushY,data=df)
summary(model)
```

```
Call:
lm(formula = Wins ~ PassY + PuntA + FGP + TurnD + PenY + OPY +
    RushY, data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-4.7636 -1.1496  0.3967  0.7301  3.7040

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.1782213 11.0207807   0.107 0.915926
PassY        0.0037776  0.0008564   4.411 0.000269 ***
PuntA       -0.2237663  0.2303610  -0.971 0.342965
FGP          0.0090954  0.0442921   0.205 0.839374
TurnD        0.0244775  0.0490619   0.499 0.623285
PenY         0.0036710  0.0034965   1.050 0.306277
OPY         -0.0033525  0.0013680  -2.451 0.023589 *
RushY        0.0047817  0.0013226   3.615 0.001725 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.034 on 20 degrees of freedom
Multiple R-squared:  0.7468,    Adjusted R-squared:  0.6582
F-statistic: 8.428 on 7 and 20 DF,  p-value: 7.995e-05
```

```r
corrplot::corrplot(cor(df))
car::vif(model)
```

```
   PassY    PuntA      FGP    TurnD     PenY      OPY    RushY
1.191923 1.361884 1.436428 1.708554 1.196951 1.073562 1.697535
```

There are four primary sources/causes of multicollinearity :

- The data collection method employed: In this case, some of the variables are typically confounded and/or the experiment/study was not well-designed.
- Constraints on the model or in the population: Sometimes, only certain combinations of levels of variables can be observed together.
- Model specification: Sometimes you have two or more variables in your model that are measuring the same thing.
- An overdefined model: You have too many variables in your model.

Make sure to check for each of these. Sometimes, regression coefficients have the wrong sign. This is likely due to one of the following:

- The range of some of the regressors is too small – if the range of some of the regressors is too small, then the variance of $\widehat{\beta}$ is high.
- Important regressors have not been included in the model.
- **Multicollinearity is present.**
- Computational errors have been made.

Multicollinearity can be cured with:

1. more data (lol often not possible),
2. model re-specification: Can you include a function of the variables that preserves the information, but aren't linearly dependent? Can you remove a variable?
3. Or, a modified version of regression, one of Lasso, ridge or elastic net regression.

8.3 Homework questions

Complete the Chapter 9 textbook questions.

Exercise 8.1. Check for multicollinearity in all of our past examples.

Exercise 8.2. Summarize the 3 multicollinearity diagnostics.

9 Variable/Model Selection

9.1 Variable Selection

In the preceding lessons we have assumed that the regressor variables included in the model are known to be important. Our focus was on techniques to ensure that the functional form of the model was correct and that the underlying assumptions were not violated. In previous lessons, we have employed the classical approach to regression model selection, which assumes that we have a very good idea of the basic form of the model and that we know all (or nearly all) of the regressors that should be used.

Our approach so far can be summarized as follows:

- Fit the full model.
- Perform a thorough analysis of this model, including a full residual analysis and investigation of multicollinearity.
- Determine if transformations of the response or of some of the regressors are necessary.
- Use the t-tests/F-tests on the individual regressors to edit the model.
- Perform a thorough analysis of the edited model, especially a residual analysis, to determine the model's adequacy.

In many problems the analyst has a rather large pool of possible candidate regressors, of which only a few are likely to be important. Finding an appropriate subset of regressors for the model is often called the **variable selection** problem .

> **Note**
>
> Variable selection can address multicollinearity, through the removal of unnecessary variables.

Building a regression model that includes only a subset of the available regressors involves two conflicting objectives:

1. We would like the model to include as many regressors as possible so that the information about Y contained in these factors can influence the predicted value of Y.
2. We want the model to include as few regressors as possible because the variance of $\hat{Y}$ increases as the number of regressors increases. Also the more regressors there are in a model, the greater the costs of data collection and model maintenance.

Therefore, we seek a model that is a compromise between these two objectives. There are several algorithms that can be used for variable selection, and these procedures frequently specify different subsets of the candidate regressors as best.

Variable selection is often developed in an idealized setting: Assumed that the correct functional specification of the regressors is known, and no outliers are present. In practice – it's messy. Residual analysis is useful in revealing functional forms of regressors that might be investigated, in pointing out new candidate regressors, and for identifying defects in the data such as outliers. The effect of influential observations should also be determined. Although ideally these problems should be solved simultaneously, an iterative approach is often employed, in which (1) a particular variable selection strategy is employed and then (2) the resulting subset model is checked for correct functional specification, outliers, and influential observations. This may indicate that step 1 must be repeated. Several iterations may be required to produce an adequate model.

None of the variable selection procedures described are guaranteed to produce the best model for a given data set. In fact, there is usually not a single best model but rather several equally good ones. Variable selection algorithms are heavily algorithmic so it is tempting to place a lot of confidence in the results of a particular procedure. Beware of this - experience, professional judgment and subjective considerations are also critical to the variable selection problem. Variable selection procedures should be used in conjunction with these.

Two key aspects of the variable selection problem are:

1. Generating the subset models.
2. Deciding if one subset is better than another.

9.2 Deciding if one subset is better than another

9.2.1 Coefficients of determination

Recall that

$$R^2 = \frac{SSModel}{SST} = 1 - \frac{SSE}{SST}.$$

We know that R^2 increases as p increases, so we cannot choose the model with the largest R^2. However, we could add variables one at a time until the marginal increase in R^2 is small.

Example 9.1. Recall example Example 6.6. Consider instead regressing the logged price per square foot onto the remaining variables. Investigate the R^2 and adjusted R^2 curve through adding the regressors into the regression model one at a time.

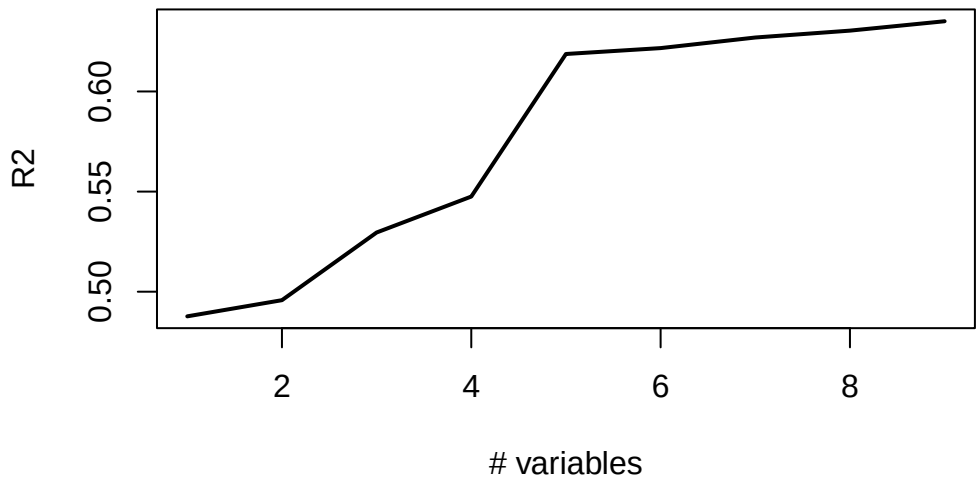
```
df=df_clean2
df['ppsq']=log(df$Sale_price/df$Fin_sqft)
df['LLS']=log(df$Lotsize)
df=df[df['Lotsize']>0, ]
df=df[df['Sale_price']>10000,]
df=df[df['Fin_sqft']>500,]
df$District=as.factor(df$District)
df2=df
```

```
var_list=c('District' ,  'Extwall' ,'Stories','Year_Built','Units','LLS','Bdrms','Fbath','Sa
#previous...
full_model=lm(ppsq~ District +  Extwall +
            Stories  +    Year_Built  +
            Units   +     Bdrms   +
            Fbath  +    LLS   + Sale_date,df2)



###########

# R2
r2=c()
for(i in 1:length(var_list)){
  vars=c(var_list[1:i],"ppsq")
  model2=lm(ppsq~.,df2[,vars])
  sm=summary(model2)
  r2=c(r2,sm$r.squared)
}
plot(r2,type='l',lwd=2,ylab="R2",xlab="# variables")
```




```
var_list[1:6]
```

```
[1] "District"   "Extwall"    "Stories"    "Year_Built" "Units"
[6] "LLS"
```

Notice how the curve levels off at 6 variables. This would indicate that we are not gaining much explained variation beyond the first 6 variables. These variables are printed above.

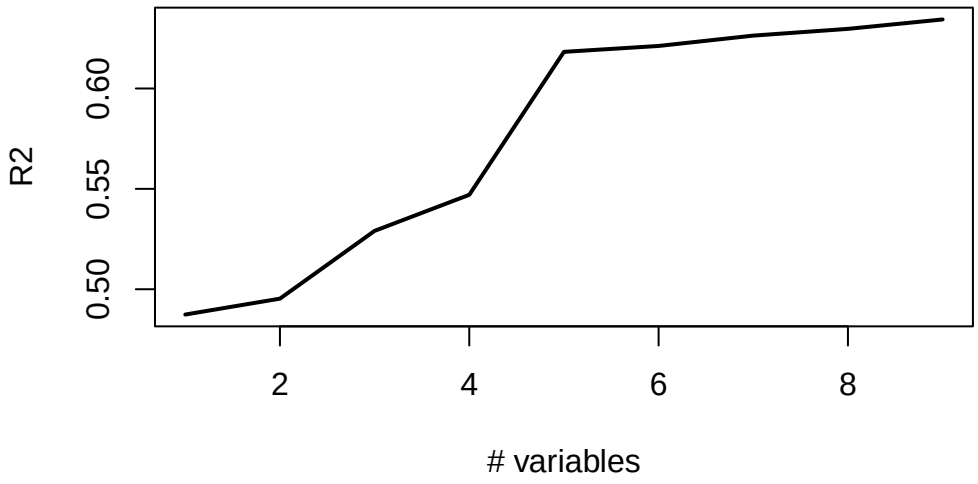
Recall that

$$\bar{R}^2 = 1 - \frac{n-1}{n-p}(1-R^2).$$

The $\bar{R}^2$ statistic does not necessarily increase as additional regressors are introduced into the model. In fact, it can be shown that if s regressors are added to the model, the new $\bar{R}^2$ will exceed the original $\bar{R}^2$ if and only if the partial F statistic for testing the significance of the s additional regressors exceeds 1. Consequently, one criterion for selection of an optimum subset model is to choose the model that has a maximum $\bar{R}^2$. In addition, note that maximizing $\bar{R}^2$ is equivalent to choosing the model with the minimum MSE.

```
# R2 adjusted
r2=c()
for(i in 1:length(var_list)){
  vars=c(var_list[1:i],"ppsq")
```

```
  model2=lm(ppsq~.,df2[,vars])
  sm=summary(model2)
  r2=c(r2,sm$adj.r.squared)
}
plot(r2,type='l',lwd=2,ylab="R2",xlab="# variables")
```




```
var_list[1:6]
```

```
[1] "District"   "Extwall"    "Stories"    "Year_Built" "Units"
[6] "LLS"
```

9.2.2 Mallows C_k criterion

Consider the complete model:

$$Y = \beta_1 + \beta_2 X_1 + \cdots + \beta_p X_{p-1} + \epsilon$$

and a reduced model with $(k-1)$ explanatory variables:

$$Y = \beta_1 + \beta_2 X_1 + \cdots + \beta_k X_{k-1} + \epsilon.$$

Mallows (1973) defined the C_k statistic as

$$C_k = \frac{SSE_R}{MSE_C} - (n - 2k)$$

where k is the number of columns in X in the reduced model.

Observe that if $k = p$ then

$$C_k = C_p = dfE_C - [n - 2p] = p = k.$$

Let Bias $= \mathrm{E}\left[\hat{Y}_i\right] - \mathrm{E}\left[Y_i\right]$ where $\hat{Y}_i$ are generated from the reduced model. It can be shown that

$$E\left[C_k \mid \text{Bias} = 0\right] = k.$$

That is, if the regression model based on the subset of $k-1$ predictors is unbiased, (that is, the model's average predictions match the true values), then we expect C_k to be roughly k. In addition, if there is bias, then $E[C_k] > k$. **Therefore, it is suggested that from all possible models, we choose the model with C_k close to k and $C_k \leq k$.** Sometimes this is a little bit subjective: For example, the smallest C_k is $C_4 = 4.1$ and the next smallest is $C_5 = 4.7$. C_4 is smallest but $C_4 > 4$ – the model may be biased. Now, C_5 is the second smallest, $C_5 < 5$ implies the model is likely not biased.

Example 9.2. Recall example Example 6.6. Investigate the Mallows C_k of the models considered in Example 9.9. In addition, compare the `model_1` and `model_2` below, using `full_model` as the complete model.

```
# Mallows C
# install.packages('olsrr')

# Recall the complete model
full_model=lm(ppsq~ District +  Extwall +
            Stories  +    Year_Built  +
            Units   +     Bdrms   +
            Fbath  +    LLS+Sale_date+District*LLS,df2)


# Potential models
model_1=lm(ppsq~ District   +LLS+ Sale_date+District*LLS ,df2)

model_2=lm(ppsq~ District +  Extwall +LLS+
                Stories  + Sale_date +District*LLS,df2)


olsrr::ols_mallows_cp(model_1,full_model)
```

```
[1] 8936.742
```

```
olsrr::ols_mallows_cp(model_2,full_model)
```

```
[1] 6119.472
```

```
for(i in 1:length(var_list)){
  vars=c(var_list[1:i],"ppsq")
  model=lm(ppsq~.+District*LLS,df2[,c(vars,'LLS')])
  print(paste0("k" ,i," mallows C_k ",
    olsrr::ols_mallows_cp(model,full_model),sep=""))
}
```

```
[1] "k1 mallows C_k 9141.49419283673"
[1] "k2 mallows C_k 8678.09518529921"
[1] "k3 mallows C_k 6378.76523011382"
[1] "k4 mallows C_k 5535.80914135633"
[1] "k5 mallows C_k 973.360231533108"
[1] "k6 mallows C_k 975.360231533108"
[1] "k7 mallows C_k 618.530222838901"
[1] "k8 mallows C_k 385.448019709613"
[1] "k9 mallows C_k 62"
```

If every potential model has a high value for Mallows C_k, this is an indication that some important predictor variables are likely missing from each model. Therefore, we may wish to revaluate the real estate model.

Example 9.3. Use the NFL data from the textbook - Consider the following models provided below. Select the best model from `model`, `model_1` and `model_2` given below, using each of the coefficient of determination criteria and Mallows C_k.

```
df=MPV::table.b1
# Note
names(df)=c("Wins","RushY","PassY",
            "PuntA","FGP","TurnD",
            "PenY","PerR","ORY","OPY")
summary(df)
```

```
      Wins            RushY          PassY          PuntA            FGP
 Min.   : 0.000   Min.   :1416   Min.   :1414   Min.   :35.10   Min.   :38.10
```

```
 1st Qu.: 4.000   1st Qu.:1896   1st Qu.:1714   1st Qu.:37.38   1st Qu.:52.42
 Median : 6.500   Median :2111   Median :2106   Median :38.85   Median :57.70
 Mean   : 6.964   Mean   :2110   Mean   :2127   Mean   :38.64   Mean   :59.40
 3rd Qu.:10.000   3rd Qu.:2303   3rd Qu.:2474   3rd Qu.:39.70   3rd Qu.:68.80
 Max.   :13.000   Max.   :2971   Max.   :2929   Max.   :42.30   Max.   :78.30
     TurnD             PenY             PerR            ORY
 Min.   :-22.00   Min.   : 576.0   Min.   :43.80   Min.   :1457
 1st Qu.: -5.75   1st Qu.: 710.5   1st Qu.:54.77   1st Qu.:1848
 Median :  1.00   Median : 787.5   Median :58.65   Median :2050
 Mean   :  0.00   Mean   : 789.9   Mean   :58.16   Mean   :2110
 3rd Qu.:  6.25   3rd Qu.: 869.8   3rd Qu.:61.10   3rd Qu.:2320
 Max.   : 19.00   Max.   :1037.0   Max.   :67.50   Max.   :2876
      OPY
 Min.   :1575
 1st Qu.:1913
 Median :2101
 Mean   :2128
 3rd Qu.:2328
 Max.   :2670
```

```r
head(df)
```

```
  Wins RushY PassY PuntA  FGP TurnD PenY PerR  ORY  OPY
1   10  2113  1985  38.9 64.7     4  868 59.7 2205 1917
2   11  2003  2855  38.8 61.3     3  615 55.0 2096 1575
3   11  2957  1737  40.1 60.0    14  914 65.6 1847 2175
4   13  2285  2905  41.6 45.3    -4  957 61.4 1903 2476
5   10  2971  1666  39.2 53.8    15  836 66.1 1457 1866
6   11  2309  2927  39.7 74.1     8  786 61.0 1848 2339
```

```r
# plot(df)


model=lm(Wins~PassY+PuntA+FGP+TurnD+PenY+OPY+RushY,data=df)
model_1=lm(Wins~ TurnD+PenY+PerR+ORY+OPY ,df)
model_2=lm(Wins~ TurnD+PenY+ORY+OPY,df)




#  R2
print("m1")
```

```
[1] "m1"
```

```
summary(model_1)$r.squared
```

```
[1] 0.5848535
```

```
print("m2")
```

```
[1] "m2"
```

```
summary(model_2)$r.squared
```

```
[1] 0.5843435
```

```
print("fm")
```

```
[1] "fm"
```

```
summary(model)$r.squared
```

```
[1] 0.7468129
```

```
# ADJ R2
print("m1")
```

```
[1] "m1"
```

```
summary(model_1)$adj.r.squared
```

```
[1] 0.490502
```

```
print("m2")
```

```
[1] "m2"
```

```r
summary(model_2)$adj.r.squared
```

```
[1] 0.5120555
```

```r
print("fm")
```

```
[1] "fm"
```

```r
summary(model)$adj.r.squared
```

```
[1] 0.6581974
```

```r
##### Mallows C
print("m1")
```

```
[1] "m1"
```

```r
olsrr::ols_mallows_cp(model_1,model) # k=6
```

```
[1] 16.79365
```

```r
print("m2")
```

```
[1] "m2"
```

```r
olsrr::ols_mallows_cp(model_2,model) # k=5
```

```
[1] 14.83393
```

```r
print("fm")
```

```
[1] "fm"
```

```
olsrr::ols_mallows_cp(model,model)
```

```
[1] 8
```

We would select `model` as the final model, since it has the lowest values of the Mallows C_k that is close to the number of variables. In addition, the R^2 jumps significantly when moving from either `model_1` or `model_2` to `model`, so again, we would select `model`. Lastly, the adjusted R^2 is the highest for `model`.

9.2.3 Akaike information criterion

- Akaike proposed an information criterion, AIC, based on maximizing the expected **entropy** of the model.
- Entropy is simply a measure of the expected information, in this case the Kullback – Leibler divergence.
- $$AIC = n \log \frac{SSE}{n} + 2p + constant$$
- The decrease in SSE is balanced by the $2p$ penalty
- AIC can be computed for other models, but should only be compared within model classes

9.2.4 Bayesian information criterion/Schwartz information criterion

- Sawa and Schwartz extended AIC, both called BIC.
- This criterion places a greater penalty on adding regressors as the sample size increases.
- $$BIC = n \log \frac{SSE}{n} + p \log n + constant$$
- BIC can be computed for other models, but should only be compared within model classes

Example 9.4. Recall example Example 6.6. Investigate the AIC and BIC of the models considered in Example 9.9. In addition, compare the `model_1`, `model_2` and the `full_model` using AIC and BIC.

```
# Mallows C
# install.packages('olsrr')

# We can consider
full_model=lm(ppsq~ District +  Extwall +
            Stories  +    Year_Built  +
            Units   +     Bdrms   +
            Fbath  +    LLS+Sale_date+District*LLS,df2)


# Potential models
model_1=lm(ppsq~ District   +LLS+ Sale_date+District*LLS ,df2)

model_2=lm(ppsq~ District +  Extwall +LLS+
                Stories  + Sale_date +District*LLS,df2)
# Aic
print('fm')
```

```
[1] "fm"
```

```
AIC(full_model)
```

```
[1] 14436.28
```

```
print('m1')
```

```
[1] "m1"
```

```
AIC(model_1)
```

```
[1] 22006.87
```

```
print('m2')
```

```
[1] "m2"
```

```
AIC(model_2)
```

```
[1] 19853
```

```
for(i in 1:length(var_list)){
  vars=c(var_list[1:i],"ppsq")
  model=lm(ppsq~.,df2[,vars])
  print(paste0("k" ,i," AIC ",
               AIC(model),sep=""))
}
```

```
[1] "k1 AIC 23263.7908086697"
[1] "k2 AIC 22892.2355569629"
[1] "k3 AIC 21206.6115714333"
[1] "k4 AIC 20259.4527688361"
[1] "k5 AIC 16091.4785165542"
[1] "k6 AIC 15904.1437297787"
[1] "k7 AIC 15579.3879301724"
[1] "k8 AIC 15363.6578186034"
[1] "k9 AIC 15053.7804753495"
```

```
# Bic
print('fm')
```

```
[1] "fm"
```

```
BIC(full_model)
```

```
[1] 14930.48
```

```
print('m1')
```

```
[1] "m1"
```

```
BIC(model_1)
```

```
[1] 22266.13
```

```
print('m2')
```

```
[1] "m2"
```

```
BIC(model_2)
```

```
[1] 20201.37
```

```
for(i in 1:length(var_list)){
  vars=c(var_list[1:i],"ppsq")
  model=lm(ppsq~.,df2[,vars])
  print(paste0("k" ,i," AIC ",
               BIC(model),sep=""))
}
```

```
[1] "k1 AIC 23393.4170718241"
[1] "k2 AIC 23086.6749516944"
[1] "k3 AIC 21425.3558905063"
[1] "k4 AIC 20486.2987293563"
[1] "k5 AIC 16342.6294014158"
[1] "k6 AIC 16163.3962560873"
[1] "k7 AIC 15911.5552295054"
[1] "k8 AIC 15736.3333251722"
[1] "k9 AIC 15434.5576233654"
```

We would select `full_model` as the final model, since it has the lowest values of both AIC and BIC.

Example 9.5. Use the NFL data from the textbook - Consider the following models provided below. Select the best model from `model`, `model_1` and `model_2` given below, using each of AIC and BIC.

```
model=lm(Wins~PassY+PuntA+FGP+TurnD+PenY+OPY+RushY,data=df)
model_1=lm(Wins~ TurnD+PenY+PerR+ORY+OPY ,df)
model_2=lm(Wins~ TurnD+PenY+ORY+OPY,df)

# Bic
BIC(model)
```

```
[1] 139.803
```

```
BIC(model_1)
```

```
[1] 146.9846
```

```
BIC(model_2)
```

```
[1] 143.6868
```

```
# Aic
AIC(model)
```

```
[1] 127.8131
```

```
AIC(model_1)
```

```
[1] 137.6592
```

```
AIC(model_2)
```

```
[1] 135.6936
```

We would select `model` as the final model, since it has the lowest values of both AIC and BIC.

9.3 Algorithms for model selection

We now discuss algorithms which auto select the "best" model.

9.3.1 All subsets regression

All subsets regression, also known as best subsets regression, is one way to select the best subset of regressors. This technique proceeds as the name implies - it involves evaluating all possible combinations of regressors to identify the model that best fits the data according to a chosen criterion. For instance, if the criterion is AIC, then we may compute the AIC for all possible 2^p models. All subsets regression would choose the model with the lowest AIC. Any of the four criterion (AIC, BIC, Mallows C_k or adjusted R^2) can be used for all subsets regression.

All subsets regression considers all possible subsets of regressors, ensuring that the best model is not missed by the algorithm. However, this has some downsides. First, it can be very computationally intensive for data sets with a large number of regressors. Second, if not properly controlled, examining many models can lead to **overfitting**, especially with small data sets.

Overfitting is when the model fits the data set so well, that it misses the underlying relationship between the independent and dependent variables. This results in a model that fits the data set well, but performs poorly at predicting new, unseen data due to its overly complex structure.

We can observe this in the example below:

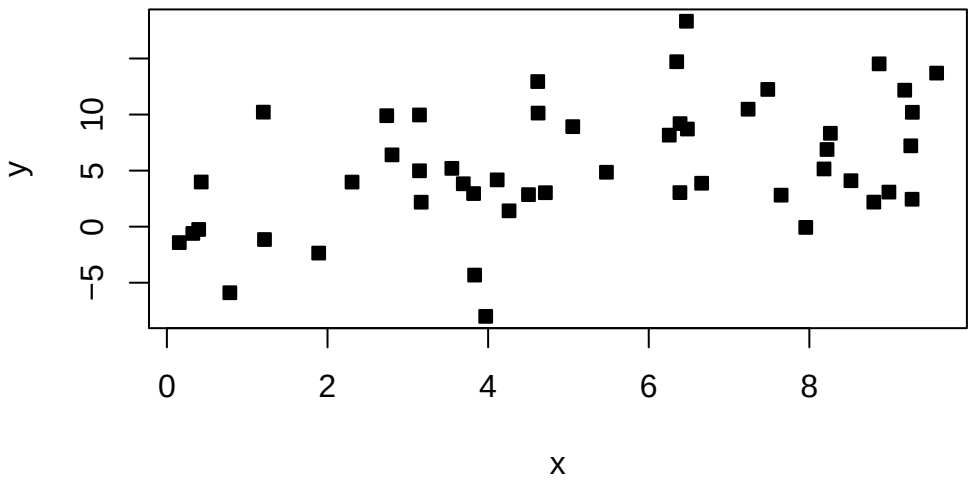
```
library(ggplot2)
```

```
Warning: package 'ggplot2' was built under R version 4.5.3
```

```
set.seed(4265)

# Generate sample data
n = 50
x = runif(n, min = 0, max = 10)
y = 1 + x + rnorm(n, sd = 5)

# Create a data frame
data = data.frame(x = x, y = y)
plot(data,pch=22,bg=1)
```




```
# complexity of the model
complexity=9

# Fit a linear regression model
linear_model = lm(y ~ x, data = data)

# Fit a polynomial regression model (degree = complexity) This model has more terms.
poly_model = lm(y ~ poly(x, complexity), data = data)

# Make predictions
linear_predictions = predict(linear_model, data)
poly_predictions = predict(poly_model, data)

# Calculate mean squared error (MSE)
linear_mse = mean((data$y - linear_predictions)^2)
poly_mse = mean((data$y - poly_predictions)^2)

# Print MSE values
cat("Linear Model MSE:", linear_mse, "\n")
```

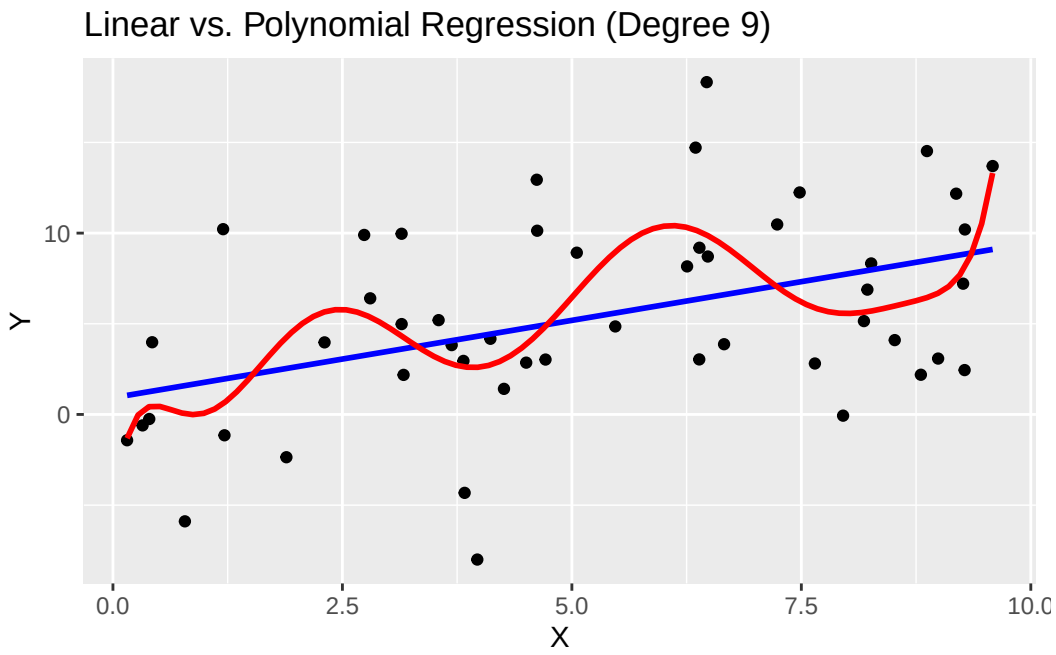
```
Linear Model MSE: 24.03022
```

```
cat("Polynomial Model MSE:", poly_mse, "\n")
```

```
Polynomial Model MSE: 19.69118
```

```
# Notice how the squared error is lower for the more complex model?


# Plot the results - this is GG plot, another way to plot in R
ggplot(data, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, color = "blue", formula = y ~ x) +
  geom_smooth(method = "lm", se = FALSE, color = "red", formula = y ~ poly(x,complexity)) +
  labs(title = "Linear vs. Polynomial Regression (Degree 9)",
       x = "X", y = "Y")
```




```
# Generate new data from the same process
N = 500
xx = runif(n, min = 0, max = 10)
yy = 1 + x + rnorm(n, sd = 5)
nd = data.frame('x'=xx,'y'=yy)
```

```
gen_linear_predictions = predict(linear_model, nd)
gen_poly_predictions = predict(poly_model, nd)

# Calculate mean squared error (MSE)
gen_linear_mse = mean((data$y - gen_linear_predictions)^2)
gen_poly_mse = mean((data$y - gen_poly_predictions)^2)

# Print MSE values
cat("Linear Model MSE:", gen_linear_mse, "\n")
```

```
Linear Model MSE: 45.06416
```

```
cat("Polynomial Model MSE:", gen_poly_mse, "\n")
```

```
Polynomial Model MSE: 50.40172
```

```
# Notice how the generalized error is lower for the linear model?
```

Above, we fit two models to the data. One that allows for polynomial functions of degree at most 9 (not just linear functions), and one that only allows linear functions. It is clear that the polynomial model has a lower error on the data set at hand. However, when we generate new data from the same process, the polynomial model has a much higher error. This is an example of overfitting - the complex model does not generalize well beyond the current data set.

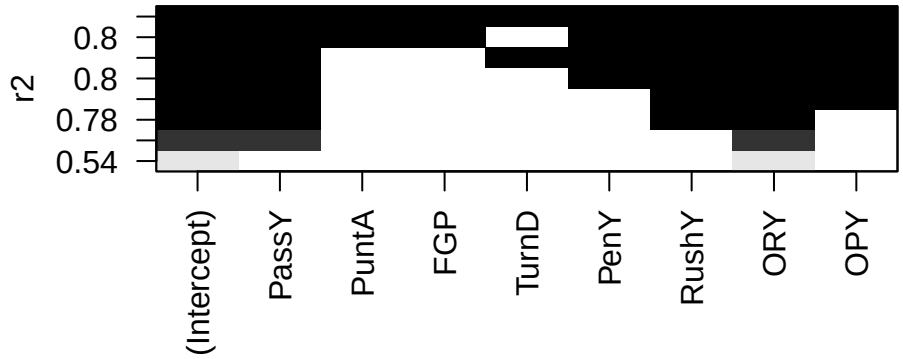
Example 9.6. Use the NFL data from the textbook - Perform all subsets regression with each of the comparison metrics described in the previous section.

```
# install.packages('leaps')

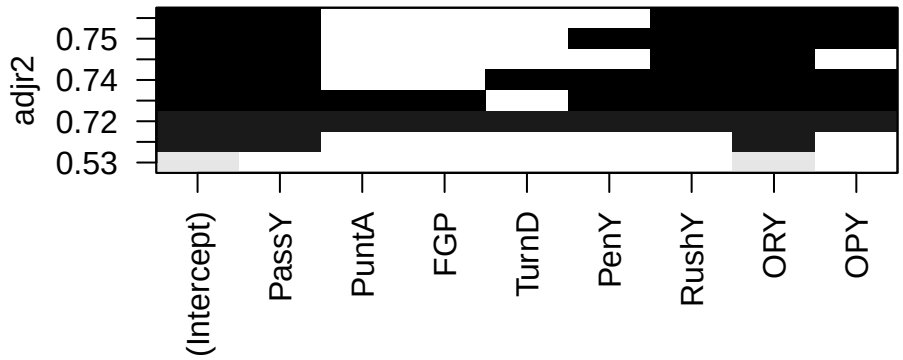
all=leaps::regsubsets(Wins~PassY+PuntA+FGP+TurnD+PenY+RushY+ORY+OPY,data=df,nvmax=10,method=

# 2^8

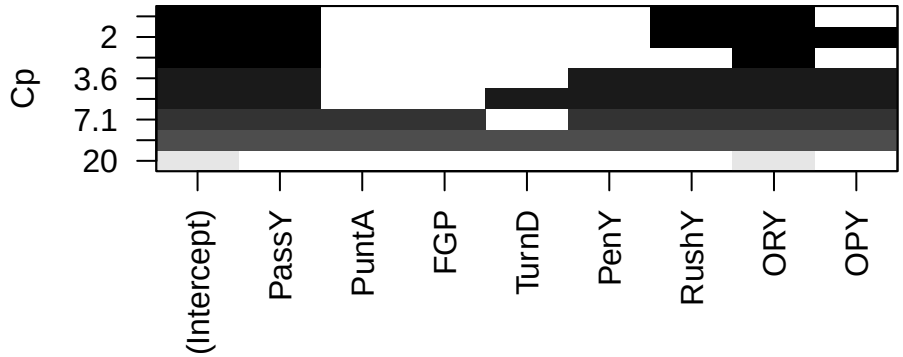
plot(all,scale='r2')
```



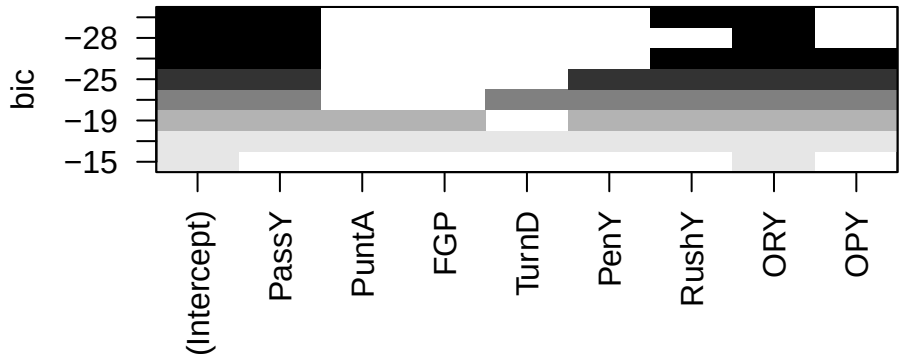

```
plot(all,scale='adjr2')
```



```
plot(all,scale='Cp')
```




```
plot(all,scale='bic')
```



The `leaps` package allows us to do all subsets regression. We see that (with the exception of the R-squared) the best model under all criterion contains passing yards, rushing yard differential, and opponent passing yards.

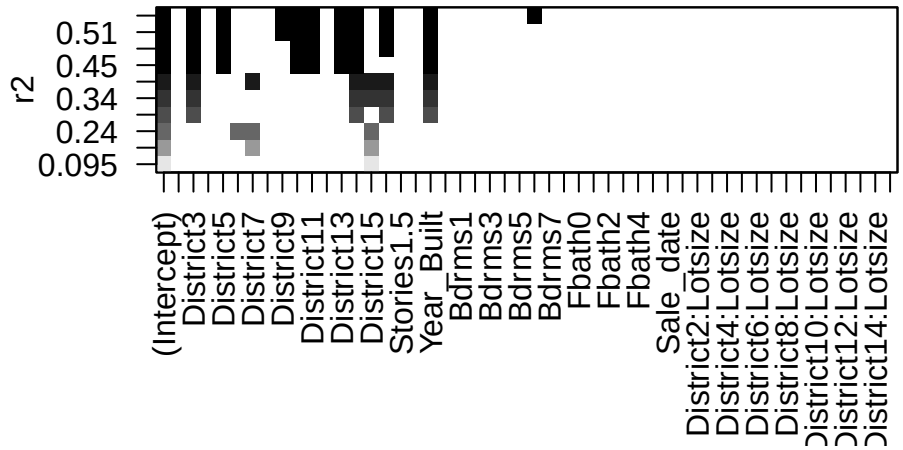
We can do the same thing on the real estate data:

```
# install.packages('leaps')
# If you run this, you will get the following error:
# all=leaps::regsubsets(ppsq~ District +  Extwall +
#             Stories  +    Year_Built  + Fin_sqft  +
#             Units   +     Bdrms   +
#             Fbath  +    Lotsize   + Sale_date +District*Lotsize,data=df2,nvmax=10,method='
# "Error in leaps.exhaustive(a, really.big) : Exhaustive search will be S L O W, must specify
# This shows how this can quickly become computationally intensive

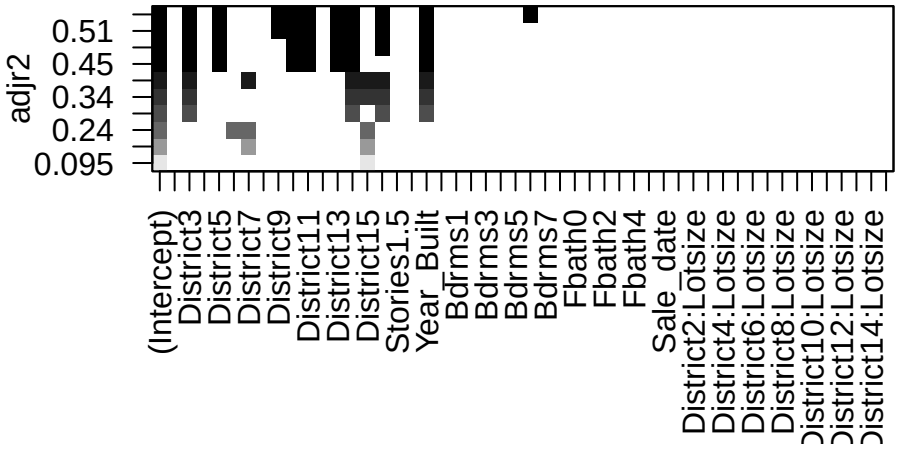
# You will see that running the following code takes very long!
all=leaps::regsubsets(ppsq~ District +
            Stories  +    Year_Built  +    Bdrms   +
            Fbath  +    LLS   + Sale_date+District*Lotsize,data=df2,nvmax=10,really.big=T,me



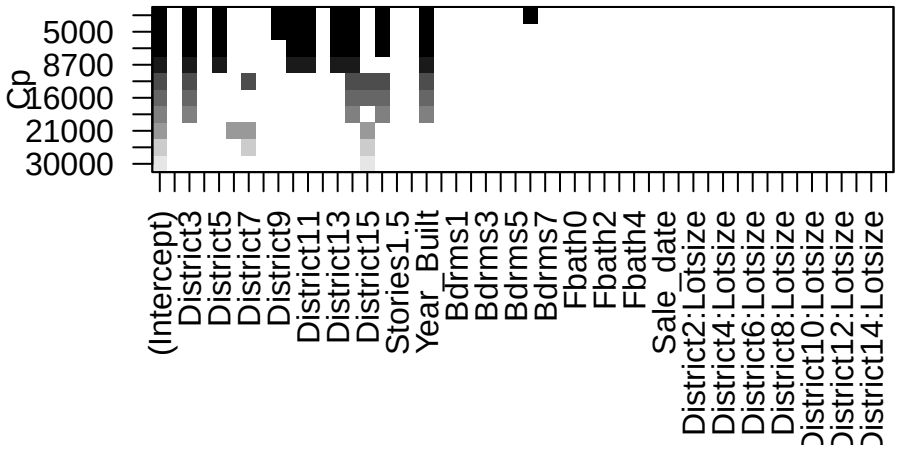
plot(all,scale='r2')
```



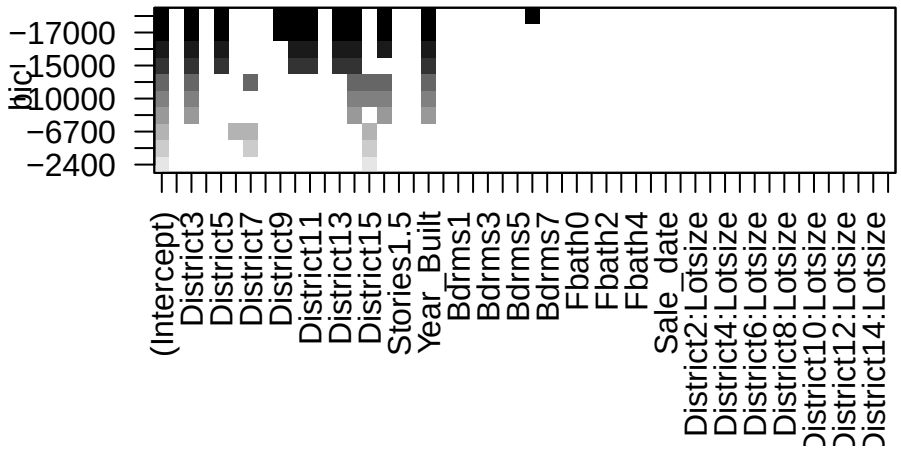
```
plot(all,scale='adjr2')
```




```
plot(all,scale='Cp')
```



```
plot(all,scale='bic')
```



The selected models appear fairly sparse!

9.3.2 Forward selection

Forward selection is another common algorithm used to select the best model. It involves adding regressors one by one until adding more regressors becomes unhelpful. Essentially, we find the most "significant" regressor to be added to the model, and add it if it is "significant enough". We keep adding regressors, until none are significant enough. However, forward selection is "greedy", in the sense that it does not consider all possible subsets of regressors. This means that it is possible that the "best" model is missed by the algorithm.

The algorithm proceeds as follows:

1. Choose α_{entry}, which is the significance level for a regressor to enter the model. Set the current model to be $Y = \beta_0 + \epsilon$.
2. Among all regressors X_i not in the current model, test $H_0 : X_i$ not entered versus $H_a : X_i$ entered.
3. Choose the covariate with smallest p-value, i.e. the most likely X_i to be entered. Say the p-value for testing $H_0 : \beta_1 = 0$ is the smallest.

4. If the chosen p-value is $\geq \alpha_{entry}$, then X_1 is not entered, and the current model is chosen as the final model and the process terminates. Otherwise, X_1 is entered and the new current model is set to the current model, with the addition of X_1.
5. Return to step 2.

Note that once an explanatory variable is entered, it will never leave the model.

Example 9.7. Suppose that we have 3 explanatory variables and α_{entry} is chosen to be 0.05. The following is how we would proceed.

Iteration 1:

Model	$H_a:$	p-value
$Y = \beta_0 + \beta_1 X_1 + \epsilon$	$\beta_1 \neq 0$	0.00033
$Y = \beta_0 + \beta_2 X_2 + \epsilon$	$\beta_2 \neq 0$	0.00490
$Y = \beta_0 + \beta_3 X_3 + \epsilon$	$\beta_3 \neq 0$	0.00101

Since 0.00033 is the smallest p-value and it is less than α_{entry}, the chosen model is $Y = \beta_0 + \beta_1 X_1 + \epsilon$, and we continue to the next iteration.

Iteration 2:

Model	$H_a:$	p-value
$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$	$\beta_2 \neq 0$	0.01330
$Y = \beta_0 + \beta_1 X_1 + \beta_3 X_3 + \epsilon$	$\beta_3 \neq 0$	0.1125

Since 0.01330 is the smallest p-value and it is less than α_{entry}, the chosen model is $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$, and we continue to the next iteration.

Iteration 3:

Model	$H_a:$	p-value
$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \epsilon$	$\beta_3 \neq 0$	0.36145

Since 0.36145 is the smallest p-value and it is greater than α_{entry}, the chosen model is $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$, and STOP.

If α_{entry} is chosen to be 0.01, observe that we will stop at Step 2 and the chosen model is $Y = \beta_0 + \beta_1 X_1 + \epsilon$. As you can see, the chosen model is easily affected by the chosen α_{entry} value.

Caution

Backward, forward and stepwise selection are useful for exploring the important variables associated with the regressor. However, because the model was selected based on p-values, or related quantities, all t and F test p-values are no longer valid. Therefore, where testing is of primary interest, as in many scientific studies, one should select the variables that make the most sense from a scientific/domain knowledge perspective. That model, and related p-values should then be used. Another alternative is LASSO or elastic net regression, which automatically selects variables of interest, and are useful when we have many explanatory variables/regressors.

9.3.3 Backward elimination/selection

In Backward elimination, otherwise known as backward selection, we start with the largest model, i.e., the one with all of the regressors, and eliminate regressors one by one until nothing can be eliminated. Like forward selection, backward selection is also greedy, and therefore, suffers from the same drawbacks.

For $p-1$ regressors, the algorithm proceeds as follows:

1. Choose: α_{stay}, which is the significance level for an explanatory variable to **stay** in the model. Set the current model to be $Y = \beta_0 + \beta_1 X_1 + ... + \beta_{p-1} X_{p-1} + \epsilon$.
2. Test $H_0 : X_i$ eliminated versus $H_a : X_i$ not eliminated, i.e. $H_0 : \beta_i = 0$ versus $H_a : \beta_i \neq 0$ with respect to the current model.
3. Choose the regressor with the largest p-value – i.e., the most likely X_i to be eliminated. Say p-value for testing $H_0 : \beta_p = 0$ is the largest.
4. If the chosen p-value is $< \alpha_{stay}$, then X_p is not eliminated, and the chosen model is the current model and the process terminates. Otherwise, X_p is eliminated and the current model is set to be the old current model, with X_p removed.
5. If there are no regressors in the current model, terminate. Otherwise, go back to step 2.

Example 9.8. Suppose that we have 3 explanatory variables and α_{stay} is chosen to be 0.05. The following is how we would proceed.

The current model is $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \epsilon$.

Iteration 1:

H_a :	p-value
$\beta_1 \neq 0$	0.0176
$\beta_2 \neq 0$	0.05627
$\beta_3 \neq 0$	0.36145

Since 0.36145 is the largest p-value and it is greater than α_{stay}, the current model is set to be $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$, and we continue to the next iteration.

Iteration 2:

Model is $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$.

H_a :	p-value
$\beta_1 \neq 0$	0.00139
$\beta_2 \neq 0$	0.01330

Since 0.01330 is the smallest p-value and it is less than α_{stay}, the final model is $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon$, and the process terminates.

If α_{stay} is chosen to be 0.01, then we will continue the process and eventually, the chosen model is $Y = \beta_0 + \beta_1 X_1 + \epsilon$. Again, the chosen model is easily affected by the chosen α_{stay} value.

In forward selection, once a regressor is entered, it will be in the final model. In backward elimination, once a regressor is eliminated, it will not be in the final model. A combination of these two processes, that allows for variables to enter and exit the model is called **stepwise Regression**.

9.3.4 Stepwise regression

In stepwise regression, we start the model from smallest model, $Y = \beta_0 + \epsilon$, and we keep adding and eliminating regressors one by one such that added regressors can be eliminated later and eliminated regressors can be added later. Like forward selection and backward selection, stepwise regression is also greedy, and therefore, suffers from the same drawbacks.

For $p - 1$ regressors, the algorithm proceeds as follows:

1. Choose: α_{enter} and α_{stay}. Set the current model to be $Y = \beta_0 + \epsilon$.
2. Set the old model to be the current model. Perform steps 2-4 of forward selection.
3. Suppose in the previous step, X_1 was added to the current model. Next, with the current model, apply steps 2-5 of backward elimination.
4. If X_1 is eliminated in backward elimination, then do not remove X_1 from the current model and set the current model to be the final model. The process then terminates. If the current model equals the old model, terminate. Otherwise, go back to step 2.

The stopping rule for stepwise regression is when we run into infinite loop (the same variable being added and eliminated) OR when we have nothing to add **and** nothing to eliminate.

> **Note**
>
> 1. One way to avoid the infinite loop is to choose $\alpha_{stay} \neq \alpha_{entry}$.
> 2. Some books suggest "easy to enter" and "tough to eliminate" strategy for selecting $\alpha_{stay}, \alpha_{entry}$ but there is no strict rule.
> 3. `R` is using the AIC criterion for entering and eliminating, rather than the F test. This is also acceptable.

Example 9.9. Recall the real estate example - Example 6.6. Run forward selection, backward selection and stepwise regression using AIC as the criterion, including all regressors. (AIC is done automatically in R.)

```
# ############ Automated methods ############

############ Forward selection with AIC ############

df3=df2[,c('District','Extwall','Stories','Year_Built','Units','Bdrms','Fbath','Sale_date','

#define intercept-only model
intercept_only = lm(ppsq~ 1, data=df3)



# define model with all predictors
all = lm(ppsq~ ., data=df3)

# perform forward stepwise regression
forward = step(intercept_only, direction='forward', scope=formula(all), trace=0)

#view results of forward stepwise regression
forward$anova
```

```
          Step  Df   Deviance Resid. Df Resid. Dev       AIC
1               NA         NA     24382   7225.476 -29654.39
2   + District -14 3523.76321     24368   3701.712 -45934.17
3      + Units  -3  825.50754     24365   2876.205 -52080.58
4 + Year_Built  -1  100.56239     24364   2775.642 -52946.36
5  + Sale_date  -1   35.95794     24363   2739.684 -53262.30
6      + Bdrms  -9   35.60684     24354   2704.078 -53563.27
7      + Fbath  -5   32.74525     24349   2671.332 -53850.34
8        + LLS  -1   21.55485     24348   2649.778 -54045.89
9    + Extwall  -8   12.84896     24340   2636.929 -54148.41
```

```
#view final model
names(forward$coefficients)
```

```
 [1] "(Intercept)"            "District2"              "District3"
 [4] "District4"              "District5"              "District6"
 [7] "District7"              "District8"              "District9"
[10] "District10"             "District11"             "District12"
[13] "District13"             "District14"             "District15"
[16] "Units1"                 "Units2"                 "Units3"
[19] "Year_Built"             "Sale_date"              "Bdrms0"
[22] "Bdrms1"                 "Bdrms2"                 "Bdrms3"
[25] "Bdrms4"                 "Bdrms5"                 "Bdrms6"
[28] "Bdrms7"                 "Bdrms8"                 "Fbath0"
[31] "Fbath1"                 "Fbath2"                 "Fbath3"
[34] "Fbath4"                 "LLS"                    "ExtwallBlock"
[37] "ExtwallBrick"           "ExtwallFiber-Cement"    "ExtwallFrame"
[40] "ExtwallMasonry / Frame" "ExtwallPrem Wood"       "ExtwallStone"
[43] "ExtwallStucco"
```

```
# ###################### Backward selection with AIC ######################
#define intercept-only model
intercept_only = lm(ppsq~ 1, data=df3)

#define model with all predictors
all = lm(ppsq~ ., data=df3)

#perform backward stepwise regression
backward = step(all, direction='backward', scope=formula(all), trace=0)

#view results of backward stepwise regression
backward$anova
```

```
       Step Df  Deviance Resid. Df Resid. Dev       AIC
1           NA        NA     24337   2636.737 -54144.18
2 - Stories  3 0.1911574     24340   2636.929 -54148.41
```

```
#view final model

names(backward$coefficients)
```

```
 [1] "(Intercept)"            "District2"              "District3"
```

```
 [4] "District4"              "District5"              "District6"
 [7] "District7"              "District8"              "District9"
[10] "District10"             "District11"             "District12"
[13] "District13"             "District14"             "District15"
[16] "ExtwallBlock"           "ExtwallBrick"           "ExtwallFiber-Cement"
[19] "ExtwallFrame"           "ExtwallMasonry / Frame" "ExtwallPrem Wood"
[22] "ExtwallStone"           "ExtwallStucco"          "Year_Built"
[25] "Units1"                 "Units2"                 "Units3"
[28] "Bdrms0"                 "Bdrms1"                 "Bdrms2"
[31] "Bdrms3"                 "Bdrms4"                 "Bdrms5"
[34] "Bdrms6"                 "Bdrms7"                 "Bdrms8"
[37] "Fbath0"                 "Fbath1"                 "Fbath2"
[40] "Fbath3"                 "Fbath4"                 "Sale_date"
[43] "LLS"
```

```r
############ Stepwise Regression with AIC ############
# Define intercept-only model
intercept_only = lm(ppsq~ 1, data=df3)

# Define model with all predictors
all = lm(ppsq~ ., data=df3)

#perform stepwise regression
both = step(intercept_only, direction='both', scope=formula(all), trace=0)

#view results of Stepwise regression
both$anova
```

```
          Step  Df   Deviance Resid. Df Resid. Dev       AIC
1               NA         NA     24382   7225.476 -29654.39
2   + District -14 3523.76321     24368   3701.712 -45934.17
3      + Units  -3  825.50754     24365   2876.205 -52080.58
4 + Year_Built  -1  100.56239     24364   2775.642 -52946.36
5  + Sale_date  -1   35.95794     24363   2739.684 -53262.30
6      + Bdrms  -9   35.60684     24354   2704.078 -53563.27
7      + Fbath  -5   32.74525     24349   2671.332 -53850.34
8        + LLS  -1   21.55485     24348   2649.778 -54045.89
9    + Extwall  -8   12.84896     24340   2636.929 -54148.41
```

```r
#view final model
names(both$coefficients)
```

```
 [1] "(Intercept)"            "District2"              "District3"
 [4] "District4"              "District5"              "District6"
 [7] "District7"              "District8"              "District9"
[10] "District10"             "District11"             "District12"
[13] "District13"             "District14"             "District15"
[16] "Units1"                 "Units2"                 "Units3"
[19] "Year_Built"             "Sale_date"              "Bdrms0"
[22] "Bdrms1"                 "Bdrms2"                 "Bdrms3"
[25] "Bdrms4"                 "Bdrms5"                 "Bdrms6"
[28] "Bdrms7"                 "Bdrms8"                 "Fbath0"
[31] "Fbath1"                 "Fbath2"                 "Fbath3"
[34] "Fbath4"                 "LLS"                    "ExtwallBlock"
[37] "ExtwallBrick"           "ExtwallFiber-Cement"    "ExtwallFrame"
[40] "ExtwallMasonry / Frame" "ExtwallPrem Wood"       "ExtwallStone"
[43] "ExtwallStucco"
```

These methods retain many more variables. It is interesting, considering all subsets contained very few.

Example 9.10. Use the NFL data from the textbook - Run forward selection and backward selection including all regressors using the F statistic criterion.

```
############ Forward selection by F value
# Threshold for p-value to determine variable inclusion
thresh = 0.05

# Initialize an empty vector to store currently selected variables
curr_vars = c()

# Create a vector of variable names excluding the dependent variable 'Wins' and 'PerR'
vars_left = names(df)[2:10]
vars_left = vars_left[vars_left != 'PerR']

# Initialize the passed flag to TRUE
passed = TRUE

# Loop until no more variables can be added (passed is FALSE) or there are no more variables
while(passed && length(vars_left) > 0) {

  # Initialize an empty vector to store p-values of models
  pvals = c()
```

```
# Loop through each remaining variable
for(var in vars_left) {
  # Create a temporary dataframe with current variables and the new candidate variable
  df_tmp = df[, c(curr_vars, var, 'Wins')]

  # Fit a linear model with 'Wins' as the dependent variable
  model = lm(Wins ~ ., df_tmp)

  # Get the summary of the model
  s = summary(model)

  # Calculate the p-value of the F-statistic for the model
  pval = 1 - pf(s$fstatistic[1], s$fstatistic[2], s$fstatistic[3])

  # Append the p-value to the pvals vector
  pvals = c(pvals, pval)
}

# Find the index of the minimum p-value
min_index = which.min(pvals)

# Get the minimum p-value
mp = pvals[min_index]

# Check if the minimum p-value is less than the threshold
passed = mp < thresh

if(passed) {
  # If passed, get the corresponding variable name
  new_var = vars_left[min_index]

  # Add the new variable to the current variables list
  curr_vars = c(curr_vars, new_var)

  # Remove the new variable from the remaining variables list
  vars_left = vars_left[vars_left != new_var]

  # Print the p-value and the variable being added
  print('pvalue')
  print(mp)
  print('adding')
  print(new_var)
```

```
  }
}
```

```
[1] "pvalue"
       value
7.380709e-06
[1] "adding"
[1] "ORY"
[1] "pvalue"
       value
4.151848e-08
[1] "adding"
[1] "PassY"
[1] "pvalue"
       value
5.286139e-08
[1] "adding"
[1] "RushY"
[1] "pvalue"
       value
1.237473e-07
[1] "adding"
[1] "OPY"
[1] "pvalue"
       value
5.151486e-07
[1] "adding"
[1] "PenY"
[1] "pvalue"
       value
2.098536e-06
[1] "adding"
[1] "TurnD"
[1] "pvalue"
       value
7.894081e-06
[1] "adding"
[1] "PuntA"
[1] "pvalue"
      value
2.52811e-05
[1] "adding"
```

```
[1] "FGP"
```

```
# Print the final list of selected variables
print(curr_vars)
```

```
[1] "ORY"   "PassY" "RushY" "OPY"   "PenY"  "TurnD" "PuntA" "FGP"
```

```
# Threshold for p-value to determine variable exclusion
thresh = 0.05

# Initialize a vector of variable names excluding the dependent variable 'Wins' and 'PerR'
vars_left = names(df)[2:10]
vars_left = vars_left[vars_left != 'PerR']

# Initialize the passed flag to TRUE
passed = TRUE

# Loop until no more variables need to be removed (passed is FALSE) or there are no more vari
while(passed && length(vars_left) > 0) {
  # Create a temporary dataframe with remaining variables and the dependent variable 'Wins'
  df_tmp = df[, c(vars_left, 'Wins')]

  # Fit a linear model with 'Wins' as the dependent variable
  model = lm(Wins ~ ., df_tmp)

  # Get the summary of the model
  s = summary(model)

  # Extract the p-values of the t-statistics for the coefficients
  tv = coef(s)[, "Pr(>|t|)"]

  # Remove the intercept p-value
  tv = tv[-1]

  # Find the index of the maximum p-value
  max_index = which.max(tv)

  # Get the maximum p-value
  mp = tv[max_index]

  # Print the maximum p-value
  print(mp)
```

```
  # Check if the maximum p-value is greater than the threshold
  passed = mp > thresh

  if(passed) {
    # If passed, get the corresponding variable name to be removed
    rem_var = names(max_index)

    # Remove the variable from the remaining variables list
    vars_left = vars_left[vars_left != rem_var]

    # Print the p-value and the variable being removed
    print('pvalue')
    print(mp)
    print('removing')
    print(rem_var)
  }
}
```

```
    TurnD
0.7312364
[1] "pvalue"
    TurnD
0.7312364
[1] "removing"
[1] "TurnD"
      FGP
0.5679556
[1] "pvalue"
      FGP
0.5679556
[1] "removing"
[1] "FGP"
    PuntA
0.6919653
[1] "pvalue"
    PuntA
0.6919653
[1] "removing"
[1] "PuntA"
     PenY
0.5147875
```

```
[1] "pvalue"
     PenY
0.5147875
[1] "removing"
[1] "PenY"
      OPY
0.1752231
[1] "pvalue"
      OPY
0.1752231
[1] "removing"
[1] "OPY"
     RushY
0.06663316
[1] "pvalue"
     RushY
0.06663316
[1] "removing"
[1] "RushY"
       PassY
0.0001775241
```

```
# Print the final list of remaining variables
print(vars_left)
```

```
[1] "PassY" "ORY"
```

Exercise 9.1. Use the NFL data from the textbook - modify the above code to perform stepwise regression, including all regressors using the F statistic criterion.

9.4 Cross Validation

We may be mainly interested in predictive performance of our model. To assess the performance of our model's predictive ability, it would be ideal to test its ability to predict the response. We could test our model out on our dataset, but our model has already 'seen' this dataset. Previously, we saw that the model's predictive ability on the dataset at hand does not neccessarily mean it will output good predictions for new data from the same population. This means that the predictive ability of the model on the current observations won't reflect the model's ability to predict observations it hasn't seen. It would be ideal to have some new data in order to measure predictive performance, but it is also ideal to use all data in building the model.

Cross validation allows us to obtain an estimate of a given model's out of sample predictive performance.

Cross-validation is a technique where we repeatedly fit our model using a subset of the dataset and then compute its predictive performance on the complementary subset of the dataset. We then average these predictive performances and use this to estimate our model's overall predictive performance.

Here is an overview of the algorithm:

1. Partition the data $\mathbb{Z}$ into k groups of size about n/k uniformly at random. Denoted $\mathbb{Z}_1, \dots, \mathbb{Z}_k$.
2. For each; $\mathbb{Z}_j$, fit the regression model on $\mathbb{Z} \backslash \mathbb{Z}_j$.
3. For each observation in the left out group $\mathbb{Z}_j$, use the previously computed model to produce a prediction $\hat{Y}_{j,1}, \dots, \hat{Y}_{j,n/k}$, resulting in n/k predictions.
4. For each prediction, compute the predictive error $(y_{j,i} - \hat{y}_{j,i})^2$ (Can also use absolute error).
5. Repeat steps 2-3 for each group $1, \dots, k$.
6. Average the n predictive errors.
7. Repeat for all candidate models and choose the model with the lowest prediction error.

To highlight the choice of k, we say k-fold Cross Validation – Each of the k subsets are known as folds. What do we choose for k? LOOCV (Leave One Out Cross Validation) is a popular technique. Here, $k = n$. This is where each run, one observation is left out of the model. This gives a low bias, but high variation estimate of the prediction error. It also has a high run time. As a result, often, k is taken to be 5 or 10. Taking lower values of k gives a result similar to that of splitting the data into training and test sets, while higher values of k gives a result similar to LOOCV. The choice of the number of folds in cross validation is a bias-variance tradeoff: too few folds may result in high bias, while too many folds may result in high variance. (This is why k is taken between 1 and n.)

Example 9.11. Using the real estate data, run cross validation in a forward selection manner. That is, run the cross validation procedure for the model with the first variable, the first and the second variable, the first three variables etc. Which model has the lowest error?

The `caret` package can be used to perform 5-fold CV.

```
############ Cross Validation Regression ############
# install.packages('caret')
library(caret)
```

```
Warning: package 'caret' was built under R version 4.5.3
```

```
Loading required package: lattice

Warning: package 'lattice' was built under R version 4.5.3
```

```r
#specify the cross-validation method 5 fold in this case.
ctrl = trainControl(method = "cv", number = 5)

#fit a regression model and use k-fold CV to evaluate performance
model = train(ppsq~., data = df3, method = "lm", trControl = ctrl)


for(i in 1:length(var_list)){
  vars=c(var_list[1:i],"ppsq")
  model = train(ppsq~., data = df3[,vars], method = "lm", trControl = ctrl)
  print(paste0("k" ,i," CV Result ", model$results$RMSE,sep=""))
}
```

```
[1] "k1 CV Result 0.39011859000292"
[1] "k2 CV Result 0.387043371742809"
[1] "k3 CV Result 0.373888622047606"
[1] "k4 CV Result 0.366761298812602"
[1] "k5 CV Result 0.336828521829276"
[1] "k6 CV Result 0.335315694585366"
[1] "k7 CV Result 0.333114794995401"
[1] "k8 CV Result 0.332012297761635"
[1] "k9 CV Result 0.329958007819844"
```

```r
#view summary of k-fold CV
print(model)
```

```
Linear Regression

24383 samples
    9 predictor

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 19506, 19506, 19507, 19507, 19506
Resampling results:

  RMSE      Rsquared   MAE
```

```
  0.329958  0.6330707  0.2384547

Tuning parameter 'intercept' was held constant at a value of TRUE
```

```
# We see that all of the variables minimize the prediction error.
```

Example 9.12. Use the NFL data from the textbook. 1. Compare the model `Wins~.` to the model `Wins~PassY+PuntA+FGP+TurnD+PenY+OPY+RushY`. 2. Run cross validation in a forward selection manner. That is, run the cross validation procedure for the model with the first variable, the first and the second variable, the first three variables etc. Which model has the lowest error? 3. Run cross validation for all subsets. Which model has the lowest error? Does it match the result in step 2?

```
############ Cross Validation Regression ############
set.seed(1251)


#specify the cross-validation method
ctrl = trainControl(method = "cv", number = 5)

#fit a regression model and use k-fold CV to evaluate performance
model_cv_1 = train(Wins~., data = df, method = "lm", trControl = ctrl)
model_cv_1
```

```
Linear Regression

28 samples
 9 predictor

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 21, 22, 22, 24, 23
Resampling results:

  RMSE      Rsquared   MAE
  1.962945  0.5925805  1.686354

Tuning parameter 'intercept' was held constant at a value of TRUE
```

```
model_cv_2 = train(Wins~PassY+PuntA+FGP+TurnD+PenY+OPY+RushY, data = df, method = "lm", trCo
model_cv_2
```

```
Linear Regression

28 samples
 7 predictor

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 23, 21, 23, 23, 22
Resampling results:

  RMSE      Rsquared   MAE
  2.666742  0.5425439  2.048828

Tuning parameter 'intercept' was held constant at a value of TRUE
```

```
for(i in 2:ncol(df)){
  # reg=sample(2:10,sample(1:9,1))
  vars=c(names(df)[2:i],'Wins')
  model = train(Wins~., data = df[,vars], method = "lm", trControl = ctrl)
  print(paste0("k=" ,i-1," vars ",paste(vars,collapse=' '),"  CV Result ",
               round(model$results$RMSE,1),sep=""))
}
```

```
[1] "k=1 vars RushY Wins CV Result 2.9"
[1] "k=2 vars RushY PassY Wins CV Result 2.3"
[1] "k=3 vars RushY PassY PuntA Wins CV Result 2.2"
[1] "k=4 vars RushY PassY PuntA FGP Wins CV Result 2.4"
[1] "k=5 vars RushY PassY PuntA FGP TurnD Wins CV Result 2.6"
[1] "k=6 vars RushY PassY PuntA FGP TurnD PenY Wins CV Result 2.5"
[1] "k=7 vars RushY PassY PuntA FGP TurnD PenY PerR Wins CV Result 2.5"
[1] "k=8 vars RushY PassY PuntA FGP TurnD PenY PerR ORY Wins CV Result 2.5"
[1] "k=9 vars RushY PassY PuntA FGP TurnD PenY PerR ORY OPY Wins CV Result 2"
```

```
# Select model k=9 vars

#
p = ncol(df)-1
l = rep(list(0:1), p)

models=expand.grid(l); dim(models)
```

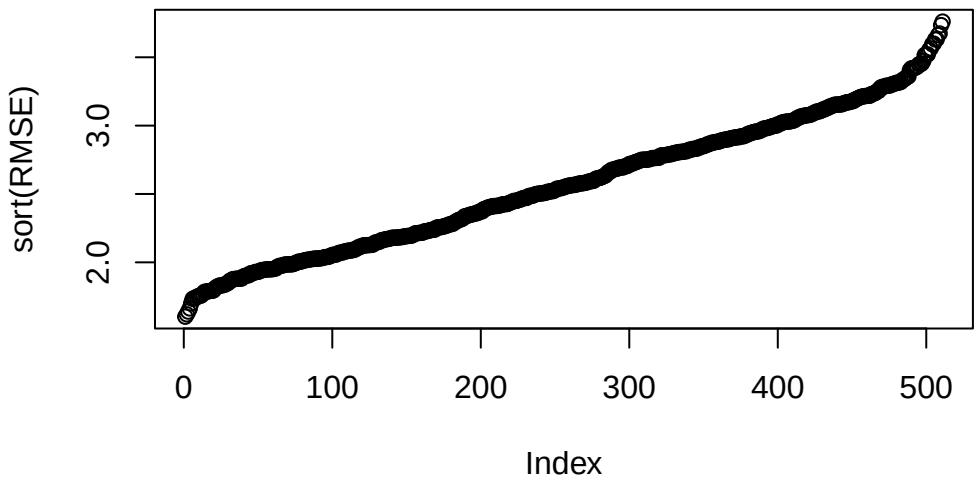
```
[1] 512   9
```

```
cv=RMSE=c()
for(i in 2:nrow(models)){
  reg=(2:(p+1))[models[i,]==1]
  vars=c(names(df)[reg],'Wins')
  model = train(Wins~., data = df[,vars], method = "lm", trControl = ctrl)
  RMSE=c(RMSE,model$results$RMSE)
  cv=c(cv,list(model))
  # print(paste0("k=" ,k," vars ",paste(vars,collapse=' '),"  CV Result ",
              # round(model$results$RMSE,1),sep=""))
}

bottom_10=order(RMSE)[1:10]
RMSE[bottom_10]
```

```
 [1] 1.602365 1.619883 1.639797 1.662903 1.699846 1.733090 1.737918 1.740185
 [9] 1.749398 1.753767
```

```
plot(sort(RMSE))
```




```
m=which.min(RMSE)
cv[which.min(RMSE)]
```

```
[[1]]
Linear Regression

28 samples
 6 predictor

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 23, 22, 21, 22, 24
Resampling results:

  RMSE      Rsquared   MAE
  1.602365  0.7865483  1.300662

Tuning parameter 'intercept' was held constant at a value of TRUE
```

```
df[1,(2:(p+1))[unlist(models[m,])==1]]
```

```
  RushY PuntA PenY PerR  ORY  OPY
1  2113  38.9  868 59.7 2205 1917
```

```
model = train(Wins~ PassY+PuntA+TurnD+PerR+OPY+RushY, data = df, method = "lm", trControl = c
RMSE=c(RMSE,model$results$RMSE)
```

9.5 LASSO Regression

Lasso regression (Least Absolute Shrinkage and Selection Operator) is a technique for linear models that adds a penalty term to the least squares loss function.

$$\hat{\beta}_{LASSO} = \underset{\beta}{\operatorname{argmin}} \|Y - X\beta\|^2 + \lambda \|\beta\|_1$$

This penalty actually "shrinks'' some coefficients exactly to zero, effectively performing variable selection. Lasso is particularly useful when dealing with high-dimensional data where the number of predictors may exceed the number of observations. The parameter λ is a tuning parameter, which controls the amount of shrinkage. It is typically chosen via cross validation. It is great for variable selection and prediction problems, but obtaining p-values and other inferential quantities is not so straightforward.

```r
#..
library(glmnet)
```

```
Warning: package 'glmnet' was built under R version 4.5.3
```

```
Loading required package: Matrix
```

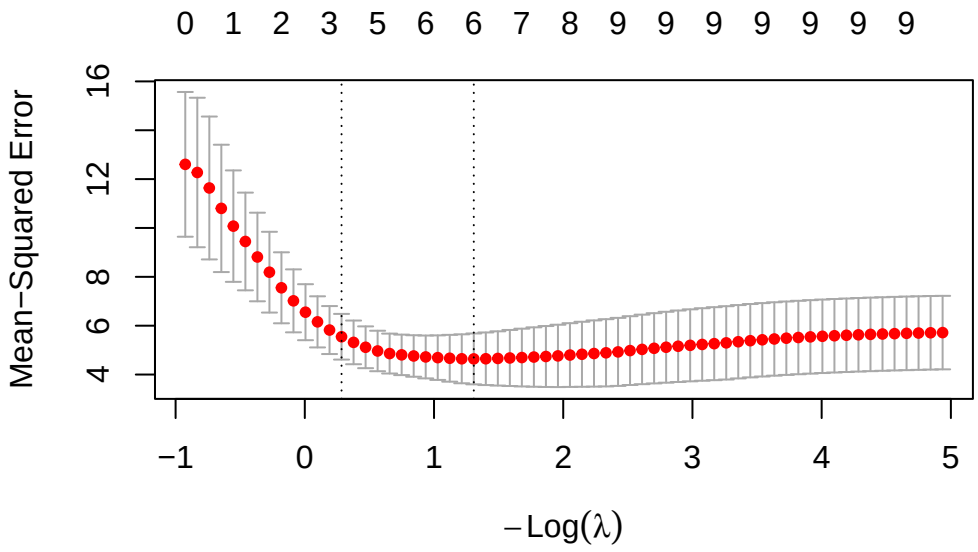
```
Loaded glmnet 5.0
```

```r
model=lm(Wins~., data = df)
X=model.matrix(model)
# Remove intercept
X=X[,-1]
cv_model = cv.glmnet(X, df$Wins, alpha = 1,nfolds=5)

best_lambda = cv_model$lambda.min
best_lambda
```

```
[1] 0.2704249
```

```r
# plot of test MSE by lambda
plot(cv_model)
```



```
# Coefficients:

coef(cv_model)
```

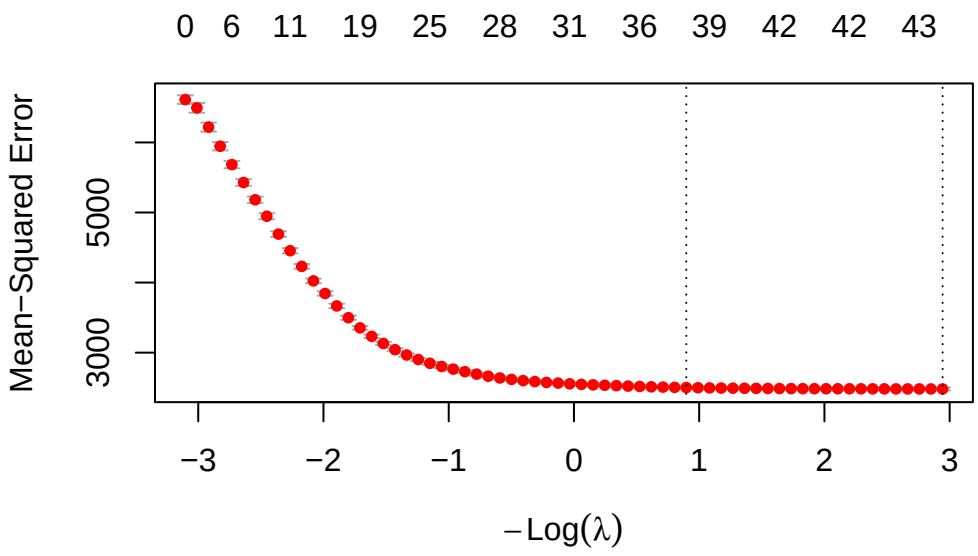
```
10 x 1 sparse Matrix of class "dgCMatrix"
               lambda.1se
(Intercept) 10.7211210917
RushY        0.0007812133
PassY        0.0016929743
PuntA        .
FGP          .
TurnD        .
PenY         .
PerR         .
ORY         -0.0042682806
OPY          .
```

We can also try the real estate data:

```
# Our example from previous units:
df5=df2[df2$District!=3,]
df5$District=droplevels(df5$District)
df2=df5
model=lm(Sale_price~District+Extwall+Stories+Year_Built+Fin_sqft+Units+Bdrms+Fbath+Lotsize+Sa
X=model.matrix(model)
# Remove intercept
X=X[,-1]
cv_model = cv.glmnet(X, sqrt(df2$Sale_price), alpha = 1,nfolds=5)
best_lambda = cv_model$lambda.min
best_lambda
```

```
[1] 0.05266812
```

```
# plot of test MSE by lambda
plot(cv_model)
```




```
# Coefficients:

coef(cv_model)
```

```
46 x 1 sparse Matrix of class "dgCMatrix"
                          lambda.1se
(Intercept)           -8.635785e+02
District2              1.134907e+01
District4             -4.028622e+01
District5              7.137695e+01
District6              .
District7             -1.954226e+01
District8              1.880525e+01
District9              4.307692e+01
District10             7.934201e+01
District11             9.296592e+01
District12             2.317774e-01
District13             9.032709e+01
District14             1.284919e+02
District15            -5.282402e+01
ExtwallBlock          -3.589080e+00
ExtwallBrick           7.581600e+00
```

```
ExtwallFiber-Cement     4.809818e+01
ExtwallFrame           -3.851441e+00
ExtwallMasonry / Frame  2.764569e+00
ExtwallPrem Wood        8.357316e+00
ExtwallStone            1.751120e+01
ExtwallStucco           5.022336e+00
Stories1               -1.520187e+01
Stories1.5              .
Stories2                1.598476e+00
Year_Built              4.653726e-01
Fin_sqft                7.077201e-02
Units1                  7.000043e+01
Units2                 -3.323116e+00
Units3                 -2.992420e+01
Bdrms0                  8.837086e+00
Bdrms1                 -1.482303e+01
Bdrms2                 -5.203189e+00
Bdrms3                  8.495290e+00
Bdrms4                  4.591476e-01
Bdrms5                  .
Bdrms6                 -1.505946e+01
Bdrms7                 -2.745586e+01
Bdrms8                 -2.561850e+01
Fbath0                  .
Fbath1                 -2.185965e+01
Fbath2                  .
Fbath3                  8.628325e+00
Fbath4                  .
Lotsize                 2.131218e-03
Sale_date               5.418531e-03
```

9.6 A final note

In modern statistics, forward/backward and stepwise are some what legacy methods for variable selection. They are easy to understand, but losing the meaning of the p-values often renders them not useful in many applications. It is better to choose one model based on scientific and domain knowledge, and interpret those p-values. This is not feasible for high dimensional data, and in that case, LASSO, elastic net regression are great for prediction and variable selection problems. P-values for high dimensional regression problems exist, but are not so straightforward to obtain readily at the current time.

9.7 Homework questions

Complete the Chapter 10 questions from the textbook.

Exercise 9.2. Load car mileage data from Example 3.8 and run 10-fold cross validation on all subset models. Which model is the best? Try LASSO regression. Try Forward and backaward selection.

10 Robust Regression

A serious problem that may dramatically impact the usefulness of a regression model is that of outlying observations. We have seen trying to locate and remove outliers, but an alternative technique is to use robust statistics. Robust statistics is a field of statistics that develops estimators that are not corrupted by outlying observations.

Recall that we minimize the squared error to obtain the least squares estimator. The best estimator was defined as the minimizer of

$$C(\beta) = \sum_{i=1}^{n} (Y_i - \beta^\top X_i)^2.$$

The squared error is an example of a **cost function**. The fact that the errors are squared implies that unusually large residuals will contribute significantly to the cost function. We may instead define the best estimator to minimize another cost function, say

$$C(\beta) = \sum_{i=1}^{n} |Y_i - \beta^\top X_i|.$$

The absolute value function grows linearly, thus, it is less affected by unusually large residuals. We can go a step further and design a cost function that is even less affected by unusually large residuals.

10.1 M-estimators

M-estimators are a group of estimators which can be defined as minimizers of cost functions. (The M stands for minimizers.) We can define an error function ρ as a function such that $\rho(|x|)$ is non-negative, non-decreasing and $\rho(0) = 0$. We can then define the **M-estimator** with error function ρ as

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}}\, C_\rho(\beta) = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^{n} \rho(Y_i - \beta^\top X_i) = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^{n} \rho(\hat{\epsilon}_i(\beta)),$$

where $\hat{\epsilon}_i(\beta) = Y_i - \beta^\top X_i$. Note that for arbitrary ρ, the above may not have a unique minimizer, which may be problematic for computational algorithms and theoretical analysis. An M-estimator is an alternative to the OLS estimator. The idea is to define ρ so that large residuals $\hat{\epsilon}_i(\beta)$ do not contribute more than they should to the cost function. To elaborate, we

would like $\rho(\hat{\epsilon}_i(\beta))$ to be somewhat large if $\hat{\epsilon}_i(\beta)$ is large, because that signals that the model defined by β does not fit the data well at that point. On the other hand, if observation i is an outlier, than we don't want to try too hard to make the model fit that point.

One drawback of M-estimators is that they are not necessarily scale invariant. For example, we have that

$$Y = X\beta + \epsilon \implies aY = aX\beta + a\epsilon.$$

Therefore, it is natural that we would expect the estimation procedure applied to (X, Y) to produce the same estimator for β as the estimation procedure applied to (aX, aY). Unfortunately, this is not always the case for M-estimators. One way to remedy this is to obtain a (robust) scale estimate s, and then define the estimate as

$$\underset{\beta}{\operatorname{argmin}} \sum_{i=1}^{n} \rho\left(\frac{Y_i - \beta^\top X_i}{s}\right).$$

For example, the median absolute deviation divided by 0.6745 is often used.

10.2 Different ρ functions

Let ψ be the derivative of ρ. Some common choices for ρ include the following:

10.2.1 Least Squares

Least squares loss, also known as quadratic loss, is the most common loss function in regression analysis. It is defined as the square of the residuals (the differences between observed and predicted values).

$$\rho(u) = u^2, \qquad \psi(u) = 2u.$$

It is highly sensitive to outliers because the squared term amplifies large residuals, making it less robust in the presence of outliers.

10.2.2 Huber's Loss

Huber's loss is a piecewise loss function that is quadratic for small residuals and linear for large residuals. It is less sensitive to outliers compared to least squares loss. This loss provides a compromise between least squares and absolute loss, offering robustness to outliers while retaining efficiency for small residuals.

$$\rho(u) = \begin{cases} u^2/2 & \text{if } |u| \leq \delta \\ \delta(|u| - \delta/2) & \text{if } |u| > \delta \end{cases}, \qquad \psi(u) = \begin{cases} u & \text{if } |u| \leq \delta \\ \delta \times sign(u) & \text{if } |u| > \delta \end{cases}.$$

The next three losses offer even more robustness to outliers, potentially at the cost of efficiency.

10.2.3 Ramsay's E Function

Ramsay's E function is used to give higher influence to residuals near zero and less influence to large residuals.

$$\rho(u) = \frac{1 - (1 + \delta|u|)\exp(-\delta|u|)}{\delta^2}, \qquad \psi(u) = sign(u)\exp(-\delta|u|)/\delta.$$

10.2.4 Andrews' Wave Function

Andrews' wave function is a loss function that limits the influence of large residuals more than Huber's loss.

$$\rho(u) = \delta\left(1 - \cos\left(\frac{u}{\delta}\right)\right), \qquad \psi(u) = \frac{\sin(u/\delta)}{u/\delta}.$$

10.2.5 Tukey's Loss

Tukey's loss function, also known as the biweight loss function, completely cuts off the influence of residuals beyond a certain point.

$$\rho(u) = \begin{cases} \frac{c^2}{6}\left[1 - \left(1 - \left(\frac{u}{c}\right)^2\right)^3\right] & \text{if } |u| \leq c \\ \frac{c^2}{6} & \text{if } |u| > c \end{cases}, \qquad \psi(u) = \begin{cases} u\left(1 - \left(\frac{u}{c}\right)^2\right)^2 & \text{if } |u| \leq c \\ 0 & \text{if } |u| > c \end{cases}.$$

Let's graph each of these functions:

```
# Define the least squares loss function
least_squares_loss <- function(u) {
  return(u^2/2)
}

# Define Huber's loss function
```

```r
hubers_loss <- function(u, delta=2) {
  return(ifelse(abs(u) <= delta, 0.5 * u^2, delta * (abs(u) - 0.5 * delta)))
}

# Define Ramsay's E function
ramsays_e_function <- function(u, delta=0.3) {
  return((1-(1+delta*abs(u))*exp(-delta*abs(u)))/(delta^2))
}

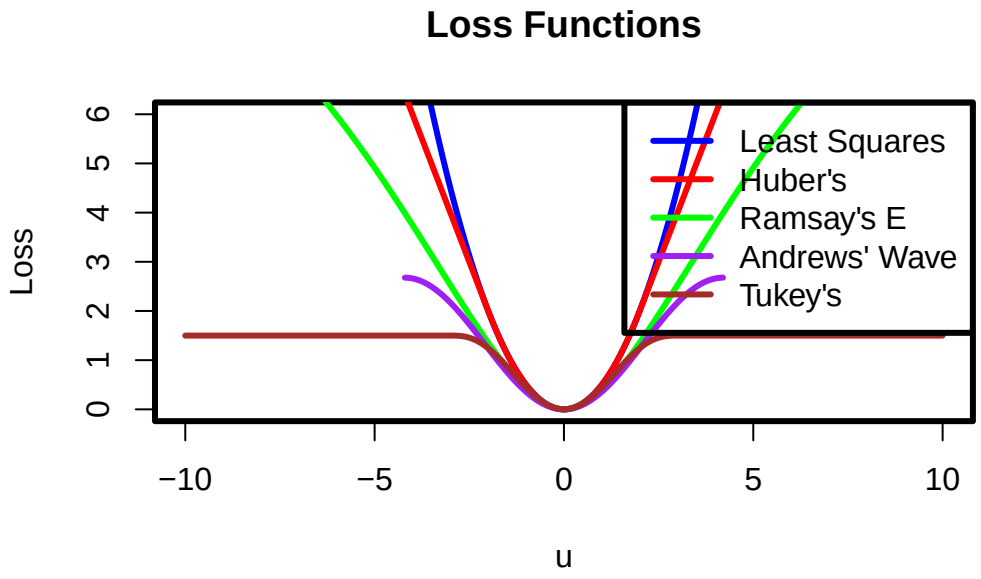
# Define Andrews' wave function
andrews_wave_function <- function(u, delta=1.339) {
  if(abs(u / delta)<pi)
    return(delta * (1 - cos(u / delta)))
  else
    return(NA)
}


# Define Tukey's loss function
tukeys_loss <- function(u, c=3) {
  abs_u <- abs(u)
  return(ifelse(abs_u <= c, (c^2 / 6) * (1 - (1 - (u / c)^2)^3), c^2 / 6))
}


u <- seq(-10,10, by = 0.1)

ls_loss <- sapply(u,least_squares_loss)
h_loss <- sapply(u,hubers_loss)
r_loss <- sapply(u,ramsays_e_function)
a_loss <- sapply(u,andrews_wave_function)
tukey_loss <- sapply(u,tukeys_loss)

par(lwd=3)
# Plotting example losses for visualization
plot(u, ls_loss, type = "l", col = "blue", ylim = c(0, 6), ylab = "Loss", xlab = "u", main =
lines(u, h_loss, col = "red")
lines(u, r_loss, col = "green")
lines(u, a_loss, col = "purple")
lines(u, tukey_loss, col = "brown")
legend("topright", legend = c("Least Squares", "Huber's", "Ramsay's E", "Andrews' Wave",  "Tu
       col = c("blue", "red", "green", "purple", "brown"), lty = 1)
```

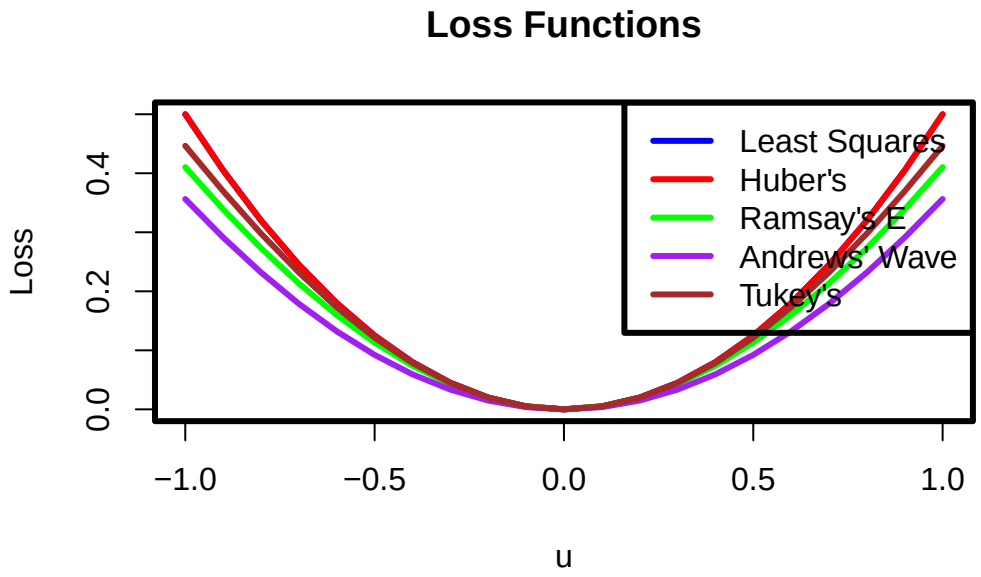



```
#Zooming in

u <- seq(-1, 1, by = 0.1)

ls_loss <- sapply(u,least_squares_loss)
h_loss <- sapply(u,hubers_loss)
r_loss <- sapply(u,ramsays_e_function)
a_loss <- sapply(u,andrews_wave_function)
tukey_loss <- sapply(u,tukeys_loss)

par(lwd=3)
# Plotting example losses for visualization
plot(u, ls_loss, type = "l", col = "blue", ylab = "Loss", xlab = "u", main = "Loss Functions"
lines(u, h_loss, col = "red")
lines(u, r_loss, col = "green")
lines(u, a_loss, col = "purple")
lines(u, tukey_loss, col = "brown")
legend("topright", legend = c("Least Squares", "Huber's", "Ramsay's E", "Andrews' Wave",  "Tu
       col = c("blue", "red", "green", "purple", "brown"), lty = 1)
```



We see that the cost attributed to large residuals varies based on the chosen function. Most of the functions appear quadratic near 0, but have quite different behaviours at large values. Tukey and Andrew's Wave function completely cut off the influence of large residuals, which the remaining functions just dampen the influence (except of course, the lease squares loss). Huber's function and the least squares function are convex functions, which make computation of the minimzer considerably easier. Huber's function is about as robust as we can be, while still maintaining a convex loss function. Data drawn from distributions with heavy tails (data with many "large" observations) require more robust loss functions, such as the Tukey biweight function.

The robustness of a regression procedure can be classified by the behavior of ψ, the derivative of ρ. The ψ function controls the weight given to each residual and is proportional to a central concept in robustness called the **influence function.** Unbounded influence functions are not desirable from a robustness perspective, as this means that one corrupted point is able to drag the estimated hyperplane arbitrarily far. The ψ function for least squares ρ is unbounded, and thus least squares tends to be "nonrobust" when used with data arising from a heavy - tailed distribution. On the other hand. the Huber loss function has a monotone ψ function and is not unbounded, making it more robust.

The other influence functions actually redescend as the residual becomes larger. Ramsay's E function is a soft redescender, that is, the ψ function is approaches zero as $|z| \to \infty$. Andrew's wave function and Tukey loss are hard redescenders. That is, the ψ function equals zero for sufficiently large $|z|$. Note that the ρ functions associated with the redescending ψ functions

are nonconvex, and this in theory can cause convergence problems in the iterative estimation procedure. We graph the functions below with their common default tuning parameters:

```
# Define the derivative of the least squares loss function
d_least_squares_loss <- function(u) {
  return(u)
}

# Define the derivative of Huber's loss function
d_hubers_loss <- function(u, delta = 2) {
  return(ifelse(abs(u) <= delta, u, delta * sign(u)))
}

# Define the derivative of Ramsay's E function
d_ramsays_e_function <- function(u, delta = 0.3) {
  return(sign(u)*exp(-delta*abs(u))/delta)
}

# Define the derivative of Andrews' wave function
d_andrews_wave_function <- function(u, delta = 1.339) {
  if(abs(u / delta) < pi)
    return(sin(u / delta))
  else
    return(NA)
}

# Define the derivative of Tukey's loss function
d_tukeys_loss <- function(u, c = 3) {
  abs_u <- abs(u)
  return(ifelse(abs_u <= c, u * (1 - (u / c)^2)^2, 0))
}

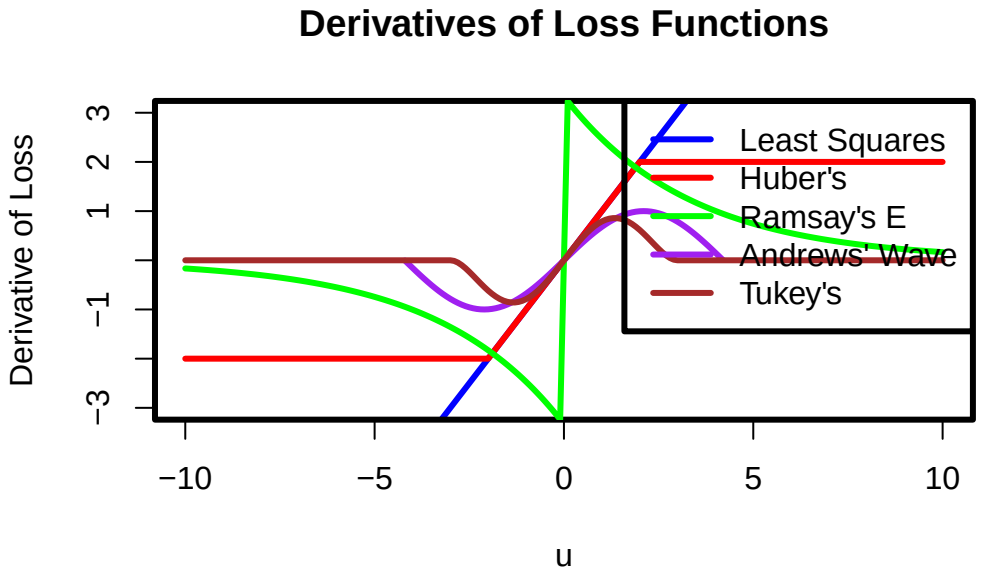


u <- seq(-10, 10, by = 0.1)

d_ls_loss <- sapply(u, d_least_squares_loss)
d_h_loss <- sapply(u, d_hubers_loss)
d_r_loss <- sapply(u, d_ramsays_e_function)
d_a_loss <- sapply(u, d_andrews_wave_function)
d_tukey_loss <- sapply(u, d_tukeys_loss)

par(lwd = 3)
# Plotting example derivatives for visualization
```

```r
plot(u, d_ls_loss, type = "l", col = "blue", ylim = c(-3, 3), ylab = "Derivative of Loss", xl
lines(u, d_h_loss, col = "red")
lines(u, d_r_loss, col = "green")
lines(u, d_a_loss, col = "purple")
lines(u, d_tukey_loss, col = "brown")
legend("topright", legend = c("Least Squares", "Huber's", "Ramsay's E", "Andrews' Wave",  "Tu
       col = c("blue", "red", "green", "purple", "brown"), lty = 1)
```



10.3 Computing M-estimators

10.3.1 Iterated re-weighted least squares

Let ψ be the derivative of ρ. A common way to compute the minimizer, of $C_\rho(\beta)$ is to use iterated re-weighted least squares. First, note that to find the minimizer, of $C_\rho(\beta)$, we solve the following system of equations:

$$\sum_{i=1}^{n} X_{ij} \psi\left(\frac{Y_i - X_i^\top \beta}{s}\right) = 0, \quad j = 0, 1, \ldots, k.$$

To do this, rewrite

$$\sum_{i=1}^{n} X_{ij}\psi\left(\frac{Y_i - X_i^\top\beta}{s}\right)\frac{Y_i - X_i^\top\beta}{s}\Big/\frac{Y_i - X_i^\top\beta}{s} = 0$$
$$\text{or, } \sum_{i=1}^{n} X_{ij}w_{i,\beta}\frac{Y_i - X_i^\top\beta}{s} = 0,$$

where

$$w_{i,\beta} = \psi\left(\frac{Y_i - X_i^\top\beta}{s}\right)\Big/\frac{Y_i - X_i^\top\beta}{s}.$$

Next, one will propose an initial estimate of the parameters α_0 and consider

$$\sum_{i=1}^{n} X_{ij}w_{i,\alpha_0}(Y_i - X_i^\top\beta) = 0.$$

Equivalently, we have $X^\top W_{\alpha_0}X\beta = X^\top W_{\alpha_0}Y$, where W_{α_0} is an $n \times n$ diagonal matrix of "weights" with diagonal elements $w_{1,\alpha_0}, \ldots, w_{n,\alpha_0}$. The algorithm proceeds as follows: iteratively compute $\alpha_i = (X^\top W_{\alpha_{i-1}}X)^{-1}X^\top W_{\alpha_{i-1}}Y$ until $\|\alpha_i - \alpha_{i-1}\| < \epsilon$ for small ϵ.

10.3.2 Gradient descent

We can also use gradient descent to compute the minimizer, of $C_\rho(\beta)$. To do this given a step size $\eta > 0$, iteratively compute

$$\alpha_i = \alpha_{i-1} - \eta \times \sum_{i=1}^{n} X_i\psi\left(\frac{Y_i - X_i^\top\alpha_{i-1}}{s}\right),$$

until $\|\alpha_i - \alpha_{i-1}\| < \epsilon$ for small ϵ.

Example 10.1. Consider the stack loss data, it records the percentage of stack loss in the operation of a plant that uses the oxidation of ammonia to produce nitric acid. The data set contains four variables and 21 observations.

Variables - **Air.Flow**: Flow rate of cooling air (in cubic meters per hour) - **Water.Temp**: Cooling water inlet temperature (in degrees Celsius) - **Acid.Conc.**: Acid concentration (percentage) - **stack.loss**: Stack loss (percentage of the ammonia lost)

Regress stack loss on the remaining variables. Compare various robust regression estimates to the OLS estimates. Add a large outlier and repeat the process. What do you observe?

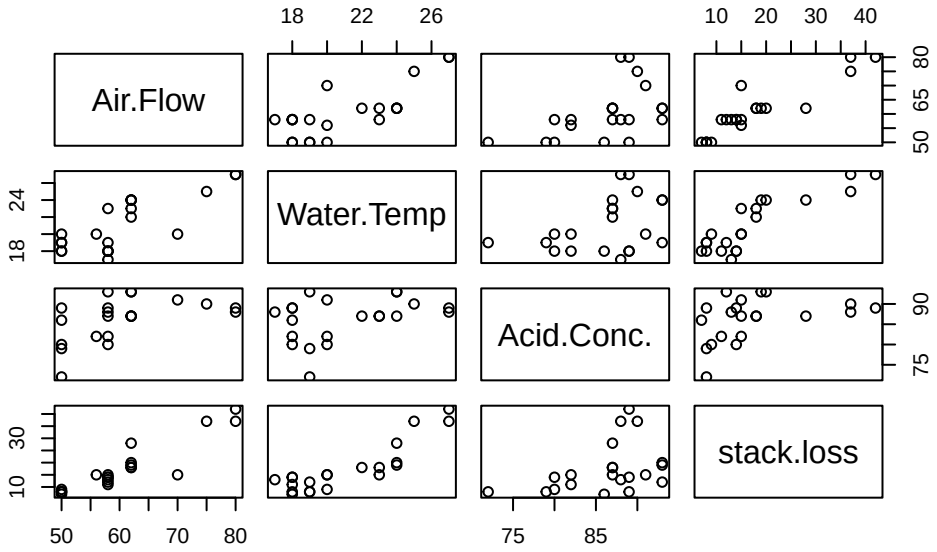
The `rlm` function in the `MASS` package allows us to run robust regression. Huber's loss is used by default.

```
# Computes how many standard errors coeficients are different by between models m1 and m2
compute_movement=function(m1,m2){
  vb=summary(m2)
  return(abs(coef(m2)-coef(m1))/vb$coef[,2])
}

library(MASS)
```

```
Warning: package 'MASS' was built under R version 4.5.3
```

```
plot(stackloss)
```




```
#fit robust regression model
# psi argument specifies psi...
# run ?rlm for more details.
OLS1=lm(stack.loss ~ ., stackloss)
RR=rlm(stack.loss ~ ., stackloss)
summary(OLS1)
```

```
Call:
```

```
lm(formula = stack.loss ~ ., data = stackloss)

Residuals:
    Min      1Q  Median      3Q     Max
-7.2377 -1.7117 -0.4551  2.3614  5.6978

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -39.9197    11.8960  -3.356  0.00375 **
Air.Flow      0.7156     0.1349   5.307  5.8e-05 ***
Water.Temp    1.2953     0.3680   3.520  0.00263 **
Acid.Conc.   -0.1521     0.1563  -0.973  0.34405
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.243 on 17 degrees of freedom
Multiple R-squared:  0.9136,    Adjusted R-squared:  0.8983
F-statistic:  59.9 on 3 and 17 DF,  p-value: 3.016e-09
```

```
summary(RR)
```

```
Call: rlm(formula = stack.loss ~ ., data = stackloss)
Residuals:
     Min       1Q   Median       3Q      Max
-8.91753 -1.73127  0.06187  1.54306  6.50163

Coefficients:
            Value    Std. Error t value
(Intercept) -41.0265   9.8073    -4.1832
Air.Flow      0.8294   0.1112     7.4597
Water.Temp    0.9261   0.3034     3.0524
Acid.Conc.   -0.1278   0.1289    -0.9922

Residual standard error: 2.441 on 17 degrees of freedom
```

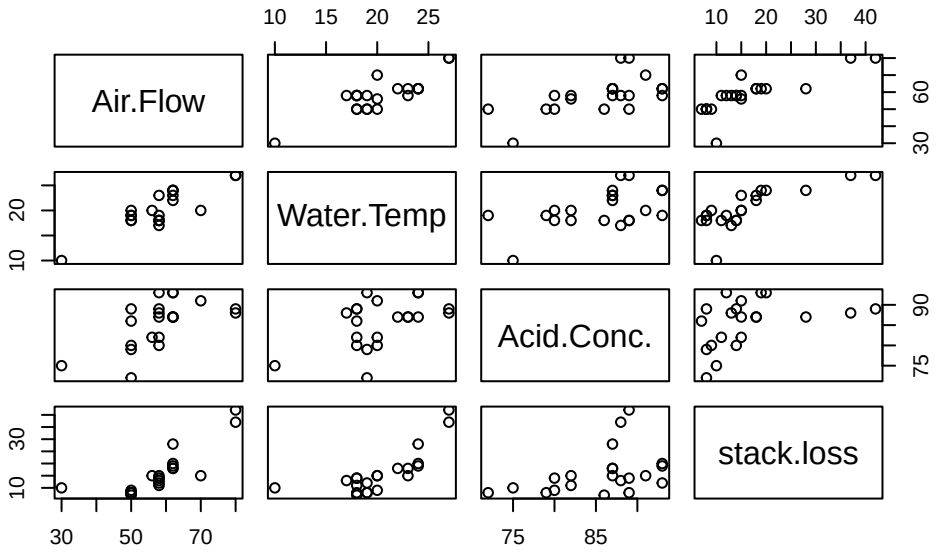
```
compute_movement(RR,OLS1)
```

```
(Intercept)    Air.Flow  Water.Temp  Acid.Conc.
 0.09304446  0.84335755  1.00313478  0.15530572
```

```
# Air flow and water temp moved one standard error.


# We can compare the estimates to their standard errors.

# Adding the outlier
stackloss2=stackloss
n=nrow(stackloss)
stackloss2[3,]=c(30,10,75,10)
plot(stackloss2)
```




```
OLS=lm(stack.loss ~ ., stackloss2)
RR_Hu=rlm(stack.loss ~ ., stackloss2, maxit = 100)
RR_Ha=rlm(stack.loss ~ ., stackloss2, psi = psi.hampel, maxit = 100)
RR_Tu=rlm(stack.loss ~ ., stackloss2, psi = psi.bisquare, maxit = 100)
```

```
Warning in rlm.default(x, y, weights, method = method, wt.method = wt.method, :
'rlm' failed to converge in 100 steps
```

```
# Error from uncorrupted estimates
error=rbind(compute_movement(OLS,OLS1),
```

```
compute_movement(RR_Hu,OLS1),
compute_movement(RR_Ha,OLS1),
compute_movement(RR_Tu,OLS1))
rownames(error)=c("OLS","Huber","Hampel","Tukey")
error
```

```
       (Intercept)    Air.Flow Water.Temp Acid.Conc.
OLS      2.3766707 1.317436984  1.4611311  0.4406644
Huber    0.5905931 0.006153154  1.4286807  0.2686428
Hampel   0.1180260 0.047197074  0.4429042  0.1073847
Tukey    0.2180443 0.716273230  2.0518634  0.5167066
```

```
rowSums(error)
```

```
     OLS    Huber   Hampel    Tukey
5.595903 2.294070 0.715512 3.502887
```

```
# summary(lm(stack.loss ~ ., stackloss2))
# summary(rlm(stack.loss ~ ., stackloss2))
# summary(rlm(stack.loss ~ ., stackloss2, psi = psi.hampel))
# summary(rlm(stack.loss ~ ., stackloss2, psi = psi.bisquare))


res=rbind(coef(OLS),
coef(RR_Hu),
coef(RR_Ha),
coef(RR_Tu),
coef(OLS1))
rownames(res)=c("OLS","Huber","Hampel","Tukey","Before Corruption")
res
```

```
                 (Intercept)  Air.Flow Water.Temp  Acid.Conc.
OLS                -11.64681 0.5379730  0.7575544 -0.22099574
Huber              -32.89398 0.7164700  0.7694970 -0.11013524
Hampel             -38.51564 0.7220051  1.1322866 -0.13533894
Tukey              -37.32582 0.8122355  0.5401506 -0.07136436
Before Corruption  -39.91967 0.7156402  1.2952861 -0.15212252
```

10.4 Homework questions

Complete question 15.5 in the textbook.

Exercise 10.1. In the previous examples given in class, add a large outlier and run both least squares and robust regression. How many standard errors did the coefficients move? How large does the outlier need to be for the regression to become corrupted? Repeat the process with a cluster of small outliers.

Exercise 10.2. Why are the OLS estimators susceptible to outliers?

Exercise 10.3. Implement gradient descent and IRLS in R for Huber's loss function.

References

Fox, John, and Georges Monette. 1992. "Generalized Collinearity Diagnostics." *Journal of the American Statistical Association* 87 (417): 178–83. https://doi.org/10.1080/01621459.1992.10475190.

Miller, Don M. 1984. "Reducing Transformation Bias in Curve Fitting." *The American Statistician* 38 (2): 124–26. http://www.jstor.org/stable/2683247.

A Introduction to R software

A.1 Some Basics

R is a Statistical Programming language, it consists of 2 types of objects: data and functions.

```
##Data
x<-2
print(x)
```

```
[1] 2
```

```
##function
log(2)
```

```
[1] 0.6931472
```

Data is stored in variables and can take many forms. To store a value in a variable use "<-", above we set the variable x equal to 2. There are many data types in R, we will go through some of them.

```
#real numbers
num=29.333
num
```

```
[1] 29.333
```

```
#Some math
#adding and subtraction
2+3-2
```

```
[1] 3
```

```
#multiplying and dividing
num<-5*(10/25)
num
```

```
[1] 2
```

```
#Strings
word<-"hello"
word
```

```
[1] "hello"
```

```
word='hello'
```

A.2 Booleans

Booleans take on either TRUE or FALSE values, and can be very useful in R. You can set booleans to the result of a comparison of two data types, some of the syntax is below:

- <,>,<=,>= corresponds to less than, greater than, less than or equal, greater than or equal
- ==, != equals, not equals
- && , written like a&&b where a and b are booleans, it is TRUE is *both* a and b are TRUE
- || , written like a||b where a and b are booleans, it is TRUE if at least *one of* a and b are TRUE

```
#booleans can be initialize in a variety of ways, for example
#must capitalize the true or false
FALSE
```

```
[1] FALSE
```

```
F
```

```
[1] FALSE
```

```
T
```

```
[1] TRUE
```

```
myBoolean<-TRUE
myBoolean
```

```
[1] TRUE
```

```
myBoolean2<- 3<4
myBoolean2
```

```
[1] TRUE
```

```
myBoolean3<-"this"=="that"
myBoolean3
```

```
[1] FALSE
```

```
## && (and) is TRUE if BOTH input booleans are true
## || (or) is TRUE if AT LEAST one input boolean is true
myBoolean4<-myBoolean2&&myBoolean
myBoolean4
```

```
[1] TRUE
```

A.3 Vectors

Vectors in R are used frequently, they are "lists" or "arrays" of all the same data type.

```
##vectors are created with c(data,data,data)
myVector<-c(2,3,4,5,6,7,8,9,10)
myVector
```

```
[1]  2  3  4  5  6  7  8  9 10
```

```
#a:b is a shortcut for a sequence from a to b adding 1
#you can create vectors of sequences using seq(), for more type ?seq in the console
myVector2<-2:10
myVector2
```

```
[1]  2  3  4  5  6  7  8  9 10
```

```
as.numeric(2:10)
```

```
[1]  2  3  4  5  6  7  8  9 10
```

```
as.double(2:10)
```

```
[1]  2  3  4  5  6  7  8  9 10
```

```
myVector2<-rep(NA,l=20)


#These do not have to be numbers, they can be vectors, Strings, booleans...
myVector<-c(myVector,myVector)
myVector
```

```
 [1]  2  3  4  5  6  7  8  9 10  2  3  4  5  6  7  8  9 10
```

```
myVector3<-c("this","is","a","vector","of","strings")
myVector3
```

```
[1] "this"    "is"      "a"       "vector"  "of"      "strings"
```

```
#access elements with square brackets []
myVector[1]
```

```
[1] 2
```

```
#more advanced accesssing
#access elements 1 to 5
myVector[1:5]
```

```
[1] 2 3 4 5 6
```

```
#access elements 1, 4 and 6
myVector[c(1,4,6)]
```

```
[1] 2 5 7
```

```
#access elements that are greater than 2
myVector[myVector>2]
```

```
 [1]  3  4  5  6  7  8  9 10  3  4  5  6  7  8  9 10
```

```
myVector[-c(1,4,6)]
```

```
 [1]  3  4  6  8  9 10  2  3  4  5  6  7  8  9 10
```

We can perform mathematical operations and comparisons on vectors

```
x<-1:10
x
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
#adds 1 to every element
x+1
```

```
 [1]  2  3  4  5  6  7  8  9 10 11
```

```
#this works for comparisons
x<4
```

```
 [1]  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
x[x<4]
```

```
[1] 1 2 3
```

```r
#multiplies element 1 to element 1 of second vectors
x*-(1:10)
```

```
 [1]   -1   -4   -9  -16  -25  -36  -49  -64  -81 -100
```

```r
#beware repetition
x-c(1,2)
```

```
 [1] 0 0 2 2 4 4 6 6 8 8
```

```r
# mathematical operations on the vector apply to each element

#squares each element
x^2
```

```
 [1]   1   4   9  16  25  36  49  64  81 100
```

```r
#log each element
log(x)
```

```
 [1] 0.0000000 0.6931472 1.0986123 1.3862944 1.6094379 1.7917595 1.9459101
 [8] 2.0794415 2.1972246 2.3025851
```

```r
#Example: Dot Product
x<-c(1,2,3)
y<-c(2,5,8)
#sum adds the elements of the vector together
sum(x*y)
```

```
[1] 36
```

A.4 Matrices

You can also use matrices in R.

```r
#you can create a matrix with matrix(vector of data,nrow=number of rows,ncol=number of colum
#You can see it will fill in the data down the columns first
myMatrix<-matrix(1:9,nrow=3,ncol=3); myMatrix
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```r
myMatrix
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```r
#rbind and cbind add a row or column repectively to the matrix
#you can create matrices with rbind(rowvector1,rowvector2,...), or with cbind(column vector

myMatrix<-rbind(c(2,3,4),c(3,4,5),c(1,2,3))
myMatrix
```

```
     [,1] [,2] [,3]
[1,]    2    3    4
[2,]    3    4    5
[3,]    1    2    3
```

```r
myMatrix2<-cbind(c(1,2,3),c(4,5,6),c(7,8,9))
myMatrix2
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```r
myMatrix3<-cbind(myMatrix2,c(10,11,12))
myMatrix3
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

```
myMatrix3<-cbind(c(10,11,12),myMatrix2)
```

We can also do Matrix math:

```
#again math functions apply to every element
myMatrix^2
```

```
     [,1] [,2] [,3]
[1,]    4    9   16
[2,]    9   16   25
[3,]    1    4    9
```

```
#multiply with '%*%
myMatrix2%*%myMatrix
```

```
     [,1] [,2] [,3]
[1,]   21   33   45
[2,]   27   42   57
[3,]   33   51   69
```

```
#we can find the inverse with 'solve()
X<-matrix(c(1,0,1,-2,3,0,1,4,2),nrow=3)
X
```

```
     [,1] [,2] [,3]
[1,]    1   -2    1
[2,]    0    3    4
[3,]    1    0    2
```

```
solve(X)
```

```
     [,1] [,2] [,3]
[1,] -1.2 -0.8  2.2
[2,] -0.8 -0.2  0.8
[3,]  0.6  0.4 -0.6
```

```r
#check dimension
dim(X)
```

```
[1] 3 3
```

```r
#We can also transpose with t()
t(X)
```

```
     [,1] [,2] [,3]
[1,]    1    0    1
[2,]   -2    3    0
[3,]    1    4    2
```

```r
#Some times to multiply vectors we have to trun them into matrix types
myVector<-c(1,2,3)
newM<-matrix(myVector,ncol=1)
```

A.5 Functions

Functions are objects that take an input and transform it into some output, just like in mathematics. We have already seen some, such as `log()`.

They are called with this format `output<-functionName(input)`.

- The input is called *parameters*, and there can be many parameters
- parameters are usually described in the documentation
- the output is what the function *returns*
- functions can only return 1 object, but this includes a list… so it could return many objects in the form of a list object

R has many, many functions, to learn more about a function type `?functionName` and the documentation will come up.

```r
#A simple function
#here the function log is called, with the parameter 2, and the output is stored in the varia
x<-log(2)
x
```

```
[1] 0.6931472
```

```r
#A more complicated function
#What are the parameters?
#not rep(a,n) gives a vector of size n where all elements are a
s<-sample(x=1:10,size=4,replace=TRUE,prob=rep(1/10,10))
s
```

```
[1] 10  8  8  7
```

We have seen other people's functions but we can also make our own! Let's see an example first:

```r
#recall the dot product example...
dotProd=function(a,b){
  value<-sum(a*b)
  return(value)
}
#calling our function
dotProd(x,y)
```

```
[1] 10.39721
```

What exactly does this code say?

- We stored the function in the variable `dotProd`
- to tell the compiler we are creating a function, we use the keyword function
- we specify the parameters in round brackets ()
- we put the names of the parameters in the () only, not what data type we expect them to be
- inside curly brackets, we put the code that the function will run when it is called
- `return()` ends the function, and sends back the variable in the brackets

Back to built in functions… `R` is a statistical software, what does that mean? It already includes many common statistical functions! For most common distributions there are functions for the pdf, cdf, inverse cdf as well as one to get a sample from that distribution. The syntax is in the format: `dDistName(x,parameters)`, `pDistName(x,parameters)`, `qDistName(x,parameters)` and `rDistName(x,parameters)` respectively. This will make more sense in the example below…

```r
#The normal distribution, sd is the standard deviation
#pdf
dnorm(c(2,3,5),mean=0,sd=1)
```

```
[1] 5.399097e-02 4.431848e-03 1.486720e-06
```

```
#cdf
pnorm(c(2,3,5),mean=0,sd=1)
```

```
[1] 0.9772499 0.9986501 0.9999997
```

```
#inverse cdf
qnorm(c(0.2,.5,.3),mean=0,sd=1)
```

```
[1] -0.8416212  0.0000000 -0.5244005
```

```
#random sample of size 10
rnorm(10,mean=0,sd=1)
```

```
 [1] -2.7021484  1.6378046  0.6735426 -0.5338139 -1.0672702 -1.6657685
 [7]  0.4911196  1.4835815  0.5874878  0.9858748
```

A.6 Plotting

R is very good for plotting! There are many types of plots in R, here are some useful plotting functions, this list is not exhaustive...

- `plot(x,y,...)` produces a scatter plot.
- `abline(a=intercept,b=slope,...)`
- `curve(expr,...)` evaluates an expression along a grid to create a curve
- `hist(data)` creates a histogram

Plot functions have many parameters, some include `col` which changes the color and add which should be set to `TRUE` if the plot should be added to the existing plot. The best way to learn plots is with examples, I have included a regression example below.

```
#simulate errors
epsilon<-rnorm(100)
x<-rexp(100)
y<-9+2*x+epsilon

#scatter plot with true line
plot(x,y)
```

```r
abline(a=9,b=2,col="blue")

#least squares line
lmm<-lm(y~x)
summary(lmm)
```

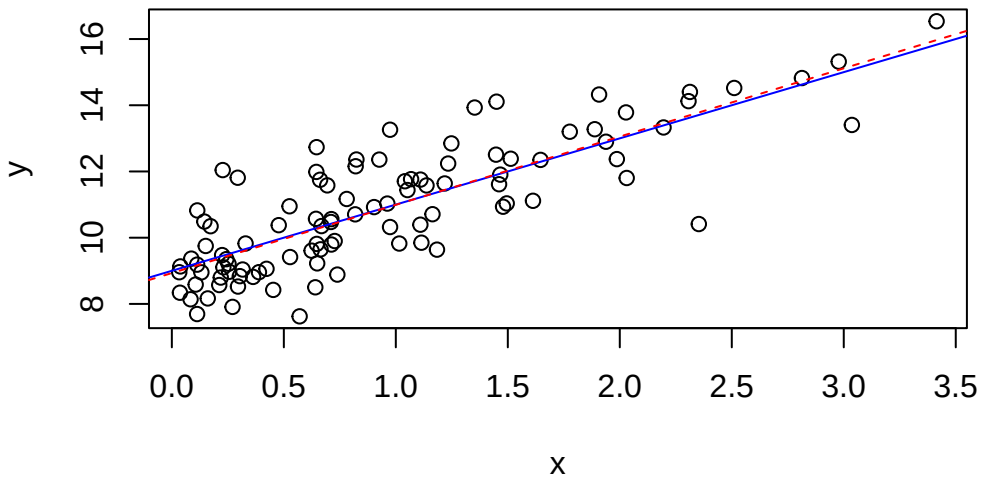
```
Call:
lm(formula = y ~ x)

Residuals:
    Min      1Q  Median      3Q     Max
-3.3687 -0.6812  0.0336  0.5961  2.6413

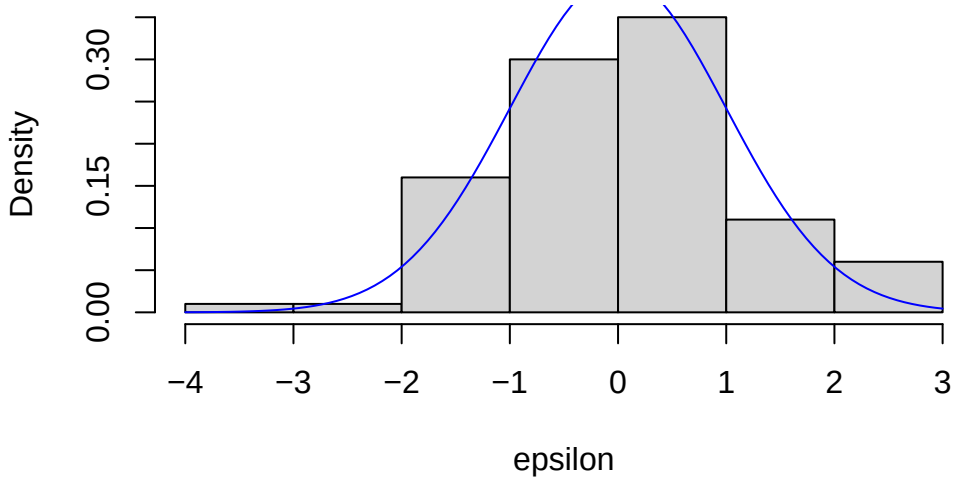
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   8.9324     0.1776   50.29   <2e-16 ***
x             2.0607     0.1462   14.10   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.112 on 98 degrees of freedom
Multiple R-squared:  0.6698,    Adjusted R-squared:  0.6664
F-statistic: 198.8 on 1 and 98 DF,  p-value: < 2.2e-16
```

```r
abline(lmm$coefficients[1],lmm$coefficients[2],col="red",lty=2)
```




```
#histogram of residuals
hist(epsilon,freq = F)
#x is what you want to evaluate the grid along
curve(dnorm(x),add=T,col="blue")
```



A.7 If Statements

If statements are essential in programming, and they are a form of 'Control Structure'. They take the form `if(boolean variable){some task}`.
When the computer runs through the code, it checks if the boolean value is `TRUE`, and if it is, it executes the code in the curly brackets, code in curly brackets is called a *block*. A simple example...

```
jim<-"nice"

if(jim=="nice"){
  alice="nice"
}
```

Placing an `else{some code}` after the if statement will execute the code in it's block if the code in the above if statement *was not* executed. The if and else must be in the same block so I have surrounded them in curly brackets.

```
jim<-"nice"
##same block
{
```

```
if(jim=="nice"){
  alice="nice"
}
else{
  alice="not nice"
}
  alice
}
```

```
[1] "nice"
```

```
jim<-"mean"
##same block
{
if(jim=="nice"){
  alice="nice"
}
else{
  alice="not nice"
}
  alice
}
```

```
[1] "not nice"
```

You may also use `else if(boolean){block}`, which executes it's block if the above (else) if statement(s) did not execute. See below:

```
jim<-"okay"
##same block
{
if(jim=="nice"){
  alice="nice"
}
else if(jim=="okay"){
  alice="okay"
}
  #Here if jim is not okay or nice, then we check if he is neutral.
else if(jim=="neutral"){
  alice="neutral"
}
```

```
else{
  alice="not nice"
}
  alice
}
```

```
[1] "okay"
```

Lastly you may put if statements inside of other if statements, called 'nested ifs'.

```
jim<-"nice"
##same block

if(jim=="nice"){
  alice=sample(c("nice","not nice"),1)
  if(alice=="nice"){
    print(alice)
  }
  else{
    print(alice)
  }
}
```

```
[1] "nice"
```

A.8 Loops

Loops execute operations within their blocks repeatedly. There are 2 types of loops you will generally use, for loops and while loops. For loops repeat the block a set number of times, while while loops repeat until a condition is satisfied. You can also nest loops, like if statements.

```
#calculate 2 to the power of ten
x<-1
#this reads for i in 1 to 10, this can be any vector that i loops through, not just a sequent
for(i in 1:10){
  x<-x*2
}

x
```

```
[1] 1024
```

```
for(i in 1:10){
  x<-x+i
}

vec=2:5

for(i in vec){
  x<-x+i
}


#calculate power of 2 less that 1000
x<-1
while(2*x<1000){
  x<-x*2
}
x
```

```
[1] 512
```

```
#nested loop
for(i in c(10,9,8,7,6,5,4,3,2,1)){
  v<-NULL
  for(j in 1:i){
    v<-c(v,"*")
  }
  print(v)
}
```

```
 [1] "*" "*" "*" "*" "*" "*" "*" "*" "*" "*"
[1] "*" "*" "*" "*" "*" "*" "*" "*" "*"
[1] "*" "*" "*" "*" "*" "*" "*" "*"
[1] "*" "*" "*" "*" "*" "*" "*"
[1] "*" "*" "*" "*" "*" "*"
[1] "*" "*" "*" "*" "*"
[1] "*" "*" "*" "*"
[1] "*" "*" "*"
[1] "*" "*"
[1] "*"
```

You can also use the replicate function, which replicates a line of code a specified number of times. This gives a 10 by 5 matrix.

```
replicate(5,rnorm(10))
```

```
            [,1]        [,2]       [,3]        [,4]       [,5]
 [1,] -0.5305250  0.09535009 -1.1784444  2.00025731 -1.0042187
 [2,] -0.9485603  0.11254290  1.3160360  2.01428169 -1.4256438
 [3,] -1.3094916 -0.68326035 -0.7718733 -0.23860536  0.4782457
 [4,] -0.4238648  1.13403817 -0.8608517 -0.32528691  0.5864753
 [5,] -1.0959917  0.10554845  1.1062442  1.14306119  0.6771475
 [6,] -0.1310219  0.36827042 -1.7824584 -1.52235683  0.3061647
 [7,] -0.4187042 -0.37325039 -0.8720916  0.50889237 -0.3404331
 [8,]  1.2825202 -1.98169543 -0.6209609 -0.37049978  0.9394566
 [9,]  0.4632472 -0.27569461 -1.2788915 -0.01171678 -0.6543562
[10,] -0.4130678  1.77425593 -0.8625001 -0.92773144  0.3561823
```

Similar functions include `sapply()` and `apply()`. `sapply(X,FUN,...)` applies the function that the parameter `FUN` is set to to individual elements of a vector. `apply(X,MARGIN,FUN,...)` applies `FUN` to the rows or columns depending on what `MARGIN` is set to, 1 for rows and 2 for columns.

A.9 Coverage Probability Example

Here we generate 10000 samples of size 100 from the exponential distribution, with $\lambda = 2$. We calculate 10000 confidence intervals for $1/\lambda$ with \$ =\$1%, using the normal approximation:

$$\sqrt{n}(\bar{X} - 1/\lambda) \sim N(0, 1/\lambda^2)$$

and interval:

$$(\bar{X} - t_{99}(0.005) * S/\sqrt{n}, \bar{X} + t_{99}(0.005) * S/\sqrt{n})$$

We then check the proportion of intervals that contain the true value of $1/\lambda$.

```
#10000 samples, each of size 100 from the exponential distribution
x<-replicate(10000,rexp(100,rate=2))
#x is 100 by 10000, each column is a sample
dim(x)
```

```
[1]   100 10000
```

```r
#calculate sample variances
S_Vector<-apply(x,2,sd);

# S_Vector

#get the t value
tval<-qt(1-0.005,99)
#calculate the means

means<-apply(x,2,mean); length(means)
```

```
[1] 10000
```

```r
# lower and upper bounds
lower<-means-S_Vector*tval/10
upper<-means+S_Vector*tval/10
intervals<-rbind(lower,upper)
#example interval
intervals[,1]
```

```
    lower     upper
0.3515522 0.6353252
```

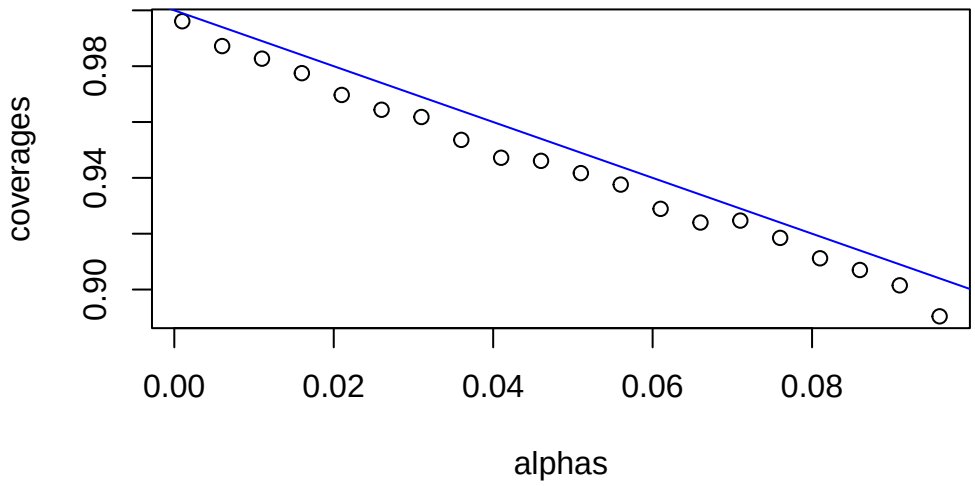
```r
#we now check each interval to see if it contains the mean
successes<-0
for(i in 1:ncol(intervals)){
  #if 0.5 is in the interval, add 1
  if((intervals[1,i]<0.5)&&(intervals[2,i]>0.5))
    successes<-successes+1
}
#here is the coverage probability...
coverage.prob<-successes/ncol(intervals)
coverage.prob
```

```
[1] 0.9814
```

Something more advanced...

```r
#Vectorzing the function changes the way the function calculates when it is a passed a vecto
#it will run the function once per element if it is vectorized instead of passing the vector
getCovProb<-Vectorize(function(alpha){
#10000 samples, each of size 100 from the exponential distribution
  x<-replicate(10000,rexp(100,rate=2))
#x is 100 by 10000, each column is a sample
  dim(x)
#calculate sample variances
  S_Vector<-apply(x,2,sd)
#get the t value
  tval<-qt(1-alpha/2,99)
#calculate the means
  means<-apply(x,2,mean)
# lower and upper bounds
  lower<-means-S_Vector*tval/10
  upper<-means+S_Vector*tval/10
  intervals<-rbind(lower,upper)
#example interval
  intervals[,1]

#we now check each interval to see if it contains the mean
successes<-0
for(i in 1:ncol(intervals)){
  #if 0.5 is in the interval, add 1
  if((intervals[1,i]<0.5)&(intervals[2,i]>0.5))
    successes<-successes+1
}
#here is the coverage probability...
coverage.prob<-successes/ncol(intervals)
return(coverage.prob)
})
#here we find the coverage probability for many alphas
alphas<-seq(from=0.001,to=0.1,by=0.005)
coverages<-getCovProb(alphas)
#adds a scatter plot
plot(alphas,coverages)
#adds a line
abline(a=1,b=-1,col="blue")
```



For more information you can visit here . It is also very easy to find tutorials on the web (Youtube is good), you could also look at the book by Lafaye, Drouilhet and Liquet (2013).